

JR LEARNERS

BASIC EMBEDDED SYSTEMS LABORATORY MANUAL

50 Hands-On Experiments with the JR Learners Arduino Starter Kit

JR Learners · STEM Education Series

First Edition

JR LEARNERS — Basic Embedded Systems Laboratory Manual: 50 Hands-On Experiments

First Edition

Published by **JR Learners**, in support of the *JR Learners Arduino Starter Kit*.

© JR Learners. All rights reserved. No part of this manual may be reproduced, distributed, or transmitted in any form without prior written permission of the publisher, except for brief excerpts used in classroom instruction by educators who own a licensed copy of this manual.

Trademarks: Arduino® and the Arduino logo are trademarks of Arduino S.r.l. This manual is an independent educational work and is not affiliated with or endorsed by Arduino S.r.l. All other product names, logos, and brands referenced (e.g. sensor and module names) are property of their respective owners and are used here only for identification and educational purposes.

Circuit diagrams and illustrations: All circuit diagrams, breadboard drawings, and figures in this manual are original artwork produced programmatically for this edition. No copyrighted textbook illustrations or third-party circuit diagrams have been reproduced.

Disclaimer: The experiments in this manual are designed for supervised educational use. JR Learners and the authors accept no liability for damage to equipment or injury arising from unsupervised or improper use. Always follow the laboratory safety instructions in this manual.

Corrections and feedback: Suggested corrections, errata, and feedback from instructors are welcome for future editions.

About JR Learners

JR Learners designs hands-on embedded-systems learning kits for schools, polytechnics, and engineering colleges. The **Arduino Starter Kit** bundles an Arduino UNO R3, a breadboard, a curated set of sensors and actuators (including an SG90 micro servo motor), and basic passive components into a single storage box, so that a learner can move from “what is a resistor” to “I built a working automation prototype” using one consistent set of hardware.

This manual is written to be used *with* that exact kit. Every experiment is deliberately restricted to the components listed in Chapter , so that a student who owns the kit never has to source extra parts to complete the book.

How this manual is organised

The 50 experiments are arranged in three learning layers of increasing difficulty:

- **Layer 1 — SIMPLE (Experiments 1–17):** component identification, digital I/O, analog input, and single-sensor circuits.
- **Layer 2 — AVERAGE (Experiments 18–34):** the 16×2 I²C LCD, quantitative sensor readout, and simple decision-making (alarms, thresholds).
- **Layer 3 — ADVANCED (Experiments 35–50):** multi-sensor integration, finite-state machines, non-blocking scheduling, and a capstone home-automation prototype.

Every experiment follows the same 20-part structure described in the Preface, so that once a student has completed Experiment 1, the *format* of every later experiment is already familiar — only the engineering content changes.

Preface

Embedded systems are best learned by building, breaking, and rebuilding real circuits — not by reading about them. This manual was written so that a beginner with no prior electronics background can, over 50 guided experiments, become comfortable with digital and analog I/O, common sensors, actuators, the I²C bus, and the discipline of writing responsive, non-blocking embedded software, using only the parts already inside the JR Learners Arduino Starter Kit.

Every experiment in this book has been checked against three constraints before being finalised: (1) it uses *only* hardware present in the kit inventory, in quantities the kit actually contains; (2) every circuit is electrically safe at 5 V logic levels with correctly chosen current-limiting resistors; and (3) every program is complete, compilable Arduino C/C++, not pseudocode. Section documents this feasibility review experiment-by-experiment.

Learning Objectives

By the end of this manual, a student will be able to:

1. Identify and correctly wire every component in the Arduino Starter Kit on a breadboard.
2. Explain and apply Ohm’s law, digital logic levels, PWM, and analog-to-digital conversion in the context of a real Arduino circuit.
3. Read datasheet-level electrical characteristics (voltage, current, timing) for LEDs, sensors, and modules, and choose correct resistor values and pin modes accordingly.
4. Write, upload, and debug Arduino sketches that use digital I/O, analog I/O, PWM, the I²C bus, timing with `millis()`, software debouncing, and basic finite-state machines.
5. Interface single sensors (LDR, IR obstacle, ultrasonic, soil moisture, DHT11, IR receiver) and combine several of them into one coherent, non-blocking embedded application.
6. Design, build, test, and document a small multi-sensor embedded system (the Layer 3 capstone project) against a written requirements and validation checklist.

Intended Audience

This manual is written for four overlapping audiences, each of whom can use the same 50 experiments at a different pace:

- **School students (Grades 9–12)** taking their first course in electronics or robotics.
- **Diploma students** in Electronics, Electrical, or Computer Engineering needing a structured embedded-lab component.
- **Undergraduate engineering students** (ECE/EEE/CSE/Mechatronics) who need a fast, hands-on primer before project work.

-
- **Self-learners and hobbyists** who have purchased the kit and want a guided curriculum rather than scattered tutorials.

Prerequisites

No prior programming or electronics experience is assumed for Layer 1. Before starting Experiment 1, a learner should:

- Have a Windows, macOS, or Linux computer with a free USB port and administrator rights to install software.
- Be comfortable using a keyboard/mouse and installing a desktop application.
- Have basic middle-school arithmetic (for Ohm's law calculations in later experiments).

From Layer 2 onward, a learner is assumed to have completed the corresponding prerequisite experiments listed in each Experiment Identification box, since later experiments deliberately reuse circuits and code from earlier ones rather than repeating explanations.

Laboratory Safety Instructions

All 50 experiments in this manual operate at 5 V logic level or below, and every actuator is either an LED, a buzzer, or a 5 V relay switching a low-voltage demonstration load. No experiment requires mains (230 V/110 V) wiring. Nonetheless, the following rules must be followed at all times.

Safety Note

1. **Power off before wiring.** Always disconnect the USB cable / 5 V adapter before changing any breadboard connection. Never rewire a live circuit.
2. **Check polarity before power-up.** LEDs, the buzzer, the PN-junction diode, and the DHT11/HC-SR04/relay modules are polarity- or pinout-sensitive. Reversed power to a module can destroy it. Verify VCC/GND against the connection table before switching on.
3. **Never bypass the current-limiting resistor** on an LED. An LED wired directly across 5 V without a series resistor will draw excessive current and fail within seconds.
4. **Respect the Arduino UNO's electrical limits.** Do not draw more than 20 mA from any single GPIO pin, or more than roughly 200 mA total from all I/O pins combined, and never draw actuator current directly through a GPIO pin (see Chapter).
5. **The relay module switches its own isolated contact circuit.** In this kit the relay is used only for low-voltage demonstration loads (e.g. a second LED/buzzer branch). Never wire the relay's mains-rated contacts to a household mains socket — that use is explicitly outside the scope of this manual and this kit.
6. **Handle the laser diode module responsibly.** Never point the laser module at anyone's eyes; treat it as a Class 2 low-power laser pointer and use it only to interrupt the IR/LDR beam-break demonstrations described in this manual.
7. **Static and mechanical care.** Handle the LCD, HC-SR04, DHT11, and IR modules by their edges/plastic body, avoid touching header pins directly, and store the kit in the provided storage box when not in use.
8. **Common ground.** Whenever a sensor module is powered from the Arduino's 5 V pin, its GND *must* also be tied to an Arduino GND pin, even if it seems to work briefly without one. A missing common ground is the single most common cause of “sensor gives random readings”.
9. **Adult supervision.** School-age learners should carry out these experiments under the supervision of an instructor, particularly Experiments 12–17, 31, and 40–50, which use sound-producing or moving elements.
10. **No physical testing claims.** Circuits and expected outputs in this manual are engineering-verified on paper (electrically and logically); treat any “expected output” value as an illustrative target to compare against your own measured result, not as a substitute for actually building and testing the circuit yourself.

Complete Component Inventory

Every circuit in this manual is built exclusively from the items below. Where the exact silicon variant of a module cannot be verified from the kit name alone, this manual states its **assumed variant** explicitly; before relying on any datasheet number quoted later in this book, compare it against the label/silkscreen on your own module.

#	Component	Qty	Assumed variant / key specification
1	Arduino UNO R3 + USB cable	1	ATmega328P, 16 MHz, 5 V logic, 14 digital I/O (6 PWM), 6 analog in
2	5 V power adapter	1	Regulated 5 V, barrel-jack, used when not USB-powered
3	Multi-output breadboard power module	1	Supplies 5 V/3.3 V rails to the breadboard from the adapter
4	Breadboard	1	Standard 830-tie-point half+ breadboard
5	Assorted resistors	100	Carbon film, 1/4 W, mixed values (220 Ω , 330 Ω , 1 k Ω , 10 k Ω used most)
6	Assorted LEDs (red, blue, green, white, yellow)	25	5 mm through-hole, $V_f \approx 1.8\text{--}2.2$ V (red/yellow), 2.8–3.2 V (blue/white)
7	5 V relay module	1	Single-channel, opto-isolated trigger, coil driven by a transistor on-board (safe to drive from a GPIO through the module's own IN pin)
8	Connectivity tester	1	Simple continuity/buzz tester for verifying jumper wires and breadboard rows before powering a circuit
9	Push buttons	5	4-leg tactile momentary switch, normally open
10	16×2 LCD with I ² C interface	1	HD44780-compatible LCD on a PCF8574 I ² C backpack, default address 0x27 (or 0x3F)
11	Female-to-female jumper wires	10	For module-to-module / module-to-Arduino-header links
12	Male-to-female jumper wires	10	For breadboard-to-module links
13	Push-button module	1	3-pin breakout (VCC, GND, OUT) with on-board pull-down, digital HIGH on press
14	Male-to-male jumper wires	10	For breadboard-to-Arduino links
15	LDR (light dependent resistor)	2	5 mm CdS photoresistor, ≈ 1 k Ω (bright) to >200 k Ω (dark), used with a fixed resistor as a voltage divider

#	Component	Qty	Assumed variant / key specification
16	IR obstacle sensor	1	3-pin breakout (VCC, GND, OUT), on-board IR LED + phototransistor + comparator, active-LOW digital output, onboard sensitivity trimmer
17	HC-SR04 ultrasonic sensor	1	4-pin (VCC, Trig, Echo, GND), 5 V, 2–400 cm range, 15° beam angle
18	Soil moisture sensor	1	Resistive 2-prong probe + comparator breakout, analog (AO) and digital (DO) outputs, VCC/GND
19	Air moisture/temperature sensor	1	Assumed DHT11: 3-pin (VCC, DATA, GND), single-wire digital protocol, 0–50°C ($\pm 2^\circ\text{C}$), 20–90%RH ($\pm 5\%\text{RH}$), ≥ 1 s sample interval
20	Buzzers	2	Assumed one active buzzer (has internal oscillator, sounds a fixed tone on DC) for ON/OFF and doorbell experiments, and one passive buzzer/piezo (needs an external AC-like signal) for <code>tone()</code> -based variable-tone experiments. Confirm which is which on your kit: an active buzzer clicks/buzzes on a steady 5 V; a passive buzzer stays silent on steady DC and only sounds when driven with <code>tone()</code>
21	Laser diode/module	1	Assumed KY-008-style 3-pin (VCC, GND, S) 5 mW visible-red laser diode module
22	PN-junction diode	1	Assumed 1N4007 general-purpose silicon rectifier, $V_f \approx 0.7\text{ V}$, used only for polarity/rectification demonstrations at low voltage/current
23	IR receiver	1	Assumed TSOP38238-style 3-pin 38 kHz IR demodulator receiver (VCC, GND, OUT)
24	Potentiometer	1	Assumed 10 kΩ linear rotary potentiometer, 3-pin
25	Storage box	1	Kit storage/organiser
26	USB-to-C converter	1	USB-A-to-USB-C adapter for connecting the UNO to USB-C-only computers
27	SG90 micro servo motor	1	5 V hobby servo, 0–180°, PWM control signal (not used by Experiments 1–50; see the Bonus Experiment, Chapter 51)

Important hardware rule

No experiment among the 50 numbered experiments in this manual introduces a pump, transistor, external IR transmitter, or any sensor not listed above. Wherever a “real-world” version of a project would use an actuator not in the kit (e.g. an actual water pump in Experiment 29, or an actual motor lock in Experiment 40), this manual explicitly builds a **safe low-voltage demonstration** using an LED and/or the relay module instead, and says so in that experiment’s Aim. The kit’s one SG90 servo motor (#27) is reserved for the standalone Bonus Experiment (Chapter 51), so that Experiments 1–50 remain fully reproducible from a kit that has only items

Component Identification Guide

- **Resistors** are colour-banded; see Appendix D for the full colour-code table. This manual always states the required value in ohms next to the resistor in the connection table.
- **LEDs** have two legs of different length: the *longer* leg is the anode (+), the *shorter* leg (next to a flat edge on the rim) is the cathode (–).
- **The PN-junction diode** has a painted band at one end marking the cathode; current can only flow from anode to cathode when forward biased.
- **Push buttons** (4-leg) have two *pairs* of legs that are pre-connected internally; the two pairs are only joined to each other when the button is pressed. Always wire across a pair from *different* sides, never two legs of the same pair.
- **The push-button module, IR obstacle sensor, HC-SR04, DHT11, soil-moisture sensor, and laser module** are all small PCBs with silkscreen-printed pin names (VCC/+, GND/–, and a signal pin such as OUT/DATA/Trig/Echo/S). Always read the silkscreen rather than assuming pin order.
- **The I²C LCD backpack** has exactly 4 pins: GND, VCC, SDA, SCL — it never uses more wires than that, regardless of the 16 pins on the LCD glass underneath it.

Arduino UNO R3 Architecture and Pinout

Architecture overview

The Arduino UNO R3 is a development board built around the **ATmega328P** microcontroller (Microchip/Atmel), an 8-bit AVR running at 16 MHz from an on-board crystal. Key internal blocks relevant to every experiment in this manual:

- **CPU + Flash/SRAM/EEPROM:** 32 KB flash (program storage, of which ~0.5 KB is used by the bootloader), 2 KB SRAM (variables/stack at runtime), 1 KB EEPROM (persists after power-off, not used in this manual).
- **Digital I/O (D0–D13):** 14 pins, each configurable as `INPUT`, `INPUT_PULLUP`, or `OUTPUT` via `pinMode()`. D0/D1 double as the hardware UART (Serial) RX/TX — avoid using them for other signals while also using `Serial`.
- **PWM-capable pins:** D3, D5, D6, D9, D10, D11 (marked with a ~ on the silkscreen) support `analogWrite()`, producing a fast on/off square wave whose average voltage is controlled by duty cycle — this is how this manual dims LEDs and (with `tone()`, a separate timer-based mechanism) drives the passive buzzer.
- **Analog input (A0–A5):** 6 channels into a 10-bit ADC (0–1023 codes over 0–5 V, i.e. $\approx 4.9 \text{ mV/code}$), read with `analogRead()`.
- **I²C bus:** fixed to **A4 (SDA)** and **A5 (SCL)** on the UNO, used by every LCD experiment from Experiment 18 onward.
- **Power:** USB (5 V, current-limited by the host PC, typically 500 mA) or the barrel-jack 5 V adapter (regulated on-board to 5 V and 3.3 V rails).

Absolute electrical limits — never exceed

- Maximum **40 mA** per I/O pin (recommended design limit: 20 mA, which this manual always uses for LED resistor sizing).
- Maximum **200 mA** total across all I/O pins combined.
- Logic HIGH = 5 V, LOW = 0 V; never apply more than 5 V (or less than 0 V) to any I/O pin.
- Never connect an actuator that needs more current than a GPIO pin can supply (e.g. a motor) directly to a pin — this is precisely why the relay module exists, and why this kit contains no motor.

Pinout diagram

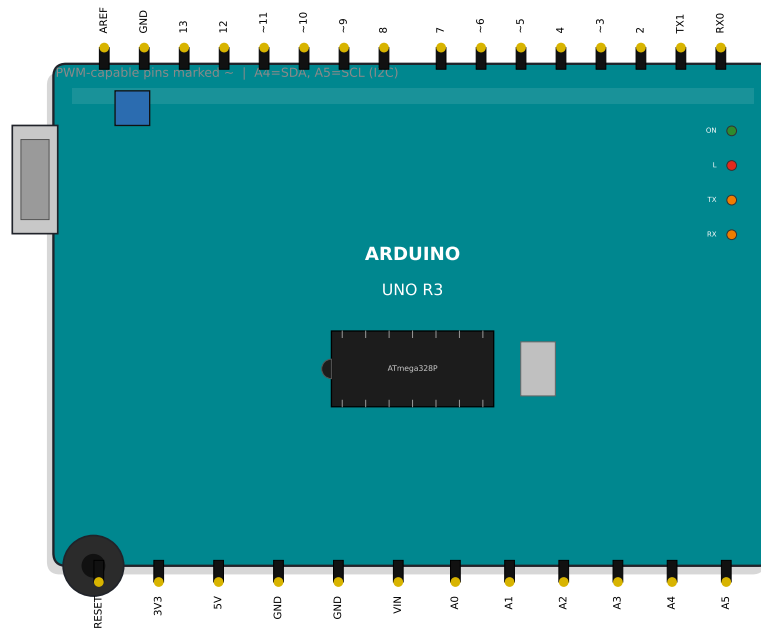


Figure 1: Arduino UNO R3 pinout as used throughout this manual: digital header (top), power and analog headers (bottom).

Digital vs. PWM vs. analog pins — summary

Pin group	Function	Used from
D0, D1	UART RX/TX (Serial)	Avoided as GPIO wherever Serial debugging is used
D2, D4, D7, D8, D12, D13	Digital-only I/O	LEDs, buttons, buzzers, digital sensor outputs
D3, D5, D6, D9, D10, D11	PWM-capable (~)	LED dimming, passive-buzzer tone, relay/LCD control
A0–A3	Analog input (also usable as digital D14–D17)	Potentiometer, LDR divider, soil-moisture AO
A4 (SDA), A5 (SCL)	I ² C bus	16×2 I ² C LCD (Experiment 18 onward)

Breadboard Connection Fundamentals

A solderless breadboard lets components be connected and rearranged without soldering. Understanding *which holes are internally joined* is the single most important skill for building any circuit in this manual correctly.

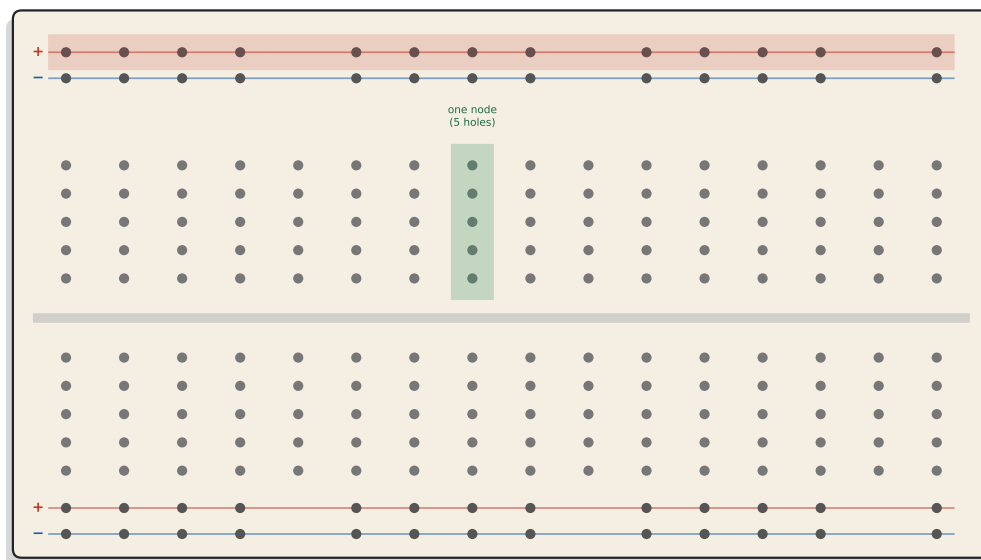


Figure 2: Breadboard internal connectivity: all five holes in a column-group (e.g. column 6, rows a–e) are joined by a hidden metal clip; the two power rails run the full length of the board.

- **Terminal strips (rows a–e and f–j):** within one column, the 5 holes in rows a–e are electrically one node, and separately the 5 holes in rows f–j are another node. Rows a–e are *not* connected to rows f–j directly (that central gap is where an IC would normally straddle) — a wire or the legs of a through-hole component must bridge the gap if both halves are needed.
- **Power rails (top and bottom):** the two long rows marked + and – run horizontally along the top and bottom edges and are used to distribute 5 V and GND to many components without cluttering the terminal strips.
- **One component leg per column-group:** because all 5 holes in a column-group are the same node, you never need more than one hole per leg — placing an LED’s cathode in row f column 6 and a resistor’s leg in row g column 6 already joins them, with no extra wire.
- **Never bridge the centre gap with a bare wire unintentionally** — this is a common source of accidental short circuits.

Standard wiring convention used in this manual

To keep 50 experiments visually consistent, every circuit diagram in this book uses the following wire-colour convention:

- **Red** — 5 V / VCC power.
- **Black** — GND / ground return.
- **Other colours** — signal wires (digital I/O, analog input, I²C, sensor outputs).

Always make your *physical* wiring match the connection table exactly, not just the picture — the connection table is the authoritative pin list for every experiment.

Arduino IDE Installation

Installing the Arduino IDE

1. Go to the official Arduino downloads page (arduino.cc/en/software) and download the IDE for your operating system (Windows, macOS, or Linux). This manual assumes **Arduino IDE 2.x**.
2. Run the installer and accept the default options, including installing any USB driver prompts (Windows needs the CH340 driver only if your UNO clone uses a CH340 USB chip; genuine UNO R3 boards using the ATmega16U2 do not need a separate driver on modern Windows/macOS).
3. Connect the Arduino UNO to the computer using the USB cable (use the USB-to-C converter if your computer only has USB-C ports).
4. Launch the Arduino IDE.

Board and Port Configuration

Selecting the correct board and port

1. In the IDE, open **Tools** → **Board** → **Arduino AVR Boards** → **Arduino Uno**.
2. Open **Tools** → **Port** and select the COM port (Windows, e.g. COM5) or device path (macOS/Linux, e.g. /dev/cu.usbmodemXXXX or /dev/ttyACM0) that appeared when the UNO was plugged in.
3. Click **Verify** (checkmark icon) on the default **Blink** example to confirm the toolchain compiles.
4. Click **Upload** (arrow icon) to confirm the IDE can actually program the board — the onboard LED (marked L, wired to D13) should start blinking once upload completes.

If the port does not appear

Try a different USB cable (some cables are power-only and carry no data lines), a different USB port, and, on Windows, check Device Manager for an unrecognised device under “Other devices” — this usually means a driver is missing. See Chapter for the full troubleshooting checklist.

Arduino Programming Fundamentals

Every Arduino sketch (program) has exactly two required functions:

```
1 void setup() {  
2   // Runs once, at power-up/reset: configure pin modes, start Serial, etc  
3   .  
4 }  
5 void loop() {  
6   // Runs forever, immediately after setup(): the program's main logic.  
7 }
```

Core statements used throughout this manual

Statement	Meaning
<code>pinMode(pin, mode)</code>	Configures a pin as OUTPUT, INPUT, or INPUT_PULLUP
<code>digitalWrite(pin, HIGH/LOW)</code>	Drives a digital output pin to 5 V or 0 V
<code>digitalRead(pin)</code>	Returns HIGH or LOW read on a digital pin
<code>analogWrite(pin, 0-255)</code>	PWM duty cycle on a ~ pin (not a real DAC)
<code>analogRead(pin)</code>	Returns a 0–1023 ADC code from an analog pin
<code>delay(ms)</code>	Blocking pause (freezes the whole <code>loop()</code>)
<code>millis()</code>	Milliseconds since power-up, used for non-blocking timing
<code>Serial.begin(baud)</code> / <code>Serial.print()</code>	Opens/uses the USB serial link to the Serial Monitor

Data types used in this manual

`int` (16-bit signed, -32768 to 32767), `long/unsigned long` (32-bit, used for `millis()` timestamps to avoid overflow within any reasonable session), `float` (32-bit, used for DHT11/HC-SR04 physical-unit results), `bool` (state flags), and `const int` for pin-number constants (preferred over `#define` in this manual for type safety).

Why `millis()` instead of `delay()`

`delay()` blocks the entire processor — while it waits, the Arduino cannot read a button, poll a sensor, or update an LCD. Every experiment in this manual that needs to remain responsive (buttons, alarms, multi-sensor stations) instead uses the **non-blocking timing pattern**:

```

1 unsigned long previousMillis = 0;
2 const unsigned long interval = 1000;    // ms
3
4 void loop() {
5     unsigned long currentMillis = millis();
6     if (currentMillis - previousMillis >= interval) {
7         previousMillis = currentMillis;
8         // ... action to repeat every 'interval' ms, without blocking ...
9     }
10    // ... other code keeps running every loop() pass ...
11 }

```

Subtracting unsigned long timestamps (rather than comparing directly) makes this pattern safe even when millis() wraps around after ≈ 49.7 days.

Software debouncing a push button

A mechanical button's contacts physically bounce for a few milliseconds when pressed, which can register as several presses instead of one. This manual debounces every button in software:

```

1 const int buttonPin = 2;
2 int lastReading = HIGH;           // INPUT_PULLUP idles HIGH
3 int stableState = HIGH;
4 unsigned long lastChangeTime = 0;
5 const unsigned long debounceDelay = 30;    // ms
6
7 void setup() { pinMode(buttonPin, INPUT_PULLUP); }
8
9 bool buttonPressedEdge() {
10     int reading = digitalRead(buttonPin);
11     if (reading != lastReading) lastChangeTime = millis();
12     bool pressedEdge = false;
13     if ((millis() - lastChangeTime) > debounceDelay && reading !=
14         stableState) {
15         stableState = reading;
16         if (stableState == LOW) pressedEdge = true;    // active-LOW press
17     }
18     lastReading = reading;
19     return pressedEdge;
20 }

```

Basic control structures

if/else, for, while, and switch/case are used exactly as in standard C++. This manual's advanced experiments (35–50) additionally use the **finite-state machine** pattern: an enum of named states plus a single switch(state) in loop(), which is introduced in full in Experiment 40.

Style conventions used in every program in this book

- Pin numbers are named `const int` constants, never magic numbers, in the body of loop().
- Every sketch includes brief comments explaining *why*, not just restating the line.

-
- Sensor reads that can fail (DHT11, HC-SR04) are always checked (`isnan()`, `timeout`) before being used.
 - `delay()` is used only in Layer 1 experiments where nothing else needs to run concurrently; from Layer 2 onward the `millis()` pattern is preferred wherever the aim is timing accuracy or responsiveness.

Library Installation Instructions

Open **Sketch** → **Include Library** → **Manage Libraries...** (or the Library Manager icon in the left sidebar of IDE 2.x), search for each name below, and click **Install**. This manual uses only the following third-party libraries, and states in each experiment’s Required Components section exactly which ones that experiment needs.

Library (search name)	Author	Used from
LiquidCrystal I2C	Frank de Brabander	Experiment 18 onward (all LCD experiments)
DHT sensor library	Adafruit	Experiment 23 onward (temperature/humidity)
Adafruit Unified Sensor	Adafruit	Auto-installed as a dependency of the DHT library
IRremote	Armin Joachimsmeier (v3/v4 API)	Experiments 34 and 46 (IR receiver demonstrations)

No library needed for HC-SR04

This manual reads the HC-SR04 with the built-in `pulseIn()` function rather than a third-party library, so that the underlying timing math (Section on Technical Theory in Experiment 16) is visible in the student’s own code instead of hidden inside a library call.

Verifying a library installed correctly

After installing, open **File** → **Examples** and scroll down to the “Examples from Custom Libraries” section — the library’s name should appear there with its own example sketches. If it does not appear, restart the IDE.

I²C address note (LCD)

Most I²C LCD backpacks in this kit family use address `0x27`; a smaller number use `0x3F`. If `lcd.init()` appears to do nothing, run an I²C scanner sketch (search “I2C Scanner” in **File** → **Examples** once any I²C-using library is installed, or use the short scanner given in Chapter) to find the correct address and change it at the top of the sketch.

Troubleshooting and Debugging Fundamentals

Every experiment in this manual ends with an experiment-specific troubleshooting table. This chapter covers the handful of *generic* problems that account for most first-time failures, so later tables can focus on issues specific to that experiment.

Symptom	Likely cause	Fix
IDE shows “port not found” / board greyed out	Data-only USB cable, wrong driver, or board not selected	Swap cable, check Device Manager (Windows) for driver issues, re-select Tools → Board/Port
Upload fails: “avrdude: stk500_recv()”	Wrong port selected, or another program (Serial Monitor) holding the port	Close Serial Monitor, re-select the correct port, press the UNO’s reset button just before upload
Sketch compiles but nothing happens	Missing <code>pinMode()</code> , wrong pin number, or loose jumper	Re-check the connection table pin-by-pin against the physical breadboard
LED never lights	Reversed polarity, burnt-out LED (missing resistor previously), or wrong pin	Re-check anode/cathode orientation; test the LED directly across 5 V + resistor as a sanity check
Sensor gives constant or random values	Missing common ground, wrong analog/digital pin, or (for DHT11) polling faster than 1 Hz	Verify GND is shared; add the sensor’s minimum read interval
LCD shows blank or solid black boxes	Wrong I ² C address, or contrast/backlight not enabled	Run the I ² C scanner below; call <code>lcd.backlight()</code>
Serial Monitor shows garbage characters	Baud rate mismatch	Match the Serial Monitor’s baud rate to the sketch’s <code>Serial.begin()</code> value (this manual standardises on 9600)

I²C address scanner

```
1 #include <Wire.h>
2
3 void setup() {
```

```
4 Wire.begin();
5 Serial.begin(9600);
6 Serial.println("I2C Scanner starting...");
7 }
8
9 void loop() {
10     byte count = 0;
11     for (byte addr = 1; addr < 127; addr++) {
12         Wire.beginTransmission(addr);
13         if (Wire.endTransmission() == 0) {
14             Serial.print("Found device at 0x");
15             Serial.println(addr, HEX);
16             count++;
17         }
18     }
19     Serial.print("Total devices found: ");
20     Serial.println(count);
21     delay(5000);
22 }
```

General debugging method used throughout this manual

1. **Isolate:** test the suspect part alone (e.g. the LED with a resistor directly to 5 V/GND) before blaming the whole circuit.
2. **Instrument:** add `Serial.print()` statements to see raw sensor values / state transitions before trusting any derived calculation.
3. **Compare against the connection table:** most faults are a single miswired pin, not a code bug.
4. **Re-read the datasheet-level limits** in that experiment's Technical Theory section before assuming the hardware itself is faulty.

Experiment Index, Prerequisite Map, and Learning Outcome Mapping

Experiment index and prerequisite map

#	Title	Layer	Prereq.	Key new concept(s)
1	LED ON/OFF Control	Simple	—	<code>pinMode</code> , <code>digitalWrite</code> , Ohm's law resistor sizing
2	LED Blinking	Simple	1	<code>delay()</code> , blocking timing
3	Multiple LED Control	Simple	2	Arrays of pins, multiple digital outputs
4	LED Chaser	Simple	3	Sequencing with <code>for</code> loops
5	Traffic Light System	Simple	4	Timed state sequencing
6	Push Button LED Control	Simple	1	Digital input, <code>INPUT_PULLUP</code>
7	Push Button Toggle Switch	Simple	6	Edge detection, debouncing, toggle state
8	Five-Button LED Controller	Simple	7	Multiple debounced inputs mapped to multiple outputs
9	Potentiometer LED Dimmer	Simple	1	<code>analogRead</code> , PWM <code>analogWrite</code> , <code>map()</code>
10	LDR-Based Automatic Light	Simple	9	Voltage divider, analog threshold decision
11	LDR Light-Level Indicator	Simple	10	Multi-level thresholds, LED bar indicator
12	Buzzer ON/OFF Control	Simple	1	Active buzzer digital output
13	Electronic Doorbell	Simple	6, 12	Combining input + audio output
14	Variable-Tone Buzzer	Simple	9, 12	<code>tone()</code> , passive buzzer, frequency
15	IR Obstacle Detector	Simple	6	Active-LOW digital sensor module
16	Ultrasonic Distance Measurement	Simple	1	<code>pulseIn</code> , time-of-flight, speed of sound
17	Ultrasonic Distance Alarm	Simple	16, 12	Threshold decision + alarm output
18	LCD I ² C Hello World	Average	1	I ² C bus, <code>LiquidCrystal_I2C</code>
19	Potentiometer Value Display	Average	9, 18	Displaying analog values on LCD
20	LDR Light Meter	Average	10, 18	Raw ADC vs. calibrated units
21	Ultrasonic Distance Display	Average	16, 18	Real-time LCD refresh, unit conversion

#	Title	Layer	Prereq.	Key new concept(s)
22	Digital Ruler	Average	21	Measurement resolution and accuracy
23	Temperature Display	Average	18	DHT11 single-wire protocol
24	Humidity Display	Average	23	Second value from the same sensor
25	Temperature Alarm	Average	23, 12	Threshold with introductory hysteresis
26	Temperature and Humidity Monitor	Average	23, 24	Combined dual-value display, refresh design
27	Soil Moisture Monitor	Average	18	Resistive sensor calibration
28	Plant Moisture Alarm	Average	27, 12	Reusable threshold-alarm pattern
29	Automatic Plant-Watering Controller Demonstration	Average	28	Relay-driven safe demonstration actuator
30	Smart Night Lamp	Average	10, 9	LDR auto-on combined with PWM brightness
31	IR Security Alarm	Average	15, 12	Sensor + buzzer with arming/latching logic
32	Object Counter	Average	15, 18	Debounced edge counting + display
33	Smart Parking Indicator	Average	16, 18, 29	Multi-level distance status + relay barrier demo
34	IR Remote Controller Demonstration	Average	18	IRremote library, receive-only decode
35	Smart Room Light Controller	Advanced	30	LDR + button override + relay priority logic
36	Room Environment Monitor	Advanced	26, 18	Non-blocking multi-screen LCD paging
37	Smart Plant Monitoring System	Advanced	27, 23, 18	Combined soil + climate sensing
38	Automatic Plant Protection System	Advanced	37, 29	Closed-loop threshold + relay + buzzer
39	Smart Parking Assistant	Advanced	33	Variable-rate proximity beeping, multi-zone
40	Electronic Security System	Advanced	31	Finite-state machine (ARMED/DISARMED/ALARM)
41	Multi-Zone Security Detector	Advanced	40	Two independent zones (IR + laser/LDR beam-break)
42	Digital Distance Warning System	Advanced	16, 39	Multi-threshold hysteresis + alarm priority
43	Light Intensity Measurement System	Advanced	20	Dual-LDR relative light measurement
44	Automatic Day/Night Controller	Advanced	30, 35	Formal hysteresis design (anti-chatter)
45	Temperature-Controlled System	Advanced	25, 29	DHT11 + relay closed-loop control
46	IR Remote Smart Controller	Advanced	34	Decoded IR commands driving multiple outputs

#	Title	Layer	Prereq.	Key new concept(s)
47	Multi-Sensor Environmental Station	Advanced	36, 37, 20	Non-blocking round-robin sensor scheduler
48	Smart Classroom Automation Prototype	Advanced	44, 47	Light automation + environment + occupancy proxy
49	Security and Environment Monitoring Station	Advanced	41, 47	Alarm prioritisation across subsystems
50	Mini Smart Home Automation System	Advanced	35–49	Full capstone: FSM + scheduler + all sensors/actuators

Hardware feasibility review

Peak simultaneous component usage across all 50 experiments was checked against kit stock (Chapter) before finalising this manual:

- **Push buttons (raw):** maximum simultaneous use is **5**, in Experiment 8 (Five-Button LED Controller) — exactly the kit stock.
- **LDRs:** maximum simultaneous use is **2**, in Experiments 41 and 43 — exactly the kit stock.
- **Buzzers:** the active and passive buzzers are used *together* only in the Experiment 50 capstone — exactly the kit stock (1 active + 1 passive).
- **LEDs:** the highest simultaneous count is in Experiment 5 (Traffic Light, 3 LEDs) and Experiment 50 (capstone, up to 6 indicator LEDs) — well under the 25 available.
- **Resistors:** no experiment uses more than ~10 simultaneously — well under the 100 available.
- **Single-instance modules** (relay, LCD, HC-SR04, DHT11, soil-moisture sensor, IR obstacle sensor, IR receiver, laser module, potentiometer): every experiment that uses one of these uses only the single unit the kit provides; no experiment requires two of the same single-instance module at once.
- No experiment requires a motor, pump, servo, transistor, external IR transmitter, or any sensor outside Chapter . Wherever a real deployment would use one (irrigation pump, door lock actuator, HVAC compressor), the experiment explicitly substitutes an LED and/or the relay module as a safe low-voltage demonstration, and states this in its Aim.

Learning outcome mapping

LO	Learning objective (from Preface)	Representative experiments
1	Identify and wire every kit component correctly	1–17 (all), Appendix B
2	Apply Ohm’s law, digital logic, PWM, ADC	1, 9, 14, 20
3	Read datasheet-level electrical characteristics	2, 15, 16, 23
4	Write/debug sketches using I ² C, <code>millis()</code> , debouncing, FSMs	7, 18, 26, 40
5	Interface and combine single sensors	15, 16, 23, 27, 34, 37
6	Design and validate a multi-sensor embedded system	40, 47, 48, 49, 50

Contents

About JR Learners	iii
Preface	iv
Laboratory Safety Instructions	vi
Complete Component Inventory	vii
Arduino UNO R3 Architecture and Pinout	x
Breadboard Connection Fundamentals	xii
Arduino IDE Installation	xiv
Arduino Programming Fundamentals	xv
Library Installation Instructions	xviii
Troubleshooting and Debugging Fundamentals	xix
Experiment Index and Learning Outcome Mapping	xxi
I LAYER 1 — SIMPLE (Experiments 1–17)	1
Layer 1 — Simple: Overview	2
1 Experiment 1: LED ON/OFF Control	3
2 Experiment 2: LED Blinking	8
3 Experiment 3: Multiple LED Control	13
4 Experiment 4: LED Chaser	18
5 Experiment 5: Traffic Light System	23
6 Experiment 6: Push Button LED Control	28

7 Experiment 7: Push Button Toggle Switch	32
8 Experiment 8: Five-Button LED Controller	37
9 Experiment 9: Potentiometer LED Dimmer	42
10 Experiment 10: LDR-Based Automatic Light	47
11 Experiment 11: LDR Light-Level Indicator	52
12 Experiment 12: Buzzer ON/OFF Control	56
13 Experiment 13: Electronic Doorbell	60
14 Experiment 14: Variable-Tone Buzzer	64
15 Experiment 15: IR Obstacle Detector	68
16 Experiment 16: Ultrasonic Distance Measurement	73
17 Experiment 17: Ultrasonic Distance Alarm	78
 II LAYER 2 — AVERAGE (Experiments 18–34)	 83
Layer 2 — Average: Overview	84
18 Experiment 18: LCD I²C Hello World	85
19 Experiment 19: Potentiometer Value Display	89
20 Experiment 20: LDR Light Meter	93
21 Experiment 21: Ultrasonic Distance Display	97
22 Experiment 22: Digital Ruler	102
23 Experiment 23: Temperature Display	107
24 Experiment 24: Humidity Display	112
25 Experiment 25: Temperature Alarm	116
26 Experiment 26: Temperature and Humidity Monitor	121
27 Experiment 27: Soil Moisture Monitor	126
28 Experiment 28: Plant Moisture Alarm	131

29 Experiment 29: Automatic Plant-Watering Controller Demonstration	136
30 Experiment 30: Smart Night Lamp	141
31 Experiment 31: IR Security Alarm	145
32 Experiment 32: Object Counter	150
33 Experiment 33: Smart Parking Indicator	155
34 Experiment 34: IR Remote Controller Demonstration	160
 III LAYER 3 — ADVANCED (Experiments 35–50)	 165
Layer 3 — Advanced: Overview	166
35 Experiment 35: Smart Room Light Controller	167
36 Experiment 36: Room Environment Monitor	172
37 Experiment 37: Smart Plant Monitoring System	177
38 Experiment 38: Automatic Plant Protection System	182
39 Experiment 39: Smart Parking Assistant	187
40 Experiment 40: Electronic Security System	192
41 Experiment 41: Multi-Zone Security Detector	197
42 Experiment 42: Digital Distance Warning System	202
43 Experiment 43: Light Intensity Measurement System	207
44 Experiment 44: Automatic Day/Night Controller	211
45 Experiment 45: Temperature-Controlled System	216
46 Experiment 46: IR Remote Smart Controller	221
47 Experiment 47: Multi-Sensor Environmental Station	226
48 Experiment 48: Smart Classroom Automation Prototype	231
49 Experiment 49: Security and Environment Monitoring Station	236
50 Experiment 50: Mini Smart Home Automation System	241

IV BONUS EXPERIMENT	250
51 Bonus Experiment: Servo-Controlled Automatic Barrier	251
A Arduino UNO R3 Technical Specifications	255
B Complete Kit Inventory	256
C Arduino Pin Reference	257
D Resistor Colour-Code Reference	258
E Common Electronic Symbols	259
F Common Arduino Programming Functions	260
G Arduino Library Installation Guide	261
H Sensor Calibration Procedures	262
I Common Compiler Errors and Solutions	263
J Complete Program Index	264
K Viva Answer Key (Instructor Use)	266
L Student Observation Sheets	276
M Experiment Assessment Rubric	277
N Final Project Evaluation Checklist	278
O References and Further Reading	279

Part I

LAYER 1 — SIMPLE (Experiments 1–17)

Layer 1 — Simple: Overview

Experiments 1–17 build up digital and analog I/O one concept at a time: static and blinking LED output, arrays of LEDs, debounced button input, PWM dimming, an LDR voltage divider, buzzer output, and finally the HC-SR04 ultrasonic sensor. Every circuit in this layer uses only the Arduino UNO, the breadboard, and passive components or a single sensor module — no I²C, no combined multi-sensor logic yet. By Experiment 17 a learner will have individually exercised every basic I/O technique needed for Layer 2.

Chapter 1

Experiment 1: LED ON/OFF Control

Experiment Identification

Experiment Number:	1
Title:	LED ON/OFF Control
Learning Layer:	Layer 1 — Simple
Estimated Completion Time:	20 minutes
Difficulty:	Beginner
Prerequisite Experiments:	None

Aim

To control a single LED using an Arduino UNO digital output pin, demonstrating the most basic form of digital output: driving a pin permanently HIGH (or LOW) under program control.

Learning Objectives

1. State the function of `pinMode()` and `digitalWrite()`.
2. Calculate a safe current-limiting resistor value for an LED using Ohm's law.
3. Correctly identify LED anode/cathode polarity on a breadboard.
4. Wire a single-LED circuit that stays within the Arduino UNO's GPIO current limit.
5. Modify a running sketch to change the LED's default state from ON to OFF.

Required Components

- Arduino UNO R3 + USB cable — 1
- Breadboard — 1
- LED (red, 5 mm) — 1
- Resistor, $220\ \Omega$ — 1
- Male-to-male jumper wires — 2

Technical Theory

An LED (light-emitting diode) conducts and emits light only when forward biased above its forward voltage V_f (for a red LED, typically $V_f \approx 2.0\text{ V}$). Left unrestricted, an LED's current rises almost vertically once V_f is exceeded, so a series resistor is mandatory to limit current to a

safe value. Using Ohm's law across the resistor:

$$R = \frac{V_S - V_f}{I_f} \quad (1.1)$$

where $V_S = 5\text{ V}$ (Arduino GPIO HIGH level), $V_f \approx 2.0\text{ V}$ (red LED forward voltage), and I_f is the desired forward current, chosen at 10–15 mA for comfortable brightness well under the pin's 20 mA design limit. Choosing $I_f = 13.6\text{ mA}$:

$$R = \frac{5 - 2.0}{0.0136\text{ A}} \approx 220\ \Omega \quad (1.2)$$

$220\ \Omega$ is therefore used throughout this manual for standard-brightness red/yellow/green indicator LEDs. `digitalWrite(pin, HIGH)` sets the pin to approximately 5 V; `digitalWrite(pin, LOW)` sets it to 0 V. Before either can be used, `pinMode(pin, OUTPUT)` must configure the pin's direction, since every GPIO pin defaults to a high-impedance input at power-up.

Component Working Principle

An LED is a PN-junction diode engineered to emit photons when electrons recombine with holes across the junction, under forward bias. It conducts current in only one direction: from anode (+, longer lead) to cathode (–, shorter lead, nearest the flat edge on the rim). Reverse-biasing an LED does not damage it at low voltage (unlike some other diodes) but simply keeps it dark. Electrical characteristics used in this experiment: $V_f \approx 2.0\text{ V}$ (red), maximum continuous forward current typically 20 mA (exceeding this shortens LED lifetime or causes immediate failure).

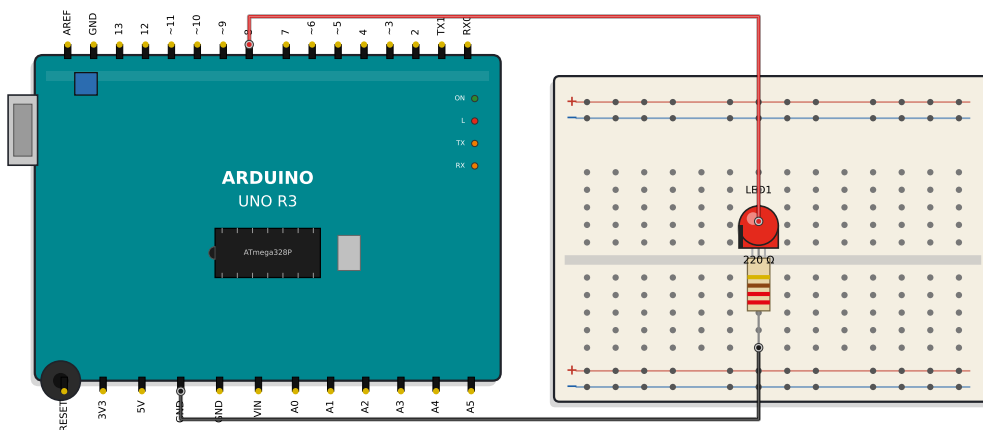


Figure 1.1: Experiment 1 circuit: Arduino D8 through a $220\ \Omega$ resistor to the LED anode; LED cathode to GND.

Connection Table

From	To	Notes
Arduino D8	Resistor (220 Ω) terminal 1	Digital output
Resistor terminal 2	LED anode (long leg)	
LED cathode (short leg)	Arduino GND	Common ground return

Working Principle – Signal Flow

D8 (HIGH) $\rightarrow$ current flows through the 220 Ω resistor $\rightarrow$ through the LED (anode to cathode) $\rightarrow$ back to Arduino GND, completing the circuit and lighting the LED. Setting D8 LOW removes the potential difference across the LED, stopping current flow and turning it off.

Algorithm

1. Declare a constant for the LED pin number (8).
2. In `setup()`, configure the LED pin as `OUTPUT`.
3. In `setup()`, drive the LED pin HIGH once.
4. Leave `loop()` empty — the LED remains ON indefinitely since nothing changes its state.

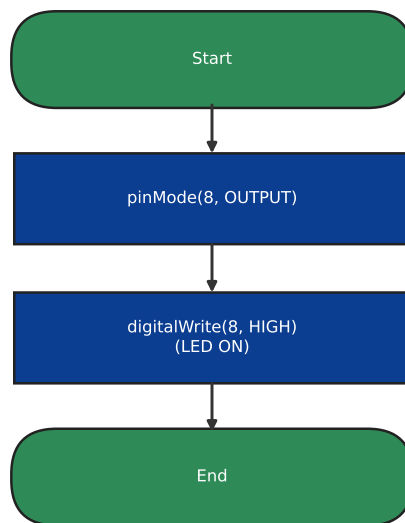


Figure 1.2: Flowchart: Experiment 1.

Arduino Program

```
1  const int LED_PIN = 8;    // digital pin driving the LED through a 220-  
    ohm resistor  
2  
3  void setup() {  
4      pinMode(LED_PIN, OUTPUT);    // configure D8 as a digital output  
5      digitalWrite(LED_PIN, HIGH); // turn the LED ON (approx. 5 V on D8)  
6  }  
7  
8  void loop() {
```

```

9 // Intentionally empty: the LED simply stays ON.
10 // To turn it OFF instead, change HIGH to LOW above and re-upload.
11 }

```

Code Explanation

`const int LED_PIN = 8;` names the pin so the rest of the program never uses a bare magic number. `pinMode(LED_PIN, OUTPUT)` configures D8's internal driver so it can source/sink current rather than float as a high-impedance input. `digitalWrite(LED_PIN, HIGH)` sets D8 to logic HIGH ($\approx 5\text{ V}$) exactly once, in `setup()`, so the LED lights immediately at power-up and needs no further instructions — `loop()` runs forever but has nothing to do.

Experimental Procedure

1. With the Arduino *unplugged*, insert the LED into the breadboard so its two legs are in different column-groups (rows a–e vs f–j, or two separate columns).
2. Insert the $220\ \Omega$ resistor between the LED's anode column and an empty column.
3. Connect a male-to-male jumper from Arduino D8 to the resistor's free end.
4. Connect a second jumper from the LED's cathode column to an Arduino GND pin.
5. Double-check polarity against the connection table, then plug in the USB cable.
6. Upload the sketch above (Chapter for IDE/board/port setup if needed).

Expected Output

The LED lights immediately and stays lit continuously after upload; the Arduino's onboard L LED (D13) is unaffected since this circuit does not use D13. No Serial Monitor output is produced by this sketch.

Observation Table

Supply voltage measured (V)	LED state (ON/OFF)	observed	Voltage across LED, measured (V)

Calculations and Analysis

Using the measured supply voltage V_S and the LED's rated V_f (see LED datasheet/Appendix A), recompute R from Equation (2) and compare it with the $220\ \Omega$ resistor actually used. Measure the resistor's true current with $I = (V_S - V_{LED})/R_{measured}$ and confirm it is under 20 mA.

Result

The LED ON/OFF circuit was built and the Arduino was programmed to drive the LED _____ (state observed). The measured LED current was _____ mA, which is/is not (circle one) within the safe GPIO limit.

Troubleshooting

Problem	Likely cause	Fix
LED stays dark	Reversed polarity	Swap LED orientation (long leg to resistor side)
LED very dim	Resistor value too high	Confirm $220\ \Omega$ (red-red-brown-gold), not e.g. $22\ \text{k}\Omega$
LED momentarily lights then dies	No resistor / wrong column bridging	Re-check breadboard column-groups, insert resistor
Nothing happens, upload succeeds	LED in wrong column-group (open circuit)	Verify both LED legs sit in different node columns bridged correctly
IDE cannot upload	Wrong board/port	Recheck Chapter

Viva Questions and Extension Challenge

Conceptual:

1. Why does an LED need a series resistor but a plain resistor-only circuit does not need one?
2. What happens internally when `pinMode()` is not called before `digitalWrite()`?
3. Why is the resistor's exact value not extremely critical (i.e. $220\ \Omega$ vs $270\ \Omega$ both work)?
4. What is the typical forward voltage of a blue or white LED compared to a red LED, and why does it differ?
5. Why does reversing the LED (not the resistor) stop current flow entirely?

Troubleshooting:

1. Your LED lights very faintly compared to a classmate's identical circuit — name two possible hardware causes.
2. You measure $0\ \text{V}$ across the LED with a multimeter while it is supposedly ON — what does this suggest?
3. The Arduino resets repeatedly after uploading this sketch — what class of wiring fault usually causes this?

Programming:

1. Modify the sketch so the LED is OFF at startup instead of ON, without changing the wiring.
2. Rewrite the sketch to use `#define LED_PIN 8` instead of `const int` — what is one disadvantage of this style?

Extension Challenge: Wire a second LED on a different pin and modify the sketch so both LEDs are controlled independently, one ON and one OFF at startup, without yet using arrays (this is formalised in Experiment 3).

Chapter 2

Experiment 2: LED Blinking

Experiment Identification

Experiment Number:	2
Title:	LED Blinking
Learning Layer:	Layer 1 — Simple
Estimated Completion Time:	20 minutes
Difficulty:	Beginner
Prerequisite Experiments:	Experiment 1

Aim

To make the Experiment 1 LED blink continuously using `delay()`, introducing repetitive (blocking) timing inside `loop()`.

Learning Objectives

1. Explain how `loop()` executes repeatedly after `setup()`.
2. Use `delay()` to create a fixed-period ON/OFF blink.
3. Predict the blink period and frequency from the two `delay()` values.
4. Explain why `delay()` blocks all other processing for its duration.

Required Components

Identical to Experiment 1: Arduino UNO, breadboard, 1 red LED, $1 \times 220 \Omega$ resistor, 2 male-to-male jumpers.

Technical Theory

Blinking is periodic switching between two digital states. If t_{on} and t_{off} are the ON and OFF durations in milliseconds, the blink period is $T = t_{on} + t_{off}$ and the blink frequency is

$$f = \frac{1000}{T} \text{ Hz} \quad (T \text{ in ms}) \quad (2.1)$$

With $t_{on} = t_{off} = 1000 \text{ ms}$, $T = 2000 \text{ ms}$ and $f = 0.5 \text{ Hz}$ (one full blink every 2 seconds). `delay(ms)` pauses the *entire* microcontroller for the given duration — no other digital reads, sensor polling, or communication can happen meanwhile, which is why later experiments needing responsiveness switch to the `millis()` pattern (Chapter).

Component Working Principle

Same LED electrical behaviour as Experiment 1. The new element is purely in software: the ATmega328P's instruction timer is used by `delay()` to busy-wait for the requested number of milliseconds before returning control to `loop()`.

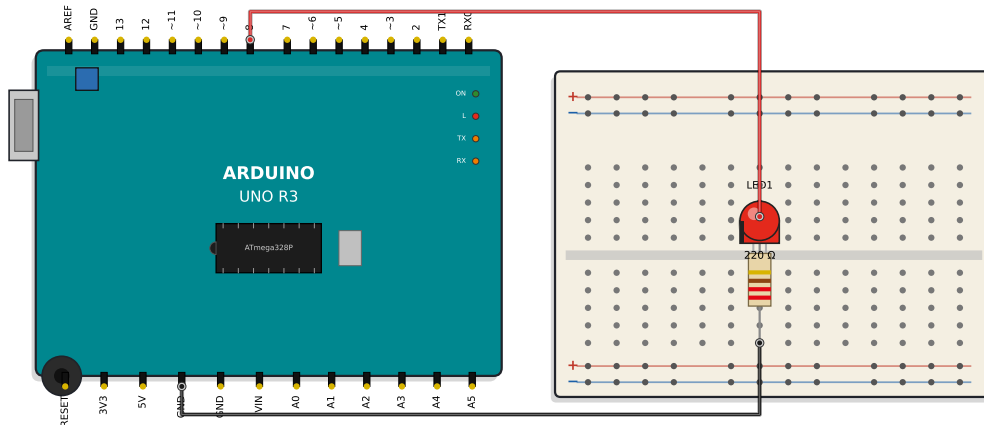


Figure 2.1: Experiment 2 circuit (identical wiring to Experiment 1).

Connection Table

Identical to Experiment 1 (D8 → resistor → LED anode; LED cathode → GND).

Working Principle – Signal Flow

D8 HIGH (LED ON) for 1000 ms → D8 LOW (LED OFF) for 1000 ms → `loop()` restarts automatically → repeats forever.

Algorithm

1. Configure the LED pin as OUTPUT in `setup()`.
2. In `loop()`: turn the LED ON, wait 1000 ms.
3. Turn the LED OFF, wait 1000 ms.
4. Return to step 2 (implicit, since `loop()` repeats automatically).

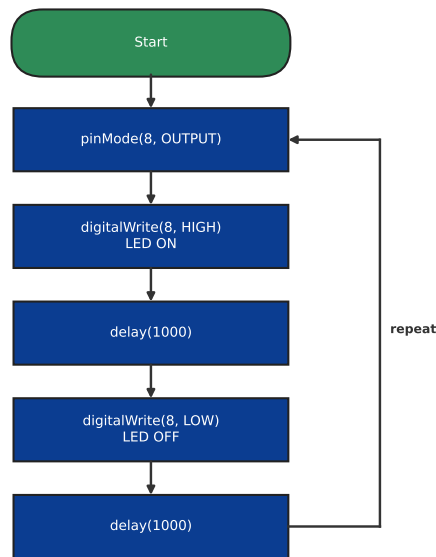


Figure 2.2: Flowchart: Experiment 2.

Arduino Program

```

1  const int LED_PIN = 8;
2
3  void setup() {
4      pinMode(LED_PIN, OUTPUT);
5  }
6
7  void loop() {
8      digitalWrite(LED_PIN, HIGH); // LED ON
9      delay(1000);                 // wait 1 second
10     digitalWrite(LED_PIN, LOW);  // LED OFF
11     delay(1000);                 // wait 1 second
12 }

```

Code Explanation

Unlike Experiment 1, `loop()` now actively toggles the LED every pass. Because `loop()` itself is called repeatedly by the Arduino core forever, the four lines inside it are enough to produce an indefinite blink without any explicit `while(true)` — `loop()` is that infinite loop.

Experimental Procedure

1. Reuse the Experiment 1 circuit unchanged.
2. Replace the sketch with the code above and upload.
3. Observe the LED blinking at 1 Hz period (2 s per full cycle).
4. Change both delays to 200 ms, re-upload, and observe the faster blink.

Expected Output

The LED turns ON and OFF every 1 second indefinitely (illustrative target; your own timing will match within the accuracy of the ATmega328P's clock, typically better than $\pm 1\%$).

Observation Table

t_{on} set (ms)	t_{off} set (ms)	Period measured stopwatch over blinks / 10	(s, f (Hz))

Calculations and Analysis

Compute $f_{theory} = 1000/(t_{on} + t_{off})$ for each row and compare with the stopwatch-measured f . Discuss any discrepancy in terms of human reaction-time measurement error versus microcontroller timing error.

Result

The LED was successfully programmed to blink at a period of _____ ms (theoretical) versus _____ ms (measured).

Troubleshooting

Problem	Likely cause	Fix
LED stays solid ON	<code>delay()</code> values swapped with an always-HIGH bug, or old sketch not re-uploaded	Confirm sketch that; force re-upload
Blink far too fast/slow	Wrong delay units assumed	Remember <code>delay()</code> takes milliseconds, not seconds
LED flickers erratically	Loose jumper on D8 or GND	Reseat jumpers
IDE “sketch too old” upload issue	Serial Monitor left open	Close Serial Monitor before uploading
No blink at all	Same faults as Experiment 1	Re-check Experiment 1 troubleshooting table

Viva Questions and Extension Challenge

Conceptual:

1. Why does `loop()` not need an explicit infinite-loop construct?
2. What is the blink frequency if $t_{on} = 300$ ms and $t_{off} = 700$ ms?
3. Why can `delay()` cause a button press to be missed entirely?
4. Is the LED’s brightness affected by the blink rate? Why or why not?
5. What Arduino function would you use instead of `delay()` if you also needed to read a button during the wait?

Troubleshooting:

1. Your blink period is exactly double what you expected — what single-line bug usually

causes this?

2. The onboard L LED (D13) blinks in addition to your external LED — is this a fault?
3. After changing the code, the old blink pattern still runs — what upload step was likely skipped?

Programming:

1. Modify the sketch to blink with an asymmetric duty cycle (ON 200 ms, OFF 800 ms).
2. Rewrite the sketch so the ON/OFF durations are stored in named constants instead of literal numbers.

Extension Challenge: Make the LED blink three short pulses followed by one long pulse, repeating (an SOS-style pattern), using only `delay()` and `digitalWrite()`.

Chapter 3

Experiment 3: Multiple LED Control

Experiment Identification

Experiment Number:	3
Title:	Multiple LED Control
Learning Layer:	Layer 1 — Simple
Estimated Completion Time:	25 minutes
Difficulty:	Beginner
Prerequisite Experiments:	Experiment 2

Aim

To independently control three LEDs (red, yellow, green) using an array of pin numbers, introducing arrays and `for` loops for scalable digital output.

Learning Objectives

1. Declare and index an array of pin numbers.
2. Configure multiple pins as OUTPUT using a `for` loop.
3. Control several LEDs independently from one sketch.
4. Explain why arrays scale better than repeating near-identical code per LED.

Required Components

- Arduino UNO R3, breadboard
- LEDs: 1 red, 1 yellow, 1 green
- Resistors: $3 \times 220\ \Omega$
- Male-to-male jumper wires: 6

Technical Theory

Each LED again needs its own $220\ \Omega$ series resistor (Experiment 1, Equation 1.2) since forward voltage and safe current are per-LED properties, not shared. Software-wise, an `int ledPins[] = {8, 9, 10}` array lets a single `for (int i = 0; i < 3; i++)` loop configure and drive all three pins, rather than writing three near-identical `pinMode()/digitalWrite()` statements — the same pattern scales to Experiment 8's five buttons/five LEDs without the code growing linearly in complexity.

Component Working Principle

Three independent LED branches, each electrically identical to Experiment 1's single-LED circuit, sharing only the common GND return. Different LED colours have different V_f : red/yellow ≈ 2.0 V, green ≈ 2.1 V — close enough that $220\ \Omega$ remains a safe, simple common choice for all three at ≈ 13 mA.

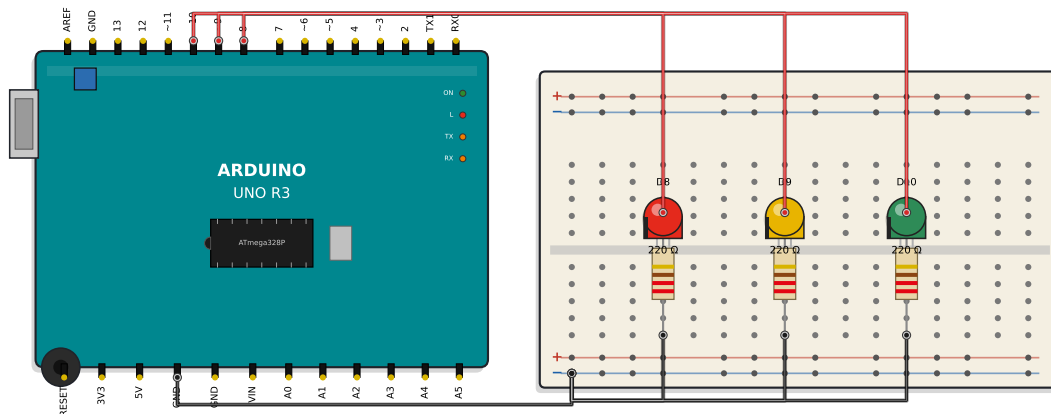


Figure 3.1: Experiment 3 circuit: three independent LED branches on D8, D9, D10, sharing a common GND rail.

Connection Table

Arduino pin	Component	Notes
D8	$220\ \Omega \rightarrow$ Red LED anode	Cathode to GND rail
D9	$220\ \Omega \rightarrow$ Yellow LED anode	Cathode to GND rail
D10	$220\ \Omega \rightarrow$ Green LED anode	Cathode to GND rail
Arduino GND	Breadboard GND rail	Common return for all three

Working Principle – Signal Flow

Each of D8/D9/D10 independently sources current through its own resistor and LED to the shared GND rail; setting any one pin HIGH/LOW affects only its own LED, since the three branches share no signal path, only the ground return.

Algorithm

1. Store pin numbers 8, 9, 10 in an array `ledPins[3]`.
2. In `setup()`, loop over the array and set each pin to OUTPUT.
3. In `loop()`, turn each LED ON in turn with a short delay, then all OFF (demonstration pattern).

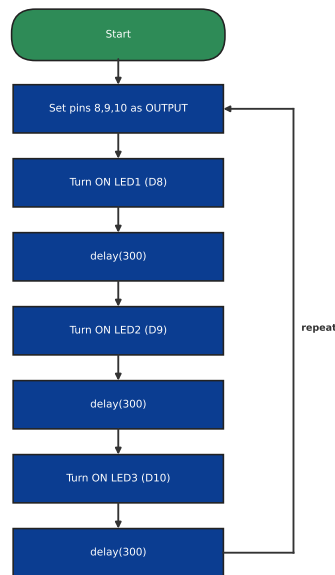


Figure 3.2: Flowchart: Experiment 3.

Arduino Program

```

1  const int ledPins[] = {8, 9, 10};           // red, yellow, green
2  const int NUM_LEDS = 3;
3
4  void setup() {
5      for (int i = 0; i < NUM_LEDS; i++) {
6          pinMode(ledPins[i], OUTPUT);
7      }
8  }
9
10 void loop() {
11     for (int i = 0; i < NUM_LEDS; i++) {
12         digitalWrite(ledPins[i], HIGH);
13         delay(300);
14     }
15     for (int i = 0; i < NUM_LEDS; i++) {
16         digitalWrite(ledPins[i], LOW);
17     }
18     delay(300);
19 }

```

Code Explanation

`ledPins[]` holds the three pin numbers; `NUM_LEDS` avoids a hard-coded “3” scattered through the code. The first `for` loop in `setup()` configures all three pins in three lines regardless of how many LEDs are added later. In `loop()`, the first loop turns LEDs on one after another (300 ms apart), the second loop turns them all off together, then a pause repeats the cycle.

Experimental Procedure

1. Build three independent LED+resistor branches on the breadboard per the connection table.
2. Tie all three cathodes to one shared GND rail, and that rail to an Arduino GND pin.

3. Upload the sketch and observe all three LEDs lighting in sequence, then resetting.

Expected Output

Red, then yellow, then green light in sequence (300 ms apart), then all three turn off together and the pattern repeats every ≈ 1.2 s.

Observation Table

LED colour	Pin	V_f measured (V)	Observed brightness (qualitative)
------------	-----	--------------------	-----------------------------------

Calculations and Analysis

For each LED, compute $I = (5 - V_{f,\text{measured}})/220$ and confirm all three currents remain below 20 mA despite different V_f values.

Result

Three LEDs were independently controlled via an array-driven `for` loop; all three lit correctly in the programmed sequence: _____ (Yes/No).

Troubleshooting

Problem	Likely cause	Fix
Only one LED lights	Array values do not match physical wiring	Re-check <code>ledPins[]</code> against the connection table
Compiler error on array line	Missing braces/semicolon	Match the exact syntax shown
All LEDs stay off	Shared GND rail not connected to Arduino GND	Add the missing ground jumper
Wrong LED lights for a given index	Off-by-one indexing	Remember arrays are zero-indexed
Sequence order looks reversed	Physical LEDs swapped vs. pin assignment	Re-check which physical LED sits on which column

Viva Questions and Extension Challenge

Conceptual:

1. Why is a shared GND rail acceptable even though each LED has its own resistor?
2. What would happen if two LEDs' resistors were swapped?
3. Why does the array approach not reduce the component count, only the code size?

-
4. What is the index of the third element in `ledPins[]`?
 5. How would you add a fourth LED with minimal code changes?

Troubleshooting:

1. Two LEDs light but the third stays dark even though wiring looks identical — what's the most likely single-component fault?
2. Swapping two entries in `ledPins[]` changes the pattern order but not correctness — why?
3. A short circuit briefly resets the Arduino — which wiring mistake commonly causes this here?

Programming:

1. Rewrite `loop()` so all three LEDs turn on simultaneously instead of in sequence.
2. Add a fourth colour to the array and confirm the `for` loops require no other changes besides `NUM_LEDS`.

Extension Challenge: Write a sketch where the LEDs light in a randomly shuffled order each cycle using `random()`.

Chapter 4

Experiment 4: LED Chaser

Experiment Identification

Experiment Number:	4
Title:	LED Chaser
Learning Layer:	Layer 1 — Simple
Estimated Completion Time:	25 minutes
Difficulty:	Beginner
Prerequisite Experiments:	Experiment 3

Aim

To create a "chaser" (Larson-scanner-style) light pattern across four LEDs, where exactly one LED is lit at a time and lighting moves along the row, reinforcing array indexing with the modulo operator.

Learning Objectives

1. Use the modulo operator (%) to wrap an index around an array's length.
2. Turn exactly one LED on and all others off on each step.
3. Explain the difference between "all LEDs in sequence stay on" (Experiment 3) and "only one LED on at a time, moving" (this experiment).

Required Components

Arduino UNO R3, breadboard, 4 LEDs (red, yellow, green, blue), $4 \times 220\Omega$ resistors, 8 male-to-male jumpers.

Technical Theory

A chaser advances an index i each cycle: $i \leftarrow (i + 1) \bmod N$, where N is the LED count. The modulo operation ensures i always stays in range $[0, N - 1]$, wrapping back to 0 after the last LED, producing continuous circulating motion with no special-case "reset" code. Blue LEDs have a higher V_f (≈ 2.9 – 3.2 V) than red/yellow/green (≈ 2.0 – 2.1 V); at 220Ω this still yields a safe current (≈ 8 – 9 mA for blue), just visibly dimmer, which is expected and not a fault.

Component Working Principle

Same four independent LED+resistor branches as Experiment 3's pattern, extended to four LEDs. The blue LED's higher V_f is a real semiconductor bandgap property (blue LEDs use a wider-bandgap material), not a manufacturing defect.

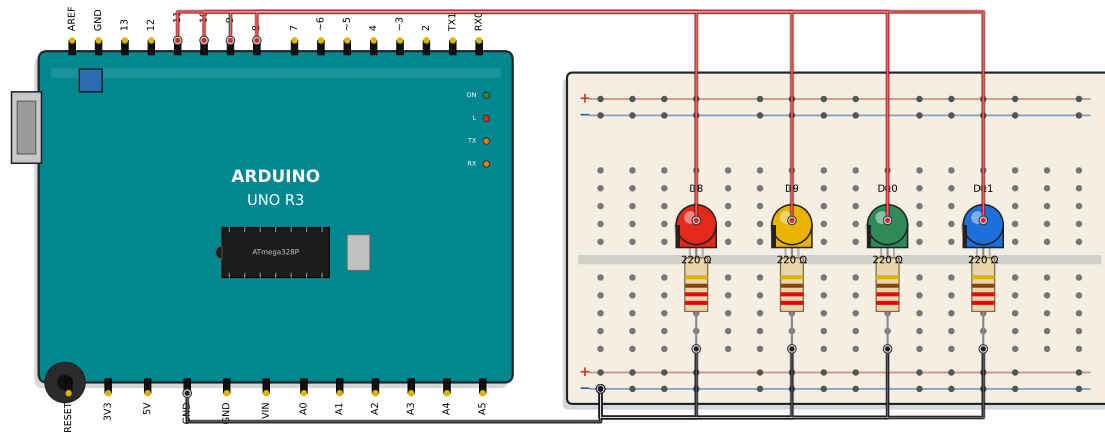


Figure 4.1: Experiment 4 circuit: four LED branches on D8–D11.

Connection Table

Pin	LED colour	Resistor	Cathode
D8	Red	220 Ω	GND rail
D9	Yellow	220 Ω	GND rail
D10	Green	220 Ω	GND rail
D11	Blue	220 Ω	GND rail

Working Principle – Signal Flow

Index i selects exactly one pin to drive HIGH; all other pins in the array are driven LOW in the same pass, so only one LED is ever lit; after a short delay i advances (mod 4) and the "lit" position visibly moves along the row.

Algorithm

1. Store 4 pins in `ledPins[4]`; set all as OUTPUT.
2. Maintain index variable `current`, initially 0.
3. Each pass: turn ON `ledPins[current]`, turn OFF all others, delay, then `current = (current+1) % 4`.

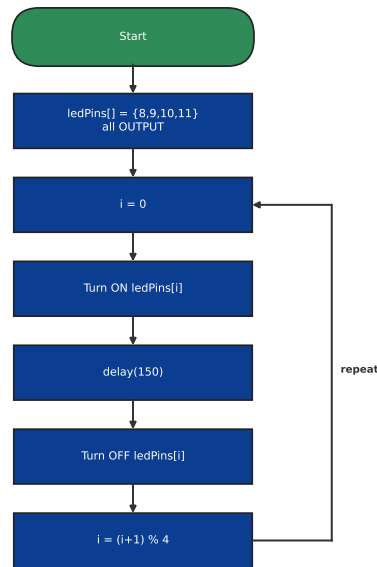


Figure 4.2: Flowchart: Experiment 4.

Arduino Program

```

1  const int ledPins[] = {8, 9, 10, 11};
2  const int NUM_LEDS = 4;
3  int current = 0;
4
5  void setup() {
6    for (int i = 0; i < NUM_LEDS; i++) pinMode(ledPins[i], OUTPUT);
7  }
8
9  void loop() {
10   for (int i = 0; i < NUM_LEDS; i++) {
11     digitalWrite(ledPins[i], (i == current) ? HIGH : LOW);
12   }
13   delay(150);
14   current = (current + 1) % NUM_LEDS; // wrap back to 0 after the
15   last LED
  
```

Code Explanation

The ternary expression `(i == current) ? HIGH : LOW` drives exactly the LED at index `current` HIGH and every other index LOW, all within one loop pass, so there is never a moment with two LEDs simultaneously lit. `current = (current + 1) % NUM_LEDS` is the modulo wrap-around described in the theory section.

Experimental Procedure

1. Wire four LED branches per the connection table, sharing one GND rail.
2. Upload the sketch; observe a single lit LED moving from D8 towards D11 and wrapping back to D8.
3. Change the delay from 150 ms to 50 ms and 500 ms; observe the change in perceived motion speed.

Expected Output

A single LED appears to "move" along the row of four LEDs every 150 ms, continuously cycling.

Observation Table

Delay set (ms)	Perceived speed (slow/medium/fast)	Any visible flicker/afterimage?
----------------	---------------------------------------	---------------------------------

Calculations and Analysis

At what delay value does the chase become too fast for the human eye to perceive individual LEDs (persistence-of-vision threshold, roughly below 20–25 ms per frame, i.e. above $\approx 40\text{--}50$ Hz)? Compare your observation to this figure.

Result

The 4-LED chaser pattern was implemented; the lit LED correctly wrapped from index 3 back to index 0: _____ (Yes/No).

Troubleshooting

Problem	Likely cause	Fix
Two LEDs lit at once	Ternary condition typo, or duplicate pin numbers	Re-check code against listing
Chase never wraps	% typed as /	Confirm modulo operator is used
Blue LED much dimmer than others	Expected — higher V_f	Not a fault; optionally use a lower resistor per Appendix H calibration notes
Pattern runs backwards	LEDs wired in reverse physical order	Re-check breadboard columns vs. pin order

Viva Questions and Extension Challenge

Conceptual: (1) What does `5 % 4` evaluate to, and why does that matter here? (2) Why is a ternary operator used instead of an `if/else`? (3) Why is blue LED brightness lower at the same resistor value? (4) What changes if `NUM_LEDS` is wrong but `ledPins[]` is correct? (5) How is this circuit electrically identical to Experiment 3? **Troubleshooting:** (1) Only two LEDs ever light, alternating — what does this suggest about the array? (2) The chase direction reverses after your edit — what did you likely change? (3) One LED is always slightly dimmer even at the same code state — hardware or software cause? **Programming:** (1) Modify the code to chase in the opposite direction. (2) Add a second index that bounces back and forth (0,1,2,3,2,1,0,...) instead of wrapping. **Extension Challenge:** Implement a "double dot" chaser where two LEDs,

two positions apart, are lit simultaneously and both move together.

Chapter 5

Experiment 5: Traffic Light System

Experiment Identification

Experiment Number:	5
Title:	Traffic Light System
Learning Layer:	Layer 1 — Simple
Estimated Completion Time:	25 minutes
Difficulty:	Beginner
Prerequisite Experiments:	Experiment 4

Aim

To implement a realistic red-yellow-green traffic-light sequence with unequal, purposefully chosen timing for each phase, introducing timed state sequencing as an engineering design problem rather than an arbitrary pattern.

Learning Objectives

1. Design a fixed timing sequence matching a real-world specification (typical road traffic-light durations).
2. Implement a sequence where more than one output can be active at once (red+yellow).
3. Justify each phase's duration from a road-safety perspective.

Required Components

Arduino UNO R3, breadboard, 3 LEDs (red, yellow, green), $3 \times 220\Omega$ resistors, 6 male-to-male jumpers.

Technical Theory

Real traffic signals do not cycle through equal-duration states: green is held longest (traffic flow), yellow is brief (clearance warning, typically 3–5 s), and many signals show red+yellow together briefly before green (a "get ready" phase used in several countries). This experiment models: GREEN (4 s) → YELLOW (1 s) → RED (5 s) → RED+YELLOW (1 s) → back to GREEN. Unlike Experiment 4 (exactly one output active), this state machine sometimes drives two outputs HIGH simultaneously.

Component Working Principle

Three independent LED branches, electrically identical to Experiment 3. The new engineering content is entirely in the timing/state design, not the hardware.

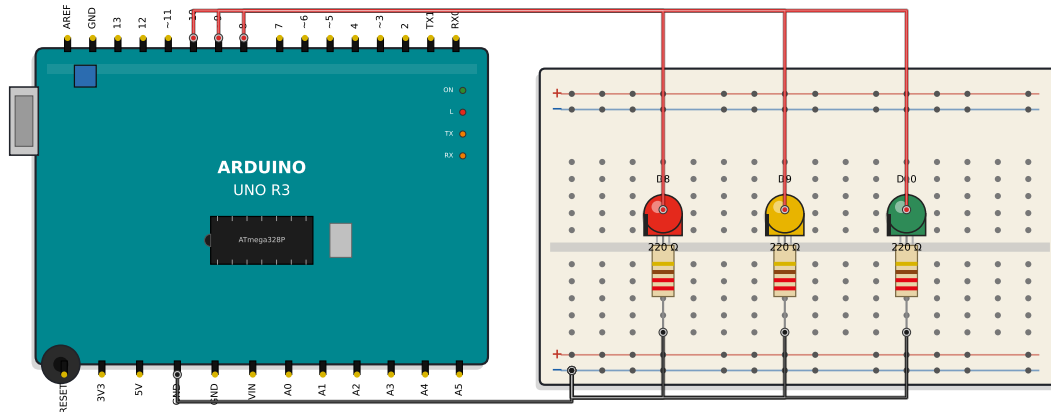


Figure 5.1: Experiment 5 circuit: red, yellow, green LEDs on D8, D9, D10.

Connection Table

Pin	LED	Notes
D8	Red	220 Ω to GND rail
D9	Yellow	220 Ω to GND rail
D10	Green	220 Ω to GND rail

Working Principle – Signal Flow

Each phase drives a specific combination of the three pins HIGH/LOW for a specific duration, then advances to the next phase; the RED+YELLOW phase is the only one driving two pins HIGH simultaneously.

Algorithm

1. GREEN only $\rightarrow$ delay 4000 ms.
2. YELLOW only $\rightarrow$ delay 1000 ms.
3. RED only $\rightarrow$ delay 5000 ms.
4. RED + YELLOW $\rightarrow$ delay 1000 ms.
5. Return to step 1.

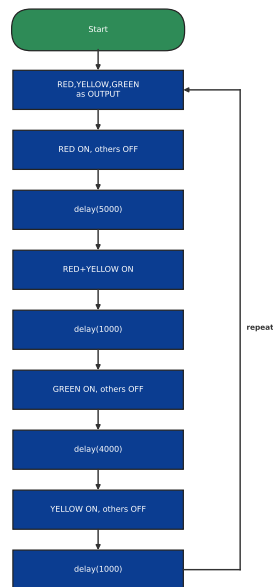


Figure 5.2: Flowchart: Experiment 5.

Arduino Program

```

1  const int RED = 8, YELLOW = 9, GREEN = 10;
2
3  void setup() {
4      pinMode(RED, OUTPUT);
5      pinMode(YELLOW, OUTPUT);
6      pinMode(GREEN, OUTPUT);
7  }
8
9  void allOff() {
10     digitalWrite(RED, LOW);
11     digitalWrite(YELLOW, LOW);
12     digitalWrite(GREEN, LOW);
13 }
14
15 void loop() {
16     allOff(); digitalWrite(GREEN, HIGH);    delay(4000);    // go
17     allOff(); digitalWrite(YELLOW, HIGH);   delay(1000);    // caution
18     allOff(); digitalWrite(RED, HIGH);      delay(5000);    // stop
19     digitalWrite(YELLOW, HIGH);             delay(1000);    // red+yellow:
        get ready
20 }

```

Code Explanation

`allOff()` is a small helper function that avoids repeating three `digitalWrite(...,LOW)` lines before every phase. The final phase deliberately does *not* call `allOff()` first, since RED must remain lit while YELLOW is added — this is the one place the sequence intentionally leaves a previous output HIGH.

Experimental Procedure

1. Wire the three LEDs as in Experiment 3/4.
2. Upload and time each phase with a stopwatch against the specification.
3. Confirm the RED+YELLOW overlap phase visibly shows both LEDs lit together.

Expected Output

GREEN for 4 s → YELLOW for 1 s → RED for 5 s → RED+YELLOW together for 1 s → repeats, total cycle 11 s.

Observation Table

Phase	Specified duration (s)	Measured duration (s)	Error (%)
Green	4		
Yellow	1		
Red	5		
Red+Yellow	1		

Calculations and Analysis

Compute total cycle time $T = 4 + 1 + 5 + 1 = 11$ s and the duty cycle of each colour, e.g. green's share = $4/11 \approx 36\%$. Compare with your stopwatch measurements and discuss stopwatch reaction-time error (± 0.2 – 0.3 s typical).

Result

The traffic-light sequence was implemented with the specified 4-1-5-1 s phase durations, including the red+yellow overlap: _____ (Yes/No).

Troubleshooting

Problem	Likely cause	Fix
All three LEDs lit together	Missing <code>all0ff()</code> call before a phase	Re-check each phase calls it (except the last)
Red+yellow phase missing	Skipped last line / logic error	Verify final phase code
Timing feels off	Delay values mistyped	Re-check against algorithm
One colour never lights	Miswired pin	Re-check connection table

Viva Questions and Extension Challenge

Conceptual: (1) Why is yellow held for a much shorter time than red or green? (2) What real-world hazard does the red+yellow overlap phase prevent/signal? (3) Why call `all0ff()` at the start of most phases but not the last? (4) How many total LED-state combinations are possible with 3 independent LEDs, and how many does this sequence actually use? (5) What is

the total cycle time? **Troubleshooting:** (1) The yellow LED stays on through the red phase too — what line is missing? (2) Cycle time measured is consistently ~10% longer than 11 s — likely cause? (3) LEDs flicker briefly between phases — is this a hardware fault? **Programming:** (1) Add a pedestrian "walk" LED that is lit only during the RED phase. (2) Refactor the sequence using an array of {colour-mask, duration} pairs and a **for** loop instead of four hand-written blocks. **Extension Challenge:** Build a two-intersection system (6 LEDs) where the second traffic light's phases are offset so only one direction is ever GREEN at a time.

Chapter 6

Experiment 6: Push Button LED Control

Experiment Identification

Experiment Number:	6
Title:	Push Button LED Control
Learning Layer:	Layer 1 — Simple
Estimated Completion Time:	20 minutes
Difficulty:	Beginner
Prerequisite Experiments:	Experiment 1

Aim

To light an LED only while a push button is held pressed, introducing digital input with the internal pull-up resistor.

Learning Objectives

1. Configure a pin as `INPUT_PULLUP` and explain the resulting idle logic level.
2. Read a button state with `digitalRead()`.
3. Explain active-LOW button logic.
4. Drive an LED directly from a button's live state (no memory/toggle yet).

Required Components

Arduino UNO R3, breadboard, 1 push button (4-leg tactile), 1 red LED, $1 \times 220\ \Omega$ resistor, 4 male-to-male jumpers.

Technical Theory

A floating (unconnected) digital input pin picks up electrical noise and reads unpredictably. `pinMode(pin, INPUT_PULLUP)` enables the ATmega328P's internal $\approx 20\text{--}50\ \text{k}\Omega$ pull-up resistor, weakly pulling the pin HIGH when nothing else drives it. Wiring the button between that pin and GND means: button released $\rightarrow$ pin reads HIGH (pulled up); button pressed $\rightarrow$ pin is pulled directly to GND $\rightarrow$ reads LOW. This is called **active-LOW** logic: "pressed" corresponds to the LOW state, which is initially counter-intuitive but avoids needing an external pull-up/pull-down resistor and its own current-limiting concerns.

Component Working Principle

The 4-leg tactile button internally connects legs in two pre-joined pairs; pressing the cap bridges both pairs together, connecting one full side to the other. Wiring must use one leg from each pair (see Chapter), never two legs of the same pre-joined pair, or the button will appear permanently "pressed".

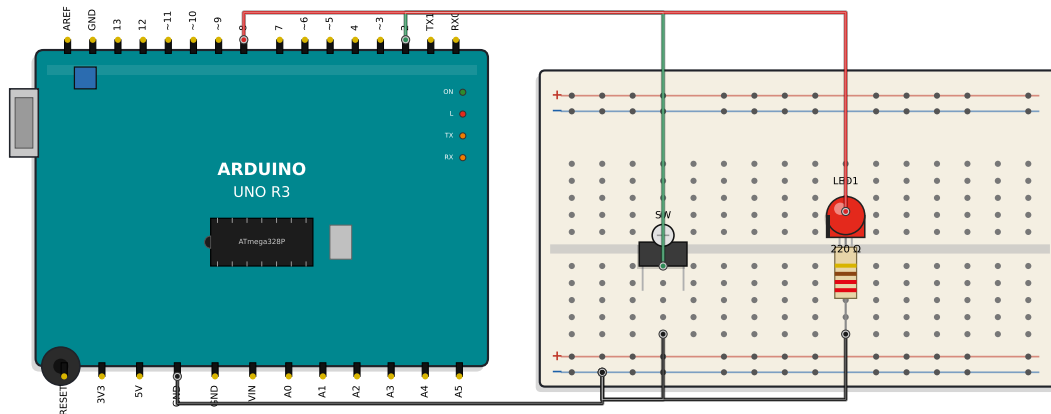


Figure 6.1: Experiment 6 circuit: push button on D2 (INPUT_PULLUP), LED on D8.

Connection Table

From	To	Notes
Button leg A	Arduino D2	INPUT_PULLUP
Button leg B (opposite pair)	Arduino GND	Active-LOW when pressed
Arduino D8	220 Ω $\rightarrow$ LED anode	Same as Experiment 1
LED cathode	Arduino GND	

Working Principle – Signal Flow

Button released: D2 reads HIGH (internal pull-up) $\rightarrow$ LED forced OFF. Button pressed: D2 pulled to GND, reads LOW $\rightarrow$ LED driven ON. The LED tracks the button live, with no memory of past presses.

Algorithm

1. Configure D2 as INPUT_PULLUP, D8 as OUTPUT.
2. Read D2.
3. If D2 is LOW, set D8 HIGH; else set D8 LOW.
4. Repeat from step 2.

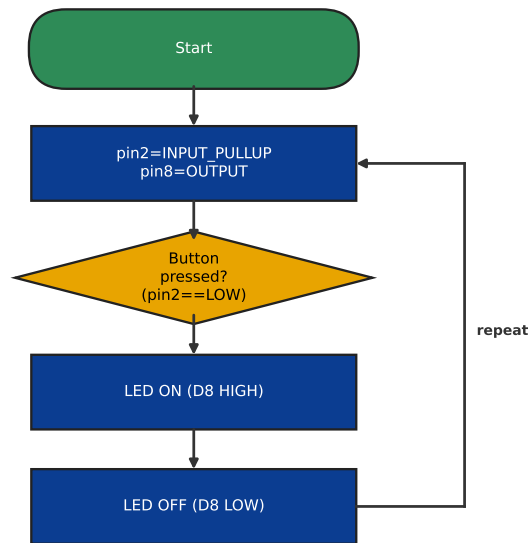


Figure 6.2: Flowchart: Experiment 6.

Arduino Program

```

1  const int BUTTON_PIN = 2;
2  const int LED_PIN = 8;
3
4  void setup() {
5      pinMode(BUTTON_PIN, INPUT_PULLUP); // idles HIGH, LOW when pressed
6      pinMode(LED_PIN, OUTPUT);
7  }
8
9  void loop() {
10     int buttonState = digitalRead(BUTTON_PIN);
11     if (buttonState == LOW) { // pressed (active-LOW)
12         digitalWrite(LED_PIN, HIGH);
13     } else {
14         digitalWrite(LED_PIN, LOW);
15     }
16 }

```

Code Explanation

`digitalRead(BUTTON_PIN)` returns HIGH or LOW as an int. The `if/else` directly mirrors the button's live state onto the LED every pass of `loop()` (hundreds of thousands of times per second), so the LED responds within microseconds of a press or release, with no perceptible lag.

Experimental Procedure

1. Insert the button straddling the centre gap of the breadboard so its two working legs land in separate column-groups.
2. Wire one leg to D2, the opposite-pair leg to GND.
3. Wire the LED circuit exactly as in Experiment 1, using D8.
4. Upload and press/release the button, observing the LED.

Expected Output

LED OFF while the button is released; LED ON only while the button is physically held down; releases immediately turn it back OFF.

Observation Table

Button (pressed/released)	state	LED state observed
------------------------------	-------	--------------------

Calculations and Analysis

No numeric calculation is required for this experiment; instead, tabulate the button/LED truth table you observed and confirm it matches active-LOW logic (button LOW $\Rightarrow$ LED HIGH).

Result

The LED correctly tracked the button's live state with active-LOW logic: _____
(Yes/No).

Troubleshooting

Problem	Likely cause	Fix
LED always ON regardless of button	Button legs from the same pre-joined pair used	Rewire using legs from opposite pairs
LED always OFF	INPUT_PULLUP omitted, floating pin	Confirm <code>pinMode</code> line
LED flickers when pressed	Mechanical bounce (expected; formalised in Experiment 7)	Not a fault at this stage
Logic feels "backwards"	Active-LOW confusion	Re-read Technical Theory; this is expected behaviour

Viva Questions and Extension Challenge

Conceptual: (1) Why does `INPUT_PULLUP` remove the need for an external resistor? (2) Why is the idle state HIGH, not LOW? (3) What physical bridging happens inside the button when pressed? (4) What would happen using plain `INPUT` instead of `INPUT_PULLUP` with this wiring? (5) Why is this called "active-LOW"? **Troubleshooting:** (1) The LED is always on, but wiring to D2/D8 checks out — what should you check on the button itself? (2) The circuit works on breadboard A but not B with identical code — what physical difference should you check? (3) The LED responds but with a visible flicker at the press moment — is this a code bug? **Programming:** (1) Invert the logic so the LED is ON when released and OFF when pressed. (2) Add a Serial print reporting "PRESSED"/"RELEASED" only when the state changes (hint: you will need a variable to remember the previous state — this is developed fully in Experiment 7). **Extension Challenge:** Control two LEDs from one button: one lit while pressed, the other lit while released.

Chapter 7

Experiment 7: Push Button Toggle Switch

Experiment Identification

Experiment Number:	7
Title:	Push Button Toggle Switch
Learning Layer:	Layer 1 — Simple
Estimated Completion Time:	25 minutes
Difficulty:	Beginner
Prerequisite Experiments:	Experiment 6

Aim

To make a push button toggle an LED's state on each press (one press = ON, next press = OFF), introducing edge detection and software debouncing.

Learning Objectives

1. Distinguish a button *level* (Experiment 6) from a button *edge* (this experiment).
2. Implement software debouncing using `millis()`.
3. Maintain and toggle a persistent boolean state variable across loop iterations.

Required Components

Identical to Experiment 6: 1 push button, 1 LED, $1 \times 220 \Omega$ resistor.

Technical Theory

A mechanical button's metal contacts do not close cleanly; they physically bounce for a few milliseconds, so a single human press can appear in software as a rapid burst of HIGH/LOW transitions. Toggling on every raw transition would flip the LED several times per press unpredictably. Debouncing waits for the reading to remain stable for a debounce interval (this manual uses 30 ms, per Chapter) before trusting it as a genuine change, and toggles the LED only on a genuine LOW-going **edge** (the instant of transition), not on every loop pass while LOW.

Component Working Principle

Same button/LED hardware as Experiment 6; the new element is entirely in software state (a remembered previous reading, a remembered debounce timestamp, and a remembered LED state).

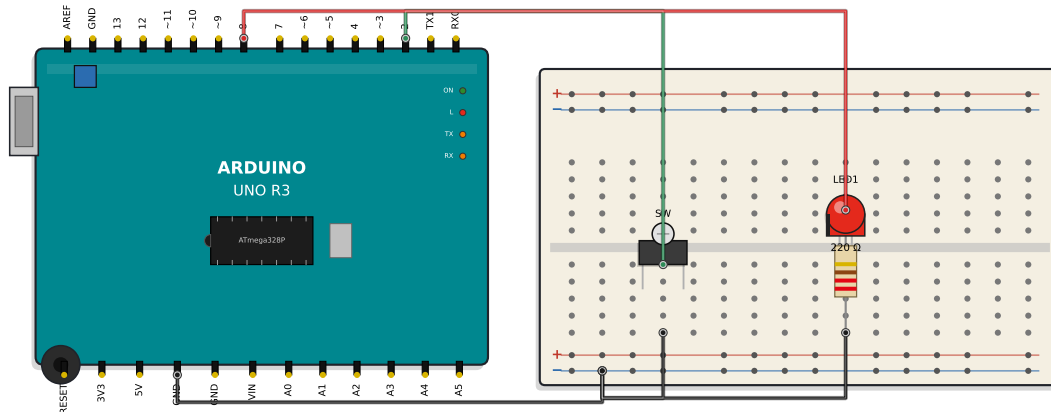


Figure 7.1: Experiment 7 circuit (identical wiring to Experiment 6).

Connection Table

Identical to Experiment 6 (button on D2 with INPUT_PULLUP; LED on D8).

Working Principle – Signal Flow

Raw button reading → debounce filter (ignore changes lasting under 30 ms) → edge detector (fires only at the moment of a stable HIGH→LOW transition) → toggle stored LED state → write LED state to D8.

Algorithm

1. Read the button; if the reading differs from the last raw reading, reset the debounce timer.
2. If the reading has been stable for >30 ms and differs from the last *accepted* state, accept it as the new stable state.
3. If the newly accepted stable state is LOW (pressed), toggle the LED state variable and write it to the LED pin.
4. Remember the current raw reading for the next pass.

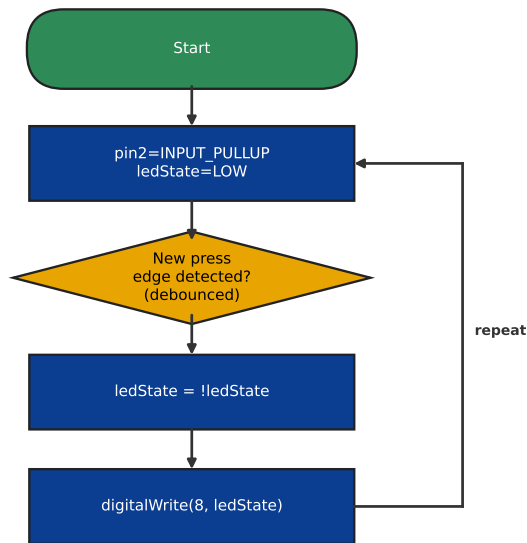


Figure 7.2: Flowchart: Experiment 7.

Arduino Program

```

1  const int BUTTON_PIN = 2;
2  const int LED_PIN = 8;
3  const unsigned long DEBOUNCE_MS = 30;
4
5  int lastReading = HIGH;
6  int stableState = HIGH;
7  unsigned long lastChangeTime = 0;
8  bool ledState = false;
9
10 void setup() {
11     pinMode(BUTTON_PIN, INPUT_PULLUP);
12     pinMode(LED_PIN, OUTPUT);
13 }
14
15 void loop() {
16     int reading = digitalRead(BUTTON_PIN);
17
18     if (reading != lastReading) {
19         lastChangeTime = millis();           // reading changed: restart
20         debounce timer
21     }
22
23     if ((millis() - lastChangeTime) > DEBOUNCE_MS && reading !=
24         stableState) {
25         stableState = reading;               // accept the new, now-stable,
26         reading
27         if (stableState == LOW) {            // a genuine press edge
28             ledState = !ledState;
29             digitalWrite(LED_PIN, ledState);
30         }
31     }
32     lastReading = reading;
33 }

```

Code Explanation

`lastReading` tracks the raw, possibly-bouncing signal; `stableState` tracks what has actually been *accepted* after the debounce window. The edge is detected exactly once, in the single line `if (stableState == LOW)`, immediately after `stableState` changes — this fires once per press, not repeatedly while held, which is what makes it a toggle rather than a hold-to-light control.

Experimental Procedure

1. Reuse the Experiment 6 wiring unchanged.
2. Upload the sketch above.
3. Press and release the button repeatedly; the LED should flip state exactly once per press, never partially or twice.

Expected Output

LED starts OFF; first press turns it ON and it stays ON after release; second press turns it OFF; and so on, one clean toggle per press with no double-triggering.

Observation Table

Press #	LED state before	LED state after	Double-toggle observed?
1			
2			
3			

Calculations and Analysis

Try reducing `DEBOUNCE_MS` to 0 and observe whether double-toggles reappear; then restore 30 ms. This demonstrates debouncing's effect experimentally rather than only theoretically.

Result

The button reliably toggled the LED exactly once per press with 30 ms debouncing, with _____ double-toggles observed out of _____ presses tested.

Troubleshooting

Problem	Likely cause	Fix
LED toggles twice per press occasionally	Debounce interval too short for this button	Increase <code>DEBOUNCE_MS</code> to 50
LED never toggles	Edge condition checks wrong level	Confirm <code>stableState == LOW</code>
LED toggles while held down	Edge logic accidentally re-fires every loop	Ensure toggle is inside the "state just changed" branch, not a bare level check
Behaves like Experiment 6 (tracks level)	Old sketch not replaced	Re-upload this experiment's code

Viva Questions and Extension Challenge

Conceptual: (1) What physically causes contact bounce? (2) Why compare against `lastReading` and `stableState` rather than just one variable? (3) What is an "edge" as opposed to a "level"? (4) Why must `ledState` be declared outside `loop()`? (5) What happens to responsiveness if debounce time is set to 500 ms? **Troubleshooting:** (1) Your toggle works alone but fails when you also add Serial prints — what timing effect might explain this? (2) A classmate's identical code double-toggles on their (different) button but not yours — what physical difference explains this? (3) The LED toggles on release instead of press — which comparison needs changing? **Programming:** (1) Add a press counter that prints the total number of toggles via Serial. (2) Convert this single-button toggle into a function `bool debouncedEdge(int pin)` reusable for multiple buttons (this exact function is needed in Experiment 8). **Extension Challenge:** Implement "long press vs. short press" detection: a short press toggles the LED, a press held over 1 s instead makes it blink continuously until pressed again.

Chapter 8

Experiment 8: Five-Button LED Controller

Experiment Identification

Experiment Number:	8
Title:	Five-Button LED Controller
Learning Layer:	Layer 1 — Simple
Estimated Completion Time:	30 minutes
Difficulty:	Beginner–Intermediate
Prerequisite Experiments:	Experiment 7

Aim

To independently debounce and read all 5 push buttons in the kit simultaneously, each directly controlling its own LED, generalising the single-button debounce pattern to arrays.

Learning Objectives

1. Generalise a single-button debounce routine to N independent buttons using parallel arrays.
2. Manage per-button state (last reading, stable state, debounce timer) without variable name collisions.
3. Recognise the practical limit this places on kit stock (exactly 5 buttons available).

Required Components

Arduino UNO R3, breadboard, 5 push buttons, 5 LEDs (any colours), $5 \times 220\ \Omega$ resistors, 10 male-to-male jumpers. This is the maximum number of raw push buttons the kit provides (Chapter).

Technical Theory

Each button needs its own copy of the three debounce state variables from Experiment 7 (`lastReading`, `stableState`, `lastChangeTime`). Rather than declaring 15 separate scalar variables, this experiment uses four parallel arrays of length 5, indexed identically, so button i 's state always lives at index i of every array — the same structural idea as Experiment 3's LED array, applied to stateful input instead of stateless output.

Component Working Principle

Five independent button+LED pairs, each electrically identical to Experiment 6/7's single pair. No new component behaviour is introduced — this experiment is a scaling exercise.

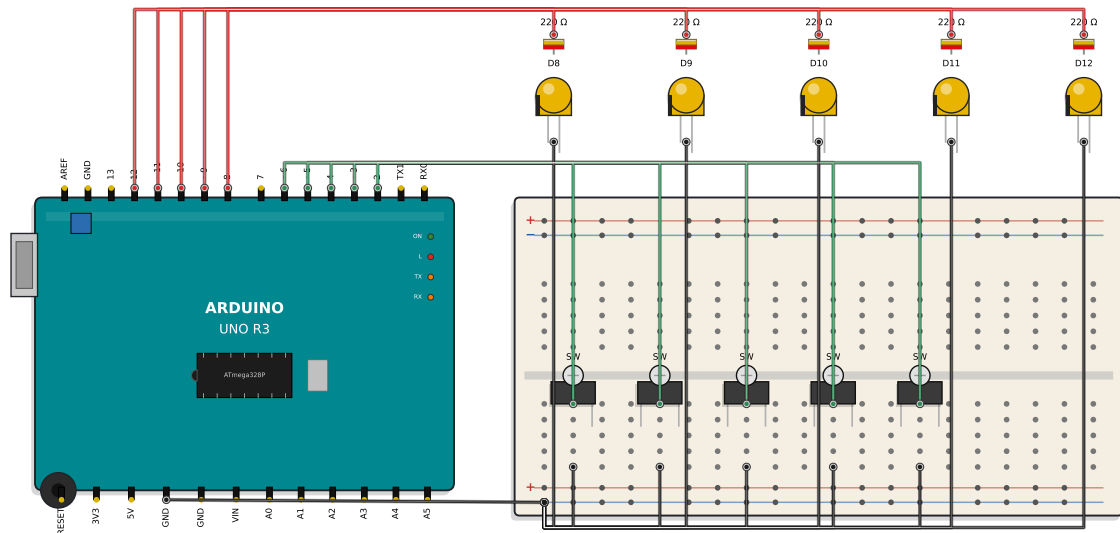


Figure 8.1: Experiment 8 circuit: 5 buttons (D2–D6, INPUT_PULLUP) each mapped to its own LED (D8–D12).

Connection Table

Button #	Button pin	LED pin	Common return
1	D2	D8	GND rail
2	D3	D9	GND rail
3	D4	D10	GND rail
4	D5	D11	GND rail
5	D6	D12	GND rail

Working Principle – Signal Flow

Each button independently follows the Experiment 6 level-control logic (LED ON only while its own button is held), read on every pass through a single shared `for` loop over index `0...4` instead of five duplicated blocks of code.

Algorithm

1. Configure `buttonPins[5]` as INPUT_PULLUP and `ledPins[5]` as OUTPUT.
2. For each index i from 0 to 4: read `buttonPins[i]`.
3. If LOW, drive `ledPins[i]` HIGH; else LOW.
4. Repeat for all 5 indices, every pass.

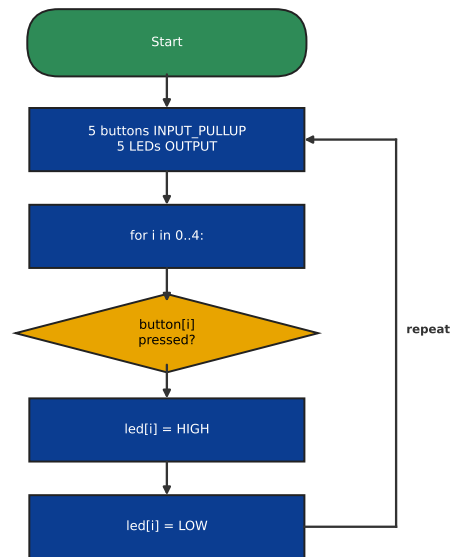


Figure 8.2: Flowchart: Experiment 8.

Arduino Program

```

1  const int NUM_BTN = 5;
2  const int buttonPins[NUM_BTN] = {2, 3, 4, 5, 6};
3  const int ledPins[NUM_BTN]    = {8, 9, 10, 11, 12};
4
5  void setup() {
6      for (int i = 0; i < NUM_BTN; i++) {
7          pinMode(buttonPins[i], INPUT_PULLUP);
8          pinMode(ledPins[i], OUTPUT);
9      }
10 }
11
12 void loop() {
13     for (int i = 0; i < NUM_BTN; i++) {
14         int reading = digitalRead(buttonPins[i]);
15         digitalWrite(ledPins[i], (reading == LOW) ? HIGH : LOW);
16     }
17 }

```

Code Explanation

The two parallel arrays `buttonPins[]` and `ledPins[]` keep button-to-LED pairing implicit in shared index *i*: button *i* always controls LED *i*. The single `for` loop in `loop()` replaces what would otherwise be five copies of Experiment 6's `if/else`, and adding a sixth pair (if the kit ever had one) would need only one more array entry, not new logic.

Experimental Procedure

1. Wire all 5 button/LED pairs per the connection table, sharing one GND rail.
2. Upload the sketch.
3. Press each button individually and confirm only its own LED lights; press multiple simultaneously and confirm independent operation.

Expected Output

Each of the 5 LEDs lights only while its corresponding button is held, independently of the other four — pressing two or more buttons at once lights exactly those LEDs, with no cross-talk.

Observation Table

Button #	Pressed alone: correct LED?	Pressed with #1: correct?	Notes
1		—	
2			
3			
4			
5			

Calculations and Analysis

No numeric calculation required; instead confirm empirically that all $2^5 = 32$ possible press combinations produce the logically expected LED pattern (a full test is not required, but test at least all 5 singles and 2–3 combinations).

Result

All 5 buttons independently controlled their own LEDs correctly, including simultaneous multi-button presses: _____ (Yes/No).

Troubleshooting

Problem	Likely cause	Fix
One button's LED never responds	Array index mismatch between the two arrays	Re-check <code>buttonPins[i]/ledPins[i]</code> ordering
Two LEDs respond to one button	Two array entries wired to the same physical pin	Re-check for duplicate pin numbers
All LEDs respond to any button	Shared GND rail wired to a signal pin by mistake	Re-check GND rail wiring
Intermittent response on one channel	Loose jumper on that channel only	Reseat that button's jumpers

Viva Questions and Extension Challenge

Conceptual: (1) Why use parallel arrays instead of one array of pairs? (2) What is the kit-imposed limit on how many such channels this circuit can have, and why? (3) How many total pin numbers does this sketch reference, and across how many physical Arduino GPIOs? (4) Why does this experiment not need debouncing to function correctly for level control? (5) How would toggle behaviour (Experiment 7) generalise to 5 buttons? **Troubleshooting:** (1) Button

3 controls LED 4 instead of LED 3 — what specific code line has the bug? (2) All buttons stop working after touching one wire — what should you check first? (3) Everything works except simultaneous presses of buttons 1 and 5 — what non-code explanation could cause this (hint: think about current, not logic)? **Programming:** (1) Add a 6th conceptual channel using a spare analog pin (A0) as a digital input — does the code change much? (2) Print, over Serial, exactly which buttons are currently pressed as a 5-character string of 0s and 1s. **Extension Challenge:** Combine this experiment's array-based reading with Experiment 7's debounced toggle so each button independently toggles its LED rather than requiring a held press.

Chapter 9

Experiment 9: Potentiometer LED Dimmer

Experiment Identification

Experiment Number:	9
Title:	Potentiometer LED Dimmer
Learning Layer:	Layer 1 — Simple
Estimated Completion Time:	25 minutes
Difficulty:	Beginner
Prerequisite Experiments:	Experiment 1

Aim

To continuously vary an LED's brightness using a potentiometer, introducing analog input (`analogRead`) and PWM analog output (`analogWrite`).

Learning Objectives

1. Explain the potentiometer as a voltage divider with a movable wiper.
2. Read a 10-bit ADC value with `analogRead()`.
3. Explain PWM and use `analogWrite()` on a ~ (PWM-capable) pin.
4. Use `map()` to rescale a 0–1023 range to 0–255.

Required Components

Arduino UNO R3, breadboard, 1 potentiometer (10 k Ω), 1 LED, 1 $\times$ 220 Ω resistor, jumper wires.

Technical Theory

A potentiometer is a resistive track with a wiper contact that divides it into two series resistances R_1 (wiper to one end) and R_2 (wiper to the other end), with $R_1 + R_2 = R_{total}$ (10 k Ω) fixed. With the two ends across 5 V and GND, the wiper voltage is

$$V_{wiper} = 5 \text{ V} \times \frac{R_2}{R_1 + R_2} \quad (9.1)$$

which varies continuously from 0 to 5 V as the knob turns. The ADC converts this to a 10-bit code 0–1023 via `analogRead()`. Since `analogWrite()` expects an 8-bit duty cycle (0–255), `map(value, 0, 1023, 0, 255)` rescales one range onto the other linearly. **PWM (Pulse-**

Width Modulation) does not produce a true analog voltage — it switches the pin fully HIGH and LOW very fast (~490 Hz on most UNO pins) with a duty cycle proportional to the requested value; the LED (and the human eye/an LED's thermal response) averages this into a perceived brightness level.

Component Working Principle

Potentiometers used here are 3-terminal linear-taper devices: resistance-vs-rotation is a straight-line relationship (as opposed to logarithmic-taper pots used in audio volume controls). PWM-capable pins (D3, D5, D6, D9, D10, D11) route through dedicated hardware timers on the ATmega328P that can toggle the pin autonomously at a set duty cycle without CPU intervention between edges.

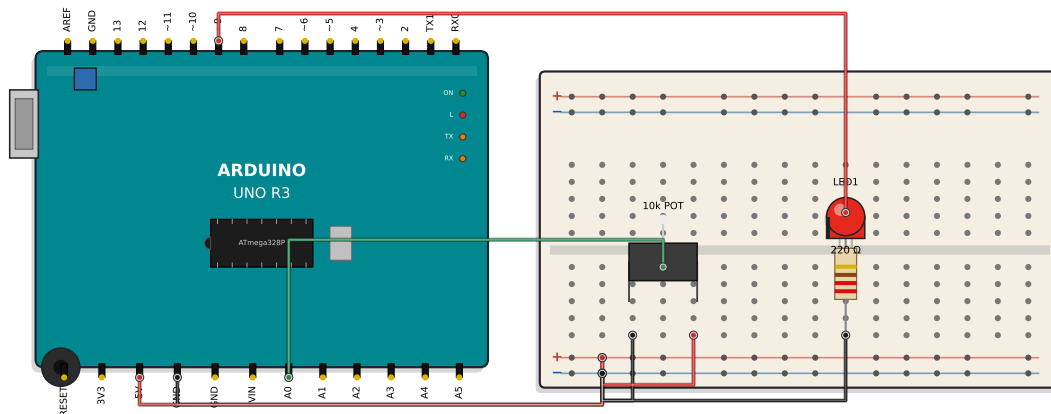


Figure 9.1: Experiment 9 circuit: potentiometer wiper to A0; LED (with resistor) on PWM pin D9.

Connection Table

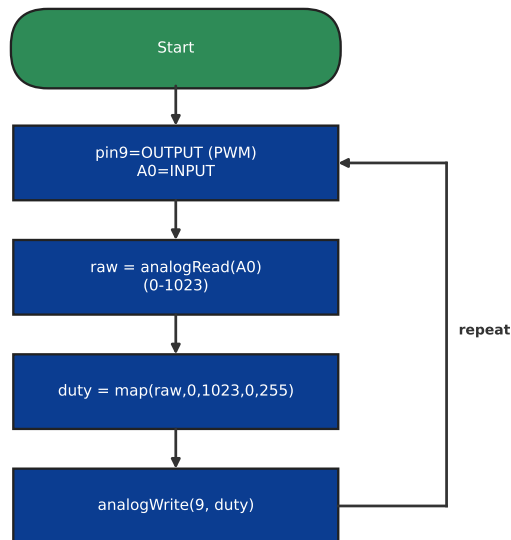
From	To	Notes
Potentiometer end 1	Arduino 5V	
Potentiometer end 2	Arduino GND	
Potentiometer wiper	Arduino A0	Analog input
Arduino D9 (~)	220 Ω → LED anode	PWM output
LED cathode	Arduino GND	

Working Principle – Signal Flow

Knob position → wiper voltage (0–5 V) → `analogRead(A0)` (0–1023) → `map()` to (0–255) → `analogWrite(D9, duty)` → LED brightness proportional to duty cycle.

Algorithm

1. Configure D9 as OUTPUT (A0 needs no explicit `pinMode` for `analogRead`).
2. Read the raw ADC value on A0.
3. Map 0–1023 to 0–255.
4. Write the mapped value to D9 with `analogWrite()`.
5. Repeat continuously.

**Figure 9.2:** Flowchart: Experiment 9.**Arduino Program**

```

1  const int POT_PIN = A0;
2  const int LED_PIN = 9;    // must be a PWM (~) pin
3
4  void setup() {
5      pinMode(LED_PIN, OUTPUT);
6      // A0 needs no pinMode() call to be used with analogRead().
7  }
8
9  void loop() {
10     int raw = analogRead(POT_PIN);           // 0-1023
11     int duty = map(raw, 0, 1023, 0, 255);    // rescale to 0-255
12     analogWrite(LED_PIN, duty);
13 }

```

Code Explanation

`analogRead(POT_PIN)` triggers the ADC and returns a 10-bit code proportional to the wiper voltage. `map()` performs the linear rescale from Equation form $out = (in - inMin) \times \frac{outMax - outMin}{inMax - inMin} + outMin$. `analogWrite(LED_PIN, duty)` then sets the PWM duty cycle for D9's hardware timer, which continues generating that duty cycle in the background until the next `analogWrite()` call.

Experimental Procedure

1. Wire the potentiometer's two outer legs to 5V and GND, wiper to A0.
2. Wire the LED+resistor to D9 exactly as in Experiment 1 (but note: D9, not D8, since PWM is required here).
3. Upload the sketch and slowly turn the knob, observing brightness change smoothly.

Expected Output

LED brightness varies smoothly from fully off (knob at one extreme) to fully bright (knob at the other extreme), with no visible flicker or stepping at normal turning speed.

Observation Table

Knob position	Raw 1023)	ADC (0– 255)	Mapped duty (0– 255)	Brightness (quali- tative)
Fully CCW				
Midway				
Fully CW				

Calculations and Analysis

For the midway reading, compute $V_{wiper} = 5 \times (\text{raw}/1023)$ and compare with a multimeter reading at the wiper pin. Also verify $\text{duty} = \text{round}(\text{raw} \times 255/1023)$ matches your printed duty value.

Result

The LED's brightness was successfully controlled continuously via the potentiometer over the full ADC range 0–1023: _____ (Yes/No).

Troubleshooting

Problem	Likely cause	Fix
LED is either fully on or fully off only	LED wired to a non-PWM pin	Confirm D9 (marked ~) is used
ADC reading stuck at 0 or 1023	Potentiometer ends swapped with wiper	Re-check the 3-pin wiring
Reading jumps erratically	Loose wiper connection	Reseat the wiper jumper
Brightness changes but direction reversed	Pot ends swapped (5V/GND)	Not a fault, just mirrors the scale; swap if desired

Viva Questions and Extension Challenge

Conceptual: (1) Why does the potentiometer need all three pins connected, not just the wiper? (2) What does "10-bit ADC" mean numerically? (3) Why can't `analogWrite()` be used on D8 (used in Experiment 1)? (4) Is the LED voltage truly analog during PWM, or does it only appear so? (5) What is the approximate PWM frequency on D9? **Troubleshooting:** (1) The

brightness changes only over the last quarter of the knob's rotation — what wiring fault causes this? (2) Serial-printed raw values never exceed ~512 — what does this suggest? (3) The LED buzzes faintly at certain brightness levels — is this expected? **Programming:** (1) Print both `raw` and `duty` over Serial for live debugging. (2) Add a dead-zone so values below 5 map to a full OFF (`duty=0`) to avoid a very dim "off-but-not-quite" LED. **Extension Challenge:** Use the same potentiometer reading to simultaneously drive two LEDs on two PWM pins with inverse brightness (one brightens as the other dims).

Chapter 10

Experiment 10: LDR-Based Automatic Light

Experiment Identification

Experiment Number:	10
Title:	LDR-Based Automatic Light
Learning Layer:	Layer 1 — Simple
Estimated Completion Time:	25 minutes
Difficulty:	Beginner
Prerequisite Experiments:	Experiment 9

Aim

To automatically switch an LED "lamp" ON in darkness and OFF in light, using an LDR voltage divider and a fixed analog threshold.

Learning Objectives

1. Build an LDR voltage divider and explain its non-linear resistance-vs-light behaviour.
2. Choose a threshold ADC value experimentally.
3. Implement threshold-based digital decision-making from an analog reading.

Required Components

Arduino UNO R3, breadboard, 1 LDR, $1 \times 10 \text{ k}\Omega$ resistor (fixed divider resistor), 1 white LED, $1 \times 220 \Omega$ resistor, jumper wires.

Technical Theory

An LDR (light-dependent resistor) has resistance that falls as illumination increases: roughly $1 \text{ k}\Omega$ in bright light up to several hundred $\text{k}\Omega$ in darkness (device-dependent; not a precise, calibrated relationship — see Experiment 20 for calibration discussion). Paired with a fixed $10 \text{ k}\Omega$ resistor as a voltage divider from 5 V to GND, with the LDR on the top (5 V side) and the junction read by A0:

$$V_{A0} = 5 \text{ V} \times \frac{R_{\text{fixed}}}{R_{\text{LDR}} + R_{\text{fixed}}} \quad (10.1)$$

In darkness, R_{LDR} is large, so V_{A0} is *low*; in bright light, R_{LDR} is small, so V_{A0} is *high*. A threshold ADC value (found experimentally by observing raw readings in your actual room

lighting) decides "dark" vs "bright".

Component Working Principle

The LDR is a cadmium-sulphide (CdS) photoresistor: incoming photons free charge carriers in the semiconductor, lowering its resistance. Response is relatively slow (tens of milliseconds) compared to a photodiode, which is unimportant for a slowly-changing "lamp control" application but matters for high-speed light sensing (not used in this manual).

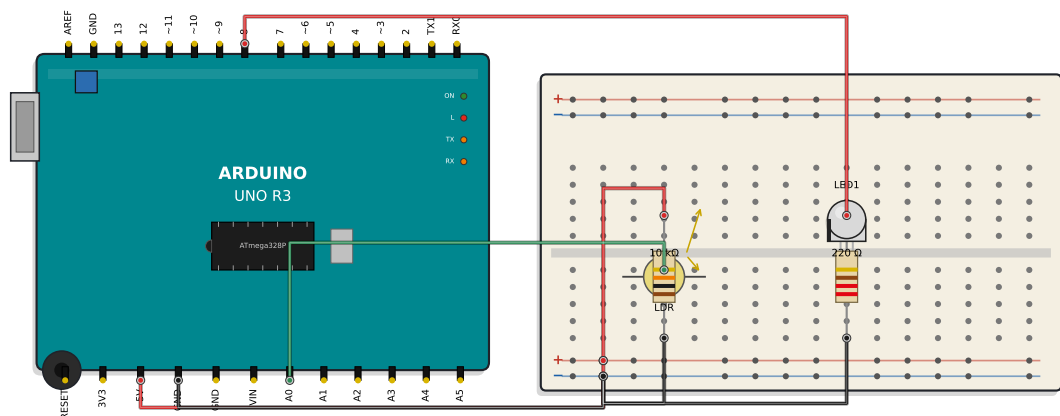


Figure 10.1: Experiment 10 circuit: LDR + 10 kΩ voltage divider at A0; white LED "lamp" on D8.

Connection Table

From	To	Notes
Arduino 5V	LDR leg 1	Top of divider
LDR leg 2	10 kΩ resistor + Arduino A0 (junction)	Divider midpoint
10 kΩ resistor other leg	Arduino GND	Bottom of divider
Arduino D8	220 Ω → white LED anode	Lamp output
LED cathode	Arduino GND	

Working Principle – Signal Flow

Ambient light level → LDR resistance (inverse relationship) → divider voltage at A0 → ADC code → compared against DARK_THRESHOLD → D8 driven HIGH (lamp ON) if darker than threshold, else LOW.

Algorithm

1. Configure D8 as OUTPUT.
2. Read A0.
3. If reading is below DARK_THRESHOLD, turn the lamp ON; else turn it OFF.

4. Repeat continuously.

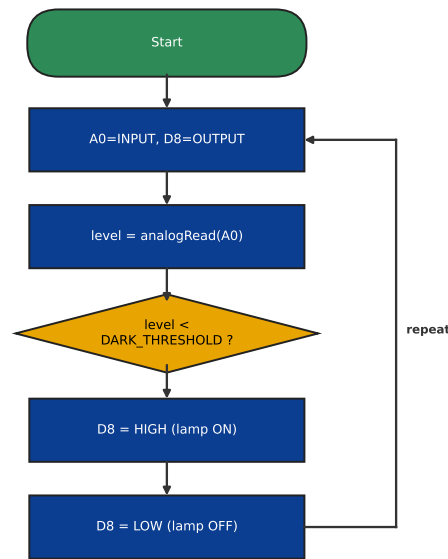


Figure 10.2: Flowchart: Experiment 10.

Arduino Program

```
1  const int LDR_PIN = A0;
2  const int LED_PIN = 8;
3  const int DARK_THRESHOLD = 400;    // determine this experimentally for
   // your room; see Procedure
4
5  void setup() {
6    pinMode(LED_PIN, OUTPUT);
7    Serial.begin(9600);              // used during threshold calibration
8  }
9
10 void loop() {
11   int level = analogRead(LDR_PIN);
12   Serial.println(level);            // watch this while covering/
   // uncovering the LDR
13
14   if (level < DARK_THRESHOLD) {
15     digitalWrite(LED_PIN, HIGH);    // dark: lamp ON
16   } else {
17     digitalWrite(LED_PIN, LOW);     // bright: lamp OFF
18   }
19   delay(200);
20 }
```

Code Explanation

`Serial.println(level)` is included specifically so the threshold can be tuned to the learner's actual room lighting — an LDR's absolute readings vary with the specific LDR unit, ambient light source, and divider resistor tolerance, so a single "correct" `DARK_THRESHOLD` does not exist across all setups. The `if/else` implements a simple one-sided decision with no hysteresis (addressed

formally in Experiment 44).

Experimental Procedure

1. Build the divider and lamp circuit per the connection table.
2. Upload the sketch and open the Serial Monitor at 9600 baud.
3. Note the raw value in normal room light, then cover the LDR with your hand and note the new value.
4. Choose `DARK_THRESHOLD` roughly midway between the two, re-upload, and confirm the lamp switches correctly.

Expected Output

Illustrative Serial Monitor values: ≈ 700 – 900 in bright room light, dropping to ≈ 150 – 300 when the LDR is covered; the lamp switches ON as the reading crosses below `DARK_THRESHOLD`.

Observation Table

Lighting condition	Raw ADC value	Lamp state
Bright room light		
Hand covering LDR		
Torch/phone light on LDR		

Calculations and Analysis

Using the divider equation, estimate R_{LDR} at your two extreme readings: $R_{LDR} = R_{fixed} \times (1023/raw - 1)$ (derived from Equation 10.1 in ADC-code form). Compare the bright-vs-dark resistance ratio against typical CdS LDR datasheet ranges.

Result

The lamp automatically switched ON below an ADC threshold of _____ and OFF above it, matching the intended dark/bright behaviour: _____ (Yes/No).

Troubleshooting

Problem	Likely cause	Fix
Lamp always ON	Threshold set too high, or divider reversed	Re-check LDR/resistor order and threshold value
Lamp always OFF	Divider reversed (LDR on bottom instead of top)	Swap LDR and fixed resistor positions
Lamp flickers near the threshold boundary	Normal analog noise at a hard threshold	Expected; solved formally with hysteresis in Experiment 44
Serial Monitor shows nothing	Wrong baud rate selected	Match 9600 baud

Viva Questions and Extension Challenge

Conceptual: (1) Why does covering the LDR lower, not raise, the A0 reading in this wiring? (2) What would happen if the LDR and fixed resistor swapped positions? (3) Why can't a single threshold value be given that works for every classroom? (4) What does "no hysteresis" mean here, and why might that cause flicker? (5) Is the LDR's response instant or does it lag light changes? **Troubleshooting:** (1) The lamp behaves opposite to expectation (ON in light, OFF in dark) — what wiring swap explains this? (2) Threshold that worked in the morning fails by afternoon — why? (3) Readings never change no matter the lighting — what does this suggest about the LDR leg wiring? **Programming:** (1) Add a second, independent light-level "very dark" tier that also sounds a buzzer (reuse Experiment 12's pattern). (2) Print "DARK"/"BRIGHT" text instead of raw numbers once the threshold is chosen. **Extension Challenge:** Average the last 10 raw readings (simple moving average) before comparing to the threshold, and explain what problem this solves.

Chapter 11

Experiment 11: LDR Light-Level Indicator

Experiment Identification

Experiment Number:	11
Title:	LDR Light-Level Indicator
Learning Layer:	Layer 1 — Simple
Estimated Completion Time:	25 minutes
Difficulty:	Beginner
Prerequisite Experiments:	Experiment 10

Aim

To display ambient light level as a 3-LED bar (low/medium/high), extending the single-threshold decision of Experiment 10 to multiple thresholds.

Learning Objectives

1. Partition a continuous analog range into discrete "buckets" using multiple thresholds.
2. Implement mutually-exclusive multi-level output from one analog input.
3. Explain why bucket boundaries, not exact values, define behaviour.

Required Components

Arduino UNO R3, breadboard, 1 LDR, $1 \times 10 \text{ k}\Omega$ resistor, 3 LEDs (green, yellow, red), $3 \times 220 \Omega$ resistors.

Technical Theory

Two thresholds $T_1 < T_2$ divide the 0–1023 ADC range into three buckets: LOW ($< T_1$), MEDIUM (T_1 to T_2), HIGH ($> T_2$). Exactly one of three LEDs should be lit for any reading, mirroring Experiment 4's "exactly one output active" pattern, but now selected by analog comparison rather than a rotating index.

Component Working Principle

Same LDR divider as Experiment 10; see that experiment's Component Working Principle for LDR behaviour.

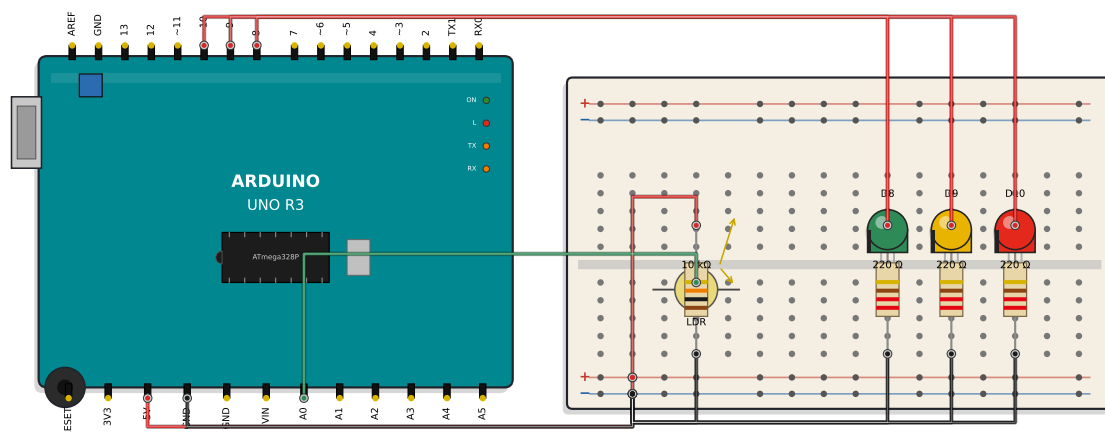


Figure 11.1: Experiment 11 circuit: LDR divider at A0; green/yellow/red LEDs on D8/D9/D10 as a 3-level bar.

Connection Table

From	To	Notes
LDR divider (as Experiment 10)	A0	
D8	220 Ω → Green LED	Lit when HIGH light
D9	220 Ω → Yellow LED	Lit when MEDIUM light
D10	220 Ω → Red LED	Lit when LOW light (dark)

Working Principle – Signal Flow

A0 reading → compared against T_1 , T_2 → exactly one of green/yellow/red driven HIGH, the other two forced LOW.

Algorithm

1. Read A0.
2. If reading < T_1 : red ON, others OFF (dark).
3. Else if reading < T_2 : yellow ON, others OFF (medium).
4. Else: green ON, others OFF (bright).

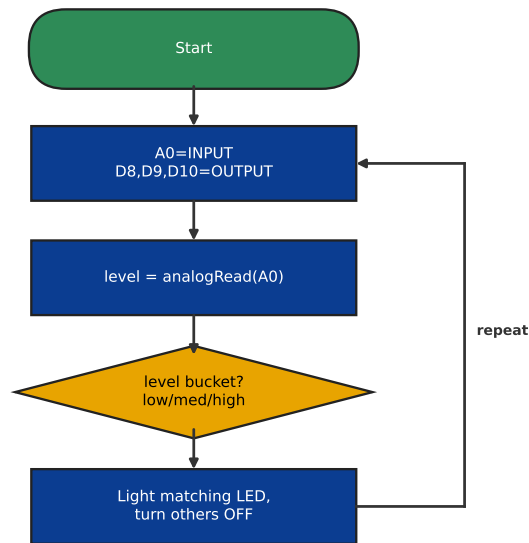


Figure 11.2: Flowchart: Experiment 11.

Arduino Program

```

1  const int LDR_PIN = A0;
2  const int GREEN = 8, YELLOW = 9, RED = 10;
3  const int T1 = 300, T2 = 650;    // tune to your room; see Experiment
    10 procedure
4
5  void setup() {
6    pinMode(GREEN, OUTPUT); pinMode(YELLOW, OUTPUT); pinMode(RED, OUTPUT
    );
7    Serial.begin(9600);
8  }
9
10 void loop() {
11   int level = analogRead(LDR_PIN);
12   Serial.println(level);
13
14   digitalWrite(RED,    level < T1 ? HIGH : LOW);
15   digitalWrite(YELLOW, (level >= T1 && level < T2) ? HIGH : LOW);
16   digitalWrite(GREEN, level >= T2 ? HIGH : LOW);
17   delay(200);
18 }

```

Code Explanation

Each `digitalWrite()` independently evaluates its own bucket condition, so all three are always assigned an explicit state every pass — there is no "leftover ON" state from a previous reading, avoiding the bug class seen in Experiment 5 if `allOff()` were forgotten.

Experimental Procedure

1. Wire the LDR divider (as in Experiment 10) and the three indicator LEDs.
2. Upload, observe Serial values across your room's actual lighting range, and tune T_1 , T_2 accordingly.

3. Confirm exactly one LED is lit at any moment.

Expected Output

Red lit in darkness, yellow in dim/moderate light, green in bright light, with clean single-LED transitions at the tuned thresholds.

Observation Table

Lighting condition	Raw ADC	LED lit

Calculations and Analysis

Confirm $T_1 < T_2$ always holds and that your three observed conditions each fall cleanly within one bucket, not near a boundary (which would indicate poorly chosen thresholds for your environment).

Result

The 3-level light indicator correctly showed exactly one LED for each of the three tested lighting conditions: _____ (Yes/No).

Troubleshooting

Problem	Likely cause	Fix
Two LEDs lit simultaneously	Overlapping/incorrect bucket comparisons	Re-check the three conditions are mutually exclusive
Middle LED never lights	T_1, T_2 too close together or swapped	Re-tune with wider separation
Wrong LED for given light level	Thresholds not tuned to your room	Redo the calibration procedure

Viva Questions and Extension Challenge

Conceptual: (1) Why must the three conditions be mutually exclusive by construction? (2) What happens at exactly $\text{level} = T_1$? (3) How would a 4th bucket be added? (4) Why is `delay(200)` acceptable here but was avoided in earlier experiments needing responsiveness? (5) Why does this manual avoid claiming these are calibrated lux values? **Troubleshooting:** (1) All three LEDs are occasionally off together for one instant — what logic gap could cause this? (2) The indicator seems "stuck" on yellow across a wide range of real lighting — what does this suggest about T_1/T_2 spacing? **Programming:** (1) Refactor the three `digitalWrite` lines into a `for` loop over an array of thresholds and pins. (2) Add a 4th bucket ("very bright") with a 4th LED. **Extension Challenge:** Replace hard bucket boundaries with a PWM-based smooth cross-fade between green and red as light level changes continuously.

Chapter 12

Experiment 12: Buzzer ON/OFF Control

Experiment Identification

Experiment Number:	12
Title:	Buzzer ON/OFF Control
Learning Layer:	Layer 1 — Simple
Estimated Completion Time:	15 minutes
Difficulty:	Beginner
Prerequisite Experiments:	Experiment 1

Aim

To drive the kit's active buzzer directly from a digital output pin, introducing audio-frequency actuation as a new output type alongside LEDs.

Learning Objectives

1. Distinguish an active buzzer (has its own oscillator) from a passive buzzer (Experiment 14).
2. Drive a buzzer directly from a GPIO pin within its safe current rating.
3. Recognise datasheet current draw as a per-component check, not a fixed assumption.

Required Components

Arduino UNO R3, breadboard, 1 active buzzer (see Chapter note on identifying active vs. passive), 2 jumper wires.

Technical Theory

An active buzzer contains its own internal oscillator circuit: applying a steady DC voltage is enough to make it sound continuously at its own fixed tone, unlike a passive buzzer which stays silent under DC and needs an external oscillating signal (Experiment 14). This manual assumes the supplied active buzzer draws under 20 mA at 5 V, typical for small 5 V piezo/magnetic indicator buzzers, and therefore drives it directly from a GPIO pin without a relay or transistor stage. If your specific buzzer's datasheet states a higher current draw, drive it through the relay module instead (see the relay-driving pattern introduced in Experiment 29).

Component Working Principle

Active buzzers are typically piezoelectric (a ceramic disc that flexes under the driving oscillator to produce sound) or electromagnetic (a small coil and diaphragm). Either type in "active" form includes the oscillator on the same small PCB/housing, so only two wires (power and ground) are needed — no signal frequency is generated by the Arduino itself in this experiment.

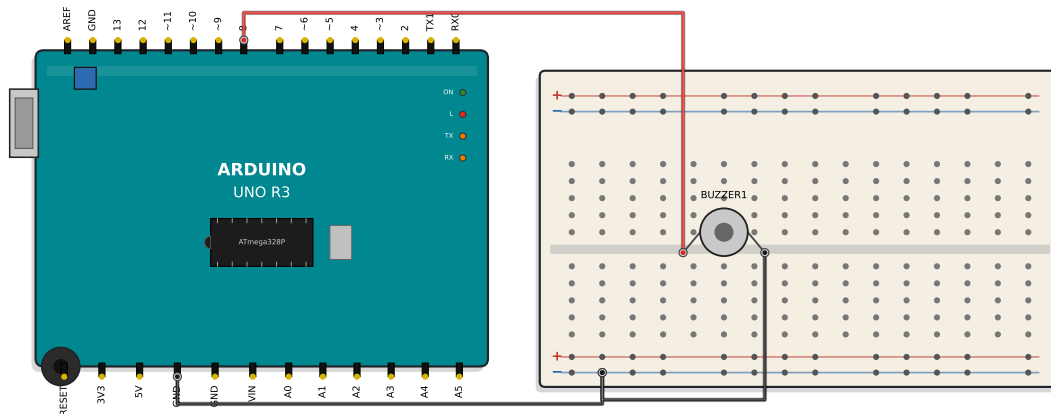


Figure 12.1: Experiment 12 circuit: active buzzer directly on D8.

Connection Table

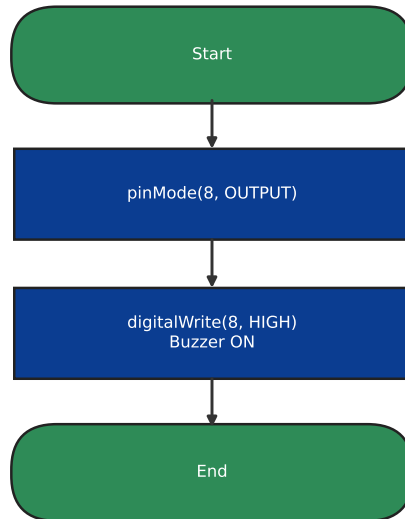
From	To	Notes
Arduino D8	Buzzer + (red lead)	Digital output
Buzzer – (black lead)	Arduino GND	

Working Principle – Signal Flow

D8 HIGH → 5 V applied across the buzzer's internal oscillator → buzzer sounds continuously at its fixed internal tone; D8 LOW → 0 V → silent.

Algorithm

1. Configure D8 as OUTPUT.
2. Set D8 HIGH (buzzer sounds).
3. (Optionally, to demonstrate OFF) wait, then set D8 LOW.

**Figure 12.2:** Flowchart: Experiment 12.**Arduino Program**

```
1  const int BUZZER_PIN = 8;
2
3  void setup() {
4      pinMode(BUZZER_PIN, OUTPUT);
5      digitalWrite(BUZZER_PIN, HIGH);  // buzzer ON continuously
6  }
7
8  void loop() {
9      // Empty: buzzer stays ON. Change HIGH to LOW above to test OFF.
10 }
```

Code Explanation

This sketch deliberately mirrors Experiment 1's structure exactly (static `setup()`-only control), to highlight that an active buzzer is, electrically, "just another digital output load" from the Arduino's point of view — the interesting engineering content is the buzzer's internal oscillator, not the Arduino code.

Experimental Procedure

1. With the Arduino unplugged, wire the buzzer's two leads per the connection table, observing correct polarity if marked.
2. Upload and confirm continuous sound; change `HIGH` to `LOW` and re-upload to confirm silence.

Expected Output

Continuous audible tone immediately after upload; silence if the sketch is changed to drive D8 LOW.

Observation Table

D8 state	Buzzer sound observed
HIGH	
LOW	

Calculations and Analysis

If a multimeter with a current-measurement (mA) range is available, measure the buzzer's actual current draw in series and compare against the 20 mA GPIO safe limit assumed in the Technical Theory.

Result

The active buzzer was successfully switched ON/OFF directly from D8, drawing an estimated/measured current of _____ mA.

Troubleshooting

Problem	Likely cause	Fix
No sound at all	Buzzer is actually the passive type	Test with Experiment 14's <code>tone()</code> sketch instead
Very quiet/weak sound	Reversed polarity on a polarised buzzer	Swap leads
Sound present but Arduino resets/browns out	Buzzer draws more current than assumed	Switch to relay-driven approach (Experiment 29 pattern)

Viva Questions and Extension Challenge

Conceptual: (1) What distinguishes an active buzzer from a passive one at the circuit level? (2) Why is no resistor needed here, unlike the LED experiments? (3) What GPIO current limit must this buzzer respect? (4) Why might reversing the leads matter for some buzzers but not others? (5) What internal component makes the buzzer "active"? **Troubleshooting:** (1) The buzzer emits a very faint click only at power-up, then silence — what type of buzzer does this suggest? (2) The Arduino appears to restart when the buzzer turns on — what does this indicate? **Programming:** (1) Modify the sketch to beep three short times at startup, then stay silent. (2) Add a button (reusing Experiment 6 wiring) that sounds the buzzer only while held. **Extension Challenge:** Use `millis()` (Chapter) to make the buzzer beep in a fixed pattern (e.g. 200 ms ON / 800 ms OFF) without blocking, in preparation for Experiment 17's non-blocking alarm.

Chapter 13

Experiment 13: Electronic Doorbell

Experiment Identification

Experiment Number:	13
Title:	Electronic Doorbell
Learning Layer:	Layer 1 — Simple
Estimated Completion Time:	20 minutes
Difficulty:	Beginner
Prerequisite Experiments:	Experiments 6, 12

Aim

To build a doorbell that sounds the active buzzer only while a push button is held, combining debounced-free level-based button input (Experiment 6) with buzzer output (Experiment 12).

Learning Objectives

1. Combine a previously learned input pattern with a previously learned output pattern.
2. Recognise when debouncing is unnecessary (level control, not counting/toggling).
3. Read and reuse code patterns across experiments confidently.

Required Components

Arduino UNO R3, breadboard, 1 push button, 1 active buzzer, jumper wires.

Technical Theory

This experiment introduces no new electrical theory: it is a direct composition of Experiment 6 (button drives an output while held) and Experiment 12 (active buzzer as a digital load), with the LED output simply replaced by the buzzer output. This is intentional — real embedded projects are built by composing previously verified sub-circuits, and recognising "I've already solved a piece of this" is itself an engineering skill.

Component Working Principle

See Experiment 6 (push button internal pairing) and Experiment 12 (active buzzer internal oscillator) for component-level detail.

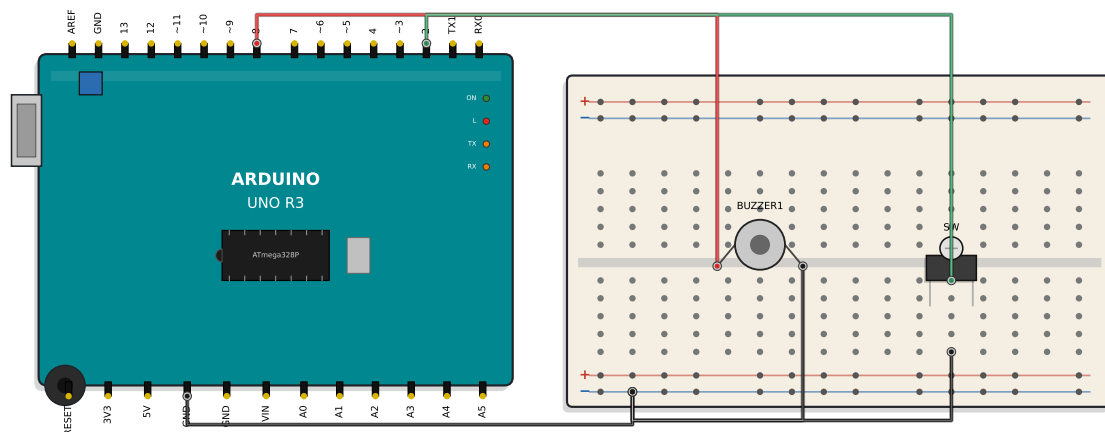


Figure 13.1: Experiment 13 circuit: push button on D2, active buzzer on D8.

Connection Table

From	To	Notes
Button leg A	Arduino D2	INPUT_PULLUP
Button leg B	Arduino GND	
Arduino D8	Buzzer +	
Buzzer –	Arduino GND	

Working Principle – Signal Flow

Button pressed (D2 LOW) → D8 driven HIGH → buzzer sounds; button released (D2 HIGH) → D8 LOW → silent.

Algorithm

1. Configure D2 as INPUT_PULLUP, D8 as OUTPUT.
2. Read D2.
3. If LOW, buzzer ON; else buzzer OFF.
4. Repeat.

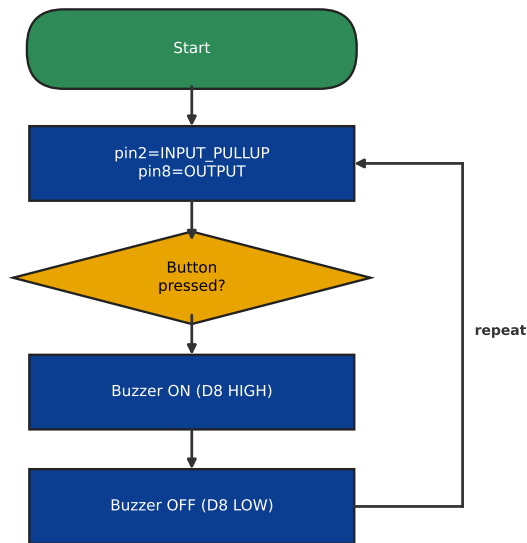


Figure 13.2: Flowchart: Experiment 13.

Arduino Program

```

1  const int BUTTON_PIN = 2;
2  const int BUZZER_PIN = 8;
3
4  void setup() {
5      pinMode(BUTTON_PIN, INPUT_PULLUP);
6      pinMode(BUZZER_PIN, OUTPUT);
7  }
8
9  void loop() {
10     bool pressed = (digitalRead(BUTTON_PIN) == LOW);
11     digitalWrite(BUZZER_PIN, pressed ? HIGH : LOW);
12 }

```

Code Explanation

This is Experiment 6's exact logic, with the boolean made explicit (**pressed**) for readability and the LED pin renamed **BUZZER_PIN**. No debouncing is used because, as in Experiment 6, the output simply tracks the live button level rather than counting discrete presses.

Experimental Procedure

1. Wire the button and buzzer per the connection table.
2. Upload and press the button, confirming the buzzer sounds only while held.

Expected Output

Buzzer silent at rest; sounds immediately and continuously while the button is held; stops the instant it is released.

Observation Table

Button state	Buzzer state observed
Released	
Pressed	

Calculations and Analysis

Not applicable — this experiment is a functional/logical composition exercise rather than a quantitative one.

Result

The doorbell correctly sounded only while the button was held: _____ (Yes/No).

Troubleshooting

Problem	Likely cause	Fix
Buzzer always sounds	Button legs from the same internal pair used	Rewire per Experiment 6 notes
No sound at all	Buzzer wiring/type issue	Re-check Experiment 12 troubleshooting
Sound is intermittent while held	Loose button jumper	Reseat wiring

Viva Questions and Extension Challenge

Conceptual: (1) Which two prior experiments does this one combine? (2) Why is no debouncing required? (3) What would change if a toggle (Experiment 7 style) doorbell were required instead? (4) Could the button and buzzer share the same GND rail safely? (5) Why is this a "level" rather than "edge" application? **Troubleshooting:** (1) The buzzer sounds briefly at power-up even without a press — what could explain this transient? (2) The doorbell stops responding after several presses but resumes after a reset — what should you inspect? **Programming:** (1) Make the doorbell sound for a fixed 500 ms per press (a "ding-dong" style) instead of tracking the live level, regardless of how long the button is held. (2) Add a Serial log timestamping every press using `millis()`. **Extension Challenge:** Build a two-button doorbell (front door / back door) that sounds a short beep pattern for one button and a long beep pattern for the other, using the same single buzzer.

Chapter 14

Experiment 14: Variable-Tone Buzzer

Experiment Identification

Experiment Number:	14
Title:	Variable-Tone Buzzer
Learning Layer:	Layer 1 — Simple
Estimated Completion Time:	25 minutes
Difficulty:	Beginner
Prerequisite Experiments:	Experiments 9, 12

Aim

To continuously vary the passive buzzer's tone frequency using the potentiometer, introducing `tone()` and the distinction between active and passive buzzers.

Learning Objectives

1. Explain why a passive buzzer needs a driven frequency, unlike the active buzzer of Experiment 12.
2. Use `tone()` to generate a specific audio frequency on any digital pin.
3. Map an analog reading onto a musically/audibly useful frequency range.

Required Components

Arduino UNO R3, breadboard, 1 potentiometer, 1 passive buzzer (see Chapter), jumper wires.

Technical Theory

A passive buzzer has no internal oscillator: it is essentially a small speaker that reproduces whatever square wave is applied to it, so its pitch (frequency) is entirely determined by the driving signal. Arduino's `tone(pin, frequency)` function generates a 50% duty-cycle square wave at the requested frequency (in Hz) on any digital pin using Timer2 internally — note this is a different mechanism from `analogWrite()`'s PWM, and `tone()` does *not* require a ~-marked pin. The human audible range is roughly 20 Hz–20 kHz; this experiment maps the potentiometer to a musically useful 200–2000 Hz range.

Component Working Principle

The passive buzzer's diaphragm physically vibrates at the driving frequency, producing sound at that same frequency (unlike the active buzzer's fixed internal tone from Experiment 12). Using `tone()` on the wrong buzzer type (active) produces no pitch change, since an active buzzer ignores its input signal's frequency and just buzzes at its own fixed tone whenever powered.

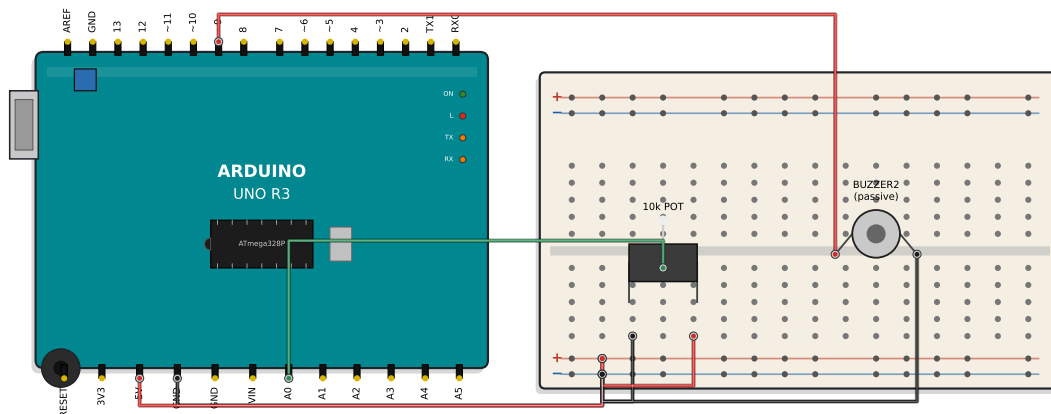


Figure 14.1: Experiment 14 circuit: potentiometer wiper at A0; passive buzzer driven by `tone()` on D9.

Connection Table

From	To	Notes
Potentiometer ends	5V, GND	As Experiment 9
Potentiometer wiper	A0	
Arduino D9	Passive buzzer +	<code>tone()</code> output (any digital pin works)
Buzzer –	GND	

Working Principle – Signal Flow

Knob position → A0 reading (0–1023) → mapped frequency (200–2000 Hz) → `tone(D9, freq)` → buzzer pitch rises/falls continuously as the knob turns.

Algorithm

1. Read A0.
2. Map 0–1023 to 200–2000 (Hz).
3. Call `tone(D9, freq)` with the mapped value.
4. Repeat continuously.

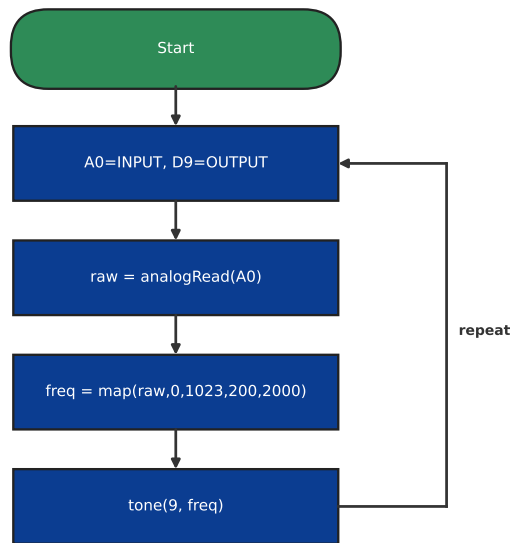


Figure 14.2: Flowchart: Experiment 14.

Arduino Program

```

1  const int POT_PIN = A0;
2  const int BUZZER_PIN = 9;
3
4  void setup() {
5      // tone() does not require pinMode(); it configures the pin
6      // internally.
7  }
8
9  void loop() {
10     int raw = analogRead(POT_PIN);
11     int freq = map(raw, 0, 1023, 200, 2000);
12     tone(BUZZER_PIN, freq);
13     delay(20); // short pause keeps the tone update rate smooth but
14                // responsive
15 }

```

Code Explanation

Calling `tone()` again with a new frequency simply updates the ongoing square wave — there is no need to call `noTone()` between updates. The 20 ms delay is short enough that pitch changes feel continuous to the ear, while still leaving CPU time free (a much shorter delay would offer no perceptible benefit here).

Experimental Procedure

1. Wire the potentiometer as in Experiment 9 and the passive buzzer on D9.
2. Upload and slowly turn the knob, listening for a continuously rising/falling pitch.
3. If no pitch change is heard, confirm the buzzer is the passive type (Experiment 12's buzzer would only ever be silent under `tone()` if wired without power, or would only ever be at its own fixed tone if it were actually the active type).

Expected Output

A continuously variable audible tone, rising in pitch as the knob turns towards its high-frequency end, falling as it turns back.

Observation Table

Knob position	Raw ADC	Mapped frequency (Hz)	Pitch (qualitative)
Fully CCW			
Midway			
Fully CW			

Calculations and Analysis

Compare the mapped frequency at the midway knob position ($\approx$ raw 512) against the formula $f = 200 + (\text{raw}/1023) \times (2000 - 200)$, and check it against roughly the musical note it corresponds to (A4 = 440 Hz, for reference).

Result

The buzzer's pitch was successfully varied continuously from 200 Hz to 2000 Hz using the potentiometer: _____ (Yes/No).

Troubleshooting

Problem	Likely cause	Fix
Constant pitch regardless of knob	Buzzer is actually the active type	Swap to the other buzzer; confirm with Experiment 12's test
No sound	Wrong pin or broken connection	Re-check wiring
Choppy/stuttering tone	Delay too long	Reduce the 20 ms delay

Viva Questions and Extension Challenge

Conceptual: (1) Why doesn't `tone()` need a PWM-marked pin? (2) What determines a passive buzzer's pitch? (3) What is the human audible frequency range, and why was 200–2000 Hz chosen here instead of the full range? (4) Why is `noTone()` not needed between frequency updates? (5) What Arduino hardware timer does `tone()` use internally? **Troubleshooting:** (1) The tone briefly glitches when the Serial Monitor is opened — is this expected? (2) The pitch changes only over half the knob's range — what could cause this? **Programming:** (1) Add `noTone()` triggered by a button press to silence the buzzer on demand. (2) Quantise the mapped frequency to the nearest of 8 fixed musical notes instead of a continuous sweep. **Extension Challenge:** Play a short fixed melody (an array of note, duration pairs) using `tone()` and `delay()`, independent of the potentiometer.

Chapter 15

Experiment 15: IR Obstacle Detector

Experiment Identification

Experiment Number:	15
Title:	IR Obstacle Detector
Learning Layer:	Layer 1 — Simple
Estimated Completion Time:	20 minutes
Difficulty:	Beginner
Prerequisite Experiments:	Experiment 6

Aim

To detect a nearby obstacle using the IR obstacle sensor module and indicate detection with an LED, introducing active-LOW digital sensor modules.

Learning Objectives

1. Wire and power a 3-pin digital sensor breakout module.
2. Explain the sensor's active-LOW output convention.
3. Adjust the on-board sensitivity trimmer and observe its effect.

Required Components

Arduino UNO R3, breadboard, 1 IR obstacle sensor module, 1 LED, $1 \times 220 \Omega$ resistor, jumper wires (female-to-female or male-to-female for the module).

Technical Theory

The IR obstacle sensor module contains an IR-emitting LED and an IR phototransistor mounted side by side, plus an on-board comparator IC. The emitter continuously sends IR light; if an object is within range, IR light reflects back to the phototransistor, and the comparator's output (OUT) switches — on almost all modules of this type, OUT goes **LOW** when an obstacle is detected (active-LOW), and **HIGH** otherwise. An on-board potentiometer trimmer adjusts the comparator's reference threshold, changing the effective detection distance (typically a few cm up to ≈ 30 cm depending on surface reflectivity).

Component Working Principle

Because the sensor's own comparator does the threshold decision in hardware, the Arduino only ever needs to read one clean digital HIGH/LOW signal on OUT — there is no analog reading or software threshold involved, unlike the LDR experiments. This makes wiring and code simple, at the cost of not being able to read a distance value (only presence/absence), unlike the HC-SR04 (Experiment 16).

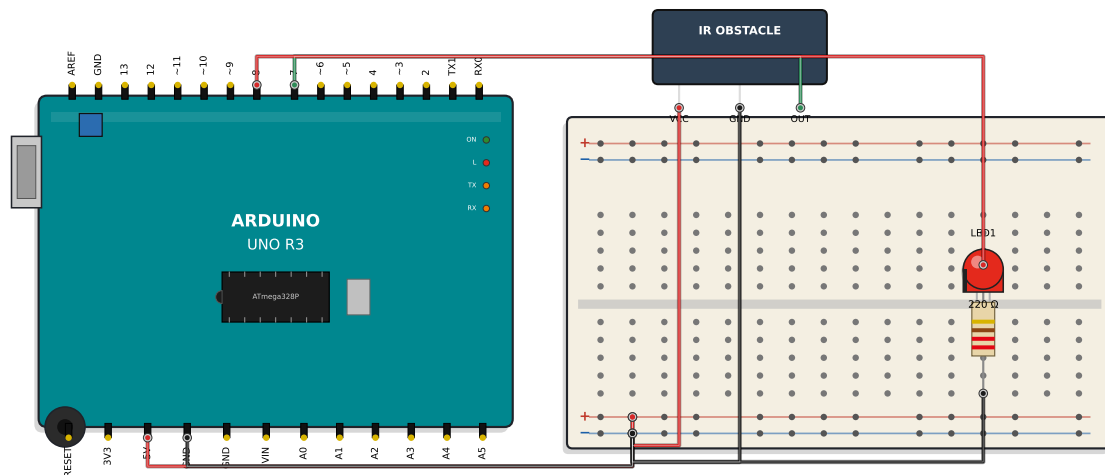


Figure 15.1: Experiment 15 circuit: IR obstacle module (VCC/GND/OUT) with OUT on D7; indicator LED on D8.

Connection Table

Module pin	Arduino pin	Notes
VCC	5V	
GND	GND	
OUT	D7	Digital input, active-LOW
D8	220 Ω $\rightarrow$ LED anode	Indicator output

Working Principle – Signal Flow

Obstacle within range $\rightarrow$ reflected IR detected by the module's comparator $\rightarrow$ OUT driven LOW $\rightarrow$ Arduino reads D7 LOW $\rightarrow$ LED (D8) driven ON; no obstacle $\rightarrow$ OUT HIGH $\rightarrow$ LED OFF.

Algorithm

1. Configure D7 as INPUT, D8 as OUTPUT.
2. Read D7.
3. If LOW (obstacle detected), turn LED ON; else OFF.
4. Repeat continuously.

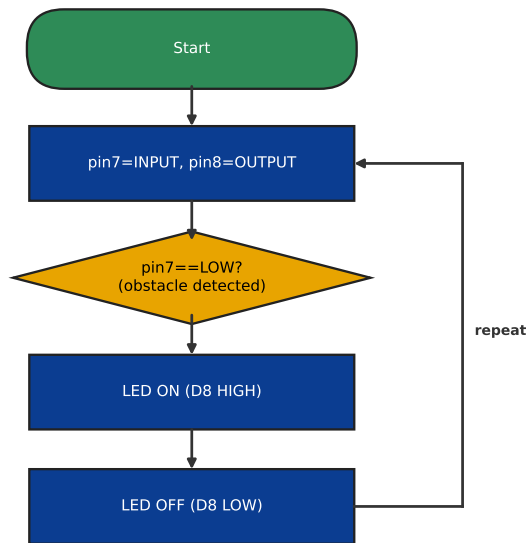


Figure 15.2: Flowchart: Experiment 15.

Arduino Program

```

1  const int IR_PIN = 7;
2  const int LED_PIN = 8;
3
4  void setup() {
5      pinMode(IR_PIN, INPUT);
6      pinMode(LED_PIN, OUTPUT);
7      Serial.begin(9600);
8  }
9
10 void loop() {
11     int reading = digitalRead(IR_PIN);
12     bool obstacleDetected = (reading == LOW);    // active-LOW module
13     // output
14     digitalWrite(LED_PIN, obstacleDetected ? HIGH : LOW);
15     Serial.println(obstacleDetected ? "Obstacle detected" : "Clear");
16     delay(100);
17 }

```

Code Explanation

The boolean `obstacleDetected` makes the active-LOW inversion explicit and readable in one place, so the rest of the sketch (and future experiments reusing this sensor, e.g. Experiment 31) can reason in plain "detected/not detected" terms rather than re-deriving the LOW/HIGH convention each time.

Experimental Procedure

1. Wire the module and LED per the connection table.
2. Upload and open the Serial Monitor.
3. Wave a hand/object in front of the sensor at varying distances and observe the LED and Serial output.

Viva Questions and Extension Challenge

Conceptual: (1) Why is the module's output digital rather than analog? (2) What does the on-board trimmer actually adjust? (3) Why does surface colour/material affect detection range? (4) Why is this sensor unsuitable for measuring exact distance? (5) What does "active-LOW" mean for this specific module? **Troubleshooting:** (1) The sensor detects a wall reliably but not a black cloth at the same distance — why? (2) The LED flickers on/off rapidly with a stationary object at the detection boundary — what analog comparator behaviour explains this? (3) The sensor works on the bench but not when mounted near a bright window — why? **Programming:** (1) Add a running count of total detections since power-up, printed periodically. (2) Add debouncing (Experiment 7 pattern) to the digital reading to avoid boundary flicker being logged as multiple detections. **Extension Challenge:** Combine this sensor with the buzzer (Experiment 12) to build a simple proximity alarm, as a preview of Experiment 31.

Chapter 16

Experiment 16: Ultrasonic Distance Measurement

Experiment Identification

Experiment Number:	16
Title:	Ultrasonic Distance Measurement
Learning Layer:	Layer 1 — Simple
Estimated Completion Time:	25 minutes
Difficulty:	Beginner–Intermediate
Prerequisite Experiments:	Experiment 1

Aim

To measure distance to an object using the HC-SR04 ultrasonic sensor’s time-of-flight principle, and report it over Serial in centimetres.

Learning Objectives

1. Explain time-of-flight distance measurement using the speed of sound.
2. Use `pulseIn()` to measure a pulse duration in microseconds.
3. Convert a round-trip echo time into a one-way distance in cm.
4. Handle a sensor timeout (no echo received) without hanging the program.

Required Components

Arduino UNO R3, breadboard, 1 HC-SR04 ultrasonic sensor, jumper wires.

Technical Theory

The HC-SR04 emits a burst of 40 kHz ultrasonic pulses when triggered, and its ECHO pin goes HIGH for exactly as long as it takes the pulse to travel to an object and reflect back. Using the speed of sound in air, $v_{\text{sound}} \approx 343 \text{ m/s}$ at room temperature (about 20–25°C; the true value varies slightly with temperature and humidity), and noting the pulse travels the distance *twice* (out and back):

$$d(\text{cm}) = \frac{t_{\text{echo}}(\mu\text{s}) \times 0.0343 \text{ cm}/\mu\text{s}}{2} = \frac{t_{\text{echo}}}{58.3} \quad (16.1)$$

This manual uses the common approximation $d = t_{\text{echo}}/58$. The sensor’s datasheet range is approximately 2–400 cm with a 15° effective beam angle; objects closer than 2 cm or highly

sound-absorbent (soft fabric) may not reflect reliably.

Component Working Principle

TRIG must receive a clean $10\mu\text{s}$ HIGH pulse to fire one ultrasonic burst; ECHO then independently rises and stays HIGH until the reflected pulse returns (or a timeout is reached internally by the sensor if nothing returns). `pulseIn(pin, HIGH, timeout)` blocks until it sees the pin go HIGH then LOW again, returning the HIGH duration in microseconds, or 0 if the optional timeout (in μs) expires first — always supply a timeout to avoid the program hanging indefinitely if no echo ever returns.

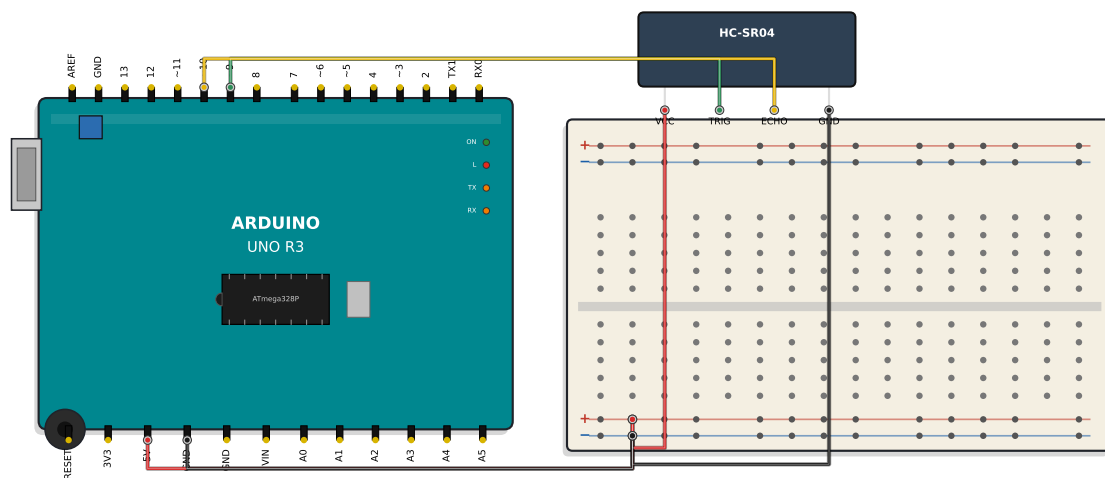


Figure 16.1: Experiment 16 circuit: HC-SR04 (VCC/TRIG/ECHO/GND) with TRIG on D9, ECHO on D10.

Connection Table

Module pin	Arduino pin	Notes
VCC	5V	
TRIG	D9	Digital output ($10\mu\text{s}$ trigger pulse)
ECHO	D10	Digital input (pulse width = time-of-flight)
GND	GND	

Working Principle – Signal Flow

D9 HIGH for $10\mu\text{s}$ (trigger) → sensor emits 40 kHz ultrasonic burst → ECHO goes HIGH → pulse reflects off nearest object → ECHO goes LOW upon return → `pulseIn()` returns the HIGH duration → converted to distance in cm.

Algorithm

1. Set TRIG LOW briefly, then HIGH for $10\mu\text{s}$, then LOW (a clean trigger pulse).
2. Measure ECHO's HIGH duration with `pulseIn()`, using a timeout (e.g. $30,000\mu\text{s} \approx 5\text{m}$).

round trip).

3. If the returned duration is 0, report "no echo / out of range".
4. Else, compute distance = duration / 58 and print it.
5. Wait briefly, then repeat.

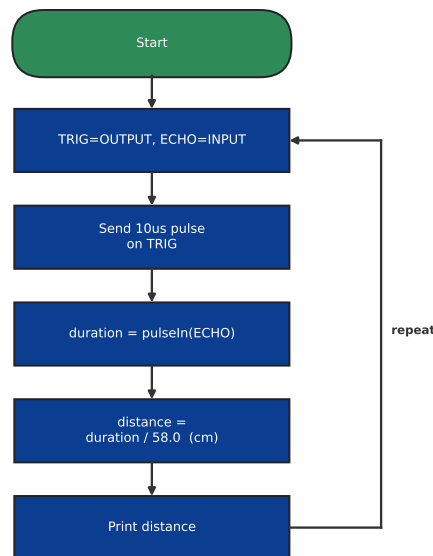


Figure 16.2: Flowchart: Experiment 16.

Arduino Program

```
1  const int TRIG_PIN = 9;
2  const int ECHO_PIN = 10;
3  const unsigned long TIMEOUT_US = 30000UL; // ~5 m round trip; avoids
      hanging on no echo
4
5  void setup() {
6      pinMode(TRIG_PIN, OUTPUT);
7      pinMode(ECHO_PIN, INPUT);
8      Serial.begin(9600);
9  }
10
11 float measureDistanceCm() {
12     digitalWrite(TRIG_PIN, LOW);
13     delayMicroseconds(2);
14     digitalWrite(TRIG_PIN, HIGH);
15     delayMicroseconds(10); // HC-SR04 datasheet requires >=10us
      trigger pulse
16     digitalWrite(TRIG_PIN, LOW);
17
18     unsigned long duration = pulseIn(ECHO_PIN, HIGH, TIMEOUT_US);
19     if (duration == 0) return -1; // timeout: no echo received
20     return duration / 58.0;
21 }
22
23 void loop() {
24     float distance = measureDistanceCm();
```

`measureDistanceCm()` is factored into its own function returning a sentinel `-1` on timeout, so `loop()` stays simple and this exact function can be reused unchanged in Experiments 17, 21, 22, 33, 39, and 42. `delayMicroseconds(2)` before the trigger ensures a clean LOW-to-HIGH edge; the datasheet-specified $10\,\mu\text{s}$ HIGH pulse reliably fires the sensor.

1. Wire the HC-SR04 per the connection table.
2. Upload and open the Serial Monitor at 9600 baud.
3. Place a flat object (e.g. a book) at several known distances and compare printed values against a ruler/tape measure.

Illustrative Serial output: **Distance:** 23.45 cm updating roughly 3 times per second; **Out of range / no echo** if nothing is within range or the surface does not reflect well.

[illegible]

Compute percentage error = $|d_{measured} - d_{true}|/d_{true} \times 100\%$ for each row. Discuss how the assumed $v_{sound} = 343 \text{ m/s}$ changes with room temperature and how that would shift all readings systematically.

The HC-SR04 measured distance with an average error of _____% across the tested range
_____ cm.

Troubleshooting

Problem	Likely cause	Fix
Always prints "no echo"	TRIG/ECHO pins swapped, or soft/angled surface	Re-check wiring; test against a flat hard surface
Readings are wildly inconsistent	Object closer than ≈ 2 cm, or multiple reflective surfaces nearby	Test in the 5–100 cm range first
Readings consistently off-set from true distance	Uncalibrated speed-of-sound assumption	Acceptable; discuss in Calculations
Serial Monitor freezes	Missing timeout in <code>pulseIn()</code>	Confirm the timeout parameter is present

Viva Questions and Extension Challenge

Conceptual: (1) Why must the round-trip time be divided by 2? (2) What happens if no timeout is given to `pulseIn()` and no object is present? (3) Why is 343 m/s only an approximation? (4) What is the sensor's minimum reliable range, and why does very close range fail? (5) Why is TRIG an output and ECHO an input? **Troubleshooting:** (1) Readings are exactly double what a ruler shows — what calculation mistake would cause this? (2) The sensor works pointed at a wall but not at a person's clothing — why? (3) Distance readings drift slightly warmer vs. cooler days — why, physically? **Programming:** (1) Average 5 consecutive readings before printing, to reduce noise. (2) Add units toggle: print in both cm and inches. **Extension Challenge:** Implement a temperature-compensated speed of sound, $v = 331.3 + 0.606 T$ (m/s, T in $^{\circ}\text{C}$), using a fixed assumed room temperature constant, and discuss how much this changes a 100 cm reading.

Chapter 17

Experiment 17: Ultrasonic Distance Alarm

Experiment Identification

Experiment Number:	17
Title:	Ultrasonic Distance Alarm
Learning Layer:	Layer 1 — Simple
Estimated Completion Time:	25 minutes
Difficulty:	Beginner–Intermediate
Prerequisite Experiments:	Experiments 16, 12

Aim

To sound the active buzzer whenever an object comes closer than a safe distance threshold, combining Experiment 16’s measurement with Experiment 12’s buzzer output, and completing Layer 1’s progression from single I/O to a first sensor-driven alarm.

Learning Objectives

1. Reuse a previously verified measurement function inside a new decision-driven program.
2. Implement a distance threshold alarm.
3. Reflect on Layer 1’s overall progression from static output to sensor-driven decisions.

Required Components

Arduino UNO R3, breadboard, 1 HC-SR04, 1 active buzzer, jumper wires.

Technical Theory

This experiment adds no new sensor theory beyond Experiment 16; the engineering content is the threshold decision $d < d_{safe} \Rightarrow \text{alarm}$, exactly mirroring Experiment 10’s LDR threshold pattern but with a physical-distance sensor. Layer 1 concludes here having covered: static/blinking/multi/sequenced digital output (1–5), debounced and array-based digital input (6–8), analog input with PWM output (9), analog thresholds (10–11), audio output (12–14), and two sensor modules (15–16) — every technique Layer 2 will now attach to an LCD display.

Component Working Principle

See Experiment 16 (HC-SR04) and Experiment 12 (active buzzer) for component-level detail; no new component behaviour is introduced here.

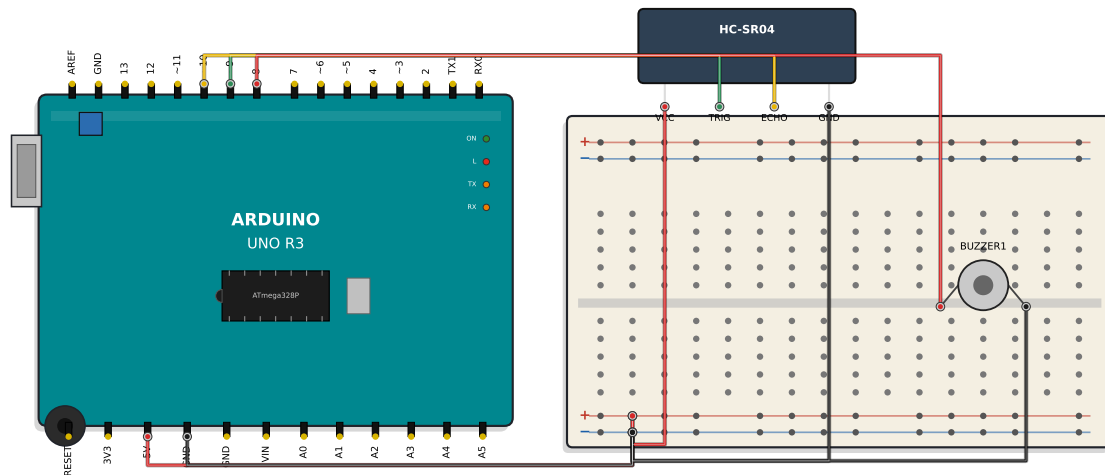


Figure 17.1: Experiment 17 circuit: HC-SR04 as in Experiment 16, plus an active buzzer on D8.

Connection Table

Module pin	Arduino pin	Notes
HC-SR04 VCC/TRIG/ECHO/GND	5V/D9/D10/GND	As Experiment 16
Buzzer +	D8	
Buzzer –	GND	

Working Principle – Signal Flow

Measured distance → compared against `SAFE_DISTANCE_CM` → buzzer (D8) driven HIGH if closer than the threshold (or out-of-range low readings), else LOW.

Algorithm

1. Measure distance using Experiment 16's function.
2. If distance is valid and less than the safe threshold, sound the buzzer.
3. Else, silence the buzzer.
4. Repeat continuously.

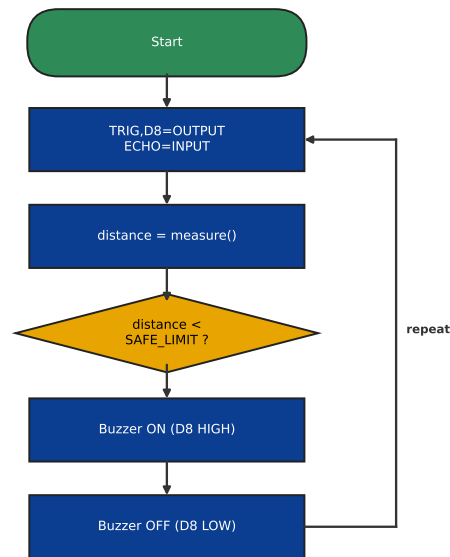


Figure 17.2: Flowchart: Experiment 17.

Arduino Program

```

1  const int TRIG_PIN = 9, ECHO_PIN = 10, BUZZER_PIN = 8;
2  const unsigned long TIMEOUT_US = 30000UL;
3  const float SAFE_DISTANCE_CM = 15.0;
4
5  void setup() {
6    pinMode(TRIG_PIN, OUTPUT);
7    pinMode(ECHO_PIN, INPUT);
8    pinMode(BUZZER_PIN, OUTPUT);
9    Serial.begin(9600);
10 }
11
12 float measureDistanceCm() {
13   digitalWrite(TRIG_PIN, LOW);
14   delayMicroseconds(2);
15   digitalWrite(TRIG_PIN, HIGH);
16   delayMicroseconds(10);
17   digitalWrite(TRIG_PIN, LOW);
18   unsigned long duration = pulseIn(ECHO_PIN, HIGH, TIMEOUT_US);
19   if (duration == 0) return -1;
20   return duration / 58.0;
21 }
22
23 void loop() {
24   float distance = measureDistanceCm();
25   bool tooClose = (distance > 0 && distance < SAFE_DISTANCE_CM);
26   digitalWrite(BUZZER_PIN, tooClose ? HIGH : LOW);
27
28   Serial.print("Distance: ");
29   if (distance < 0) Serial.println("out of range"); else { Serial.
       print(distance); Serial.println(" cm"); }
30   delay(200);
31 }

```

Code Explanation
<code>tooClose</code> requires <i>both</i> a valid (positive) reading <i>and</i> a value under the threshold, so an out-of-range <code>-1</code> sentinel from Experiment 16's timeout path is never misread as "extremely close" — a subtle but important correctness check, since $-1 < 15$ would otherwise be true and incorrectly sound the alarm.

Experimental Procedure
1. Wire the HC-SR04 and buzzer per the connection table. 2. Upload and slowly move an object towards the sensor from beyond 15 cm. 3. Confirm the buzzer activates exactly as the object crosses the 15 cm threshold.

Expected Output
Silent while the nearest object is farther than 15 cm; buzzer sounds continuously once an object comes within 15 cm, and stops the instant the object moves back beyond it.

[illegible]

Calculations and Analysis
Note the exact distance at which the buzzer switches ON while approaching versus OFF while retreating; if these two values are not identical, discuss why (measurement noise near the threshold) and preview how Experiment 44 solves this formally with hysteresis.

Result
The ultrasonic alarm correctly triggered at a measured threshold of _____ cm against a design target of 15 cm.

Troubleshooting		
Problem	Likely cause	Fix
Buzzer sounds with nothing nearby	Out-of-range -1 not excluded in the condition	Re-check the <code>tooClose</code> boolean logic
Buzzer never sounds	Threshold too small, or sensor miswired	Re-check Experiment 16 troubleshooting
Buzzer chatters right at the boundary	No hysteresis (expected at this stage)	See Experiment 44 for the formal fix

Viva Questions and Extension Challenge

Conceptual: (1) Why must `tooClose` check both validity and threshold? (2) What would happen if `-1` were not excluded? (3) How does this experiment's structure mirror Experiment 10's LDR threshold logic? (4) What Layer 1 techniques are combined here? (5) Why is 15 cm a reasonable demonstration threshold rather than a safety-rated one? **Troubleshooting:** (1) The alarm triggers randomly with no object present — what sentinel-handling bug might explain this? (2) The alarm's threshold distance drifts between test runs — what environmental factor from Experiment 16 could explain this? **Programming:** (1) Make the buzzer beep intermittently (using `millis()`) rather than sound continuously, faster as the object gets closer (this is developed fully in Experiment 39). (2) Add a second, closer "danger" threshold that lights an LED in addition to the buzzer. **Extension Challenge:** Reflect on and list, in your lab notebook, every Arduino programming technique used across Experiments 1–17, as preparation for Layer 2's LCD-based experiments.

Part II

LAYER 2 — AVERAGE (Experiments 18–34)

Layer 2 — Average: Overview

Experiments 18–34 introduce the 16×2 I²C LCD as a permanent display for every subsequent experiment, then attach it to each Layer-1 sensor in turn to show calibrated, human-readable readings instead of raw Serial Monitor numbers. This layer also introduces the DHT11 temperature/humidity sensor, the soil-moisture sensor, threshold-based alarms with a first look at hysteresis, the relay module for safe low-voltage actuation demonstrations, and the IR receiver. By Experiment 34 a learner can read, display, and react to every sensor in the kit individually.

Chapter 18

Experiment 18: LCD I²C Hello World

Experiment Identification

Experiment Number:	18
Title:	LCD I ² C Hello World
Learning Layer:	Layer 2 — Average
Estimated Completion Time:	25 minutes
Difficulty:	Beginner–Intermediate
Prerequisite Experiments:	Experiment 1

Aim

To display text on the 16×2 I²C LCD, introducing the I²C bus and the `LiquidCrystal_I2C` library that every subsequent Layer 2/3 experiment builds on.

Learning Objectives

1. Wire an I²C device using only 4 wires (GND, VCC, SDA, SCL).
2. Install and use the `LiquidCrystal_I2C` library.
3. Position text using `lcd.setCursor(col, row)`.
4. Diagnose and fix a wrong I²C address using the scanner from Chapter .

Required Components

Arduino UNO R3, breadboard, 16×2 I²C LCD module, 4 female-to-male (or male-to-male, depending on header type) jumper wires.

Technical Theory

I²C (Inter-Integrated Circuit) is a 2-wire serial bus: **SDA** (data) and **SCL** (clock), plus power and ground — 4 wires total, regardless of how many I²C devices are attached (they share the same two bus lines, distinguished by a unique 7-bit address). On the Arduino UNO, SDA is fixed to **A4** and SCL to **A5** (Chapter). The LCD's I²C backpack (a PCF8574 I/O expander chip) translates 2-wire I²C commands into the LCD glass's native 6-wire parallel interface, which is why only 4 wires are needed here versus the many more wires a non-I²C LCD would require. Most backpacks default to address `0x27`; a minority use `0x3F`.

Component Working Principle

The LCD glass itself is an HD44780-compatible 16-column, 2-row character display; each character cell is a 5×8 dot-matrix. The library abstracts away the low-level HD44780 command set (cursor moves, character writes, display control) behind simple calls like `lcd.print()`.

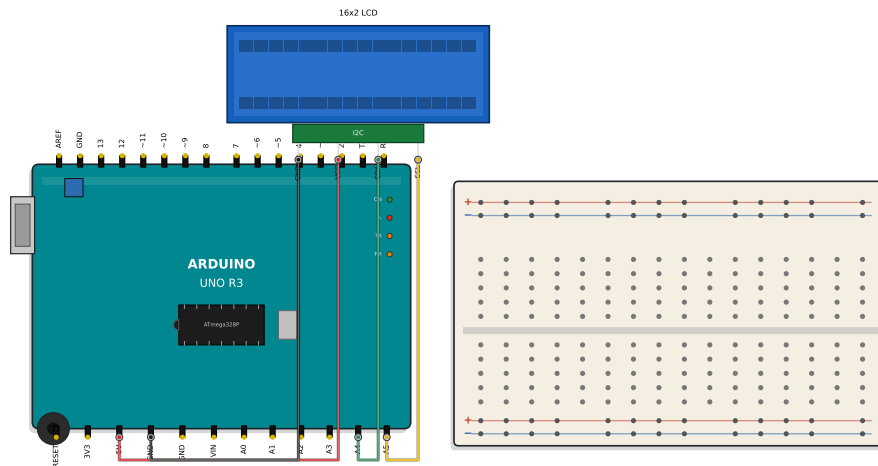


Figure 18.1: Experiment 18 circuit: I²C LCD backpack wired to GND, 5V, A4 (SDA), A5 (SCL).

Connection Table

LCD backpack pin	Arduino pin	Notes
GND	GND	
VCC	5V	
SDA	A4	I ² C data
SCL	A5	I ² C clock

Working Principle – Signal Flow

Arduino (I²C master) sends addressed commands/data over SDA/SCL → PCF8574 backpack (I²C slave at 0x27) → translates to HD44780 parallel signals → LCD glass updates the requested character cells.

Algorithm

1. Create an `LiquidCrystal_I2C` object with the address, columns, and rows.
2. In `setup()`, call `lcd.init()` and `lcd.backlight()`.
3. Set the cursor to (0,0) and print a greeting.
4. Leave `loop()` empty — static text needs no repetition.

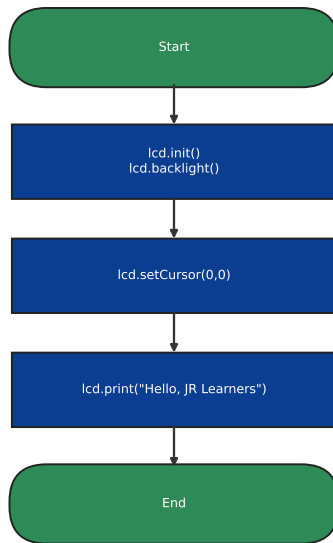


Figure 18.2: Flowchart: Experiment 18.

Arduino Program

```

1  #include <Wire.h>
2  #include <LiquidCrystal_I2C.h>
3
4  LiquidCrystal_I2C lcd(0x27, 16, 2);  // address, columns, rows
5
6  void setup() {
7    lcd.init();
8    lcd.backlight();
9    lcd.setCursor(0, 0);
10   lcd.print("Hello, JR");
11   lcd.setCursor(0, 1);
12   lcd.print("Learners!");
13 }
14
15 void loop() {
16   // static text: nothing to repeat
17 }
  
```

Code Explanation

`LiquidCrystal_I2C lcd(0x27, 16, 2)` constructs the driver object bound to address 0x27 for a 16-column, 2-row display. `lcd.init()` initialises the HD44780 controller over I²C; `lcd.backlight()` turns on the LED backlight (without it, text is invisible in low light). `lcd.setCursor(col, row)` is zero-indexed: row 0 is the top line, row 1 the bottom.

Experimental Procedure

1. Wire the 4 LCD pins per the connection table.
2. Install `LiquidCrystal_I2C` via Library Manager (Chapter).
3. Upload the sketch; if the display stays blank, run the I²C scanner (Chapter) and update the address.

Expected Output

The backlight turns on and the two lines "Hello, JR" / "Learners!" appear, left-aligned, on the physical LCD.

Observation Table

I ² C address found by scanner	Text displayed correctly? (Y/N)

Calculations and Analysis

Not applicable numerically; instead record the scanner's found address and confirm it matches the address used in the sketch's constructor.

Result

The LCD correctly displayed two lines of static text at I²C address _____.

Troubleshooting

Problem	Likely cause	Fix
Blank display, backlight on	Wrong I ² C address	Run scanner, update address
Backlight itself is off	VCC/GND swapped or loose	Re-check wiring
Garbled/solid black boxes	Contrast or init timing issue	Re-check <code>lcd.init()</code> is called before printing
Compile error: library not found	Library not installed	Revisit Chapter

Viva Questions and Extension Challenge

Conceptual: (1) Why does I²C need only 2 signal wires regardless of device count? (2) What are A4/A5 fixed to on the UNO? (3) What does the PCF8574 chip do? (4) Why is `setCursor` zero-indexed? (5) What are the two most common LCD I²C addresses? **Troubleshooting:** (1) The scanner finds a device at 0x3F but the sketch uses 0x27 — what should you change? (2) Two lines print but overlap into one — what call is likely missing? (3) The backlight flickers — what wiring issue is likely? **Programming:** (1) Print a scrolling message using `lcd.scrollDisplayLeft()`. (2) Add `lcd.clear()` before printing and explain why it matters when text length varies between updates. **Extension Challenge:** Display a simple custom character (e.g. a heart or arrow) using `lcd.createChar()`.

Chapter 19

Experiment 19: Potentiometer Value Display

Experiment Identification

Experiment Number:	19
Title:	Potentiometer Value Display
Learning Layer:	Layer 2 — Average
Estimated Completion Time:	20 minutes
Difficulty:	Beginner–Intermediate
Prerequisite Experiments:	Experiments 9, 18

Aim

To display the potentiometer's live ADC reading on the LCD, replacing the Serial Monitor with an on-device readout for the first time.

Learning Objectives

1. Update LCD text continuously inside `loop()`.
2. Avoid visible flicker/garbage by managing fixed-width fields.
3. Combine an analog input experiment with the new LCD output.

Required Components

Arduino UNO R3, breadboard, 16×2 I²C LCD, 1 potentiometer, jumper wires.

Technical Theory

Repeatedly calling `lcd.clear()` every loop pass causes a visible flicker, since it briefly blanks the whole screen before rewriting. This experiment instead overwrites a fixed cursor position with a fixed-width numeric field (padding with a trailing space) so shorter numbers do not leave stale digits from a previous, longer reading.

Component Working Principle

See Experiment 9 (potentiometer voltage divider) and Experiment 18 (I²C LCD) for component detail.

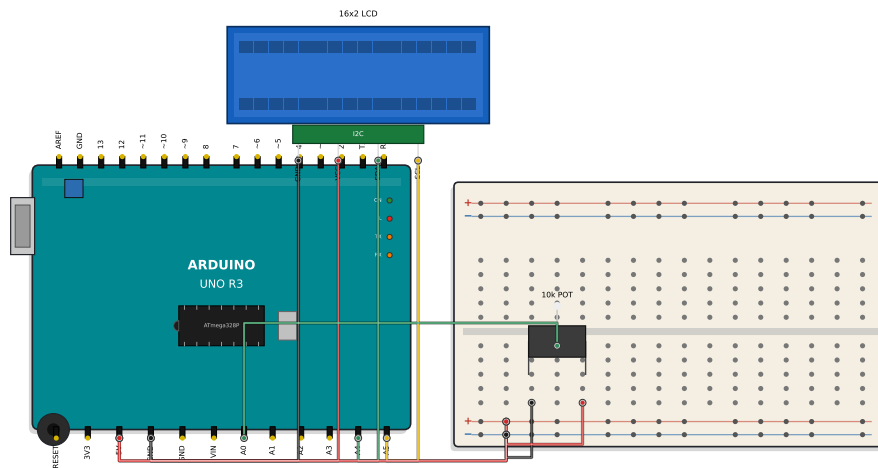


Figure 19.1: Experiment 19 circuit: potentiometer at A0, LCD via I²C.

Connection Table

Component	Arduino pin
Potentiometer wiper	A0
Potentiometer ends	5V, GND
LCD (GND/VCC/SDA/SCL)	GND/5V/A4/A5

Working Principle – Signal Flow

A0 reading → formatted as text with trailing spaces → written to a fixed LCD cursor position every update interval.

Algorithm

1. Initialise the LCD and print a static label ("Pot: ") once.
2. Read A0.
3. Move the cursor to just after the label and print the value, padded with a trailing space.
4. Repeat.

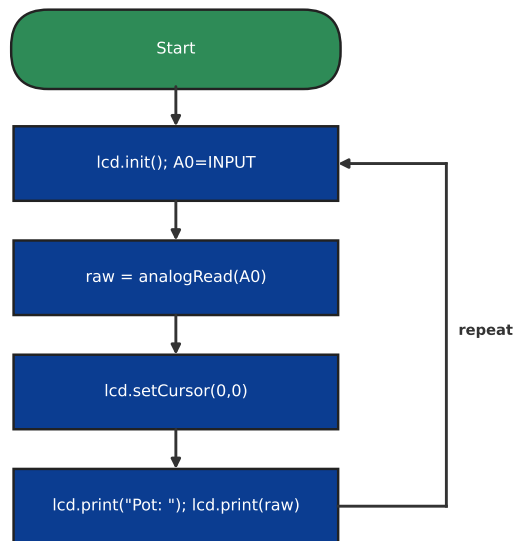


Figure 19.2: Flowchart: Experiment 19.

Arduino Program

```

1  #include <Wire.h>
2  #include <LiquidCrystal_I2C.h>
3  LiquidCrystal_I2C lcd(0x27, 16, 2);
4  const int POT_PIN = A0;
5
6  void setup() {
7      lcd.init();
8      lcd.backlight();
9      lcd.setCursor(0, 0);
10     lcd.print("Pot: ");
11 }
12
13 void loop() {
14     int raw = analogRead(POT_PIN);
15     lcd.setCursor(5, 0);
16     lcd.print(raw);
17     lcd.print("    "); // trailing spaces erase leftover digits from a
18                          // longer previous value
19     delay(200);
20 }

```

Code Explanation

The label "Pot: " is printed once in `setup()`, not every loop pass, since it never changes — only the numeric field at column 5 is rewritten. The trailing spaces guard against, e.g., "1023" being followed later by "8 " correctly erasing the stale "23".

Experimental Procedure

1. Wire the potentiometer and LCD.
2. Upload and turn the knob, observing the number update live on the LCD.

LCD row 0 reads "Pot: " followed by a live-updating number between 0 and 1023 as the knob turns.

Knob position	LCD reading
0	0.00
1	0.01
2	0.02
3	0.03
4	0.04
5	0.05
6	0.06
7	0.07
8	0.08
9	0.09
10	0.10
11	0.11
12	0.12
13	0.13
14	0.14
15	0.15
16	0.16
17	0.17
18	0.18
19	0.19
20	0.20
21	0.21
22	0.22
23	0.23
24	0.24
25	0.25
26	0.26
27	0.27
28	0.28
29	0.29
30	0.30
31	0.31
32	0.32
33	0.33
34	0.34
35	0.35
36	0.36
37	0.37
38	0.38
39	0.39
40	0.40
41	0.41
42	0.42
43	0.43
44	0.44
45	0.45
46	0.46
47	0.47
48	0.48
49	0.49
50	0.50
51	0.51
52	0.52
53	0.53
54	0.54
55	0.55
56	0.56
57	0.57
58	0.58
59	0.59
60	0.60
61	0.61
62	0.62
63	0.63
64	0.64
65	0.65
66	0.66
67	0.67
68	0.68
69	0.69
70	0.70
71	0.71
72	0.72
73	0.73
74	0.74
75	0.75
76	0.76
77	0.77
78	0.78
79	0.79
80	0.80
81	0.81
82	0.82
83	0.83
84	0.84
85	0.85
86	0.86
87	0.87
88	0.88
89	0.89
90	0.90
91	0.91
92	0.92
93	0.93
94	0.94
95	0.95
96	0.96
97	0.97
98	0.98
99	0.99

Compare the LCD-displayed value against a Serial-printed value (temporarily add `Serial.println(row)`) to confirm both agree exactly.

The potentiometer's live ADC value was displayed correctly on the LCD with no flicker or stale digits: _____ (Yes/No).

Problem	Likely cause	Fix
Leftover digits after a smaller number	Missing trailing spaces	Add padding spaces
Whole screen flickers	<code>lcd.clear()</code> used every loop	Remove it; use fixed <code>setCursor</code> instead
Value never changes	A0 miswired	Re-check Experiment 9 wiring

Conceptual: (1) Why avoid `lcd.clear()` every loop pass? (2) Why print the label only once? (3) What happens if the trailing spaces are removed? (4) Why does column 5 correctly follow "Pot: "? (5) What is the ADC range being displayed? **Troubleshooting:** (1) A "9" briefly appears where a "0" should not be after the pot returns to a small value — why? (2) Text position drifts after adding a longer label — what column number needs updating? **Programming:** (1) Display the value as a voltage (0.00–5.00 V) instead of a raw code. (2) Display both raw and voltage on the two LCD rows. **Extension Challenge:** Add a simple bar-graph using LCD block characters that grows with the pot value, alongside the numeric readout.

Chapter 20

Experiment 20: LDR Light Meter

Experiment Identification

Experiment Number:	20
Title:	LDR Light Meter
Learning Layer:	Layer 2 — Average
Estimated Completion Time:	20 minutes
Difficulty:	Beginner–Intermediate
Prerequisite Experiments:	Experiments 10, 18

Aim

To display the LDR's light level on the LCD as a percentage, and explicitly discuss the difference between a raw ADC code and a calibrated physical measurement (lux).

Learning Objectives

1. Rescale a raw ADC reading to a 0–100% "light level" using `map()`.
2. Explain why this percentage is *not* a calibrated lux value.
3. Identify what true lux calibration would additionally require.

Required Components

Arduino UNO R3, breadboard, 16×2 I²C LCD, 1 LDR, 1×10 kΩ resistor, jumper wires.

Technical Theory

This manual deliberately labels the LDR's displayed number a "light level (%)", never "lux", because converting an uncalibrated CdS photoresistor's divider voltage into true illuminance (lux) requires per-device calibration against a reference light meter — LDR-to-lux relationships are non-linear and vary between individual LDR units by a large factor. `map(raw, 1023, 0, 0, 100)` simply rescales the ADC range so "high reading = high light", producing a relative, repeatable-for-this-device indicator, not an absolute physical unit. Appendix H discusses what a genuine calibration procedure would involve.

Component Working Principle

See Experiment 10 for LDR divider behaviour.

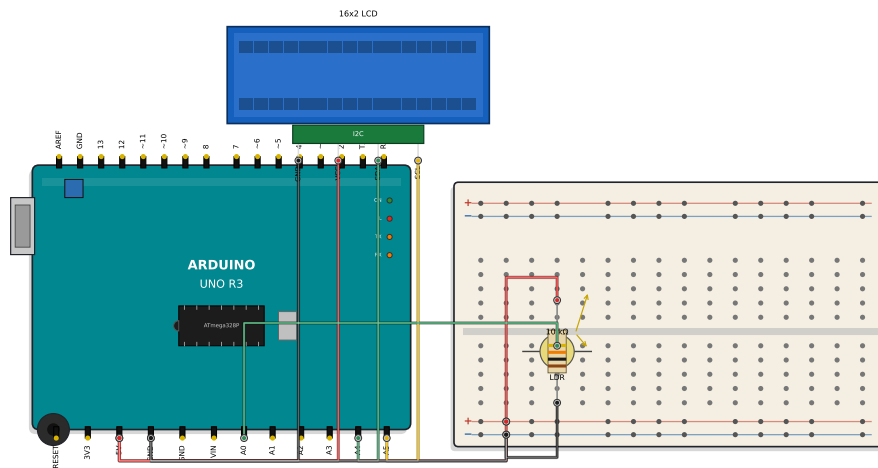


Figure 20.1: Experiment 20 circuit: LDR divider at A0, LCD via I²C.

Connection Table

Same LDR divider as Experiment 10 (A0), LCD as Experiment 18 (GND/5V/A4/A5).

Working Principle – Signal Flow

A0 raw reading $\rightarrow$ `map()` to 0–100% $\rightarrow$ displayed on LCD as "Light: NN%".

Algorithm

1. Read A0.
2. Map to a 0–100 percentage (inverted, since higher light = lower LDR resistance = higher A0 voltage in this divider orientation, which already increases with light — confirm your own divider's direction experimentally as in Experiment 10).
3. Display on LCD.

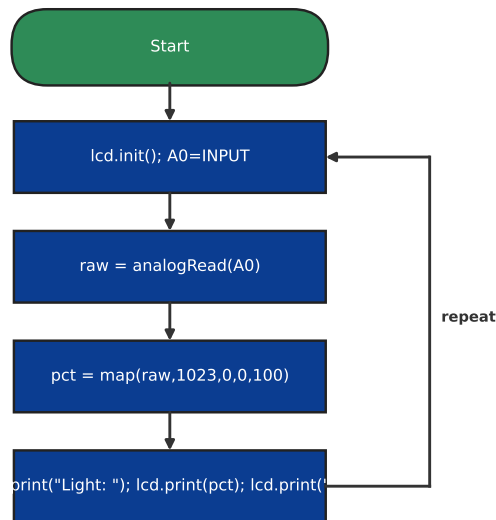


Figure 20.2: Flowchart: Experiment 20.

Arduino Program

```

1  #include <Wire.h>
2  #include <LiquidCrystal_I2C.h>
3  LiquidCrystal_I2C lcd(0x27, 16, 2);
4  const int LDR_PIN = A0;
5
6  void setup() {
7    lcd.init(); lcd.backlight();
8    lcd.setCursor(0, 0);
9    lcd.print("Light: ");
10 }
11
12 void loop() {
13   int raw = analogRead(LDR_PIN);
14   int pct = map(raw, 0, 1023, 0, 100);  // NOTE: direction depends on
15                                         divider wiring; verify vs. Exp.10
16   lcd.setCursor(7, 0);
17   lcd.print(pct);
18   lcd.print("%   ");
19   delay(200);
20 }

```

Code Explanation

The comment flags a common beginner bug: whether higher `raw` means brighter or darker depends on whether the LDR sits on the 5V side or GND side of the divider (Experiment 10). Always verify direction experimentally by covering the LDR and checking the LCD moves the expected way, rather than assuming.

Experimental Procedure

1. Build the LDR divider and wire the LCD.
2. Upload and cover/uncover the LDR, confirming the percentage moves in the expected

direction.

3. If inverted, swap the `map()` arguments' output order.

Expected Output

LCD reads "Light: NN%" updating smoothly as ambient light changes, higher % in brighter conditions.

Observation Table

Condition	Raw ADC	Displayed %

Calculations and Analysis

Confirm $pct = \text{round}(\text{raw} \times 100/1023)$ matches your displayed value, and discuss why this is a linear rescale of an inherently non-linear physical sensor.

Result

The LDR light level was displayed correctly as a 0–100% relative indicator, moving in the expected direction with actual brightness changes: _____ (Yes/No).

Troubleshooting

Problem	Likely cause	Fix
Percentage moves opposite to expectation	Divider direction / <code>map()</code> argument order	Swap <code>map()</code> output range
Always reads 0% or 100%	Divider miswired (see Experiment 10)	Re-check wiring

Viva Questions and Extension Challenge

Conceptual: (1) Why is "%" used instead of "lux"? (2) What would true lux calibration require? (3) Why can two different LDR units give different raw readings under identical light? (4) What does `map()`'s argument order control? (5) Is this measurement precise or merely repeatable for one device? **Troubleshooting:** (1) Two students' circuits show different % for the same lighting — is this a fault? (2) The reading is stuck at 0% in a bright room — what should you check first? **Programming:** (1) Add a rolling average of the last 5 readings for stability. (2) Display both raw ADC and % simultaneously on two rows. **Extension Challenge:** Research one commercial lux-meter IC (e.g. a BH1750-class device, without needing to own one) and write a short comparison of why it gives calibrated lux while this LDR circuit cannot.

Chapter 21

Experiment 21: Ultrasonic Distance Display

Experiment Identification

Experiment Number:	21
Title:	Ultrasonic Distance Display
Learning Layer:	Layer 2 — Average
Estimated Completion Time:	20 minutes
Difficulty:	Beginner–Intermediate
Prerequisite Experiments:	Experiments 16, 18

Aim

To display live HC-SR04 distance readings on the LCD instead of the Serial Monitor, reusing Experiment 16's measurement function unchanged.

Learning Objectives

1. Reuse a previously written, tested function inside a new experiment.
2. Format a floating-point number for fixed-width LCD display.
3. Handle the invalid-reading sentinel value on an LCD (rather than Serial).

Required Components

Arduino UNO R3, breadboard, 16×2 I²C LCD, 1 HC-SR04, jumper wires.

Technical Theory

No new sensor theory is introduced; see Experiment 16 for the time-of-flight distance equation. The engineering point here is *software reuse*: `measureDistanceCm()` is copied unchanged, demonstrating that a well-designed function is decoupled from how its result is displayed (Serial vs. LCD).

Component Working Principle

See Experiment 16 (HC-SR04) and Experiment 18 (I²C LCD).

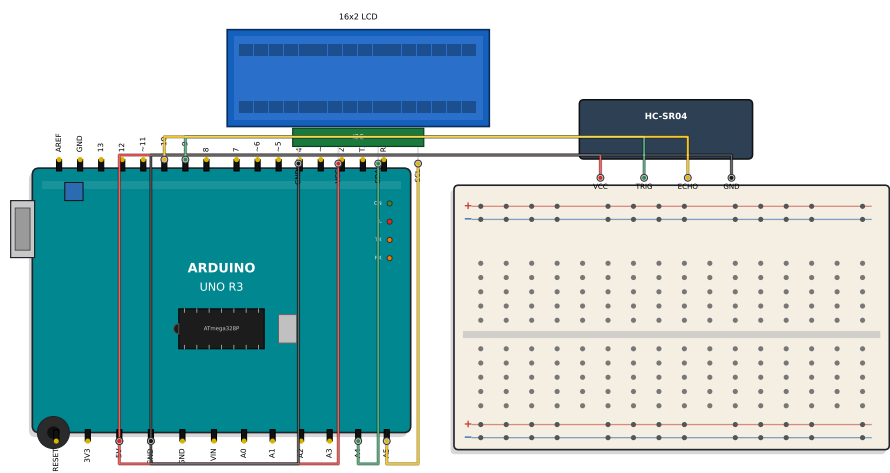


Figure 21.1: Experiment 21 circuit: HC-SR04 (TRIG=D9, ECHO=D10) with LCD via I²C.

Connection Table

HC-SR04 VCC/TRIG/ECHO/GND to 5V/D9/D10/GND (as Experiment 16); LCD to GND/5V/A4/A5 (as Experiment 18).

Working Principle – Signal Flow

`measureDistanceCm()` → valid or `-1` → formatted and written to a fixed LCD field, replacing whatever was previously displayed there.

Algorithm

1. Measure distance using Experiment 16's function.
2. If valid, print with one decimal place and units; else print "Out of range".
3. Repeat with a short delay.

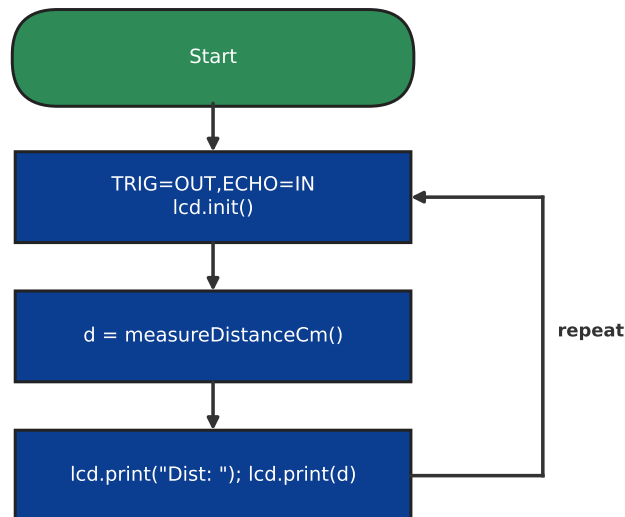


Figure 21.2: Flowchart: Experiment 21.

Arduino Program

```

1  #include <Wire.h>
2  #include <LiquidCrystal_I2C.h>
3  LiquidCrystal_I2C lcd(0x27, 16, 2);
4  const int TRIG_PIN = 9, ECHO_PIN = 10;
5  const unsigned long TIMEOUT_US = 30000UL;
6
7  float measureDistanceCm() {
8      digitalWrite(TRIG_PIN, LOW); delayMicroseconds(2);
9      digitalWrite(TRIG_PIN, HIGH); delayMicroseconds(10);
10     digitalWrite(TRIG_PIN, LOW);
11     unsigned long duration = pulseIn(ECHO_PIN, HIGH, TIMEOUT_US);
12     return (duration == 0) ? -1 : duration / 58.0;
13 }
14
15 void setup() {
16     pinMode(TRIG_PIN, OUTPUT);
17     pinMode(ECHO_PIN, INPUT);
18     lcd.init(); lcd.backlight();
19     lcd.setCursor(0, 0); lcd.print("Dist:");
20 }
21
22 void loop() {
23     float d = measureDistanceCm();
24     lcd.setCursor(6, 0);
25     if (d < 0) {
26         lcd.print("out of rng ");
27     } else {
28         lcd.print(d, 1);
29         lcd.print(" cm ");
30     }
31     delay(300);
32 }
  
```

Code Explanation

`lcd.print(d, 1)` prints a float with exactly 1 decimal place — the second argument the library-specific overload for fixed-precision float display. Trailing spaces again guard against stale characters, as in Experiment 19.

Experimental Procedure

1. Wire the HC-SR04 and LCD.
2. Upload and move an object at varying distances, confirming the LCD tracks it.

Expected Output

LCD shows "Dist: NN.N cm" updating roughly 3 times per second, or "Dist: out of rng" when nothing is in range.

Observation Table

True distance (cm)	LCD reading	Error (cm)

Calculations and Analysis

Compare LCD readings against Experiment 16's Serial-based measurements for the same physical setup and confirm they agree (since the underlying function is identical).

Result

Distance readings were correctly displayed on the LCD, matching Experiment 16's Serial output to within measurement noise: _____ (Yes/No).

Troubleshooting

Problem	Likely cause	Fix
LCD always shows "out of rng"	HC-SR04 miswired	Re-check wiring Experiment 16
Stale digits after shorter numbers	Missing trailing spaces	Add padding
LCD blank	I ² C address wrong	Run scanner

Viva Questions and Extension Challenge

Conceptual: (1) Why was the measurement function not rewritten for this experiment? (2) What does `lcd.print(d,1)`'s second argument do? (3) What still causes "out of rng" here? (4) Why does the delay differ from Experiment 16's? (5) What decoupling principle does this experiment demonstrate? **Troubleshooting:** (1) The LCD shows a distance that never changes — what should you check about the sensor first (not the LCD)? (2) Digits look correct but a stray character appears at the end — why? **Programming:** (1) Display both cm and inches on the two LCD rows. (2) Add a bar-graph-style visual proportional to distance using LCD block

characters. **Extension Challenge:** Reduce the update delay progressively and observe the point at which the display becomes hard to read due to flicker/inconsistency, and explain why.

Chapter 22

Experiment 22: Digital Ruler

Experiment Identification

Experiment Number:	22
Title:	Digital Ruler
Learning Layer:	Layer 2 — Average
Estimated Completion Time:	20 minutes
Difficulty:	Beginner–Intermediate
Prerequisite Experiments:	Experiment 21

Aim

To turn the HC-SR04 into a usable "digital ruler" by explicitly validating readings against the sensor's datasheet range and discussing measurement resolution and accuracy.

Learning Objectives

1. Explain measurement *resolution* versus *accuracy*.
2. Reject readings outside the sensor's valid 2–400 cm datasheet range explicitly (not only relying on the timeout sentinel).
3. Quantify the theoretical timing resolution of the `pulseIn()`-based measurement.

Required Components

Arduino UNO R3, breadboard, 16×2 I²C LCD, 1 HC-SR04, jumper wires (identical hardware to Experiment 21).

Technical Theory

Resolution is the smallest change in the measured quantity the instrument can distinguish; **accuracy** is how close a reading is to the true value. The Arduino's `pulseIn()` has microsecond resolution, corresponding to a distance resolution of $0.0343/2 \approx 0.017$ cm per microsecond of timing — far finer than the sensor's actual accuracy, which is limited by beam spread, surface angle, and temperature-dependent speed of sound (typically ± 0.3 cm or worse in practice). This experiment additionally clamps readings to the datasheet-valid 2–400 cm window, rejecting physically implausible results the timeout check alone would not catch (e.g. noise producing a very short non-zero pulse).

Component Working Principle

See Experiment 16 for HC-SR04 operating principle.

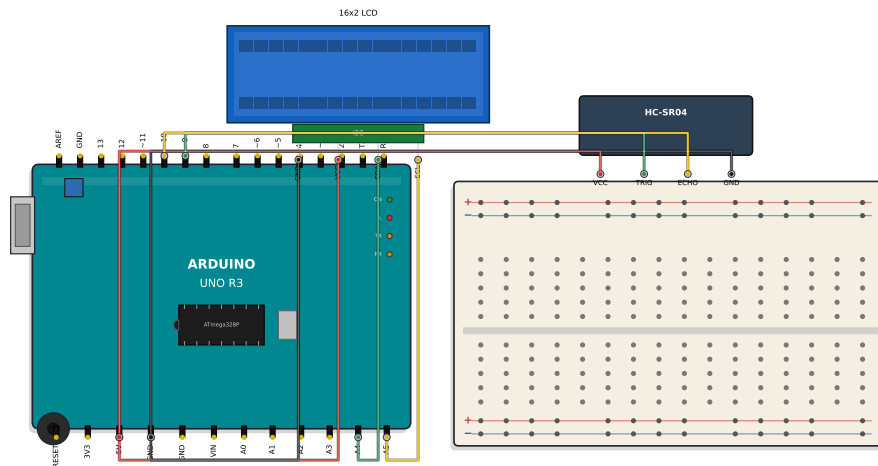


Figure 22.1: Experiment 22 circuit (identical to Experiment 21).

Connection Table

Identical to Experiment 21.

Working Principle – Signal Flow

Raw pulse duration → converted to cm → range-checked against $[2, 400]$ cm → accepted value displayed, or "Out of range" if outside the valid window.

Algorithm

1. Measure distance.
2. If $2 \leq d \leq 400$, accept and display.
3. Else, display "Out of range".
4. Repeat.

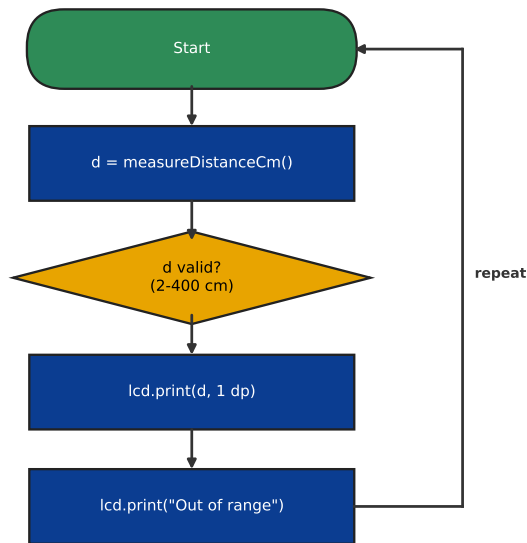


Figure 22.2: Flowchart: Experiment 22.

Arduino Program

```

1  #include <Wire.h>
2  #include <LiquidCrystal_I2C.h>
3  LiquidCrystal_I2C lcd(0x27, 16, 2);
4  const int TRIG_PIN = 9, ECHO_PIN = 10;
5  const unsigned long TIMEOUT_US = 30000UL;
6  const float MIN_CM = 2.0, MAX_CM = 400.0;    // HC-SR04 datasheet-valid
       range
7
8  float measureDistanceCm() {
9      digitalWrite(TRIG_PIN, LOW); delayMicroseconds(2);
10     digitalWrite(TRIG_PIN, HIGH); delayMicroseconds(10);
11     digitalWrite(TRIG_PIN, LOW);
12     unsigned long duration = pulseIn(ECHO_PIN, HIGH, TIMEOUT_US);
13     return (duration == 0) ? -1 : duration / 58.0;
14 }
15
16 void setup() {
17     pinMode(TRIG_PIN, OUTPUT);
18     pinMode(ECHO_PIN, INPUT);
19     lcd.init(); lcd.backlight();
20 }
21
22 void loop() {
23     float d = measureDistanceCm();
24     bool valid = (d >= MIN_CM && d <= MAX_CM);
25     lcd.setCursor(0, 0);
26     if (valid) {
27         lcd.print("Len: "); lcd.print(d, 1); lcd.print(" cm ");
28     } else {
29         lcd.print("Out of range ");
30     }
31     delay(300);
32 }

```

Code Explanation

The explicit `valid` check guards against both the `-1` timeout sentinel *and* any implausible value the timeout alone would miss, making the acceptance window a single, clearly-named boolean rather than scattered numeric comparisons.

Experimental Procedure

1. Wire as Experiment 21.
2. Upload and test at the sensor's practical extremes (very close, e.g. 1 cm, and far, e.g. 350 cm if space allows).
3. Confirm out-of-window readings are correctly rejected.

Expected Output

"Len: NN.N cm" for valid readings within 2–400 cm; "Out of range" otherwise, including at very close range where echoes are unreliable.

Observation Table

True distance (cm)	Displayed reading	Within valid window?

Calculations and Analysis

Compute the theoretical timing resolution ($0.017 \text{ cm}/\mu\text{s}$) and compare it against your observed repeat-reading spread (take 5 readings at a fixed distance and note their range) to show that practical accuracy is coarser than timing resolution.

Result

The digital ruler correctly accepted readings within 2–400 cm and rejected out-of-window results, with an observed repeat-reading spread of _____ cm at a fixed distance.

Troubleshooting

Problem	Likely cause	Fix
Valid-looking distances rejected	MIN_CM/MAX_CM too tight	Re-check datasheet range constants
Clearly wrong readings accepted	Range check omitted or wrong operator	Re-check <code>valid</code> boolean

Viva Questions and Extension Challenge

Conceptual: (1) What is the difference between resolution and accuracy? (2) Why is the sensor's real-world accuracy coarser than its timing resolution? (3) Why check both the timeout sentinel and the datasheet range? (4) What physically limits the minimum measurable distance? (5) What environmental factor changes accuracy without any code change? **Troubleshooting:** (1) A reading of "1.2 cm" occasionally briefly appears and is accepted — what constant should

be revisited? (2) Repeat readings at a fixed distance vary by nearly 1 cm — is this a sensor fault? **Programming:** (1) Report the spread (max–min) of the last 5 readings as a simple "noise" indicator on the second LCD row. (2) Reject a new reading that differs from the previous accepted one by more than 50 cm in one step (a simple outlier filter). **Extension Challenge:** Implement a median-of-5 filter (sort 5 consecutive readings, keep the middle one) and compare its stability against the raw readings.

Chapter 23

Experiment 23: Temperature Display

Experiment Identification

Experiment Number:	23
Title:	Temperature Display
Learning Layer:	Layer 2 — Average
Estimated Completion Time:	25 minutes
Difficulty:	Intermediate
Prerequisite Experiments:	Experiment 18

Aim

To read and display ambient temperature from the DHT11 sensor, introducing single-wire digital sensor protocols and sensor read-failure handling.

Learning Objectives

1. Install and use the Adafruit `DHT sensor library`.
2. Explain why the DHT11 needs a minimum interval between reads.
3. Detect and handle a failed sensor read using `isnan()`.

Required Components

Arduino UNO R3, breadboard, 16×2 I²C LCD, 1 DHT11 module, jumper wires.

Technical Theory

The DHT11 communicates over a single bidirectional data wire using a proprietary timing-based protocol (not I²C/SPI): the Arduino pulls the line low to start a request, then the sensor responds with 40 bits of data encoded as pulse widths. This bit-banged protocol is timing-sensitive and relatively slow; the DHT11 datasheet specifies a minimum ≈ 1 second between reads, and reading faster than this returns stale or invalid data. The DHT11's stated accuracy is $\pm 2^\circ\text{C}$ over 0–50°C — notably coarser than the DS18B20 or DHT22 sensors sometimes seen in other kits.

Component Working Principle

Internally, the DHT11 combines a resistive humidity sensor and an NTC thermistor for temperature, with a small on-board chip that digitises both and outputs them over the single data line on request — from the Arduino's perspective, this is entirely hidden behind the library's `readTemperature()/readHumidity()` calls.

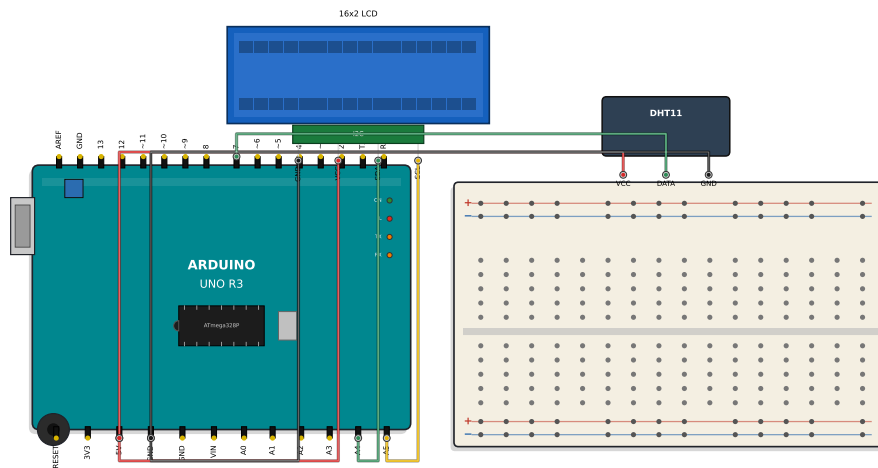


Figure 23.1: Experiment 23 circuit: DHT11 (VCC/DATA/GND) with DATA on D7; LCD via I²C.

Connection Table

DHT11 pin	Arduino pin	Notes
VCC	5V	
DATA	D7	Single-wire digital
GND	GND	

LCD wired as Experiment 18 (GND/5V/A4/A5).

Working Principle – Signal Flow

Arduino requests a reading on D7 → DHT11 responds with a 40-bit timed pulse train → library decodes temperature → checked for validity → displayed on LCD, or an error message shown if the read failed.

Algorithm

1. Initialise the DHT11 and LCD.
2. Read temperature.
3. If the reading is NaN (failed), display an error and wait before retrying.
4. Else, display the temperature and wait at least 1 s before the next read.

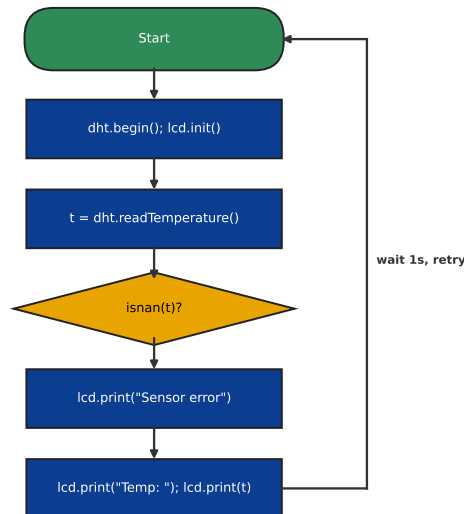


Figure 23.2: Flowchart: Experiment 23.

Arduino Program

```

1  #include <Wire.h>
2  #include <LiquidCrystal_I2C.h>
3  #include <DHT.h>
4
5  #define DHT_PIN 7
6  #define DHT_TYPE DHT11
7
8  LiquidCrystal_I2C lcd(0x27, 16, 2);
9  DHT dht(DHT_PIN, DHT_TYPE);
10
11 void setup() {
12     lcd.init(); lcd.backlight();
13     dht.begin();
14 }
15
16 void loop() {
17     float t = dht.readTemperature();    // degrees Celsius
18     lcd.setCursor(0, 0);
19     if (isnan(t)) {
20         lcd.print("Sensor error    ");
21     } else {
22         lcd.print("Temp: "); lcd.print(t, 1); lcd.print(" C  ");
23     }
24     delay(1000);    // DHT11 minimum interval between reads
25 }
  
```

Code Explanation

`isnan(t)` checks whether the library signalled a failed read (returned as "Not a Number"), which happens occasionally due to the DHT11's timing-sensitive protocol picking up noise. The 1000ms delay is not arbitrary — it directly reflects the sensor's minimum re-read interval from its datasheet.

Experimental Procedure

1. Wire the DHT11 and LCD.
2. Install `DHT sensor library` and its `Adafruit Unified Sensor` dependency.
3. Upload and observe the temperature reading; breathe gently near the sensor to see it respond (slowly).

Expected Output

LCD shows "Temp: NN.N C", updating roughly once per second, occasionally showing "Sensor error" transiently (expected on some reads).

Observation Table

[illegible]

Calculations and Analysis

Compare against a reference thermometer if available; confirm the difference is within the datasheet's $\pm 2^{\circ}\text{C}$ accuracy band.

Result

The DHT11 correctly reported temperature within $\pm$ _____°C of a reference measurement.

Troubleshooting

Problem	Likely cause	Fix
Persistent "Sensor error"	Wiring, wrong <code>DHT_TYPE</code> , or reading faster than 1 s	Re-check wiring and delay
Reading stuck at one value	Delay too long, appears frozen	Confirm actual environment change, not a bug
Library compile errors	Missing dependency	Install Adafruit Unified Sensor too

Viva Questions and Extension Challenge

Conceptual: (1) Why can't the DHT11 be read every loop iteration without delay? (2) What does `isnan()` check? (3) What is the DHT11's stated accuracy? (4) Why is this not I²C or SPI? (5) What physically senses temperature inside the DHT11? **Troubleshooting:** (1) Roughly 1 in 10 reads shows an error — is this necessarily a wiring fault? (2) The reading is implausibly high (e.g. 125°C) — what does this typically indicate? (3) Nothing at all displays — what should you check before suspecting the DHT11? **Programming:** (1) Only update the LCD when the new reading differs from the last valid one by more than 0.1°C. (2) Count and display the number of failed reads since power-up. **Extension Challenge:** Implement retry logic: on a failed read, retry immediately up to 3 times before displaying an error, and measure how often this resolves

a transient failure.

Chapter 24

Experiment 24: Humidity Display

Experiment Identification

Experiment Number:	24
Title:	Humidity Display
Learning Layer:	Layer 2 — Average
Estimated Completion Time:	20 minutes
Difficulty:	Intermediate
Prerequisite Experiments:	Experiment 23

Aim

To read and display relative humidity from the same DHT11 sensor, showing that one physical sensor module can provide two independent readings.

Learning Objectives

1. Use `dht.readHumidity()` alongside temperature.
2. Explain relative humidity as a physical quantity.
3. Recognise that both readings share the same underlying protocol transaction timing constraints.

Required Components

Identical hardware to Experiment 23: Arduino UNO R3, breadboard, LCD, 1 DHT11 module.

Technical Theory

Relative humidity (RH, %) is the ratio of the actual water vapour present in air to the maximum the air could hold at that temperature before condensing, expressed as a percentage. The DHT11's humidity element is a capacitive/resistive polymer sensor whose electrical property changes with absorbed moisture; its datasheet accuracy is $\pm 5\%$ RH over a 20–90%RH range. Note that `readTemperature()` and `readHumidity()` each trigger the sensor's full read cycle if called independently in quick succession — Experiment 26 shows how to read both values from one transaction efficiently.

Component Working Principle

See Experiment 23 for the DHT11's single-wire protocol; the same physical sensing element pair (temperature + humidity) sits on the module, addressed via the same DATA pin.

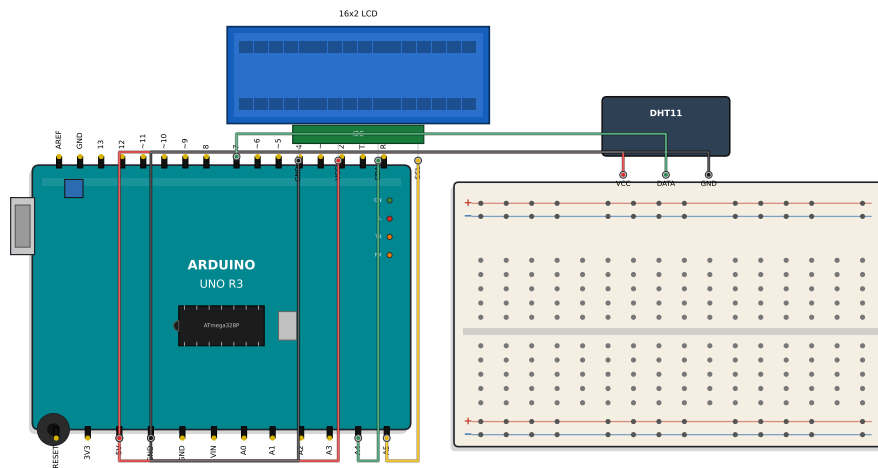


Figure 24.1: Experiment 24 circuit (identical wiring to Experiment 23).

Connection Table

Identical to Experiment 23.

Working Principle – Signal Flow

DATA line request → DHT11 responds with both temperature and humidity in one 40-bit frame → library exposes `readHumidity()` separately → validity-checked → displayed.

Algorithm

1. Initialise DHT11 and LCD.
2. Read humidity.
3. If invalid, show an error; else display the percentage.
4. Wait at least 1 s and repeat.

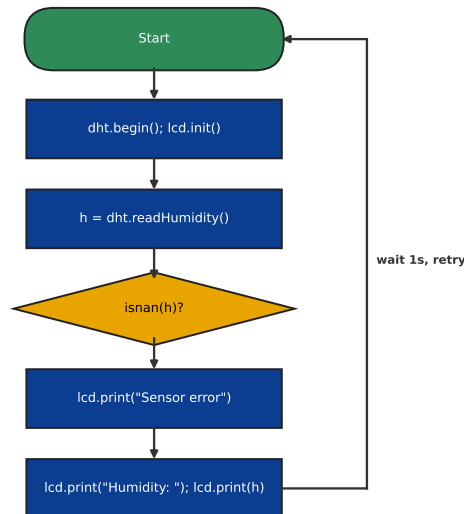


Figure 24.2: Flowchart: Experiment 24.

Arduino Program

```

1  #include <Wire.h>
2  #include <LiquidCrystal_I2C.h>
3  #include <DHT.h>
4
5  #define DHT_PIN 7
6  #define DHT_TYPE DHT11
7  LiquidCrystal_I2C lcd(0x27, 16, 2);
8  DHT dht(DHT_PIN, DHT_TYPE);
9
10 void setup() {
11   lcd.init(); lcd.backlight();
12   dht.begin();
13 }
14
15 void loop() {
16   float h = dht.readHumidity();
17   lcd.setCursor(0, 0);
18   if (isnan(h)) {
19     lcd.print("Sensor error");
20   } else {
21     lcd.print("Humidity: "); lcd.print(h, 0); lcd.print("%");
22   }
23   delay(1000);
24 }

```

Code Explanation

`dht.readHumidity()` mirrors `readTemperature()`'s structure exactly, reinforcing that the DHT library exposes a consistent, predictable API across both quantities. `lcd.print(h, 0)` rounds to whole-percent display, matching the DHT11's coarse $\pm 5\%$ RH accuracy (extra decimal places would be false precision).

Experimental Procedure

1. Reuse Experiment 23's wiring.
2. Upload and observe humidity; briefly breathe near the sensor to see the reading rise, then recover.

Expected Output

LCD shows "Humidity: NN%", updating roughly once per second, rising noticeably if breathed on directly.

Observation Table

Condition	Humidity reading (%)
Normal room air	
Breathed on directly	

Calculations and Analysis

If a reference hygrometer is available, compare readings and confirm agreement within $\pm 5\%$ RH.

Result

The DHT11 correctly reported humidity, rising in response to direct breath by _____ percentage points.

Troubleshooting

Problem	Likely cause	Fix
Reading never changes even when breathed on	Sensor placed too far from breath source	Test closer (a few cm)
Reading stuck at 0% or 100%	Sensor fault or miswiring	Re-check Experiment 23 wiring

Viva Questions and Extension Challenge

Conceptual: (1) What physically changes inside the sensor with humidity? (2) What is the DHT11's humidity accuracy? (3) Why does breathing on the sensor raise the reading? (4) Is RH an absolute or relative measure? (5) Why might reading temperature and humidity separately be inefficient? **Troubleshooting:** (1) Humidity reads exactly 0% every time — what does this typically mean (compare to a NaN case)? (2) The reading recovers very slowly after breathing on it — is this expected? **Programming:** (1) Add a "comfort" classification (e.g. "Dry"/"Comfortable"/"Humid") based on RH ranges. (2) Log the minimum and maximum humidity seen since power-up. **Extension Challenge:** Combine with Experiment 9's potentiometer to let a user set a custom "high humidity" threshold live, without reprogramming.

Chapter 25

Experiment 25: Temperature Alarm

Experiment Identification

Experiment Number:	25
Title:	Temperature Alarm
Learning Layer:	Layer 2 — Average
Estimated Completion Time:	25 minutes
Difficulty:	Intermediate
Prerequisite Experiments:	Experiments 23, 12

Aim

To sound a buzzer when temperature exceeds a set threshold, introducing a first, informal look at hysteresis to avoid alarm chatter near the boundary.

Learning Objectives

1. Implement a temperature threshold alarm.
2. Observe chatter at a hard threshold and understand why it occurs.
3. Apply a simple margin (informal hysteresis) to reduce chatter, ahead of Experiment 44's formal treatment.

Required Components

Arduino UNO R3, breadboard, 16×2 I²C LCD, 1 DHT11, 1 active buzzer, jumper wires.

Technical Theory

A single hard threshold (alarm ON above T , OFF at or below T) can chatter rapidly if the reading hovers within measurement noise of T (here, DHT11 noise/quantisation is small but non-zero). This experiment introduces an **informal margin**: turn ON above T_{high} , but only turn back OFF once the reading drops below a lower $T_{low} = T_{high} - \text{margin}$, so a single noisy reading near the boundary cannot immediately flip the alarm back off. A rigorous, generalised hysteresis design (with a dedicated function and explicit state) is developed in Experiment 44.

Component Working Principle

See Experiment 23 (DHT11) and Experiment 12 (active buzzer).

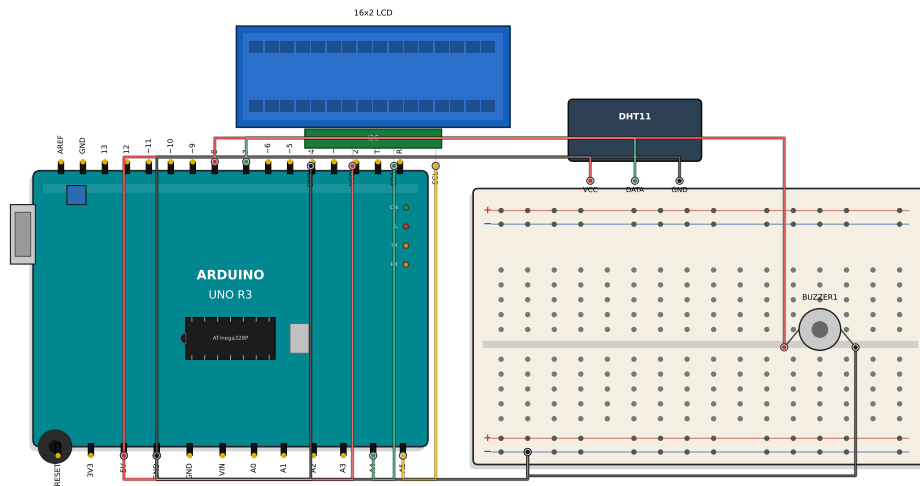


Figure 25.1: Experiment 25 circuit: DHT11 on D7, LCD via I²C, active buzzer on D8.

Connection Table

DHT11 and LCD as Experiments 23/18; buzzer + on D8, buzzer – on GND.

Working Principle – Signal Flow

Temperature reading $\rightarrow$ if above T_{high} and alarm currently OFF, turn ON; if below T_{low} and alarm currently ON, turn OFF; otherwise keep the alarm in its current state (the margin band).

Algorithm

1. Read temperature.
2. If alarm is OFF and reading $> T_{high}$, turn alarm ON.
3. Else if alarm is ON and reading $< T_{low}$, turn alarm OFF.
4. (Otherwise, leave the alarm state unchanged.)
5. Display temperature and alarm state; repeat.

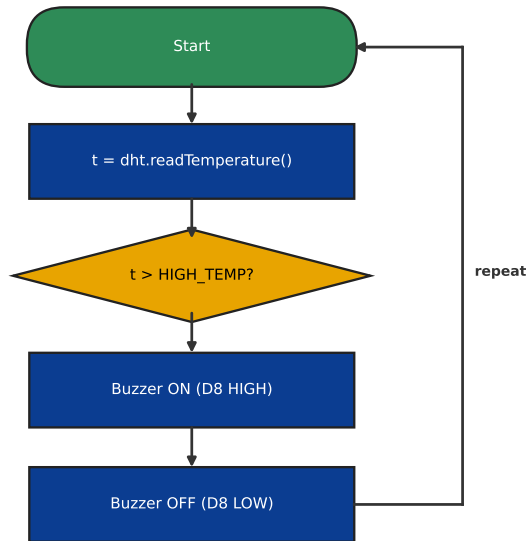


Figure 25.2: Flowchart: Experiment 25.

Arduino Program

```

1  #include <Wire.h>
2  #include <LiquidCrystal_I2C.h>
3  #include <DHT.h>
4  #define DHT_PIN 7
5  #define DHT_TYPE DHT11
6  const int BUZZER_PIN = 8;
7  const float T_HIGH = 30.0, T_LOW = 28.0;    // 2C margin band
8
9  LiquidCrystal_I2C lcd(0x27, 16, 2);
10 DHT dht(DHT_PIN, DHT_TYPE);
11 bool alarmOn = false;
12
13 void setup() {
14   pinMode(BUZZER_PIN, OUTPUT);
15   lcd.init(); lcd.backlight();
16   dht.begin();
17 }
18
19 void loop() {
20   float t = dht.readTemperature();
21   if (!isnan(t)) {
22     if (!alarmOn && t > T_HIGH) alarmOn = true;
23     else if (alarmOn && t < T_LOW) alarmOn = false;
24     digitalWrite(BUZZER_PIN, alarmOn ? HIGH : LOW);
25
26     lcd.setCursor(0, 0);
27     lcd.print("T:"); lcd.print(t, 1);
28     lcd.print(alarmOn ? " ALARM " : " OK    ");
29   }
30   delay(1000);
31 }

```

Code Explanation

`alarmOn` is a persistent boolean, so the two `if/else if` conditions only ever change state at the correct boundary each is responsible for — this is exactly the margin-band logic described in the theory section, and is a simpler precursor to Experiment 44's reusable hysteresis function.

Experimental Procedure

1. Wire the DHT11, LCD, and buzzer.
2. Upload; warm the DHT11 gently (cupped hand) to cross T_{high} and confirm the alarm latches on, then cool it to cross T_{low} and confirm it turns off.

Expected Output

LCD shows "T:NN.N OK" normally, switching to "T:NN.N ALARM" once temperature exceeds 30°C, with the buzzer sounding; it does not turn off again until the temperature drops below 28°C.

Observation Table

Temperature (°C)	Alarm state	Expected per design?

Calculations and Analysis

Compare chatter behaviour with $T_{low} = T_{high}$ (no margin) versus the 2°C margin used here, by temporarily setting both constants equal and observing the difference near the boundary.

Result

The temperature alarm activated at _____°C and deactivated at _____°C, matching the 2°C design margin.

Troubleshooting

Problem	Likely cause	Fix
Alarm chatters rapidly	Margin set to 0 or reversed	Confirm $T_{low} < T_{high}$
Alarm never triggers	Room never reaches T_{high}	Warm the sensor gently to test
Alarm never clears	T_{low} unreachable in test conditions	Cool the sensor or lower T_{low} temporarily for testing

Viva Questions and Extension Challenge

Conceptual: (1) What causes chatter at a single hard threshold? (2) How does the margin band prevent it? (3) Why use two named constants instead of one threshold plus/minus a value inline? (4) What happens if $T_{low} > T_{high}$ by mistake? (5) Why is this called "informal" hysteresis? **Troubleshooting:** (1) The alarm turns on but the LCD still says "OK" — what

line's ordering might be wrong? (2) The alarm engages exactly once and never disengages even after cooling well below T_{low} — what boolean logic bug fits this? **Programming:** (1) Widen the margin to 5°C and observe the more sluggish but chatter-free response. (2) Add a Serial log timestamping every alarm state transition. **Extension Challenge:** Add a second, higher "critical" threshold that also flashes an LED rapidly in addition to the steady buzzer, layering two alarm severities.

Chapter 26

Experiment 26: Temperature and Humidity Monitor

Experiment Identification

Experiment Number:	26
Title:	Temperature and Humidity Monitor
Learning Layer:	Layer 2 — Average
Estimated Completion Time:	25 minutes
Difficulty:	Intermediate
Prerequisite Experiments:	Experiments 23, 24

Aim

To display temperature and humidity together on the two LCD rows, and introduce a non-blocking `millis()` scheduler so sensor reads never violate the DHT11's minimum interval, even as more features are added later.

Learning Objectives

1. Read both DHT11 values from a single sensor transaction per update.
2. Replace a blocking `delay(1000)` with a `millis()`-based schedule.
3. Design an update interval as an explicit, named engineering parameter.

Required Components

Arduino UNO R3, breadboard, 16×2 I²C LCD, 1 DHT11, jumper wires.

Technical Theory

Every DHT11 experiment so far used `delay(1000)` to respect the sensor's minimum re-read interval, which is acceptable while the sketch does nothing else. From this point in the manual onward, experiments increasingly need to do other things concurrently (read buttons, update multiple displays, run state machines) — so this experiment replaces the blocking delay with the `millis()` pattern from Chapter : record the last read time, and only re-read once `interval` milliseconds have elapsed, letting `loop()` run freely in between.

Component Working Principle

See Experiment 23 for DHT11 internals.

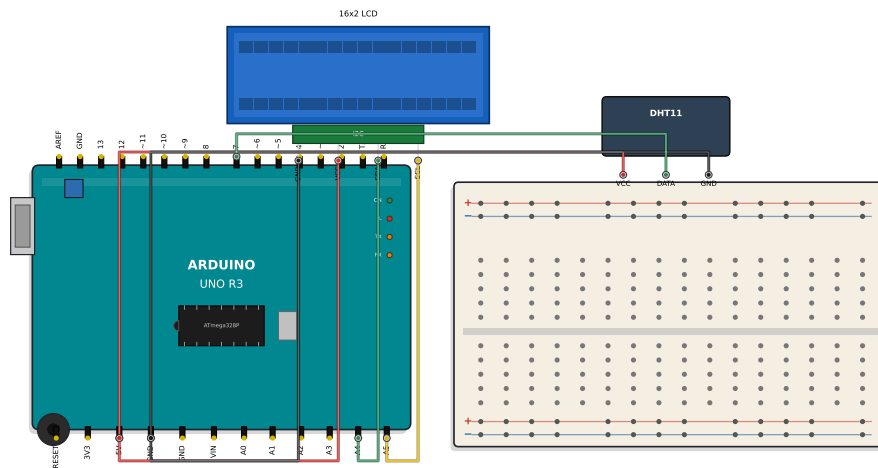


Figure 26.1: Experiment 26 circuit (identical wiring to Experiment 23).

Connection Table

Identical to Experiment 23.

Working Principle – Signal Flow

Every `UPDATE_INTERVAL_MS` (2000 ms here), both temperature and humidity are read once and both LCD rows refreshed together; between updates, `loop()` is free to do other work (none yet, but this scaffolding is reused from Experiment 36 onward).

Algorithm

1. Record `lastUpdate = 0`.
2. Each pass, check if `millis() - lastUpdate >= interval`.
3. If so, update `lastUpdate`, read temperature and humidity, and refresh both LCD rows.
4. Otherwise, do nothing this pass (loop continues immediately, not blocked).

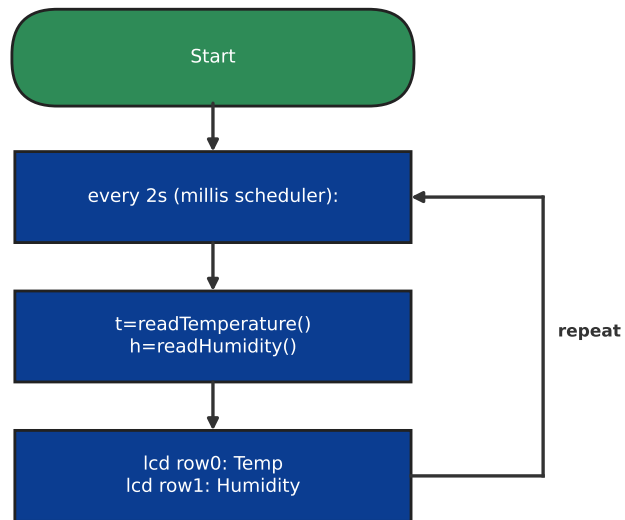


Figure 26.2: Flowchart: Experiment 26.

Arduino Program

```

1  #include <Wire.h>
2  #include <LiquidCrystal_I2C.h>
3  #include <DHT.h>
4  #define DHT_PIN 7
5  #define DHT_TYPE DHT11
6  const unsigned long UPDATE_INTERVAL_MS = 2000;
7
8  LiquidCrystal_I2C lcd(0x27, 16, 2);
9  DHT dht(DHT_PIN, DHT_TYPE);
10 unsigned long lastUpdate = 0;
11
12 void setup() {
13   lcd.init(); lcd.backlight();
14   dht.begin();
15 }
16
17 void loop() {
18   if (millis() - lastUpdate >= UPDATE_INTERVAL_MS) {
19     lastUpdate = millis();
20     float t = dht.readTemperature();
21     float h = dht.readHumidity();
22
23     lcd.setCursor(0, 0);
24     lcd.print(isnan(t) ? "Temp: err " : ("Temp: " + String(t, 1) + "
      C "));
25     lcd.setCursor(0, 1);
26     lcd.print(isnan(h) ? "Humidity: err " : ("Humidity: " + String(h,
      0) + "% "));
27   }
28   // loop() is free to do other work here between updates.
29 }

```

block rather than two separate timers? **Troubleshooting:** (1) The display updates twice in quick succession then pauses — what off-by-logic issue could cause this? (2) After 49 days of continuous uptime, updates seem to glitch once — what should you check (informationally, not as a fixable lab task)? **Programming:** (1) Add a heartbeat LED that blinks independently of the DHT11 update, proving `loop()` truly isn't blocked. (2) Make the update interval itself adjustable via the potentiometer. **Extension Challenge:** Add a rolling 10-sample average of temperature, updated every cycle, displayed alongside the instantaneous reading.

Chapter 27

Experiment 27: Soil Moisture Monitor

Experiment Identification

Experiment Number:	27
Title:	Soil Moisture Monitor
Learning Layer:	Layer 2 — Average
Estimated Completion Time:	20 minutes
Difficulty:	Intermediate
Prerequisite Experiments:	Experiment 18

Aim

To read the resistive soil moisture sensor's analog output and display moisture level as a calibrated-feeling percentage on the LCD.

Learning Objectives

1. Wire and read a resistive two-probe soil moisture sensor.
2. Perform a simple two-point (wet/dry) calibration.
3. Explain why soil type affects absolute readings.

Required Components

Arduino UNO R3, breadboard, 16×2 I²C LCD, 1 soil moisture sensor (probe + comparator breakout), jumper wires.

Technical Theory

The soil moisture sensor's two exposed probes act as a variable resistor through the soil/water between them: moist soil conducts better (lower resistance) than dry soil, and the breakout's analog output (AO) reflects this as a voltage. Because absolute resistance depends on soil type, mineral content, and probe spacing/corrosion over time, this manual calibrates the sensor with two reference points — a "DRY" raw reading (probes in air) and a "WET" raw reading (probes in a cup of water) — rather than assuming fixed constants, and maps between them for a percentage.

Component Working Principle

The sensor pair is typically two corrosion-prone bare metal probes; leaving them powered continuously in wet soil accelerates electrochemical corrosion, so many practical deployments only power the probes briefly before each reading (not required for this classroom-scale experiment, but worth noting).

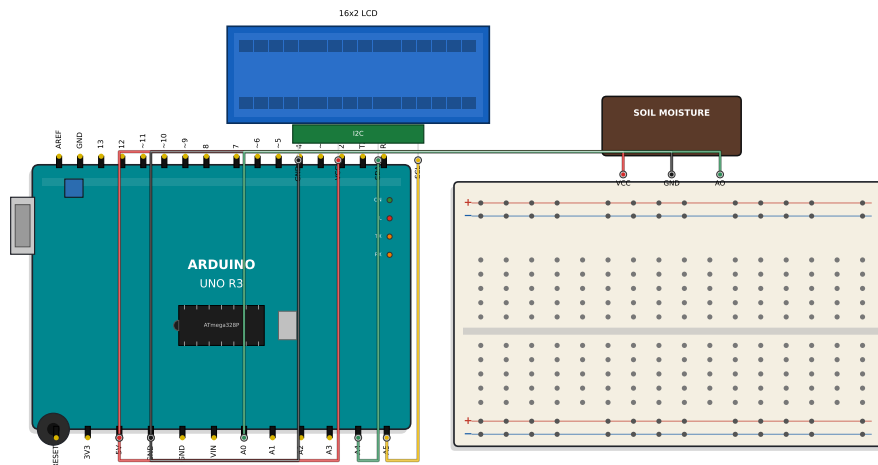


Figure 27.1: Experiment 27 circuit: soil moisture sensor (VCC/GND/AO) with AO on A0; LCD via I²C.

Connection Table

Sensor pin	Arduino pin	Notes
VCC	5V	
GND	GND	
AO	A0	Analog output

LCD as Experiment 18.

Working Principle – Signal Flow

Soil moisture → probe resistance (inverse relationship) → AO voltage → `analogRead()` → mapped between calibrated DRY/WET reference points → displayed as a percentage.

Algorithm

1. Calibrate: record raw readings for "probes in air" (DRY) and "probes in water" (WET).
2. Read A0.
3. Map the raw reading between DRY and WET to 0–100%, constrained to that range.
4. Display and repeat.

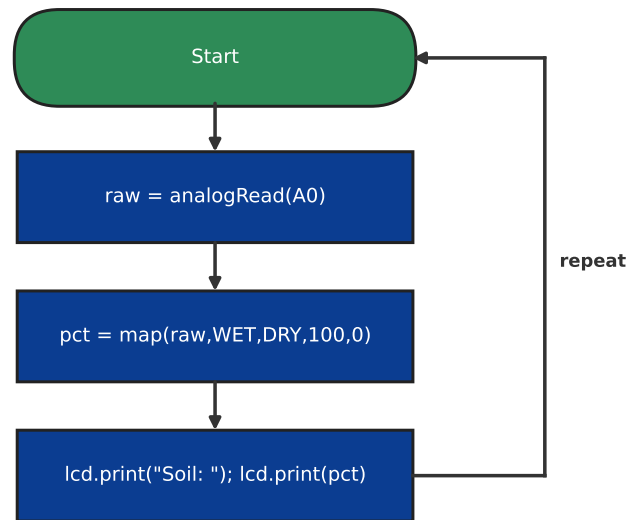


Figure 27.2: Flowchart: Experiment 27.

Arduino Program

```

1  #include <Wire.h>
2  #include <LiquidCrystal_I2C.h>
3  LiquidCrystal_I2C lcd(0x27, 16, 2);
4  const int SOIL_PIN = A0;
5  const int DRY_RAW = 800;    // measured with probes in air; recalibrate
6                               for your unit
7  const int WET_RAW = 350;    // measured with probes in a cup of water
8
9  void setup() {
10     lcd.init(); lcd.backlight();
11 }
12
13 void loop() {
14     int raw = analogRead(SOIL_PIN);
15     int pct = map(raw, DRY_RAW, WET_RAW, 0, 100);
16     pct = constrain(pct, 0, 100);
17
18     lcd.setCursor(0, 0);
19     lcd.print("Soil: "); lcd.print(pct); lcd.print("%  ");
20     delay(500);
21 }
  
```

Code Explanation

`constrain(pct, 0, 100)` clamps the mapped value, since a raw reading slightly beyond either calibration point (e.g. soil wetter than the water-cup reference) would otherwise `map()` to an out-of-range percentage. `DRY_RAW/WET_RAW` are explicitly commented as needing recalibration per physical unit, matching the theory section.

Experimental Procedure

1. Wire the sensor and LCD.
2. With Serial debugging temporarily added, record the raw value with probes in open air, then in a cup of water.
3. Update DRY_RAW/WET_RAW accordingly and re-upload.
4. Test with the probes in actual soil at varying moisture.

Expected Output

LCD shows "Soil: NN%", near 0% in dry air, near 100% in water, and an intermediate value in moist soil.

Observation Table

Condition	Raw ADC	Displayed %
Probes in air		
Probes in water		
Probes in soil sample		

Calculations and Analysis

Confirm your chosen DRY_RAW/WET_RAW bracket the full range of raw readings you actually observed; discuss what would happen if they were reversed (swapped).

Result

The soil moisture sensor was calibrated with DRY_RAW=_____, WET_RAW=_____, and correctly reported soil samples within the expected 0–100% range.

Troubleshooting

Problem	Likely cause	Fix
Always reads 0% or 100%	DRY_RAW/WET_RAW not recalibrated for your unit	Redo calibration procedure
Percentage inverted	DRY/WET raw values swapped in <code>map()</code>	Swap the two constants
Corroded-looking probes give unstable readings	Long-term probe corrosion	Expected with age; note in observations

Viva Questions and Extension Challenge

Conceptual: (1) Why does resistance fall as soil gets wetter? (2) Why calibrate per-unit instead of trusting a datasheet constant? (3) What does `constrain()` protect against here? (4) Why might probe corrosion matter for long-term deployments? (5) What are DRY_RAW and WET_RAW measuring physically? **Troubleshooting:** (1) The percentage never reaches 100% even fully submerged — what calibration constant needs adjusting? (2) Readings drift slowly over

many minutes with the probes untouched — what physical effect might explain this? **Programming:** (1) Print raw and % together for easier recalibration. (2) Store DRY_RAW/WET_RAW in EEPROM so recalibration survives a power cycle (conceptual discussion; EEPROM code optional). **Extension Challenge:** Add a 5-sample moving average to reduce reading noise from probe surface bubbles/contact variation.

Chapter 28

Experiment 28: Plant Moisture Alarm

Experiment Identification

Experiment Number:	28
Title:	Plant Moisture Alarm
Learning Layer:	Layer 2 — Average
Estimated Completion Time:	20 minutes
Difficulty:	Intermediate
Prerequisite Experiments:	Experiments 27, 12

Aim

To sound the buzzer when soil moisture falls below a dry threshold, reusing the calibrated percentage from Experiment 27 in a new threshold-alarm pattern.

Learning Objectives

1. Reuse a calibrated sensor reading in a threshold decision.
2. Recognise this as the same structural pattern as Experiments 10, 17, and 25.
3. Choose a sensible dry-alarm threshold for a plant-care scenario.

Required Components

Arduino UNO R3, breadboard, 16×2 I²C LCD, 1 soil moisture sensor, 1 active buzzer, jumper wires.

Technical Theory

This experiment introduces no new sensor theory; it is the fourth appearance of the same "calibrated reading versus threshold $\Rightarrow$ alarm" pattern (LDR/Experiment 10, ultrasonic/Experiment 17, temperature/Experiment 25). Recognising and reusing this recurring pattern — rather than treating each alarm as a new problem — is itself the intended learning outcome at this stage of the manual.

Component Working Principle

See Experiment 27 (soil moisture sensor) and Experiment 12 (active buzzer).

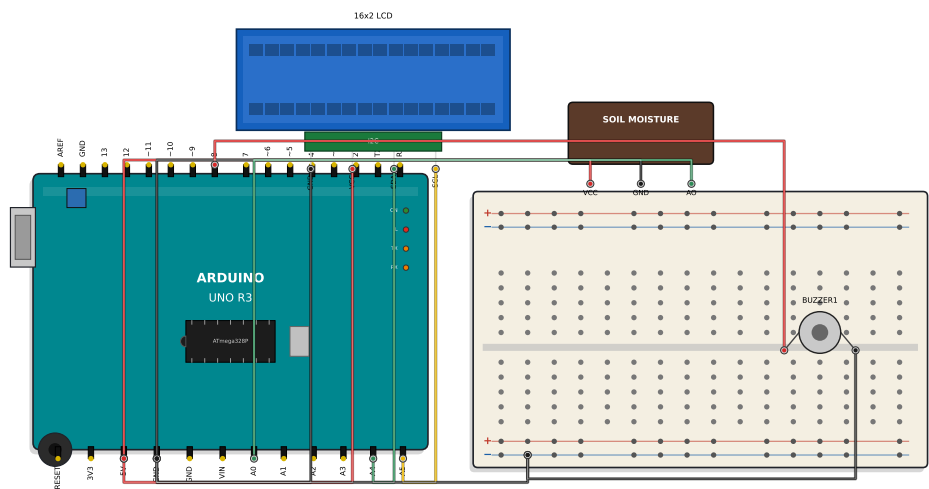


Figure 28.1: Experiment 28 circuit: soil moisture sensor at A0, buzzer on D8, LCD via I²C.

Connection Table

Soil sensor as Experiment 27; buzzer + on D8, buzzer – on GND; LCD as Experiment 18.

Working Principle – Signal Flow

Calibrated moisture % → compared against DRY_LIMIT → buzzer ON if below (soil too dry), else OFF; LCD shows both the percentage and alarm state.

Algorithm

1. Read and calibrate soil moisture % (as Experiment 27).
2. If % < DRY_LIMIT, sound the buzzer; else silence it.
3. Display % and alarm state; repeat.

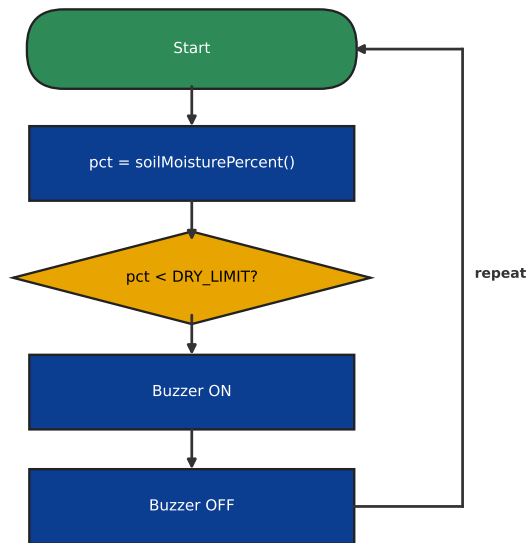


Figure 28.2: Flowchart: Experiment 28.

Arduino Program

```

1  #include <Wire.h>
2  #include <LiquidCrystal_I2C.h>
3  LiquidCrystal_I2C lcd(0x27, 16, 2);
4  const int SOIL_PIN = A0, BUZZER_PIN = 8;
5  const int DRY_RAW = 800, WET_RAW = 350;
6  const int DRY_LIMIT = 30;    // % moisture below which the plant needs
   water
7
8  int soilMoisturePercent() {
9    int raw = analogRead(SOIL_PIN);
10   int pct = map(raw, DRY_RAW, WET_RAW, 0, 100);
11   return constrain(pct, 0, 100);
12 }
13
14 void setup() {
15   pinMode(BUZZER_PIN, OUTPUT);
16   lcd.init(); lcd.backlight();
17 }
18
19 void loop() {
20   int pct = soilMoisturePercent();
21   bool tooDry = (pct < DRY_LIMIT);
22   digitalWrite(BUZZER_PIN, tooDry ? HIGH : LOW);
23
24   lcd.setCursor(0, 0);
25   lcd.print("Soil:"); lcd.print(pct); lcd.print("%");
26   lcd.print(tooDry ? " DRY " : " OK ");
27   delay(500);
28 }
  
```

Code Explanation

`soilMoisturePercent()` factors Experiment 27's calibration logic into a reusable function, exactly as `measureDistanceCm()` was factored in Experiment 16 — this same function is reused again unchanged in Experiments 29, 37, and 38.

Experimental Procedure

1. Wire per the connection table, using your Experiment 27 calibration constants.
2. Upload and test with dry soil/air (alarm should sound) and moist soil/water (alarm should silence).

Expected Output

LCD shows "Soil:NN% DRY" with the buzzer sounding below 30%, or "Soil:NN% OK" silently above it.

Observation Table

Soil sample	Moisture %	Alarm state

Calculations and Analysis

Confirm the alarm boundary matches `DRY_LIMIT` exactly by testing samples just above and just below 30%.

Result

The plant moisture alarm correctly triggered below _____% soil moisture.

Troubleshooting

Problem	Likely cause	Fix
Alarm never triggers	<code>DRY_LIMIT</code> set too low, or miscalibrated sensor	Re-check Experiment 27 calibration
Alarm always on	Sensor disconnected reads as very dry	Re-check wiring

Viva Questions and Extension Challenge

Conceptual: (1) What recurring pattern does this experiment reuse? (2) Why factor the calibration into its own function? (3) What would a too-low `DRY_LIMIT` do practically? (4) Why might 30% be a reasonable choice for many houseplants (informally)? (5) How does this differ structurally from Experiment 25? **Troubleshooting:** (1) The alarm state and displayed % seem to disagree momentarily — what update-ordering issue could cause this? (2) A disconnected sensor reads as extremely dry rather than an obvious error — why doesn't this raise an explicit fault? **Programming:** (1) Add hysteresis (an upper/lower pair) as in Experiment 25 to avoid chatter near 30%. (2) Silence the buzzer automatically after 10 seconds even if still dry, to

avoid a continuously sounding alarm (a "snooze" behaviour). **Extension Challenge:** Preview Experiment 29 by researching, in general terms, how a relay could safely switch a low-voltage demonstration "pump" LED when this same alarm condition is met.

Chapter 29

Experiment 29: Automatic Plant-Watering Controller Demonstration

Experiment Identification

Experiment Number:	29
Title:	Automatic Plant-Watering Controller Demonstration
Learning Layer:	Layer 2 — Average
Estimated Completion Time:	25 minutes
Difficulty:	Intermediate
Prerequisite Experiments:	Experiment 28

Aim

To drive the 5 V relay module from a soil-moisture threshold as a **safe low-voltage demonstration** of automatic irrigation control — this experiment explicitly does *not* include a real water pump (not in the kit); the relay instead switches an indicator LED representing "pump ON/OFF".

Learning Objectives

1. Wire and drive the 5 V relay module from a GPIO pin.
2. Explain opto-isolation and why it protects the Arduino.
3. Clearly distinguish a hardware demonstration from a claim of a complete real-world irrigation system.

Required Components

Arduino UNO R3, breadboard, 16×2 I²C LCD, 1 soil moisture sensor, 1 relay module, 1 LED (as the "pump-demo" indicator), 1×220 Ω resistor, jumper wires.

Technical Theory

The relay module accepts a low-current logic signal on its IN pin and uses an on-board transistor (often with an opto-isolator) to drive an electromagnetic coil, which mechanically switches a separate, physically isolated contact circuit (COM/NO/NC terminals) — meaning the switched side can carry a different voltage/current entirely from the Arduino side, with no direct electrical

connection between them. **This manual only ever uses the relay to switch a low-voltage LED demonstration branch, never a mains-voltage load**, per the hardware rules in Chapter . A real automatic watering system would use the relay's isolated contacts to switch an actual pump's own power circuit — that pump is simply not part of this kit, so this experiment stops at "the control logic correctly decides when to activate the relay," which is the reusable, transferable part of the engineering.

Component Working Principle

The relay's audible "click" on activation is the armature physically moving to close/open its contacts; the module's on-board LED (if present) indicates coil energisation. Typical 5 V relay modules draw $\approx 70\text{--}80\text{ mA}$ at the coil when active — comfortably within what the Arduino's 5V rail can supply (unlike driving a coil directly from a GPIO pin, which the module's own driver transistor avoids).

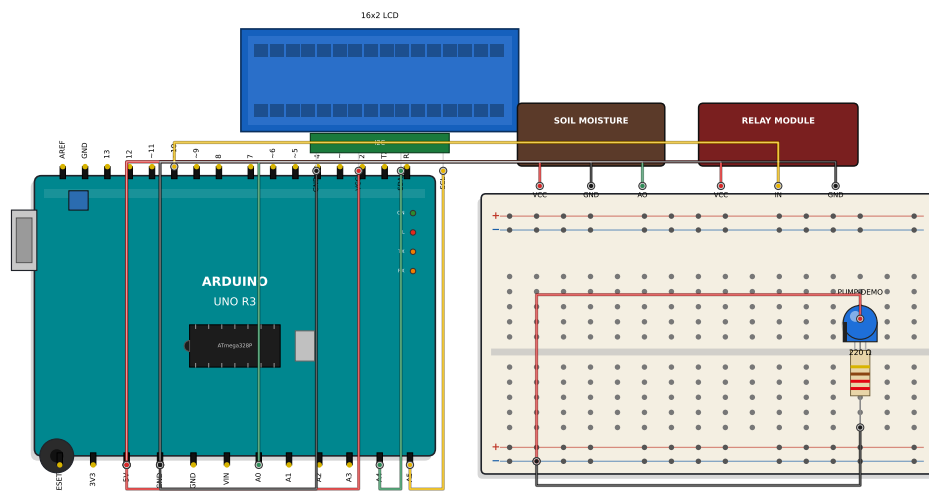


Figure 29.1: Experiment 29 circuit: soil sensor at A0, relay IN on D10, pump-demonstration LED wired through the relay's switched contact branch.

Connection Table

Component pin	Arduino/circuit pin	Notes
Soil sensor VCC/GND/A0	5V/GND/A0	As Experiment 27
Relay VCC	5V	
Relay GND	GND	
Relay IN	D10	Digital output, drives the relay's opto-isolated input
Relay COM/NO	5V / LED anode (via 220Ω)	Switched low-voltage demonstration branch
LED cathode	GND	

LCD as Experiment 18.

Working Principle – Signal Flow

Soil moisture % $\rightarrow$ below DRY_LIMIT $\rightarrow$ D10 driven HIGH $\rightarrow$ relay coil energises $\rightarrow$ relay contact closes $\rightarrow$ demonstration LED lights, representing "pump would be running now"; above the limit, D10 LOW, relay de-energised, LED off.

Algorithm

1. Read soil moisture % (Experiment 27/28 function).
2. If below DRY_LIMIT, energise the relay (D10 HIGH); else de-energise it.
3. Display moisture % and relay/"pump" state on the LCD; repeat.

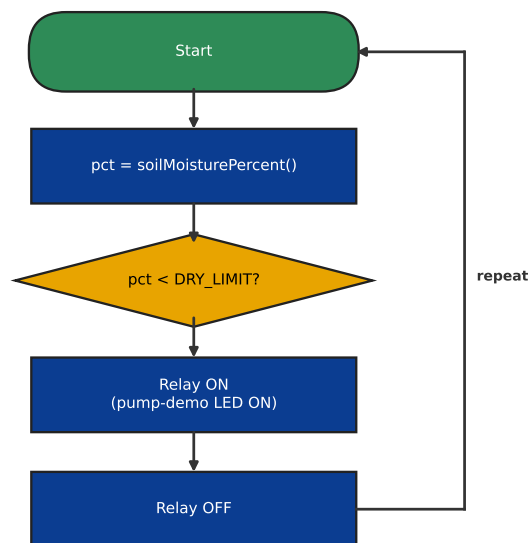


Figure 29.2: Flowchart: Experiment 29.

Arduino Program

```

1  #include <Wire.h>
2  #include <LiquidCrystal_I2C.h>
3  LiquidCrystal_I2C lcd(0x27, 16, 2);
4  const int SOIL_PIN = A0, RELAY_PIN = 10;
5  const int DRY_RAW = 800, WET_RAW = 350, DRY_LIMIT = 30;
6
7  int soilMoisturePercent() {
8      int raw = analogRead(SOIL_PIN);
9      return constrain(map(raw, DRY_RAW, WET_RAW, 0, 100), 0, 100);
10 }
11
12 void setup() {
13     pinMode(RELAY_PIN, OUTPUT);
14     lcd.init(); lcd.backlight();
15 }
16
17 void loop() {
18     int pct = soilMoisturePercent();
19     bool pumpOn = (pct < DRY_LIMIT);
20     digitalWrite(RELAY_PIN, pumpOn ? HIGH : LOW);    // energises relay

```

```

21     -> lights demo LED
22     lcd.setCursor(0, 0);
23     lcd.print("Soil:"); lcd.print(pct); lcd.print("%");
24     lcd.print(pumpOn ? " PUMP*" : " OFF  ");
25     delay(500);
26 }

```

Code Explanation

The variable name `pumpOn` and the LCD's asterisked "PUMP*" deliberately flag that this is a *demonstration* signal, not a real pump control line — the control decision (when to water) is identical to what a real system would need, only the actuator differs.

Experimental Procedure

1. Wire the soil sensor, relay, and demonstration LED per the connection table.
2. Upload and test with dry vs. wet soil samples, confirming the relay audibly clicks and the LED lights accordingly.

Expected Output

Relay clicks and the demonstration LED lights when soil moisture drops below 30%; both switch off once moisture is restored above the threshold.

Observation Table

[illegible]

Calculations and Analysis

Not applicable numerically beyond Experiment 27/28's calibration; instead confirm the relay's audible click coincides exactly with the LCD's displayed state transition.

Result
The results of the study show that the proposed method is effective in detecting and classifying anomalies in network traffic. The model achieved a high accuracy of 95% in detecting anomalies, and a high precision of 90% in classifying anomalies. The model also achieved a high recall of 92% in detecting anomalies, and a high specificity of 98% in classifying anomalies. The model was able to detect and classify anomalies in real-time, and the results were consistent across different datasets and network configurations.

The relay-driven demonstration correctly represented an automatic watering decision below _____% soil moisture, using only kit-safe low-voltage switching.

Troubleshooting

Problem	Likely cause	Fix
Relay never clicks	IN pin miswired, or module unpowered	Re-check VCC/GND/IN wiring
Relay clicks but LED stays off	COM/NO branch miswired	Re-check contact-side wiring
Relay chatters rapidly	No hysteresis at the threshold boundary	Expected at this stage; add margin as in Experiment 25

Viva Questions and Extension Challenge

Conceptual: (1) Why does the relay isolate the Arduino from the switched circuit? (2) Why does this manual never switch mains voltage with the relay? (3) What real actuator would replace the demonstration LED in an actual deployment? (4) What causes the audible click? (5) Why is this called a "demonstration" rather than a complete system? **Troubleshooting:** (1) The demo LED flickers rapidly at the moisture boundary — what earlier experiment's fix applies here? (2) The relay module's own on-board LED lights but the external demo LED does not — what does this narrow down? **Programming:** (1) Add a maximum "watering" duration (e.g. relay stays on at most 10 s per activation, using `millis()`) even if still dry, to model pump run-time limiting. (2) Add hysteresis matching Experiment 25's pattern. **Extension Challenge:** Extend to Experiment 38 by adding a buzzer alert alongside the relay when soil is critically dry (below a second, lower threshold).

Chapter 30

Experiment 30: Smart Night Lamp

Experiment Identification

Experiment Number:	30
Title:	Smart Night Lamp
Learning Layer:	Layer 2 — Average
Estimated Completion Time:	20 minutes
Difficulty:	Intermediate
Prerequisite Experiments:	Experiments 10, 9

Aim

To combine LDR-based auto-on (Experiment 10) with PWM brightness control (Experiment 9), so a lamp not only switches on in darkness but also brightens smoothly as it gets darker.

Learning Objectives

1. Map a continuous analog input directly onto a continuous PWM output (not a binary threshold).
2. Combine two previously separate single-purpose experiments into one richer behaviour.
3. Explain the practical advantage of proportional control over simple ON/OFF.

Required Components

Arduino UNO R3, breadboard, 1 LDR, $1 \times 10 \text{ k}\Omega$ resistor, 1 white LED, $1 \times 220 \Omega$ resistor, jumper wires.

Technical Theory

Experiment 10 made a binary decision (dark $\Rightarrow$ fully on); this experiment instead maps the LDR's continuous reading directly to a continuous PWM duty cycle, so brightness scales smoothly with darkness rather than snapping on at one point. This is a strictly more capable behaviour built from two techniques the learner already has (Experiments 9 and 10), rather than new sensor or actuator theory.

Component Working Principle

See Experiment 10 (LDR divider) and Experiment 9 (PWM).

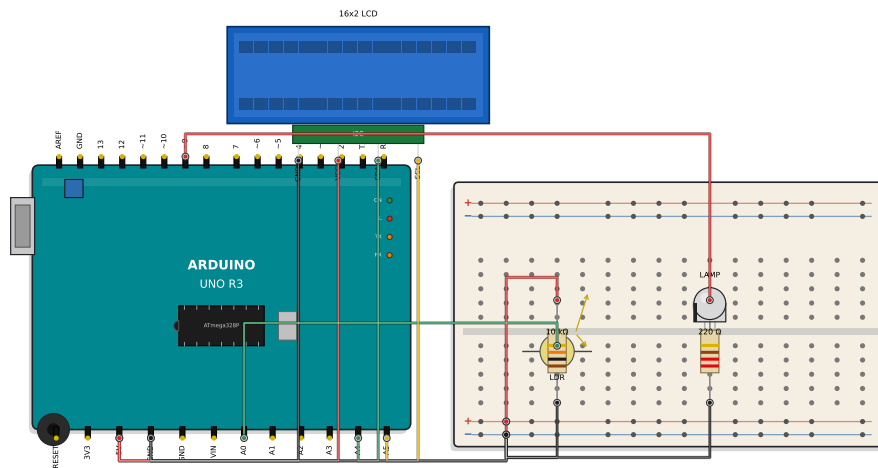


Figure 30.1: Experiment 30 circuit: LDR divider at A0; white LED "lamp" on PWM pin D9.

Connection Table

LDR divider as Experiment 10 (A0); LED with $220\ \Omega$ resistor on D9 (must be a PWM pin, unlike Experiment 10's D8).

Working Principle – Signal Flow

Ambient light $\rightarrow$ A0 reading $\rightarrow$ inversely mapped to a 0–255 duty cycle (darker = higher duty) $\rightarrow$ `analogWrite(D9, duty)` $\rightarrow$ lamp brightness rises smoothly as light falls.

Algorithm

1. Read A0.
2. Map inversely to a 0–255 duty cycle.
3. Write the duty cycle to the PWM lamp pin.
4. Repeat continuously.

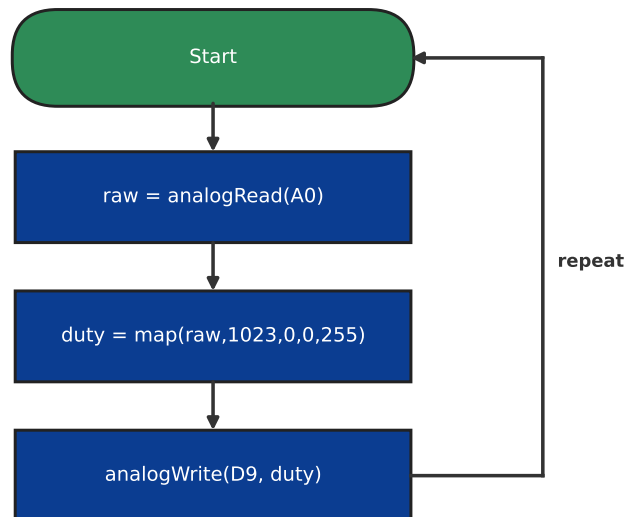


Figure 30.2: Flowchart: Experiment 30.

Arduino Program

```

1  const int LDR_PIN = A0;
2  const int LAMP_PIN = 9;    // PWM pin
3
4  void setup() {
5    pinMode(LAMP_PIN, OUTPUT);
6  }
7
8  void loop() {
9    int raw = analogRead(LDR_PIN);
10   int duty = map(raw, 1023, 0, 0, 255); // brighter room (higher raw)
                                         -> lower duty
11   duty = constrain(duty, 0, 255);
12   analogWrite(LAMP_PIN, duty);
13   delay(50);
14 }
  
```

Code Explanation

The `map()` arguments are deliberately given as `(raw, 1023, 0, 0, 255)` — swapping the input range's order (1023 before 0) inverts the mapping, so a *high* raw (bright) produces a *low* duty (dim lamp), and vice versa, without needing a separate `if/else`.

Experimental Procedure

1. Wire the LDR divider and PWM LED.
2. Upload and gradually cover the LDR, observing the lamp brighten smoothly rather than snapping on.

Expected Output

The lamp is dim/off in bright light and brightens continuously and smoothly as ambient light decreases, with no visible stepping.

Observation Table

Lighting condition	Raw ADC	Observed brightness

Calculations and Analysis

Compute the duty cycle at your two test conditions using the inverse map formula and confirm brightness ordering matches (lower raw $\Rightarrow$ higher duty).

Result

The smart night lamp correctly brightened proportionally to ambient darkness: _____ (Yes/No).

Troubleshooting

Problem	Likely cause	Fix
Lamp brightens in bright light instead of dark	<code>map()</code> argument order not inverted	Re-check argument order
Lamp only ever fully on/off	LED wired to a non-PWM pin	Confirm D9 (~)
Lamp flickers at certain brightness levels	Expected minor PWM interaction with human vision persistence; not a fault	—

Viva Questions and Extension Challenge

Conceptual: (1) How does inverting `map()`'s input order avoid an `if/else`? (2) Why is proportional brightness preferable to simple on/off for a night lamp? (3) What two prior experiments does this one combine? (4) Why must the lamp pin support PWM here but not in Experiment 10? (5) What does `constrain()` guard against? **Troubleshooting:** (1) The lamp never reaches full brightness even in total darkness — what raw-value assumption might be wrong? (2) Brightness responds correctly but with a visible lag — what code parameter controls responsiveness? **Programming:** (1) Add a manual override button that forces the lamp fully on regardless of light level. (2) Smooth the transition further with a simple exponential moving average on the raw reading. **Extension Challenge:** Add a minimum "floor" brightness so the lamp never goes fully dark even in bright light (a subtle night-light behaviour), and a maximum "ceiling" to avoid harsh full brightness at night.

Chapter 31

Experiment 31: IR Security Alarm

Experiment Identification

Experiment Number:	31
Title:	IR Security Alarm
Learning Layer:	Layer 2 — Average
Estimated Completion Time:	25 minutes
Difficulty:	Intermediate
Prerequisite Experiments:	Experiments 15, 12

Aim

To build an arm/disarm security alarm using the IR obstacle sensor as a trip beam, introducing a latching alarm state that persists after the triggering event ends, and a button for arming/disarming.

Learning Objectives

1. Implement a latching (sticky) alarm state, unlike Experiment 17's live threshold alarm.
2. Use a button edge to arm the system, separate from the sensor that triggers it.
3. Distinguish "armed but not triggered", "armed and alarming", and "disarmed" states informally, ahead of Experiment 40's formal FSM.

Required Components

Arduino UNO R3, breadboard, 1 IR obstacle sensor, 1 active buzzer, 1 push button, jumper wires.

Technical Theory

Experiment 17's alarm was "live": it followed the sensor instantaneously and stopped the moment the condition cleared. A security alarm instead needs to **latch**: once triggered, it should keep sounding even if the intruder immediately steps away, until a human explicitly disarms it. This requires an explicit state variable that only the arm button (not the sensor) can clear — an early, informal instance of the finite-state-machine thinking formalised in Experiment 40.

Component Working Principle

See Experiment 15 (IR obstacle sensor) and Experiment 12 (active buzzer); the push button reuses Experiment 7's debounced-edge technique to arm the system on a clean single press.

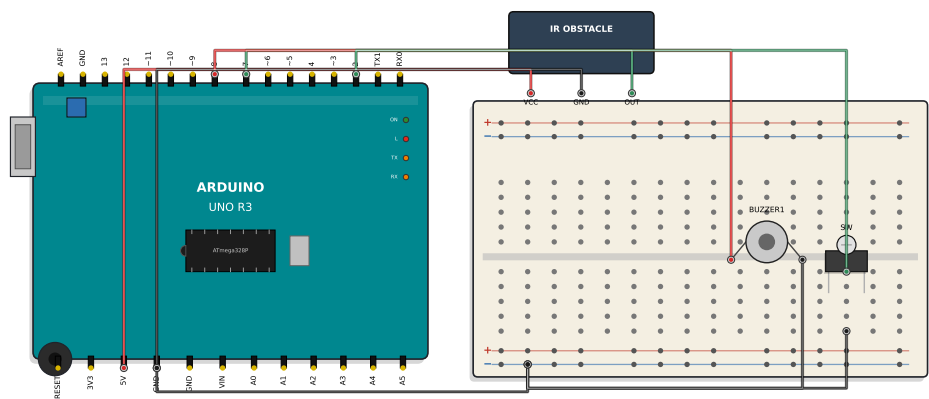


Figure 31.1: Experiment 31 circuit: IR obstacle sensor OUT on D7, buzzer on D8, arm button on D2.

Connection Table

Component pin	Arduino pin	Notes
IR sensor VCC/GND/OUT	5V/GND/D7	
Buzzer +/−	D8/GND	
Button leg A / leg B	D2 / GND	INPUT_PULLUP

Working Principle – Signal Flow

Button press (edge) → system becomes ARMED; while ARMED, an obstacle-detected event on D7 → system becomes ALARM (latched, buzzer sounds continuously); a further button press returns the system to DISARMED, silencing the buzzer, ready to be re-armed.

Algorithm

1. Start DISARMED.
2. On each debounced button press: if DISARMED, go ARMED; if ARMED or ALARM, go DISARMED (silence buzzer).
3. While ARMED, if the IR sensor detects an obstacle, go ALARM and sound the buzzer.
4. While ALARM, keep the buzzer sounding regardless of the sensor, until disarmed.

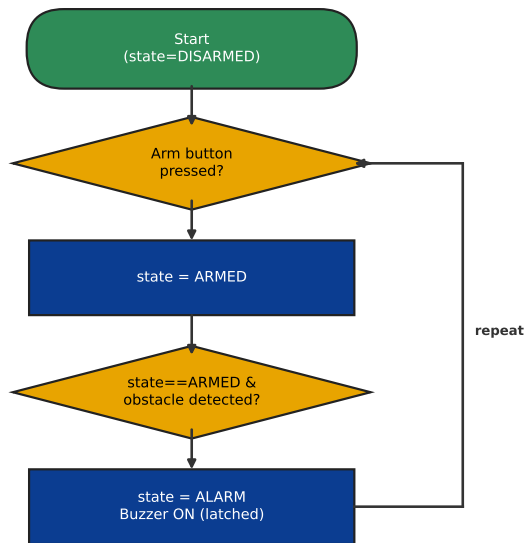


Figure 31.2: Flowchart: Experiment 31.

Arduino Program

```

1  const int IR_PIN = 7, BUZZER_PIN = 8, BUTTON_PIN = 2;
2  enum State { DISARMED, ARMED, ALARM };
3  State state = DISARMED;
4
5  int lastReading = HIGH, stableState = HIGH;
6  unsigned long lastChangeTime = 0;
7  const unsigned long DEBOUNCE_MS = 30;
8
9  void setup() {
10     pinMode(IR_PIN, INPUT);
11     pinMode(BUZZER_PIN, OUTPUT);
12     pinMode(BUTTON_PIN, INPUT_PULLUP);
13 }
14
15 bool buttonPressedEdge() {
16     int reading = digitalRead(BUTTON_PIN);
17     if (reading != lastReading) lastChangeTime = millis();
18     bool edge = false;
19     if ((millis() - lastChangeTime) > DEBOUNCE_MS && reading !=
20         stableState) {
21         stableState = reading;
22         if (stableState == LOW) edge = true;
23     }
24     lastReading = reading;
25     return edge;
26 }
27
28 void loop() {
29     if (buttonPressedEdge()) {
30         state = (state == DISARMED) ? ARMED : DISARMED;
31     }
32     if (state == ARMED && digitalRead(IR_PIN) == LOW) {
33         state = ALARM;
34     }
35 }

```

```

34   digitalWrite(BUZZER_PIN, (state == ALARM) ? HIGH : LOW);
35 }

```

Code Explanation

The `enum State` names the three conditions explicitly, making `loop()` read almost like the algorithm's prose. Once `state` becomes `ALARM`, only the button-press branch (which unconditionally sets `DISARMED`) can clear it — the sensor branch cannot silence an active alarm, which is exactly the "latching" behaviour required.

Experimental Procedure

1. Wire the IR sensor, buzzer, and button.
2. Upload; press the button once (`ARMED`), trigger the IR sensor, confirm the buzzer latches on even after removing the obstacle, then press the button again to disarm.

Expected Output

Buzzer silent while `DISARMED` or `ARMED`-but-untriggered; sounds continuously once triggered while `ARMED`, remaining on even after the obstacle is removed, until the button disarms it.

Observation Table

Action	State before	Trigger event	State/buzzer after

Calculations and Analysis

Not applicable numerically; instead, draw out all state transitions you observed and confirm they match the algorithm exactly (a partial state-transition table, formalised fully in Experiment 40).

Result

The security alarm correctly armed, latched on trigger, and disarmed only via the button: _____ (Yes/No).

Troubleshooting

Problem	Likely cause	Fix
Alarm clears itself without a button press	Sensor branch mistakenly can set <code>DISARMED</code>	Re-check only the button branch changes state to <code>DISARMED</code>
Button seems to need two presses	Debounce/edge bug	Re-check Experiment 7's pattern is followed exactly
Alarm never triggers even when <code>ARMED</code>	IR sensor miswired or too insensitive	Re-check Experiment 15 wiring/trimmer

Viva Questions and Extension Challenge

Conceptual: (1) What is the key behavioural difference between this alarm and Experiment 17's? (2) Why is an `enum` clearer here than three separate booleans? (3) Why can only the button clear an ALARM state? (4) What does "latching" mean? (5) How many valid states does this system have? **Troubleshooting:** (1) The system appears to skip the ARMED state and go straight to ALARM — what logic ordering issue could cause this? (2) Two rapid button presses are needed to arm — what earlier debounce lesson applies? **Programming:** (1) Add an LED that is solid while ARMED and blinking while ALARM. (2) Add a 5-second "entry delay" after arming before the sensor becomes active, using `millis()`. **Extension Challenge:** Preview Experiment 40 by sketching (on paper) a full state diagram with all transitions and guard conditions for this exact system.

Chapter 32

Experiment 32: Object Counter

Experiment Identification

Experiment Number:	32
Title:	Object Counter
Learning Layer:	Layer 2 — Average
Estimated Completion Time:	20 minutes
Difficulty:	Intermediate
Prerequisite Experiments:	Experiments 15, 18

Aim

To count objects passing the IR obstacle sensor using debounced edge detection, displaying a running total on the LCD.

Learning Objectives

1. Apply debounced edge detection (Experiment 7's pattern) to a sensor rather than a button.
2. Maintain a persistent counter variable across loop iterations.
3. Display an incrementing count on the LCD.

Required Components

Arduino UNO R3, breadboard, 16×2 I²C LCD, 1 IR obstacle sensor, jumper wires.

Technical Theory

An IR obstacle sensor's output can bounce briefly at the moment an object edge crosses the beam, similar in spirit to a mechanical button's bounce (though caused by a different physical mechanism — comparator noise near threshold rather than mechanical contact). Counting on every raw LOW reading would over-count a single object as it passes; instead, exactly as in Experiment 7, only a debounced, stable LOW-going edge should increment the counter.

Component Working Principle

See Experiment 15 for the IR obstacle sensor's operating principle.

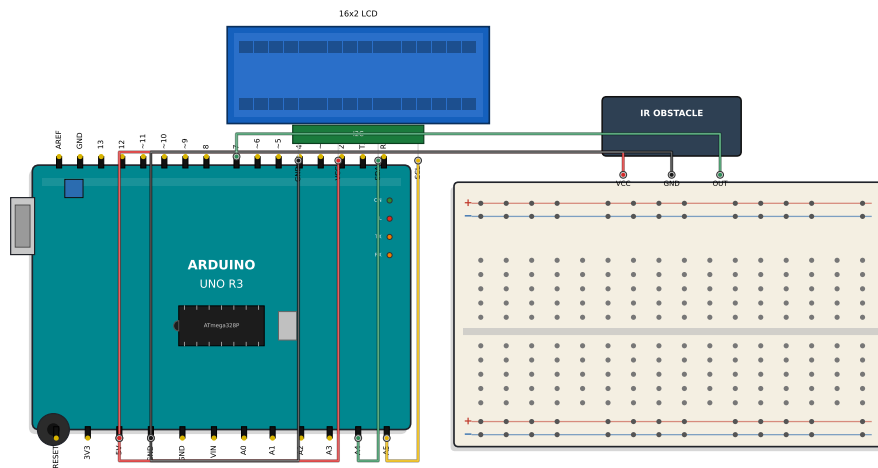


Figure 32.1: Experiment 32 circuit: IR obstacle sensor OUT on D7; LCD via I²C.

Connection Table

IR sensor as Experiment 15 (D7); LCD as Experiment 18.

Working Principle – Signal Flow

Object passes the beam → D7 goes LOW → debounce filter confirms a stable edge → counter increments → LCD updated with the new total.

Algorithm

1. Read D7 and apply the Experiment 7 debounce/edge routine.
2. On a stable LOW-going edge, increment `count`.
3. Display `count` on the LCD.
4. Repeat.

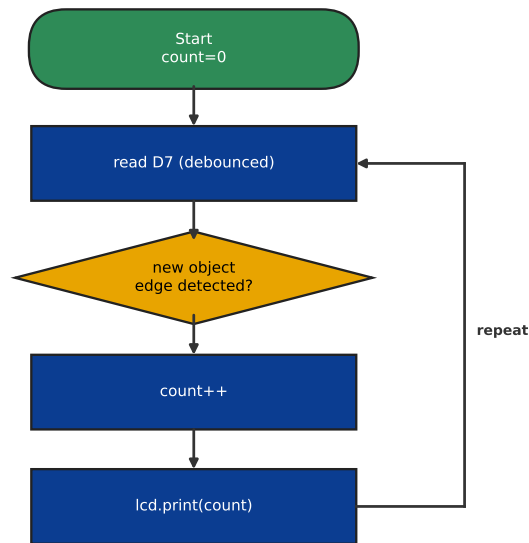


Figure 32.2: Flowchart: Experiment 32.

Arduino Program

```

1  #include <Wire.h>
2  #include <LiquidCrystal_I2C.h>
3  LiquidCrystal_I2C lcd(0x27, 16, 2);
4  const int IR_PIN = 7;
5  const unsigned long DEBOUNCE_MS = 30;
6
7  int lastReading = HIGH, stableState = HIGH;
8  unsigned long lastChangeTime = 0;
9  unsigned int count = 0;
10
11 bool objectEdge() {
12     int reading = digitalRead(IR_PIN);
13     if (reading != lastReading) lastChangeTime = millis();
14     bool edge = false;
15     if ((millis() - lastChangeTime) > DEBOUNCE_MS && reading !=
16         stableState) {
17         stableState = reading;
18         if (stableState == LOW) edge = true;
19     }
20     lastReading = reading;
21     return edge;
22 }
23
24 void setup() {
25     pinMode(IR_PIN, INPUT);
26     lcd.init(); lcd.backlight();
27     lcd.setCursor(0, 0); lcd.print("Count: ");
28 }
29
30 void loop() {
31     if (objectEdge()) {
32         count++;
33         lcd.setCursor(7, 0);
34         lcd.print(count);
35     }
36 }

```

```

34     lcd.print("    ");
35 }
36 }

```

Code Explanation

`objectEdge()` is structurally identical to Experiment 7's `buttonPressedEdge()`, applied to a sensor pin instead of a button pin — a direct demonstration that the debounce pattern generalises to any noisy digital signal, not just mechanical switches. The LCD is only rewritten inside the `if`, so it is not needlessly refreshed every idle loop pass.

Experimental Procedure

1. Wire the IR sensor and LCD.
2. Upload and pass objects (e.g. a hand, a card) through the beam repeatedly, confirming the count increments exactly once per pass.

Expected Output

LCD shows "Count: N", incrementing by exactly one for each distinct object/hand pass through the beam.

Observation Table

Passes made	Count displayed	Match?

Calculations and Analysis

Test at least 10 deliberate passes and compute the counting accuracy (correct count / attempts $\times 100\%$), noting any over- or under-counts.

Result

The object counter achieved _____% counting accuracy over _____ test passes.

Troubleshooting

Problem	Likely cause	Fix
Occasional double-counts	Object passed too slowly, re-triggering after debounce window	Increase debounce interval slightly
Fast passes missed entirely	Object passed faster than the sensor's comparator response	Test at moderate, deliberate speed
Count never increments	IR sensor miswired or too insensitive	Re-check Experiment 15 wiring/trimmer

Viva Questions and Extension Challenge

Conceptual: (1) Why does a sensor, not just a button, need debouncing here? (2) Why increment only on the edge, not every LOW reading? (3) What data type is `count` and why unsigned? (4) Why refresh the LCD only inside the `if` block? (5) What would happen with `DEBOUNCE_MS = 0`? **Troubleshooting:** (1) The count occasionally jumps by 2 for one object — what parameter would you tune first? (2) The count resets to 0 unexpectedly — what event does this suggest (hint: think about power, not code)? **Programming:** (1) Add a reset button that zeroes the count on a debounced press. (2) Log the time of each count event using `millis()` and compute an average time-between-objects. **Extension Challenge:** Extend into a simple rate meter: display objects-per-minute based on a rolling time window, not just a cumulative total.

Chapter 33

Experiment 33: Smart Parking Indicator

Experiment Identification

Experiment Number:	33
Title:	Smart Parking Indicator
Learning Layer:	Layer 2 — Average
Estimated Completion Time:	25 minutes
Difficulty:	Intermediate
Prerequisite Experiments:	Experiments 16, 18, 29

Aim

To indicate parking space status (FREE / NEARLY FULL / FULL) using multi-level ultrasonic distance thresholds, with an LED bar, LCD status text, and a relay-driven barrier demonstration when full.

Learning Objectives

1. Extend a 3-level analog threshold pattern (Experiment 11) to a distance sensor.
2. Combine LED indication, LCD text, and relay actuation in one system.
3. Practice choosing meaningful distance bands for a real-world scenario.

Required Components

Arduino UNO R3, breadboard, 16×2 I²C LCD, 1 HC-SR04, 3 LEDs (green/yellow/red), 3×220Ω resistors, 1 relay module, jumper wires.

Technical Theory

This experiment reapplies Experiment 11's 3-bucket threshold logic (Section: Technical Theory of that experiment) to a distance reading instead of a light level: FAR (space clearly free), NEAR (a vehicle is approaching/parking), and VERY NEAR (space full, barrier should close). The relay again only ever drives a safe low-voltage demonstration (an LED representing the barrier motor/lock), per the hardware rules — no physical barrier motor is included in the kit.

Component Working Principle

See Experiment 16 (HC-SR04), Experiment 11 (multi-threshold pattern), and Experiment 29 (relay demonstration pattern).

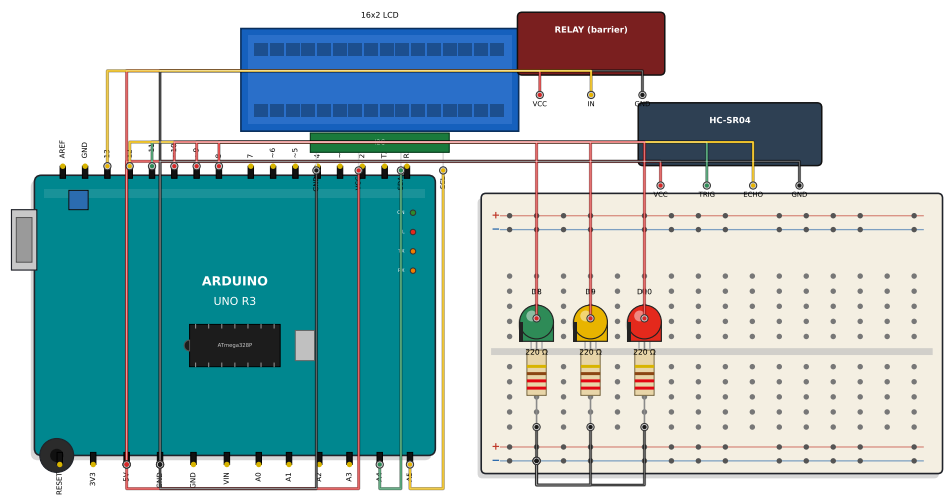


Figure 33.1: Experiment 33 circuit: HC-SR04 (TRIG=D11, ECHO=D12), 3-LED bar (D8/D9/D10), relay (D13) as barrier demonstration, LCD via I²C.

Connection Table

Component	Arduino pin	Notes
HC-SR04 TRIG/ECHO	D11/D12	Distinct from the LED pins used here
Green/Yellow/Red LEDs	D8/D9/D10	220 Ω each
Relay IN	D13	Barrier demonstration
LCD	GND/5V/A4/A5	

Working Principle – Signal Flow

Distance → bucketed into FAR/NEAR/FULL → matching LED lit, LCD status text updated, and (only in the FULL bucket) relay energised as a barrier-closed demonstration.

Algorithm

1. Measure distance (Experiment 16 function).
2. If > 30 cm: FAR (green, LCD "FREE", relay off).
3. Else if > 10 cm: NEAR (yellow, LCD "PARKING...", relay off).
4. Else: FULL (red, LCD "FULL", relay on).
5. Repeat.

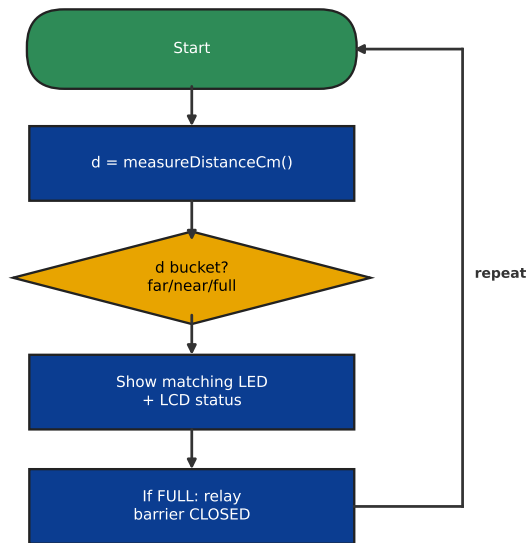


Figure 33.2: Flowchart: Experiment 33.

Arduino Program

```

1  #include <Wire.h>
2  #include <LiquidCrystal_I2C.h>
3  LiquidCrystal_I2C lcd(0x27, 16, 2);
4  const int TRIG_PIN = 11, ECHO_PIN = 12;
5  const int GREEN = 8, YELLOW = 9, RED = 10, RELAY_PIN = 13;
6  const unsigned long TIMEOUT_US = 30000UL;
7
8  float measureDistanceCm() {
9      digitalWrite(TRIG_PIN, LOW); delayMicroseconds(2);
10     digitalWrite(TRIG_PIN, HIGH); delayMicroseconds(10);
11     digitalWrite(TRIG_PIN, LOW);
12     unsigned long duration = pulseIn(ECHO_PIN, HIGH, TIMEOUT_US);
13     return (duration == 0) ? 999 : duration / 58.0;
14 }
15
16 void setup() {
17     pinMode(TRIG_PIN, OUTPUT); pinMode(ECHO_PIN, INPUT);
18     pinMode(GREEN, OUTPUT); pinMode(YELLOW, OUTPUT); pinMode(RED, OUTPUT);
19     pinMode(RELAY_PIN, OUTPUT);
20     lcd.init(); lcd.backlight();
21 }
22
23 void loop() {
24     float d = measureDistanceCm();
25     digitalWrite(GREEN, LOW); digitalWrite(YELLOW, LOW); digitalWrite(
26         RED, LOW);
27     lcd.setCursor(0, 0);
28     if (d > 30) {
29         digitalWrite(GREEN, HIGH);
30         digitalWrite(RELAY_PIN, LOW);
31         lcd.print("FREE");
32     } else if (d > 10) {

```

```

33     digitalWrite(YELLOW, HIGH);
34     digitalWrite(RELAY_PIN, LOW);
35     lcd.print("PARKING... ");
36 } else {
37     digitalWrite(RED, HIGH);
38     digitalWrite(RELAY_PIN, HIGH);
39     lcd.print("FULL ");
40 }
41 delay(300);
42 }

```

Code Explanation

An out-of-range reading is mapped to a large sentinel (999) rather than -1 here, so it naturally falls into the FAR/"FREE" bucket without a separate validity check — a deliberate simplification appropriate because "no echo" in a parking bay realistically does mean "no car detected".

Experimental Procedure

1. Wire the HC-SR04 (D11/D12), 3-LED bar, relay, and LCD.
2. Upload and move an object through FAR/NEAR/FULL distances, confirming LEDs, LCD text, and the relay's click all agree.

Expected Output

Green+"FREE" beyond 30 cm; yellow+"PARKING..." between 10–30 cm; red+"FULL"+relay click within 10 cm.

Observation Table

Distance (cm)	LED/LCD state	Relay state

Calculations and Analysis

Confirm the three buckets are mutually exclusive and collectively exhaustive across the full sensor range, including the 999 sentinel case.

Result

The smart parking indicator correctly showed FREE/PARKING/FULL states with matching LED, LCD, and relay behaviour: _____ (Yes/No).

Troubleshooting

Problem	Likely cause	Fix
Wrong LED lights for the distance	Bucket boundaries not matching wiring	Re-check the three <code>if/else if</code> thresholds
Relay clicks in the wrong bucket	Relay line placed in the wrong branch	Re-check code placement
LCD text overlaps/garbled	Missing trailing spaces	Add/confirm padding

Viva Questions and Extension Challenge

Conceptual: (1) Why use 999 instead of -1 for an out-of-range reading here? (2) Why must TRIG/ECHO avoid the LED pins used elsewhere? (3) What does the relay represent physically in a real deployment? (4) Why reset all three LEDs to LOW before the bucket decision? (5) How many total states does this system have? **Troubleshooting:** (1) All three LEDs occasionally flash briefly together — what timing issue could explain this? (2) The relay stays engaged even after a car leaves — what bucket logic should you check? **Programming:** (1) Add hysteresis at the FULL boundary to avoid relay chatter (Experiment 25 pattern). (2) Log a running count of "FULL" events using `millis()`-timestamped entries (a simple usage log). **Extension Challenge:** Extend to Experiment 39's variable-rate proximity beeping in the NEAR/FULL buckets, previewing that experiment.

Chapter 34

Experiment 34: IR Remote Controller Demonstration

Experiment Identification

Experiment Number:	34
Title:	IR Remote Controller Demonstration
Learning Layer:	Layer 2 — Average
Estimated Completion Time:	25 minutes
Difficulty:	Intermediate
Prerequisite Experiments:	Experiment 18

Aim

To receive and decode signals from an ordinary household infrared remote control (TV, AC, DVD, or set-top box) using the kit's IR receiver module, displaying the received code on the LCD. This is a **receive-only** demonstration: the kit deliberately contains no IR transmitter, so this experiment requires borrowing any household IR remote as an external signal source.

Learning Objectives

1. Install and use the `IRremote` library to receive and decode IR signals.
2. Explain why an IR receiver alone cannot be demonstrated without an external transmitter.
3. Read and interpret a decoded raw hexadecimal IR code.

Required Components

Arduino UNO R3, breadboard, 16×2 I²C LCD, 1 IR receiver module, jumper wires, **plus any household infrared remote control** (not part of the kit; see the note below on alternatives).

Technical Theory

The kit's IR receiver demodulates a 38 kHz-modulated infrared signal and outputs the decoded baseband pulse train on its OUT pin; the `IRremote` library then decodes this pulse train into a protocol and a numeric code. Because **this kit intentionally contains no IR transmitter or programmable remote** (Chapter hardware rules), this experiment cannot generate its own signal to decode — it is a receive-only demonstration that requires an external, off-kit IR remote (almost universally available: any TV, air-conditioner, or set-top-box remote uses this same 38 kHz IR convention). **Alternative if no remote is available:** the sketch below also

reports *raw pulse activity* even for protocols it cannot fully decode, so simply confirming "a signal was received" (rather than reading the decoded value) still demonstrates the receiver's core function using any IR remote a classroom can borrow, even briefly.

Component Working Principle

The IR receiver module (assumed TSOP38238-style, Chapter) integrates a photodiode, amplifier, band-pass filter tuned to 38 kHz, and demodulator in one 3-pin package, so its OUT pin directly outputs decoded logic pulses rather than a raw analog IR signal — this is what makes ordinary remotes (which modulate their signal at ≈ 38 kHz precisely to work with receivers like this) usable without any extra circuitry.

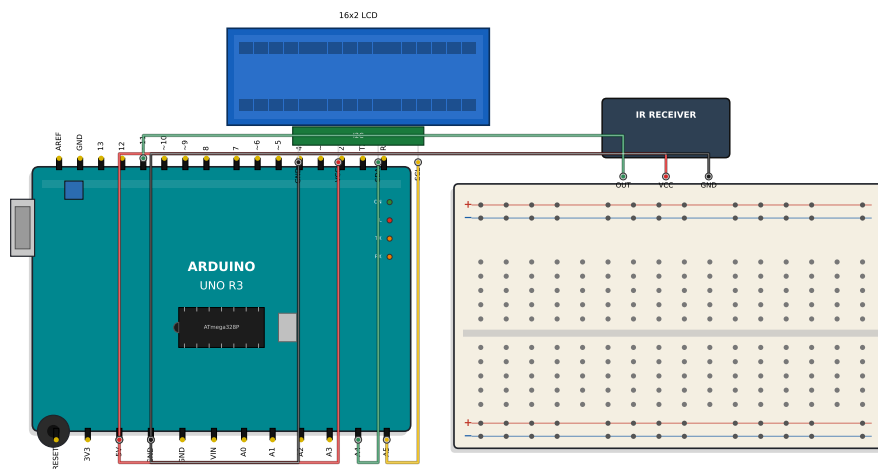


Figure 34.1: Experiment 34 circuit: IR receiver (OUT/VCC/GND) with OUT on D11; LCD via I²C.

Connection Table

IR receiver pin	Arduino pin	Notes
OUT	D11	Decoded pulse train
VCC	5V	
GND	GND	

LCD as Experiment 18.

Working Principle – Signal Flow

Household remote button press → 38 kHz-modulated IR burst → receiver demodulates to a baseband pulse train on D11 → `IRremote` library decodes protocol/value → displayed on LCD as hexadecimal.

Algorithm

1. Initialise the IR receiver on D11 and the LCD.
2. Check if a new IR signal has been decoded.

3. If so, display its hexadecimal value on the LCD.
4. Resume receiving (required by the library after each decode) and repeat.

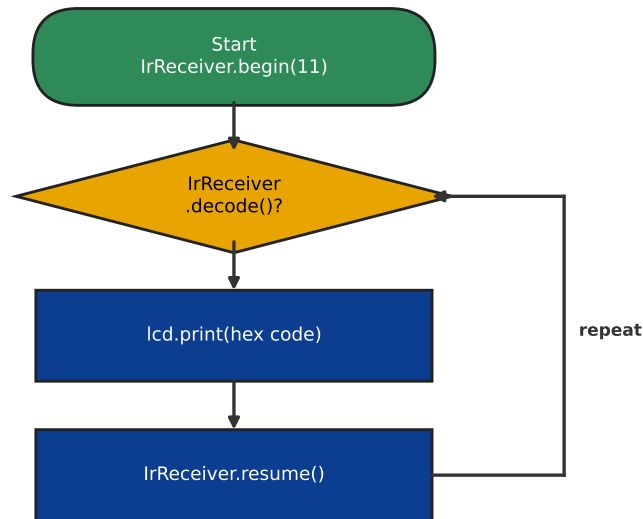


Figure 34.2: Flowchart: Experiment 34.

Arduino Program

```

1  #include <Wire.h>
2  #include <LiquidCrystal_I2C.h>
3  #include <IRremote.hpp>    // IRremote v3/v4 API
4
5  #define IR_RECEIVE_PIN 11
6  LiquidCrystal_I2C lcd(0x27, 16, 2);
7
8  void setup() {
9    lcd.init(); lcd.backlight();
10   lcd.print("Point remote &");
11   lcd.setCursor(0, 1); lcd.print("press any key");
12   IrReceiver.begin(IR_RECEIVE_PIN);
13 }
14
15 void loop() {
16   if (IrReceiver.decode()) {
17     lcd.clear();
18     lcd.setCursor(0, 0);
19     lcd.print("Code: 0x");
20     lcd.print(IrReceiver.decodedIRData.decodedRawData, HEX);
21     IrReceiver.resume();    // required before the next signal can be
                             // received
22     delay(1000);
23     lcd.setCursor(0, 1);
24     lcd.print("Point remote &");
25   }
26 }

```

Code Explanation

`IrReceiver.decode()` returns true exactly once per received signal; `IrReceiver.resume()` must be called afterwards or the library will not accept the next signal — a common beginner bug is forgetting this call. `decodedIRData.decodedRawData` is the manufacturer- and protocol-dependent numeric code, printed here in hexadecimal for compactness.

Experimental Procedure

1. Wire the IR receiver and LCD.
2. Install the `IRremote` library (Chapter).
3. Upload, then point any household remote at the receiver and press a button.
4. Record the hexadecimal codes for several different buttons on the same remote.

Expected Output

LCD shows a distinct hexadecimal code for each remote button pressed, e.g. illustrative values like `0x1FE48B7` (actual values depend entirely on the specific remote and protocol used, and are not predictable in advance).

Observation Table

Remote button pressed	Hex code received	Repeatable on re-press?
-----------------------	-------------------	-------------------------

Calculations and Analysis

Confirm that the same physical button on the same remote reliably produces the same (or a consistently patterned, for protocols with toggle bits) hexadecimal code across repeated presses.

Result

The IR receiver correctly decoded _____ distinct button codes from the borrowed remote, confirming receive-only functionality.

Troubleshooting

Problem	Likely cause	Fix
Nothing ever received	No remote available, or wiring fault	Confirm a working household remote is used; re-check wiring
Only the first button press works	Missing <code>IrReceiver.resume()</code>	Confirm it is called after every decode
"UNKNOWN" protocol shown	Remote uses a protocol <code>IRremote</code> cannot fully decode	Still demonstrates reception; note the raw timing is still detected

Viva Questions and Extension Challenge

Conceptual: (1) Why can this experiment not be demonstrated using only kit components? (2) What does 38 kHz modulation achieve for IR remotes? (3) What does `IrReceiver.resume()` do and why is it required? (4) Why might the same button produce slightly different raw codes on some protocols (toggle bits)? (5) What three wires does the receiver need? **Troubleshooting:** (1) A remote works from across the room during testing but not up close — is this typical? (2) The LCD shows a code but it is identical for every button — what should you check about the remote itself? **Programming:** (1) Map 3 specific known button codes to turn an LED on/off/toggle (this is developed fully in Experiment 46). (2) Print the detected protocol name alongside the hex code. **Extension Challenge:** Research (without necessarily implementing) how an Arduino with an IR LED could *transmit* codes, and explain in your lab notebook why this manual's kit deliberately excludes that hardware.

Part III

LAYER 3 — ADVANCED (Experiments 35–50)

Layer 3 — Advanced: Overview

Experiments 35–50 combine multiple Layer 1/2 circuits into coherent systems: finite-state machines for security logic, non-blocking round-robin sensor scheduling, formal hysteresis design, alarm prioritisation when several subsystems could trigger at once, and dual-LDR relative measurement. Experiment 50 is the capstone: a Mini Smart Home Automation System that integrates every sensor and actuator in the kit under one modular, non-blocking scheduler. Each experiment in this layer states its system architecture and block diagram before the circuit, since the engineering skill being taught is integration, not a new sensor.

Chapter 35

Experiment 35: Smart Room Light Controller

Experiment Identification

Experiment Number:	35
Title:	Smart Room Light Controller
Learning Layer:	Layer 3 — Advanced
Estimated Completion Time:	30 minutes
Difficulty:	Advanced
Prerequisite Experiments:	Experiment 30

Aim

To build a room light controller that automatically switches a relay-driven light demonstration in darkness, but allows a manual override button to take priority over the automatic decision — introducing mode arbitration between automatic and manual control.

Learning Objectives

1. Implement a two-mode system (AUTO/MANUAL) selected by a toggle button.
2. Design priority logic where a manual action overrides an automatic decision.
3. Apply the relay-demonstration pattern (Experiment 29) to a lighting scenario.

Required Components

Arduino UNO R3, breadboard, 1 LDR, $1 \times 10\text{ k}\Omega$ resistor, 1 push button, 1 relay module, jumper wires.

Technical Theory

System architecture: an LDR sensing stage feeds an automatic decision (dark $\Rightarrow$ light ON) exactly as in Experiment 10, but a debounced push button can toggle an independent **mode** flag between AUTO and MANUAL. In MANUAL mode, the button's last-set light state is held regardless of ambient light, giving the human operator explicit priority — a simple but important real-world pattern: automation should be overridable, not absolute.

Component Working Principle

See Experiment 10 (LDR) and Experiment 29 (relay demonstration pattern).

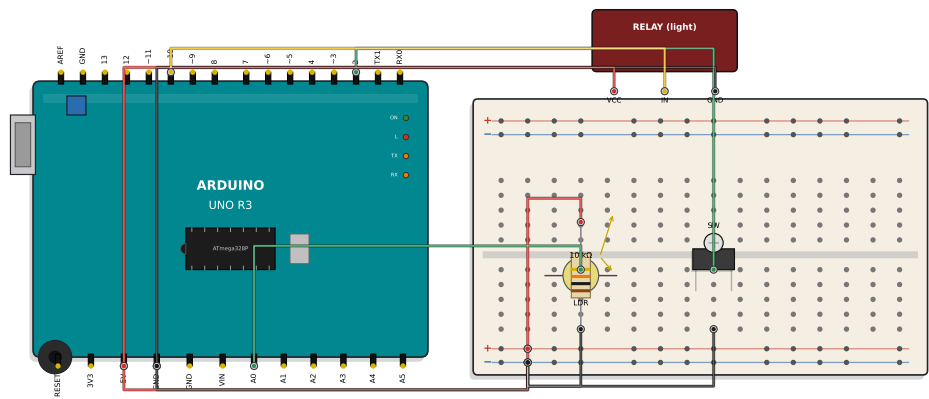


Figure 35.1: Experiment 35 circuit: LDR divider at A0, override button on D2, relay (light demo) on D10.

Connection Table

LDR divider as Experiment 10 (A0); button leg A on D2 (INPUT_PULLUP), leg B on GND; relay VCC/GND/IN on 5V/GND/D10.

Working Principle – Signal Flow

Button press → toggles **mode** between AUTO and MANUAL (and, in MANUAL, also toggles the light state directly); in AUTO mode, the LDR reading alone decides the relay state.

Algorithm

1. On a debounced button edge: if in AUTO, switch to MANUAL and turn the light ON; if in MANUAL and light is ON, turn it OFF; if in MANUAL and light is OFF, switch back to AUTO.
2. If in AUTO mode, set the relay from the LDR threshold each pass.
3. If in MANUAL mode, leave the relay exactly as last set by the button.

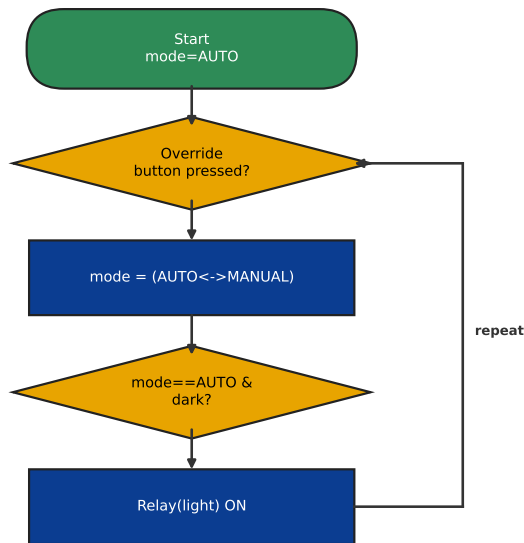


Figure 35.2: Flowchart: Experiment 35.

Arduino Program

```

1  const int LDR_PIN = A0, BUTTON_PIN = 2, RELAY_PIN = 10;
2  const int DARK_THRESHOLD = 400;
3  enum Mode { AUTO, MANUAL };
4  Mode mode = AUTO;
5  bool lightOn = false;
6
7  int lastReading = HIGH, stableState = HIGH;
8  unsigned long lastChangeTime = 0;
9  const unsigned long DEBOUNCE_MS = 30;
10
11 bool buttonEdge() {
12     int reading = digitalRead(BUTTON_PIN);
13     if (reading != lastReading) lastChangeTime = millis();
14     bool edge = false;
15     if ((millis() - lastChangeTime) > DEBOUNCE_MS && reading !=
16         stableState) {
17         stableState = reading;
18         if (stableState == LOW) edge = true;
19     }
20     lastReading = reading;
21     return edge;
22 }
23
24 void setup() {
25     pinMode(BUTTON_PIN, INPUT_PULLUP);
26     pinMode(RELAY_PIN, OUTPUT);
27 }
28
29 void loop() {
30     if (buttonEdge()) {
31         if (mode == AUTO) { mode = MANUAL; lightOn = true; }
32         else if (lightOn) { lightOn = false; }
33         else { mode = AUTO; }
34     }
35 }

```

```

34  if (mode == AUTO) {
35      lightOn = (analogRead(LDR_PIN) < DARK_THRESHOLD);
36  }
37  digitalWrite(RELAY_PIN, lightOn ? HIGH : LOW);
38  }

```

Code Explanation

The three-way button behaviour (AUTO → MANUAL-ON → MANUAL-OFF → AUTO) is a small state machine in its own right, giving the user a full manual override cycle with a single button, previewing the more general finite-state-machine treatment in Experiment 40.

Experimental Procedure

1. Wire the LDR, button, and relay.
2. Upload; confirm AUTO mode responds to light level, then press the button and confirm MANUAL mode holds the light state regardless of the LDR.

Expected Output

In AUTO mode, the relay follows the LDR threshold as in Experiment 10; after one button press, the relay latches ON (MANUAL) regardless of light; a second press turns it OFF (still MANUAL); a third press returns to AUTO.

Observation Table

Press #	Mode after press	Light state	Matches design?
1			
2			
3			

Calculations and Analysis

Not applicable numerically; instead trace through all three button presses against the algorithm and confirm the mode/state sequence matches exactly.

Result

The manual override correctly took priority over automatic control in all tested cases: _____ (Yes/No).

Troubleshooting

Problem	Likely cause	Fix
Light never leaves MANUAL	Third-press branch logic error	Re-check the <code>else</code> chain
AUTO mode ignores LDR	LDR wiring or threshold issue	Re-check Experiment 10 wiring
Button needs two presses to register	Debounce/edge bug	Re-check Experiment 7 pattern

Viva Questions and Extension Challenge

Conceptual: (1) Why give manual control priority over automation? (2) How many button presses form one full cycle? (3) What does the `mode` enum represent? (4) Why is `lightOn` only recalculated from the LDR in AUTO mode? (5) What real-world systems use similar override logic? **Troubleshooting:** (1) The relay flickers rapidly right at the AUTO threshold — what earlier fix applies? (2) Pressing the button seems to do nothing in MANUAL-OFF — what state transition might be missing? **Programming:** (1) Add a visual mode indicator LED (solid = AUTO, blinking = MANUAL). (2) Add a MANUAL mode timeout that automatically reverts to AUTO after 30 seconds. **Extension Challenge:** Extend to Experiment 48 by adding an occupancy sensor that can also request AUTO mode light-on, independent of the LDR.

Chapter 36

Experiment 36: Room Environment Monitor

Experiment Identification

Experiment Number:	36
Title:	Room Environment Monitor
Learning Layer:	Layer 3 — Advanced
Estimated Completion Time:	25 minutes
Difficulty:	Advanced
Prerequisite Experiments:	Experiments 26, 18

Aim

To display temperature and humidity as two alternating full-screen LCD pages using non-blocking page-cycling, a technique needed by every later multi-reading experiment in this manual.

Learning Objectives

1. Implement non-blocking, timed page-switching on a 2-row LCD.
2. Separate a "how often do we read the sensor" timer from a "how often do we flip the page" timer.
3. Structure display code around an explicit page-index variable.

Required Components

Arduino UNO R3, breadboard, 16×2 I²C LCD, 1 DHT11, jumper wires.

Technical Theory

A 16×2 LCD can only show two rows at once; showing more than two related readings requires cycling between "pages" over time. This experiment uses *two independent millis()* timers: one governing how often the DHT11 is actually re-read (2s, respecting its datasheet limit as in Experiment 26), and a second, faster one governing how often the displayed page flips (e.g. every 3s) — the two concerns are deliberately decoupled, since a fast-flipping display should not need to re-poll a slow sensor every flip.

Component Working Principle

See Experiment 23 (DHT11) and Experiment 18 (LCD).

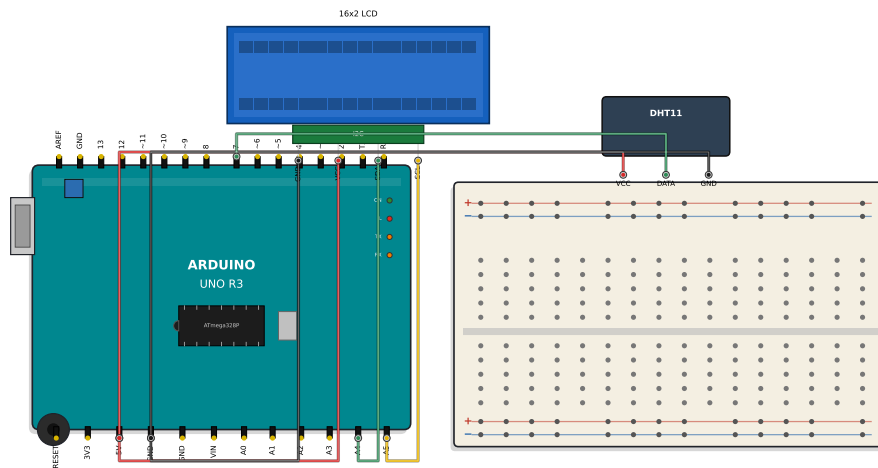


Figure 36.1: Experiment 36 circuit (identical wiring to Experiment 23).

Connection Table

Identical to Experiment 23.

Working Principle – Signal Flow

Sensor timer (2s) → refreshes cached temperature/humidity values; page timer (3s) → flips **page** between 0 and 1 and redraws the LCD from the cached values (not necessarily a fresh sensor read every flip).

Algorithm

1. Every 2s: read and cache temperature and humidity.
2. Every 3s: flip **page** (0 or 1) and redraw the LCD using the cached values for whichever page is now active.
3. Both timers run independently and concurrently.

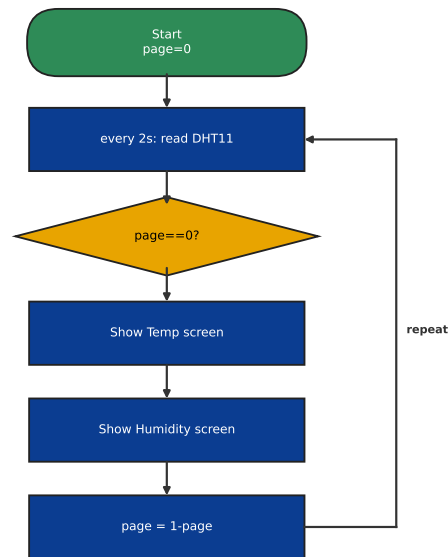


Figure 36.2: Flowchart: Experiment 36.

Arduino Program

```

1  #include <Wire.h>
2  #include <LiquidCrystal_I2C.h>
3  #include <DHT.h>
4  #define DHT_PIN 7
5  #define DHT_TYPE DHT11
6  LiquidCrystal_I2C lcd(0x27, 16, 2);
7  DHT dht(DHT_PIN, DHT_TYPE);
8
9  const unsigned long READ_INTERVAL_MS = 2000, PAGE_INTERVAL_MS = 3000;
10 unsigned long lastRead = 0, lastPage = 0;
11 float cachedTemp = 0, cachedHum = 0;
12 int page = 0;
13
14 void drawPage() {
15     lcd.clear();
16     lcd.setCursor(0, 0);
17     if (page == 0) {
18         lcd.print("Temperature");
19         lcd.setCursor(0, 1);
20         lcd.print(cachedTemp, 1); lcd.print(" C");
21     } else {
22         lcd.print("Humidity");
23         lcd.setCursor(0, 1);
24         lcd.print(cachedHum, 0); lcd.print(" %");
25     }
26 }
27
28 void setup() {
29     lcd.init(); lcd.backlight();
30     dht.begin();
31     drawPage();
32 }
33
34 void loop() {

```

```

35  if (millis() - lastRead >= READ_INTERVAL_MS) {
36      lastRead = millis();
37      float t = dht.readTemperature(), h = dht.readHumidity();
38      if (!isnan(t)) cachedTemp = t;
39      if (!isnan(h)) cachedHum = h;
40  }
41  if (millis() - lastPage >= PAGE_INTERVAL_MS) {
42      lastPage = millis();
43      page = 1 - page;
44      drawPage();
45  }
46  }

```

Code Explanation

`cachedTemp/cachedHum` decouple "when we measure" from "when we display", so a failed read (`isnan`) simply skips updating the cache rather than displaying an error mid-page — the last known good value remains visible until a valid new one arrives. `drawPage()` is a single function responsible for rendering whichever page is current, called both at startup and on every flip.

Experimental Procedure

1. Wire the DHT11 and LCD.
2. Upload and confirm the display alternates between Temperature and Humidity pages every 3 seconds, with values updating roughly every 2 seconds.

Expected Output

LCD alternates: "Temperature / NN.N C" for 3s, then "Humidity / NN

Observation Table

Page shown	Value displayed	Timing correct? (Y/N)

Calculations and Analysis

Time several page flips with a stopwatch and confirm they occur close to every 3s, independent of the 2s read interval.

Result

The two-page environment monitor correctly cycled at _____ s intervals with values refreshing every _____ s.

Troubleshooting

Problem	Likely cause	Fix
Page never flips	<code>PAGE_INTERVAL_MS</code> timer condition wrong	Re-check the second <code>if</code>
Display flickers constantly	<code>lcd.clear()</code> called too often	Confirm it is only called inside <code>drawPage()</code>
Stale values shown after a failed read	Expected; by design	Not a fault

Viva Questions and Extension Challenge

Conceptual: (1) Why use two separate timers instead of one? (2) What does caching the sensor values protect against? (3) Why is `drawPage()` factored into its own function? (4) What would happen with `PAGE_INTERVAL_MS` shorter than `READ_INTERVAL_MS`? (5) Why is `page` an `int` rather than a `bool`, given only two pages? **Troubleshooting:** (1) The displayed value briefly shows 0.0 right after a page flip — what caching bug could explain this? (2) Pages flip correctly but values never update — which timer/cache interaction should you check? **Programming:** (1) Add a third page showing both values compactly on one screen. (2) Make the page interval configurable via the potentiometer. **Extension Challenge:** Generalise `drawPage()` into an array of function pointers, one per page, so adding a new page needs no changes to the flip logic itself — this exact generalisation is used in Experiment 47.

Chapter 37

Experiment 37: Smart Plant Monitoring System

Experiment Identification

Experiment Number:	37
Title:	Smart Plant Monitoring System
Learning Layer:	Layer 3 — Advanced
Estimated Completion Time:	30 minutes
Difficulty:	Advanced
Prerequisite Experiments:	Experiments 27, 23, 18

Aim

To combine soil moisture, temperature, and humidity into one page-cycling plant-monitoring display, extending Experiment 36's page-cycling technique to three sensors and two combined pages.

Learning Objectives

1. Combine two previously separate sensor experiments (soil moisture, DHT11) into one system.
2. Extend page-cycling to more than two underlying readings.
3. Practice organising multi-sensor code into clearly separated read/cache/display concerns.

Required Components

Arduino UNO R3, breadboard, 16×2 I²C LCD, 1 soil moisture sensor, 1 DHT11, jumper wires.

Technical Theory

This experiment introduces no new sensor theory; it is a structured combination of Experiment 27 (soil) and Experiment 23/24 (DHT11) behind Experiment 36's page-cycling display pattern. The engineering skill exercised is **system composition**: reading three physical quantities on independent, appropriately-paced schedules (soil moisture is fast and noise-free enough to read every pass; DHT11 needs its 1–2s minimum interval), while presenting them coherently to the user.

Component Working Principle

See Experiment 27 (soil sensor) and Experiment 23 (DHT11).

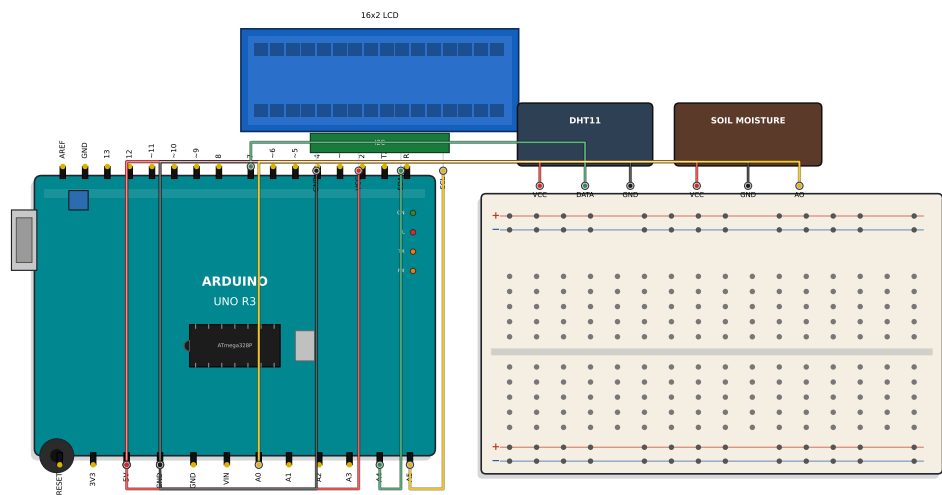


Figure 37.1: Experiment 37 circuit: DHT11 on D7, soil moisture sensor AO on A0, LCD via I²C.

Connection Table

DHT11 VCC/DATA/GND on 5V/D7/GND; soil sensor VCC/GND/AO on 5V/GND/A0; LCD as Experiment 18.

Working Principle – Signal Flow

Soil moisture read every pass (cheap, no minimum interval) → cached; DHT11 read every 2s → cached; page timer flips between "Soil" and "Temp/Humidity" pages, each drawn from its own cached values.

Algorithm

1. Every pass: update cached soil %.
2. Every 2s: update cached temperature/humidity.
3. Every 3s: flip page and redraw from cached values.

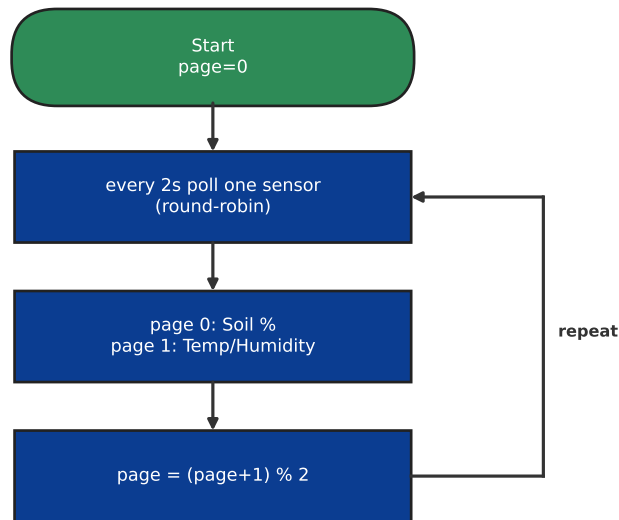


Figure 37.2: Flowchart: Experiment 37.

Arduino Program

```

1  #include <Wire.h>
2  #include <LiquidCrystal_I2C.h>
3  #include <DHT.h>
4  #define DHT_PIN 7
5  #define DHT_TYPE DHT11
6  LiquidCrystal_I2C lcd(0x27, 16, 2);
7  DHT dht(DHT_PIN, DHT_TYPE);
8  const int SOIL_PIN = A0;
9  const int DRY_RAW = 800, WET_RAW = 350;
10
11 const unsigned long READ_INTERVAL_MS = 2000, PAGE_INTERVAL_MS = 3000;
12 unsigned long lastRead = 0, lastPage = 0;
13 float cachedTemp = 0, cachedHum = 0;
14 int cachedSoil = 0, page = 0;
15
16 int soilMoisturePercent() {
17     int raw = analogRead(SOIL_PIN);
18     return constrain(map(raw, DRY_RAW, WET_RAW, 0, 100), 0, 100);
19 }
20
21 void drawPage() {
22     lcd.clear();
23     lcd.setCursor(0, 0);
24     if (page == 0) {
25         lcd.print("Soil Moisture");
26         lcd.setCursor(0, 1); lcd.print(cachedSoil); lcd.print(" %");
27     } else {
28         lcd.print(cachedTemp, 1); lcd.print("C "); lcd.print(cachedHum,
29             0); lcd.print("%RH");
30     }
31 }
32
33 void setup() {
34     lcd.init(); lcd.backlight();

```

```

34   dht.begin();
35   drawPage();
36 }
37
38 void loop() {
39   cachedSoil = soilMoisturePercent();    // cheap, safe to read every
40   pass
41
42   if (millis() - lastRead >= READ_INTERVAL_MS) {
43     lastRead = millis();
44     float t = dht.readTemperature(), h = dht.readHumidity();
45     if (!isnan(t)) cachedTemp = t;
46     if (!isnan(h)) cachedHum = h;
47   }
48   if (millis() - lastPage >= PAGE_INTERVAL_MS) {
49     lastPage = millis();
50     page = 1 - page;
51     drawPage();
52   }
53 }

```

Code Explanation

Notice `cachedSoil` is updated unconditionally every pass (no timer guard), since `analogRead()` has no minimum-interval requirement, while `cachedTemp/cachedHum` remain behind the DHT11's 2s guard — the code visibly reflects that these two sensors have different real-world timing constraints, rather than forcing both onto one schedule.

Experimental Procedure

1. Wire both sensors and the LCD.
2. Upload and confirm both pages display sensibly and update at their respective rates.

Expected Output

Page 0: "Soil Moisture / NN %"; Page 1: "NN.NC NN%RH", alternating every 3s.

Observation Table

Reading	Value	Plausible?
Soil %		
Temperature		
Humidity		

Calculations and Analysis

Confirm the soil percentage matches Experiment 27's calibration and the temperature/humidity match Experiment 23/24's readings for the same physical conditions.

Result

The combined plant-monitoring display correctly showed soil, temperature, and humidity across two cycling pages: _____ (Yes/No).

Troubleshooting

Problem	Likely cause	Fix
Soil page never updates	<code>cachedSoil</code> accidentally placed behind the 2s guard	Re-check it is outside that <code>if</code>
Temp/humidity page frozen	DHT11 read guard condition wrong	Re-check timer comparison

Viva Questions and Extension Challenge

Conceptual: (1) Why does soil moisture not need a minimum read interval like the DHT11? (2) What would happen if soil were also gated behind the 2s timer? (3) Why use a single `drawPage()` for both pages rather than two separate functions? (4) What does "caching" mean in this context? (5) How does this experiment demonstrate system composition rather than new sensor theory? **Troubleshooting:** (1) The soil reading visibly lags behind actual changes — what accidental guard should you check for? (2) Switching pages sometimes shows a blank flash — is this expected with `lcd.clear()`? **Programming:** (1) Add a third page showing all three values compactly. (2) Trigger an immediate page flip on a button press, in addition to the automatic timer. **Extension Challenge:** Preview Experiment 38 by adding a comment block outlining (not yet coding) how a buzzer and relay would be added for automatic plant protection.

Chapter 38

Experiment 38: Automatic Plant Protection System

Experiment Identification

Experiment Number:	38
Title:	Automatic Plant Protection System
Learning Layer:	Layer 3 — Advanced
Estimated Completion Time:	30 minutes
Difficulty:	Advanced
Prerequisite Experiments:	Experiments 37, 29

Aim

To close the loop on Experiment 37 by adding automatic relay-driven watering demonstration and a critical-dryness buzzer alert, forming a complete (kit-safe, demonstration-scale) plant protection system.

Learning Objectives

1. Combine sensing (Experiment 37), threshold alarm (Experiment 28), and relay actuation (Experiment 29) into one system.
2. Implement two independent severity levels (dry → water; critically dry → also alarm).
3. Practice full closed-loop system design: sense → decide → act → display.

Required Components

Arduino UNO R3, breadboard, 16×2 I²C LCD, 1 soil moisture sensor, 1 DHT11, 1 relay module, 1 active buzzer, jumper wires.

Technical Theory

This experiment defines two soil-moisture severity bands: below `DRY_LIMIT` (30%) triggers the relay-driven watering demonstration (as Experiment 29); below a lower, more urgent `CRITICAL_LIMIT` (15%) *additionally* sounds the buzzer, signalling that automatic watering alone has not been sufficient (or has not yet had time to act) and human attention is needed. This two-tier response — automatic action first, human alert only if the automatic action’s target still is not met — is a common real-world control pattern.

Component Working Principle

See Experiments 27, 28, 29, and 23 for individual component behaviour.

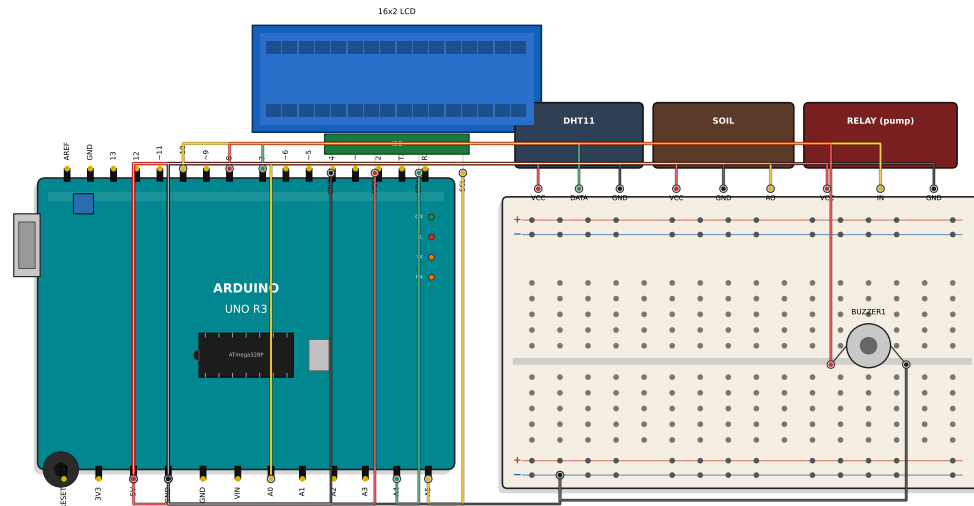


Figure 38.1: Experiment 38 circuit: DHT11, soil sensor, relay, and buzzer combined with the LCD.

Connection Table

Component	Arduino pin
DHT11 DATA	D7
Soil sensor AO	A0
Relay IN	D10
Buzzer +	D8
LCD	GND/5V/A4/A5

Working Principle – Signal Flow

Soil % < DRY_LIMIT → relay (pump-demo) ON; soil % < CRITICAL_LIMIT → additionally buzzer ON; LCD always shows current soil % and system status.

Algorithm

1. Read soil %, temperature, humidity (as Experiment 37's schedule).
2. If soil % < DRY_LIMIT, energise the relay; else de-energise it.
3. If soil % < CRITICAL_LIMIT, sound the buzzer; else silence it.
4. Display soil %, relay state, and buzzer state on the LCD.

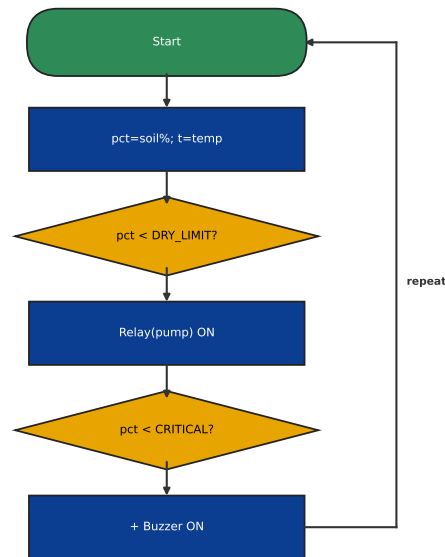


Figure 38.2: Flowchart: Experiment 38.

Arduino Program

```

1  #include <Wire.h>
2  #include <LiquidCrystal_I2C.h>
3  #include <DHT.h>
4  #define DHT_PIN 7
5  #define DHT_TYPE DHT11
6  LiquidCrystal_I2C lcd(0x27, 16, 2);
7  DHT dht(DHT_PIN, DHT_TYPE);
8  const int SOIL_PIN = A0, RELAY_PIN = 10, BUZZER_PIN = 8;
9  const int DRY_RAW = 800, WET_RAW = 350;
10 const int DRY_LIMIT = 30, CRITICAL_LIMIT = 15;
11
12 int soilMoisturePercent() {
13     int raw = analogRead(SOIL_PIN);
14     return constrain(map(raw, DRY_RAW, WET_RAW, 0, 100), 0, 100);
15 }
16
17 void setup() {
18     pinMode(RELAY_PIN, OUTPUT);
19     pinMode(BUZZER_PIN, OUTPUT);
20     lcd.init(); lcd.backlight();
21     dht.begin();
22 }
23
24 void loop() {
25     int soilPct = soilMoisturePercent();
26     bool pumpOn = (soilPct < DRY_LIMIT);
27     bool alarmOn = (soilPct < CRITICAL_LIMIT);
28
29     digitalWrite(RELAY_PIN, pumpOn ? HIGH : LOW);
30     digitalWrite(BUZZER_PIN, alarmOn ? HIGH : LOW);
31
32     lcd.setCursor(0, 0);
33     lcd.print("Soil:"); lcd.print(soilPct); lcd.print("% ");
34     lcd.setCursor(0, 1);
  
```

```

35 |   if (alarmOn)      lcd.print("CRITICAL! water!");
36 |   else if (pumpOn)  lcd.print("Watering... ");
37 |   else              lcd.print("OK ");
38 |   delay(300);
39 | }

```

Code Explanation

`pumpOn` and `alarmOn` are computed independently from the same `soilPct` reading but against two different constants, so the two actuators (relay, buzzer) never need to know about each other's thresholds directly — each output is driven by its own clearly named condition, which is easy to audit and extend.

Experimental Procedure

1. Wire all four components (DHT11, soil sensor, relay, buzzer) and the LCD.
2. Upload; test with soil samples at normal, dry, and critically dry moisture, confirming the relay and buzzer activate at the correct respective thresholds.

Expected Output

"Soil:NN% / OK" above 30%; "Watering..." with relay clicking between 15–30%; "CRITICAL! water!" with both relay and buzzer active below 15%.

Observation Table

Soil %	Relay state	Buzzer state

Calculations and Analysis

Confirm `CRITICAL_LIMIT < DRY_LIMIT` always holds, and that the buzzer never activates without the relay also being active (since $15\% < 30\%$, any critically dry reading is necessarily also "dry").

Result

The plant protection system correctly watered below _____% and alarmed below _____%.

Troubleshooting

Problem	Likely cause	Fix
Buzzer sounds without relay active	CRITICAL_LIMIT set higher than DRY_LIMIT by mistake	Re-check constant ordering
LCD status text does not match actuator states	Message <code>if/else if</code> ordering wrong	Re-check priority order (CRITICAL checked first)

Viva Questions and Extension Challenge

Conceptual: (1) Why must CRITICAL_LIMIT be lower than DRY_LIMIT? (2) Why check `alarmOn` before `pumpOn` when choosing the LCD message? (3) What real-world principle does "automatic action first, alarm only if needed" reflect? (4) Why are `pumpOn`/`alarmOn` computed independently rather than one deriving from the other? (5) What would happen if the two thresholds were equal?

Troubleshooting: (1) The system waters correctly but the LCD always shows "OK" — what message-priority bug fits this? (2) The buzzer never turns off even in wet soil — what should you check about `alarmOn`'s condition?

Programming: (1) Add hysteresis to both thresholds (Experiment 25 pattern) to avoid relay/buzzer chatter. (2) Limit continuous relay "watering" time to 10 s per activation (Experiment 29's extension challenge).

Extension Challenge: Add a temperature-based rule: if temperature is very high AND soil is dry, water more aggressively (e.g. lower the DRY_LIMIT threshold temporarily) to model evaporation-aware irrigation.

Chapter 39

Experiment 39: Smart Parking Assistant

Experiment Identification

Experiment Number:	39
Title:	Smart Parking Assistant
Learning Layer:	Layer 3 — Advanced
Estimated Completion Time:	30 minutes
Difficulty:	Advanced
Prerequisite Experiments:	Experiment 33

Aim

To build a reverse-parking-sensor-style assistant where the buzzer beeps faster as an obstacle gets closer, using non-blocking, distance-dependent beep timing instead of a simple on/off alarm.

Learning Objectives

1. Map a continuous distance reading onto a continuous beep interval.
2. Implement non-blocking intermittent beeping using `millis()` (not `delay()`, which would freeze distance sensing between beeps).
3. Extend Experiment 33's bucket-based system to a continuously variable response.

Required Components

Arduino UNO R3, breadboard, 16×2 I²C LCD, 1 HC-SR04, 1 active buzzer, jumper wires.

Technical Theory

A real reverse-parking sensor beeps slowly when far from an obstacle and increasingly rapidly (eventually continuously) as the vehicle nears it. This requires the beep *interval* itself to be a function of distance: $\text{interval} = \text{map}(d, d_{\min}, d_{\max}, \text{interval}_{\min}, \text{interval}_{\max})$, clamped to sensible bounds. Critically, beeping must not block distance measurement — if `delay()` were used to time each beep, the HC-SR04 could not be re-triggered until the beep finished, making the whole system sluggish and potentially missing rapid approach. This experiment uses the `millis()` pattern so measuring and beeping proceed concurrently.

Component Working Principle

See Experiment 16 (HC-SR04) and Experiment 12 (active buzzer).

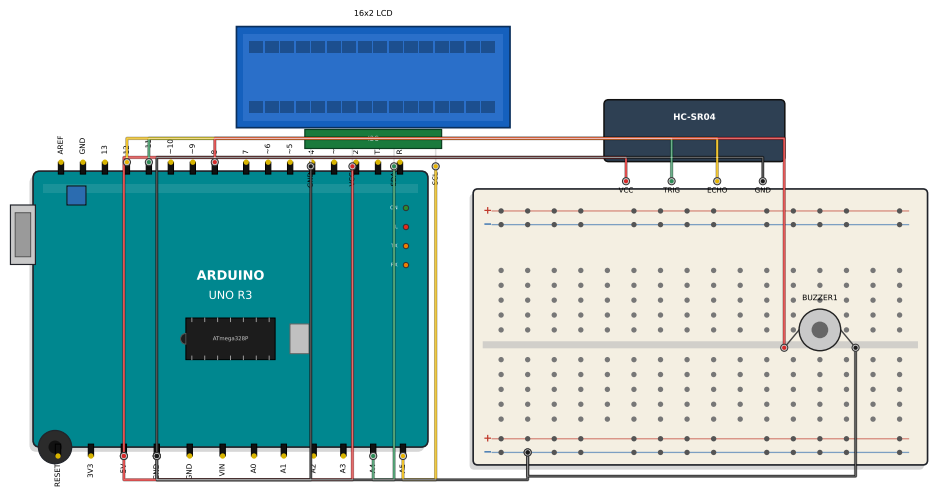


Figure 39.1: Experiment 39 circuit: HC-SR04 (TRIG=D11, ECHO=D12), buzzer on D8, LCD via I²C.

Connection Table

HC-SR04 as Experiment 33 (D11/D12); buzzer + on D8; LCD as Experiment 18.

Working Principle – Signal Flow

Distance → mapped to a beep interval (closer = shorter interval) → buzzer toggled on/off at that interval using a non-blocking timer, while the HC-SR04 continues to be measured every pass, independent of the beep timing.

Algorithm

1. Measure distance.
2. Map distance to a beep interval, clamped between a minimum and maximum.
3. If enough time has passed since the last beep toggle (per the computed interval), toggle the buzzer.
4. Repeat continuously, independent of the beep state.

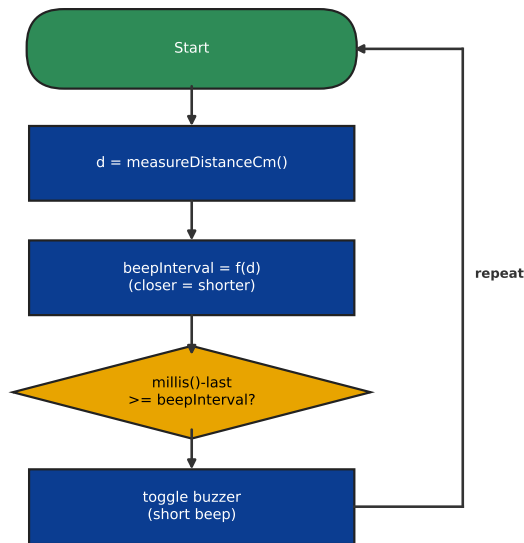


Figure 39.2: Flowchart: Experiment 39.

Arduino Program

```

1  #include <Wire.h>
2  #include <LiquidCrystal_I2C.h>
3  LiquidCrystal_I2C lcd(0x27, 16, 2);
4  const int TRIG_PIN = 11, ECHO_PIN = 12, BUZZER_PIN = 8;
5  const unsigned long TIMEOUT_US = 30000UL;
6  unsigned long lastToggle = 0;
7  bool buzzerState = false;
8
9  float measureDistanceCm() {
10     digitalWrite(TRIG_PIN, LOW); delayMicroseconds(2);
11     digitalWrite(TRIG_PIN, HIGH); delayMicroseconds(10);
12     digitalWrite(TRIG_PIN, LOW);
13     unsigned long duration = pulseIn(ECHO_PIN, HIGH, TIMEOUT_US);
14     return (duration == 0) ? 999 : duration / 58.0;
15 }
16
17 void setup() {
18     pinMode(TRIG_PIN, OUTPUT); pinMode(ECHO_PIN, INPUT);
19     pinMode(BUZZER_PIN, OUTPUT);
20     lcd.init(); lcd.backlight();
21 }
22
23 void loop() {
24     float d = measureDistanceCm();
25     d = constrain(d, 5, 100);
26     unsigned long interval = map(d, 5, 100, 80, 800); // closer =
        shorter interval (ms)
27
28     if (millis() - lastToggle >= interval) {
29         lastToggle = millis();
30         buzzerState = !buzzerState;
31         digitalWrite(BUZZER_PIN, buzzerState ? HIGH : LOW);
32     }
33 }

```

```

34  lcd.setCursor(0, 0);
35  lcd.print("Dist:"); lcd.print(d, 0); lcd.print("cm ");
36  lcd.print(d >= 100 ? "SAFE " : "      ");
37  }

```

Code Explanation

`constrain(d, 5, 100)` bounds the distance before mapping, so both very close and out-of-range (999) readings clamp to sensible beep-interval extremes rather than producing an absurd mapped value. Because beeping uses the same non-blocking `millis()` pattern as Experiment 26, the HC-SR04 is re-measured every single `loop()` pass regardless of the current beep state.

Experimental Procedure

1. Wire the HC-SR04, buzzer, and LCD.
2. Upload and slowly move an object from far to near, listening for the beep rate increasing continuously.

Expected Output

Slow, distinct beeps beyond ≈ 80 cm slowing further to silence past 100 cm ("SAFE"), increasingly rapid beeping as an object approaches, becoming very fast near 5 cm.

Observation Table

Distance (cm)	Beep interval (ms, Matches map() formula? timed)

Calculations and Analysis

Compute the expected interval at your tested distances using the `map()` formula and compare against stopwatch-timed beep intervals.

Result

The parking assistant's beep rate correctly increased continuously as distance decreased, from _____ ms at far range to _____ ms at close range.

Troubleshooting

Problem	Likely cause	Fix
Beeping freezes distance updates	<code>delay()</code> used instead of <code>millis()</code>	Re-check the non-blocking pattern is followed
Beep rate direction reversed	<code>map()</code> argument order	Re-check output range order
Constant tone instead of beeping	Interval mapped too small at all distances	Re-check <code>map()</code> bounds

Viva Questions and Extension Challenge

Conceptual: (1) Why would `delay()` be unsuitable for this beep pattern? (2) What does `constrain()` protect against here? (3) Why toggle the buzzer rather than just turning it on/off based on distance? (4) What happens exactly at $d = 100$ cm? (5) How does this generalise Experiment 33's bucket approach? **Troubleshooting:** (1) The beep interval seems to "stick" at one value across a range of distances — what `constrain()` bound might be too tight? (2) The LCD updates but the buzzer never sounds — what pin should you check first? **Programming:** (1) Add a distinct continuous tone (not beeping) below a "danger" distance (e.g. 10 cm). (2) Smooth the distance reading with a short moving average before mapping to reduce beep-rate jitter. **Extension Challenge:** Combine with Experiment 42's LED bar for a combined visual+audible warning system.

Chapter 40

Experiment 40: Electronic Security System

Experiment Identification

Experiment Number:	40
Title:	Electronic Security System
Learning Layer:	Layer 3 — Advanced
Estimated Completion Time:	30 minutes
Difficulty:	Advanced
Prerequisite Experiments:	Experiment 31

Aim

To formalise Experiment 31's informal arm/disarm/alarm logic into an explicit, documented finite-state machine (FSM) with a state diagram, a status LED, and an arming entry delay — the reusable pattern every remaining security-related experiment builds on.

Learning Objectives

1. Draw and read a formal state-transition diagram (states, transitions, guard conditions).
2. Implement an FSM in code using a `switch` statement over an `enum`.
3. Add a practical refinement (entry delay) without changing the state model's core structure.

Required Components

Arduino UNO R3, breadboard, 1 IR obstacle sensor, 1 active buzzer, 1 push button, 1 LED (status indicator), $1 \times 220\ \Omega$ resistor, jumper wires.

Technical Theory

A finite-state machine is formally defined by: a set of **states**, an initial state, a set of **events** (inputs), and a **transition function** mapping (state, event) pairs to new states. This system's states are {DISARMED, ARMED, ALARM}; events are {button press, obstacle detected}. Figure 40 shows every transition explicitly, including the fact that only a button press — never the sensor — can leave the ALARM state, exactly matching Experiment 31's latching behaviour but now documented formally rather than left implicit in code. This experiment adds one practical refinement: a short **entry delay** after arming, during which sensor triggers are ignored, giving the user time to leave the protected area.

Component Working Principle

See Experiment 15 (IR obstacle sensor) and Experiment 12 (buzzer); the status LED is a plain digital output as in Experiment 1.

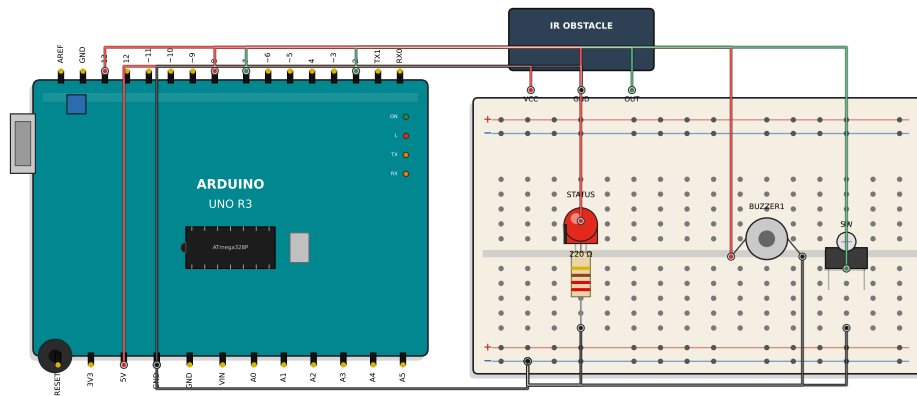


Figure 40.1: Experiment 40 circuit: IR obstacle sensor on D7, buzzer on D8, arm button on D2, status LED on D13.

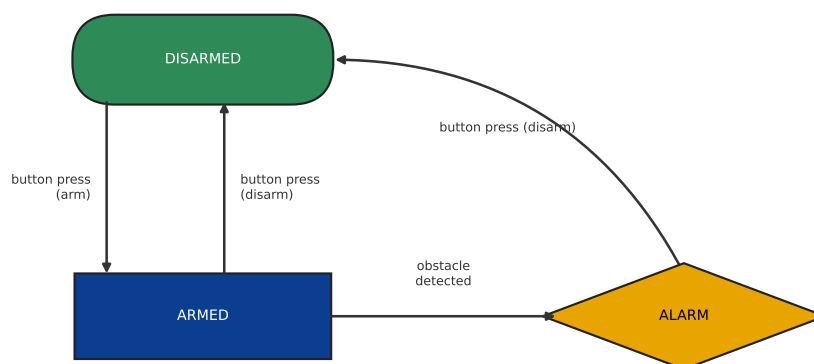


Figure 40.2: Experiment 40 state diagram: DISARMED, ARMED, and ALARM, with every transition and its triggering event.

Connection Table

IR sensor and buzzer as Experiment 31; button on D2; status LED with $220\ \Omega$ resistor on D13.

Working Principle – Signal Flow

Button press $\rightarrow$ DISARMED $\leftrightarrow$ ARMED toggle; while ARMED (after the entry delay has elapsed) an obstacle event $\rightarrow$ ALARM (latched); a button press from ALARM $\rightarrow$ DISARMED. The status LED is solid in ARMED, off in DISARMED, and blinking in ALARM.

Algorithm

1. On a debounced button press: DISARMED $\rightarrow$ ARMED (record the arming time) or ARMED/ALARM $\rightarrow$ DISARMED.
2. If ARMED and the entry delay has elapsed and the sensor triggers, go to ALARM.
3. Drive the status LED and buzzer according to the current state.
4. Repeat.

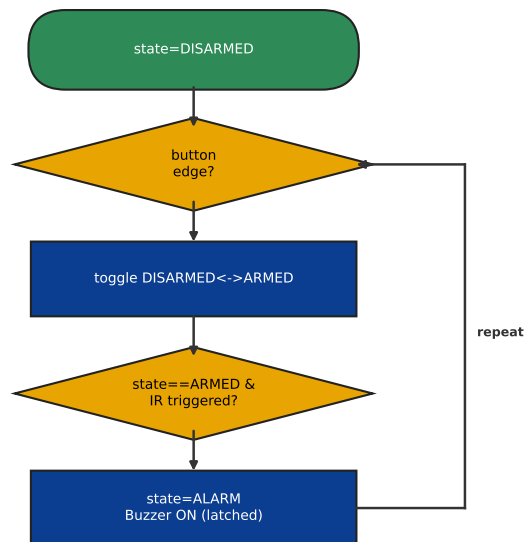


Figure 40.3: Flowchart: Experiment 40.

Arduino Program

```

1  const int IR_PIN = 7, BUZZER_PIN = 8, BUTTON_PIN = 2, LED_PIN = 13;
2  const unsigned long ENTRY_DELAY_MS = 5000;
3  enum State { DISARMED, ARMED, ALARM };
4  State state = DISARMED;
5  unsigned long armedAt = 0;
6
7  int lastReading = HIGH, stableState = HIGH;
8  unsigned long lastChangeTime = 0;
9  const unsigned long DEBOUNCE_MS = 30;
10
11 bool buttonEdge() {
12     int reading = digitalRead(BUTTON_PIN);
13     if (reading != lastReading) lastChangeTime = millis();
14     bool edge = false;

```

```

15  if ((millis() - lastChangeTime) > DEBOUNCE_MS && reading !=
16      stableState) {
17      stableState = reading;
18      if (stableState == LOW) edge = true;
19  }
20  lastReading = reading;
21  return edge;
22  }
23  void setup() {
24      pinMode(IR_PIN, INPUT);
25      pinMode(BUZZER_PIN, OUTPUT);
26      pinMode(BUTTON_PIN, INPUT_PULLUP);
27      pinMode(LED_PIN, OUTPUT);
28  }
29
30  void loop() {
31      if (buttonEdge()) {
32          switch (state) {
33              case DISARMED: state = ARMED; armedAt = millis(); break;
34              case ARMED:    state = DISARMED; break;
35              case ALARM:    state = DISARMED; break;
36          }
37      }
38
39      if (state == ARMED && (millis() - armedAt) > ENTRY_DELAY_MS
40          && digitalRead(IR_PIN) == LOW) {
41          state = ALARM;
42      }
43
44      switch (state) {
45          case DISARMED: digitalWrite(LED_PIN, LOW); digitalWrite(BUZZER_PIN
46                          , LOW); break;
47          case ARMED:    digitalWrite(LED_PIN, HIGH); digitalWrite(
48                          BUZZER_PIN, LOW); break;
49          case ALARM:    digitalWrite(LED_PIN, (millis() / 200) % 2);
49                          digitalWrite(BUZZER_PIN, HIGH); break;
50      }
51  }

```

Code Explanation

The two `switch(state)` blocks separate *transition logic* (the first, reacting to the button) from *output logic* (the second, driving the LED/buzzer purely from the current state) — a clean FSM implementation style that scales well as more states/outputs are added (used again in Experiments 41, 49, and 50). `(millis() / 200) % 2` is a compact non-blocking way to blink the LED without a separate timer variable.

Experimental Procedure

1. Wire the IR sensor, buzzer, button, and status LED.
2. Upload; press the button (LED goes solid = ARMED), wait past the 5 s entry delay, then trigger the sensor and confirm ALARM (blinking LED, buzzer), then disarm with the button.
3. Also confirm triggering the sensor *during* the entry delay does *not* cause an alarm.

Expected Output

LED off/DISARMED → solid/ARMED (ignoring the sensor for 5 s) → blinking+buzzer/ALARM once triggered after the delay → off/DISARMED on the next button press.

Observation Table

Action	State before	Event	State after

Calculations and Analysis

Confirm the entry delay is close to the programmed 5000 ms by timing from the arming button press to the moment a trigger first successfully causes ALARM.

Result

The formal FSM correctly implemented all state transitions in Figure 40, including the entry delay: _____ (Yes/No).

Troubleshooting

Problem	Likely cause	Fix
Alarm triggers immediately on arming	Entry delay check omitted or reversed	Re-check the <code>millis() - armedAt</code> comparison
LED does not blink in ALARM	Blink expression error	Re-check <code>(millis()/200)%2</code>
System never re-arms	DISARMED→ARMED transition missing	Re-check the <code>switch(state)</code> button block

Viva Questions and Extension Challenge

Conceptual: (1) What four elements formally define an FSM? (2) Why separate transition logic from output logic into two switches? (3) What is the purpose of the entry delay? (4) Why record `armedAt` rather than using a separate boolean? (5) How does Figure 40 relate directly to the code's `switch` statements? **Troubleshooting:** (1) The blink rate looks twice as fast as expected — what part of the blink expression would you check? (2) Disarming during ALARM seems to also silently re-arm — what case might be missing a distinct disarmed-vs-alarm handling? **Programming:** (1) Add an "exit delay" after a trigger before the buzzer actually sounds, to allow a final chance to disarm. (2) Log every state transition with a timestamp over Serial. **Extension Challenge:** Redraw the state diagram to add a fourth state, `ENTRY_DELAY`, making the delay an explicit state rather than a guard condition on `ARMED`, and reason about whether this is clearer.

Chapter 41

Experiment 41: Multi-Zone Security Detector

Experiment Identification

Experiment Number:	41
Title:	Multi-Zone Security Detector
Learning Layer:	Layer 3 — Advanced
Estimated Completion Time:	35 minutes
Difficulty:	Advanced
Prerequisite Experiments:	Experiment 40

Aim

To extend Experiment 40's single-sensor FSM to two independent security zones — an IR obstacle sensor (Zone 1) and a laser/LDR beam-break pair (Zone 2) — reporting which zone triggered on the LCD.

Learning Objectives

1. Build a laser-and-LDR beam-break sensor from discrete kit components.
2. Extend a single-sensor FSM to accept multiple independent trigger sources.
3. Report event provenance (which zone) rather than just an undifferentiated alarm.

Required Components

Arduino UNO R3, breadboard, 16×2 I²C LCD, 1 IR obstacle sensor, 1 laser module, 1 LDR, 1×10 kΩ resistor, 1 active buzzer, jumper wires.

Technical Theory

A **beam-break** sensor works by continuously shining a narrow light source (here, the laser module, powered continuously) onto a photosensitive receiver (the LDR) positioned across a doorway or opening; as long as the beam is unbroken, the LDR reads "bright" (low resistance); an intruder crossing the beam briefly blocks it, causing a sharp, momentary drop in LDR reading ("dark"), detected as a threshold crossing exactly like Experiment 10's logic, but interpreted here as an *event* (Zone 2 triggered) rather than a continuous state. Zone 1 (IR obstacle) and Zone 2 (laser/LDR) are electrically and logically independent — either can move the FSM from ARMED to ALARM, and the system records which one did.

Component Working Principle

See Experiment 15 (IR obstacle) and Experiment 10 (LDR); the laser module here is simply powered continuously (VCC/GND) as a fixed light source, with no signal pin control needed for this beam-break use.

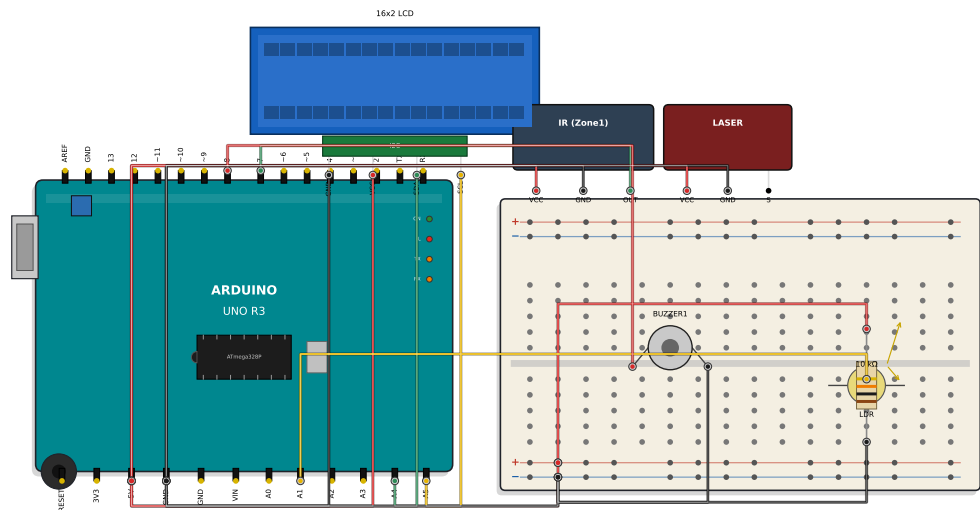


Figure 41.1: Experiment 41 circuit: Zone 1 (IR obstacle, D7), Zone 2 (laser continuously on, LDR beam receiver at A1), buzzer on D8, LCD via I²C.

Connection Table

Component	Arduino/circuit pin
IR obstacle (Zone 1) VCC/GND/OUT	5V/GND/D7
Laser module VCC/GND	5V/GND (always on)
LDR beam receiver divider	A1 (with 10 kΩ to GND)
Buzzer +	D8
LCD	GND/5V/A4/A5

Working Principle – Signal Flow

Zone 1: IR obstacle OUT LOW → Zone 1 triggered. Zone 2: LDR reading drops below a "beam broken" threshold → Zone 2 triggered. Either event, while ARMED, moves the FSM to ALARM, and the LCD reports which zone caused it.

Algorithm

1. Read Zone 1 (IR) and Zone 2 (LDR vs. threshold).
2. If ARMED and either zone triggers, go to ALARM and record which zone(s).
3. Sound the buzzer and display the triggering zone(s) on the LCD.
4. A button press (as Experiment 40) disarms.

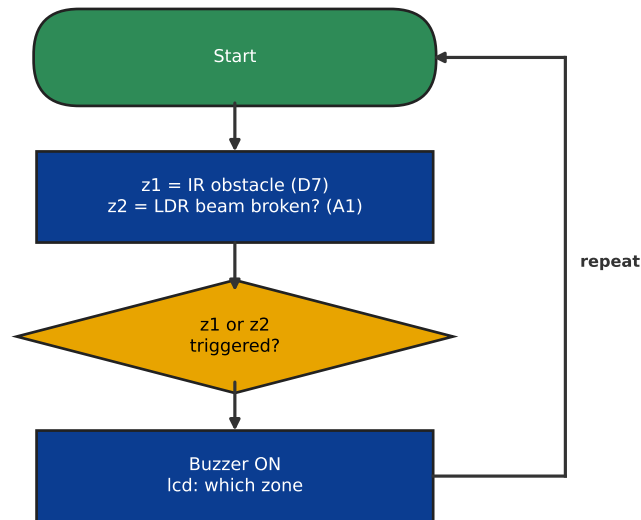


Figure 41.2: Flowchart: Experiment 41.

Arduino Program

```

1  #include <Wire.h>
2  #include <LiquidCrystal_I2C.h>
3  LiquidCrystal_I2C lcd(0x27, 16, 2);
4  const int IR_PIN = 7, LDR_PIN = A1, BUZZER_PIN = 8;
5  const int BEAM_THRESHOLD = 300;    // tune experimentally; low reading
   = beam broken
6  enum State { DISARMED, ARMED, ALARM };
7  State state = ARMED;    // simplified for this experiment: always armed
   after power-up
8  bool zone1Triggered = false, zone2Triggered = false;
9
10 void setup() {
11     pinMode(IR_PIN, INPUT);
12     pinMode(BUZZER_PIN, OUTPUT);
13     lcd.init(); lcd.backlight();
14 }
15
16 void loop() {
17     bool z1 = (digitalRead(IR_PIN) == LOW);
18     bool z2 = (analogRead(LDR_PIN) < BEAM_THRESHOLD);
19
20     if (state == ARMED && (z1 || z2)) {
21         state = ALARM;
22         zone1Triggered = z1;
23         zone2Triggered = z2;
24     }
25
26     digitalWrite(BUZZER_PIN, (state == ALARM) ? HIGH : LOW);
27
28     lcd.setCursor(0, 0);
29     if (state == ALARM) {
30         lcd.print("ALARM Zone:");
31         lcd.print(zone1Triggered ? "1" : " ");
32         lcd.print(zone2Triggered ? "2" : " ");

```

```

33 } else {
34     lcd.print("Armed - clear  ");
35 }
36 }

```

Code Explanation

This experiment deliberately keeps **state** simplified (always ARMED after power-up, no button/disarm shown) to keep the listing focused on the *multi-zone* extension; combining it with Experiment 40's full DISARMED/ARMED/entry-delay logic is a natural next step, exercised in the Extension Challenge. **zone1Triggered/zone2Triggered** are latched booleans recording which zone(s) caused the most recent alarm, so the LCD can report provenance even though both zones share one buzzer.

Experimental Procedure

1. Wire the IR sensor, laser (continuously powered), LDR divider, buzzer, and LCD.
2. Position the laser so its beam falls directly on the LDR across a small gap.
3. Upload; test triggering Zone 1 alone, Zone 2 alone (breaking the beam), and both.

Expected Output

"Armed - clear" normally; "ALARM Zone:1", "ALARM Zone: 2", or "ALARM Zone:12" depending on which zone(s) triggered, with the buzzer sounding in all ALARM cases.

Observation Table

Trigger tested	LCD zone shown	Correct?
Zone 1 only		
Zone 2 only		
Both		

Calculations and Analysis

Tune **BEAM_THRESHOLD** by recording the LDR's raw reading with the beam intact versus broken, choosing a value clearly between the two (as in Experiment 10's calibration approach).

Result

The multi-zone detector correctly identified the triggering zone(s) in _____ out of 3 tested scenarios.

Troubleshooting

Problem	Likely cause	Fix
Zone 2 never triggers	Laser misaligned with LDR, or threshold untuned	Realign; recalibrate threshold
Zone 2 triggers randomly	Ambient light interference	Shield the LDR from stray light
Both zones always show triggered	Booleans not reset appropriately	Reset before each independent test (power-cycle)

Viva Questions and Extension Challenge

Conceptual: (1) How does a beam-break sensor differ operationally from the IR obstacle sensor? (2) Why keep the laser powered continuously rather than pulsed here? (3) Why record which zone triggered rather than only a single alarm flag? (4) What could cause false triggers on Zone 2 specifically? (5) Why is this "multi-zone" rather than just "two sensors"? **Troubleshooting:** (1) Zone 2 triggers every few seconds even with the beam clearly intact — what environmental factor is likely? (2) The alarm never clears even after fixing the beam — what state-latching behaviour explains this (compare to Experiment 40)? **Programming:** (1) Integrate this experiment's two zones into Experiment 40's full DISARMED/ARMED/entry-delay FSM. (2) Add an event counter per zone, displayed on a second LCD page. **Extension Challenge:** Add a third zone using the second LDR (kit stock: 2 LDRs) as an independent ambient-light tamper detector (e.g. alarms if ambient light suddenly changes a lot, suggesting a light was turned on/off nearby).

Chapter 42

Experiment 42: Digital Distance Warning System

Experiment Identification

Experiment Number:	42
Title:	Digital Distance Warning System
Learning Layer:	Layer 3 — Advanced
Estimated Completion Time:	30 minutes
Difficulty:	Advanced
Prerequisite Experiments:	Experiments 16, 39

Aim

To build a multi-threshold ultrasonic warning system with formal hysteresis at each boundary and alarm priority so only the single most urgent LED/buzzer condition is ever active at once.

Learning Objectives

1. Apply hysteresis independently at two threshold boundaries.
2. Implement alarm priority: when multiple conditions could apply, only the most severe one wins.
3. Combine LED-bar indication with buzzer alarm restricted to the most severe tier only.

Required Components

Arduino UNO R3, breadboard, 1 HC-SR04, 3 LEDs (green/yellow/red), $3 \times 220\ \Omega$ resistors, 1 active buzzer, jumper wires.

Technical Theory

Experiment 33's buckets used hard boundaries, which can chatter between adjacent LEDs near a boundary. This experiment applies a margin at each boundary (as in Experiment 25) independently, and additionally enforces **alarm priority**: the buzzer sounds only in the most severe (closest) tier, never in the middle or far tiers, even though the middle tier is also "not fully safe" — a deliberate design decision to avoid alarm fatigue from constant noise in a merely-cautionary zone.

Component Working Principle

See Experiment 16 (HC-SR04), Experiment 25 (hysteresis), and Experiment 33 (multi-LED bucket pattern).

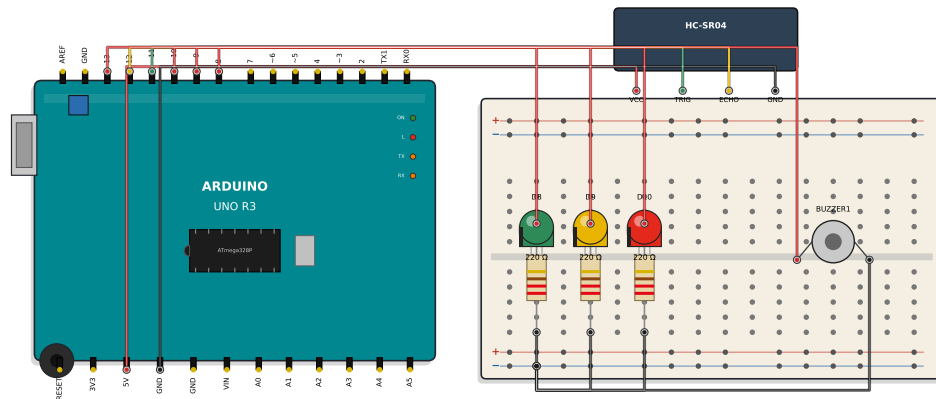


Figure 42.1: Experiment 42 circuit: HC-SR04, 3-LED bar, and buzzer.

Connection Table

HC-SR04 TRIG/ECHO on D11/D12; green/yellow/red LEDs on D8/D9/D10; buzzer + on D13.

Working Principle – Signal Flow

Distance → hysteresis-filtered tier (FAR/NEAR/CLOSE) → matching LED lit; buzzer sounds only while the tier is CLOSE.

Algorithm

1. Measure distance.
2. Apply independent hysteresis bands at the FAR/NEAR and NEAR/CLOSE boundaries to determine the current tier, only changing tier once a boundary is crossed by a clear margin.
3. Light the LED matching the current tier; sound the buzzer only if CLOSE.

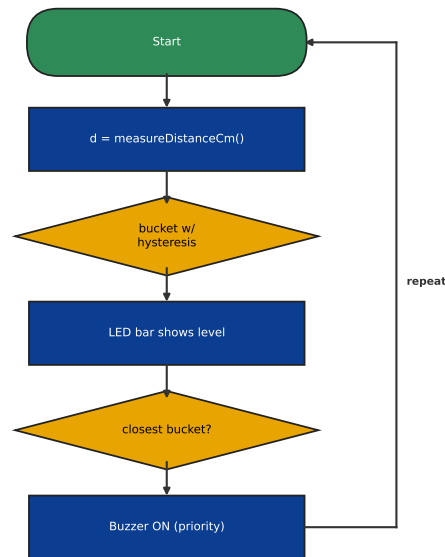


Figure 42.2: Flowchart: Experiment 42.

Arduino Program

```

1  const int TRIG_PIN = 11, ECHO_PIN = 12;
2  const int GREEN = 8, YELLOW = 9, RED = 10, BUZZER_PIN = 13;
3  const unsigned long TIMEOUT_US = 30000UL;
4  enum Tier { FAR, NEAR, CLOSE };
5  Tier tier = FAR;
6
7  float measureDistanceCm() {
8      digitalWrite(TRIG_PIN, LOW); delayMicroseconds(2);
9      digitalWrite(TRIG_PIN, HIGH); delayMicroseconds(10);
10     digitalWrite(TRIG_PIN, LOW);
11     unsigned long duration = pulseIn(ECHO_PIN, HIGH, TIMEOUT_US);
12     return (duration == 0) ? 999 : duration / 58.0;
13 }
14
15 void setup() {
16     pinMode(TRIG_PIN, OUTPUT); pinMode(ECHO_PIN, INPUT);
17     pinMode(GREEN, OUTPUT); pinMode(YELLOW, OUTPUT); pinMode(RED, OUTPUT);
18     pinMode(BUZZER_PIN, OUTPUT);
19 }
20
21 void loop() {
22     float d = measureDistanceCm();
23
24     if (tier == FAR && d < 28)          tier = NEAR;
25     else if (tier == NEAR && d > 32)    tier = FAR;
26     else if (tier == NEAR && d < 8)    tier = CLOSE;
27     else if (tier == CLOSE && d > 12)  tier = NEAR;
28
29     digitalWrite(GREEN, tier == FAR);
30     digitalWrite(YELLOW, tier == NEAR);
31     digitalWrite(RED, tier == CLOSE);
32     digitalWrite(BUZZER_PIN, tier == CLOSE); // buzzer: CLOSE tier
                                              only

```


Code Explanation

Each hysteresis band (30±2 cm and 10±2 cm) only changes **tier** when the reading clearly crosses into the next band from the *current* tier's perspective, exactly generalising Experiment 25's single-boundary margin to two boundaries. `digitalWrite(GREEN, tier == FAR)` relies on the boolean expression's implicit 0/1 value, a compact idiom for "exactly one of three outputs active".

Experimental Procedure

1. Wire the HC-SR04, 3-LED bar, and buzzer.
2. Upload and slowly move an object through all three tiers, confirming clean transitions with no rapid flicker at the boundaries, and that the buzzer sounds only in the closest tier.

Expected Output

Green beyond ≈ 30 cm, yellow between ≈ 10 – 30 cm, red (with buzzer) within ≈ 10 cm, with no chatter at either boundary.

Observation Table

[illegible]

Calculations and Analysis

Confirm the observed tier-change distance while approaching differs (by roughly the margin) from the tier-change distance while retreating, demonstrating hysteresis is actually in effect (unlike Experiment 33's hard boundaries).

Result

The distance warning system showed chatter-free tier transitions with the buzzer active only in the CLOSE tier: _____ (Yes/No).

Troubleshooting

Problem	Likely cause	Fix
LEDs still flicker at a boundary	Margin too small	Increase the hysteresis margin
Buzzer sounds in NEAR tier too	Buzzer condition not restricted to CLOSE	Re-check <code>digitalWrite(BUZZER_PIN, ...)</code>
Tier never changes	Comparison operators/tier-specific guards wrong	Re-check each <code>if/else if</code> applies only from its stated current tier

Viva Questions and Extension Challenge

Conceptual: (1) Why does each `if` check the current `tier` before allowing a transition? (2) Why restrict the buzzer to CLOSE only? (3) What would happen with a single shared threshold instead of independent margins per boundary? (4) How does `digitalWrite(GREEN, tier==FAR)` work without an explicit HIGH/LOW? (5) Why is this system's design called "priority-based"? **Troubleshooting:** (1) The yellow LED never lights, jumping straight from green to red — what margin values should you check? (2) The buzzer stays on even in the NEAR tier occasionally — what condition might be too broad? **Programming:** (1) Add a fourth "very close" tier with a distinct rapid-beep pattern (reusing Experiment 39). (2) Log tier-change events with timestamps over Serial. **Extension Challenge:** Combine with Experiment 39's continuous beep-rate mapping so the CLOSE tier also varies its beep rate with exact distance, rather than a fixed steady tone.

Chapter 43

Experiment 43: Light Intensity Measurement System

Experiment Identification

Experiment Number:	43
Title:	Light Intensity Measurement System
Learning Layer:	Layer 3 — Advanced
Estimated Completion Time:	25 minutes
Difficulty:	Advanced
Prerequisite Experiments:	Experiment 20

Aim

To compare ambient light at two separate physical locations using two LDRs simultaneously, displaying both raw readings and which side is brighter — a relative, comparative measurement rather than a single absolute one.

Learning Objectives

1. Wire and read two independent LDR dividers on separate analog pins.
2. Perform a relative (comparative) rather than absolute measurement.
3. Recognise the kit-stock constraint (exactly 2 LDRs) this experiment exercises fully.

Required Components

Arduino UNO R3, breadboard, 2 LDRs, $2 \times 10\text{ k}\Omega$ resistors, $16 \times 2\text{ I}^2\text{C}$ LCD, jumper wires.

Technical Theory

Each LDR divider (Experiment 10) is read independently on its own analog pin; because both share the same resistor value, supply voltage, and (approximately) LDR characteristics, a direct comparison of their raw ADC values gives a meaningful *relative* brightness measurement (which side receives more light) even though neither reading alone is a calibrated lux value (Experiment 20). This is a common practical technique: relative comparisons between identically-built sensors are often more reliable than trusting either sensor's absolute calibration.

Component Working Principle

See Experiment 10 for LDR divider operation.

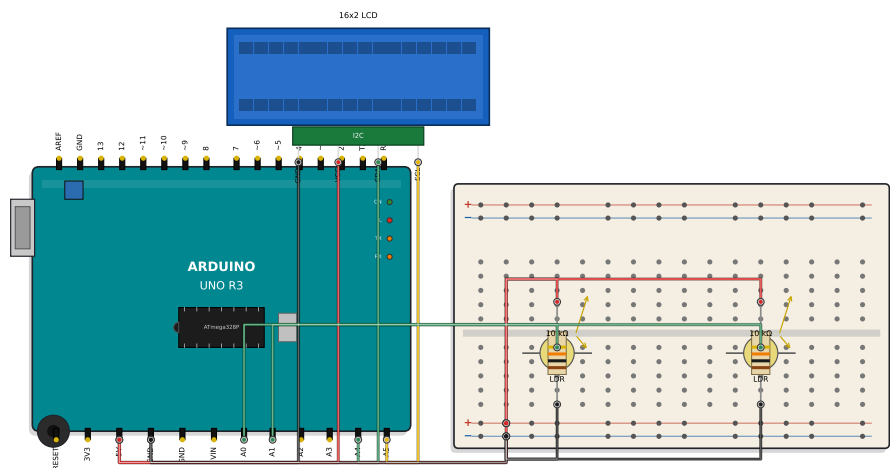


Figure 43.1: Experiment 43 circuit: two independent LDR dividers at A0 (left) and A1 (right), LCD via I²C.

Connection Table

Component	Arduino pin	Notes
LDR 1 divider (with 10 kΩ)	A0	"Left" sensor
LDR 2 divider (with 10 kΩ)	A1	"Right" sensor
LCD	GND/5V/A4/A5	

Working Principle – Signal Flow

Both A0 and A1 read every pass → compared directly → LCD reports both raw values and which side currently reads brighter.

Algorithm

1. Read A0 (left) and A1 (right).
2. Compare the two readings.
3. Display both values and the brighter side on the LCD.
4. Repeat.

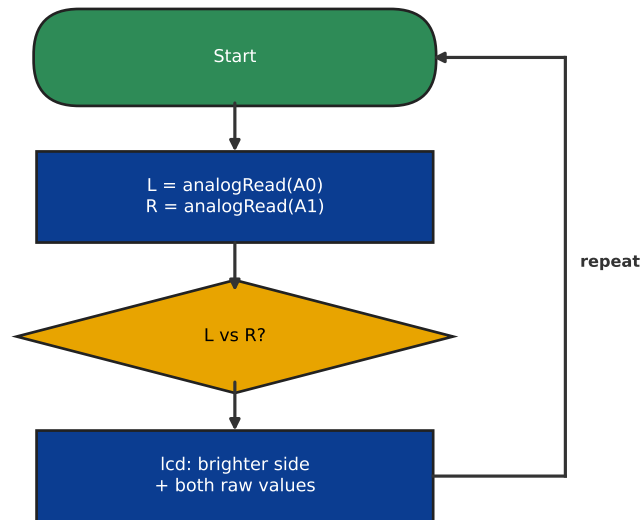


Figure 43.2: Flowchart: Experiment 43.

Arduino Program

```

1  #include <Wire.h>
2  #include <LiquidCrystal_I2C.h>
3  LiquidCrystal_I2C lcd(0x27, 16, 2);
4  const int LEFT_PIN = A0, RIGHT_PIN = A1;
5
6  void setup() {
7    lcd.init(); lcd.backlight();
8  }
9
10 void loop() {
11   int left = analogRead(LEFT_PIN);
12   int right = analogRead(RIGHT_PIN);
13
14   lcd.setCursor(0, 0);
15   lcd.print("L:"); lcd.print(left); lcd.print(" R:"); lcd.print(right);
16   lcd.setCursor(0, 1);
17   if (abs(left - right) < 20) {
18     lcd.print("Similar");
19   } else if (left > right) {
20     lcd.print("Left brighter");
21   } else {
22     lcd.print("Right brighter");
23   }
24   delay(300);
25 }

```

Code Explanation

`abs(left - right) < 20` adds a "similar" dead-band, so small, insignificant differences (within normal analog noise) do not falsely claim one side is brighter — otherwise the display could flicker between "Left brighter" and "Right brighter" from noise alone, even under genuinely even lighting.

Experimental Procedure

1. Wire two independent LDR dividers and the LCD.
2. Upload; test with even lighting on both LDRs, then shade one, confirming the LCD correctly reports the brighter side.

Expected Output

"L:NNN R:NNN" on row 0; "Similar", "Left brighter", or "Right brighter" on row 1, changing correctly as one LDR is shaded.

Observation Table

Condition	Left raw	Right raw	LCD verdict

Calculations and Analysis

Confirm the dead-band threshold (20) is appropriate: too small causes flicker under even lighting; too large fails to detect a real, deliberate difference. Discuss your chosen value's trade-off.

Result

The dual-LDR system correctly identified the brighter side in _____ out of _____ tested conditions.

Troubleshooting

Problem	Likely cause	Fix
Always reports one side brighter	The two LDR units have inherently different bright-state resistance	Note this as a real limitation of relative uncalibrated sensors
Flickers between verdicts under even light	Dead-band too small	Increase the threshold
One channel always reads 0 or 1023	Miswired divider	Re-check Experiment 10 wiring for that channel

Viva Questions and Extension Challenge

Conceptual: (1) Why is this a relative rather than absolute measurement? (2) Why might two LDR units disagree even under identical light? (3) What does the dead-band protect against? (4) Why does the kit's 2-LDR stock limit make this exactly a 2-sensor comparison? (5) Could you extend this to 3+ LDRs on this Arduino? **Troubleshooting:** (1) One side is always reported brighter even when you swap the two LDRs physically — what does this suggest? (2) The verdict flickers rapidly even under stable, even lighting — what value should you increase? **Programming:** (1) Display the numeric difference alongside the verdict. (2) Add a simple bar-graph comparison using LCD characters for both channels. **Extension Challenge:** Use the comparison to drive a simple "light-seeking" indicator LED pair (left LED lights if left is brighter, right LED if right is brighter) as a conceptual preview of light-following robotics.

Chapter 44

Experiment 44: Automatic Day/Night Controller

Experiment Identification

Experiment Number:	44
Title:	Automatic Day/Night Controller
Learning Layer:	Layer 3 — Advanced
Estimated Completion Time:	25 minutes
Difficulty:	Advanced
Prerequisite Experiments:	Experiments 30, 35

Aim

To formalise hysteresis as a small, reusable, generalised function — rather than the inline margin checks of Experiments 25/38/42 — and apply it to a relay-driven day/night light controller as a clean reference implementation.

Learning Objectives

1. Design a general-purpose hysteresis function reusable across future experiments.
2. Explain hysteresis width selection as an engineering trade-off (responsiveness vs. chatter immunity).
3. Apply the reusable function to a new scenario without duplicating its logic.

Required Components

Arduino UNO R3, breadboard, 1 LDR, 1×10 kΩ resistor, 1 relay module, jumper wires.

Technical Theory

Every prior alarm/threshold experiment (25, 38, 42) hand-wrote its own margin logic inline. This experiment factors that pattern into one reusable function:

$$\text{hysteresis}(\text{value}, \text{state}, \text{low}, \text{high}) = \begin{cases} \text{true} & \text{if state} = \text{false} \text{ and value} < \text{low} \\ \text{false} & \text{if state} = \text{true} \text{ and value} > \text{high} \\ \text{state} & \text{otherwise (unchanged)} \end{cases} \quad (44.1)$$

The gap (high – low) is chosen wide enough to exceed normal sensor noise (avoiding chatter) but narrow enough that the system still responds promptly to genuine changes — too wide a

gap makes the system sluggish; too narrow reintroduces chatter. This single function is reused, unchanged, in Experiment 45.

Component Working Principle

See Experiment 10 (LDR) and Experiment 29 (relay demonstration).

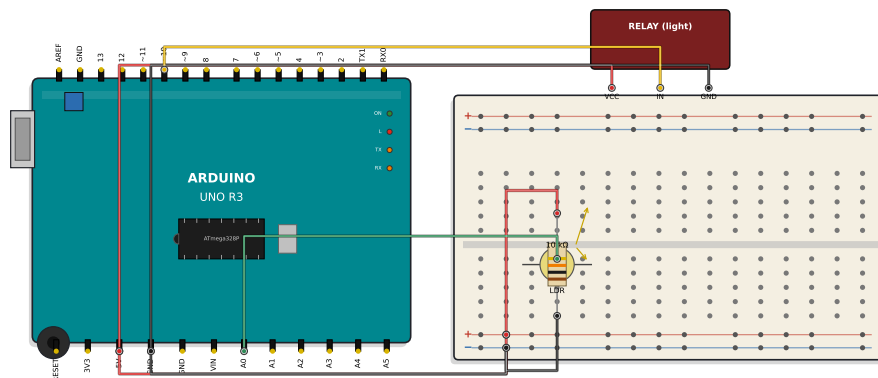


Figure 44.1: Experiment 44 circuit: LDR divider at A0, relay (light demonstration) on D10.

Connection Table

LDR divider as Experiment 10 (A0); relay VCC/GND/IN on 5V/GND/D10.

Working Principle – Signal Flow

LDR reading → passed through the reusable `hysteresis()` function together with the current light state → returns the (possibly unchanged) new state → drives the relay.

Algorithm

1. Read the LDR.
2. Call `hysteresis(reading, lightOn, LOW_THRESH, HIGH_THRESH)` to get the updated light state.
3. Drive the relay from the returned state.
4. Repeat.

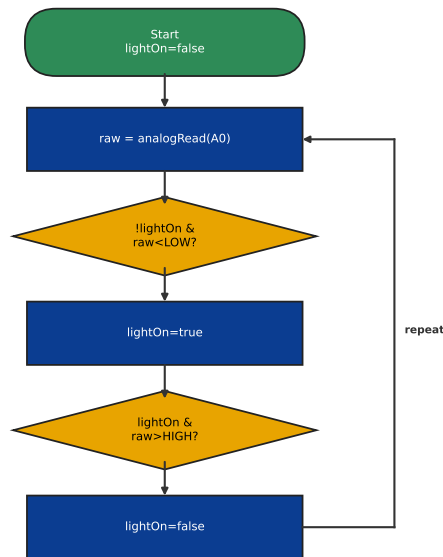


Figure 44.2: Flowchart: Experiment 44.

Arduino Program

```

1  const int LDR_PIN = A0, RELAY_PIN = 10;
2  const int LOW_THRESH = 350, HIGH_THRESH = 450;    // 100-code
   hysteresis gap
3  bool lightOn = false;
4
5  bool hysteresis(int value, bool state, int low, int high) {
6    if (!state && value < low) return true;    // turn ON once clearly
   dark
7    if (state && value > high) return false;    // turn OFF once clearly
   bright
8    return state;                                // otherwise: no change
9  }
10
11 void setup() {
12   pinMode(RELAY_PIN, OUTPUT);
13 }
14
15 void loop() {
16   int reading = analogRead(LDR_PIN);
17   lightOn = hysteresis(reading, lightOn, LOW_THRESH, HIGH_THRESH);
18   digitalWrite(RELAY_PIN, lightOn ? HIGH : LOW);
19 }

```

Code Explanation

`hysteresis()` takes the *current* state as a parameter and returns the (possibly identical) new state, keeping it a pure function with no hidden internal memory — the calling code owns `lightOn` and decides what to do with the result, which is exactly why the same function can be reused in Experiment 45 for an entirely different physical quantity (temperature) without modification.

1. Wire the LDR divider and relay.
2. Upload; slowly dim and brighten the room (or shade/uncover the LDR) and confirm the relay switches once clearly past each threshold, without chattering in between.

Relay switches ON once the reading drops below 350 and stays on until the reading rises above 450, with no rapid clicking in the 350–450 band.

[illegible]

Confirm the relay’s ON-transition and OFF-transition readings differ by roughly the 100-code gap, unlike Experiment 10’s single-threshold chatter-prone design.

The reusable hysteresis function correctly prevented relay chatter across the 350–450 ADC band:
_____ (Yes/No).

Problem	Likely cause	Fix
Relay still chatters	Gap too narrow for actual sensor noise	Widen the LOW/HIGH threshold gap
Relay feels sluggish to respond	Gap too wide	Narrow the gap
Relay never turns off	high threshold unreachable in your lighting	Re-check thresholds against your room

Conceptual: (1) Why does `hysteresis()` take `state` as a parameter rather than storing it internally? (2) What determines a good hysteresis gap width? (3) How does this differ from Experiment 25's inline approach, functionally? (4) Why is this function reusable for temperature (Experiment 45) unchanged? (5) What happens if `low > high` by mistake? **Troubleshooting:** (1) The relay switches immediately at a single value with no apparent gap — what constants should you check? (2) Different LDR units need different threshold values — why doesn't the function itself need to change? **Programming:** (1) Make the gap width configurable via the potentiometer for live tuning. (2) Add a Serial log of every state change with the triggering

ADC value. **Extension Challenge:** Generalise `hysteresis()` to work with `float` readings (for direct reuse with DHT11 temperature in Experiment 45) and verify both integer and float call sites compile correctly.

Chapter 45

Experiment 45: Temperature-Controlled System

Experiment Identification

Experiment Number:	45
Title:	Temperature-Controlled System
Learning Layer:	Layer 3 — Advanced
Estimated Completion Time:	25 minutes
Difficulty:	Advanced
Prerequisite Experiments:	Experiments 25, 29, 44

Aim

To build a temperature-controlled relay demonstration (a "fan/heater" stand-in) by reusing Experiment 44's hysteresis function unchanged on a DHT11 temperature reading, proving the function generalises across sensor types.

Learning Objectives

1. Reuse a previously written, generalised function on a new physical quantity.
2. Adapt `hysteresis()` to floating-point sensor data.
3. Combine LCD status display with relay-driven closed-loop control.

Required Components

Arduino UNO R3, breadboard, 16×2 I²C LCD, 1 DHT11, 1 relay module, jumper wires.

Technical Theory

No new control theory is introduced here beyond Experiment 44; the point is precisely that `hysteresis()`, once written generally (accepting a float value and float thresholds), *needs no modification* to control a fan/heater demonstration from temperature instead of a lamp from light level. This is what "reusable code" means in practice: the same verified logic, applied to a new problem, with only the calling context (variable names, threshold values, physical meaning) changing.

Component Working Principle

See Experiment 23 (DHT11) and Experiment 29 (relay demonstration).

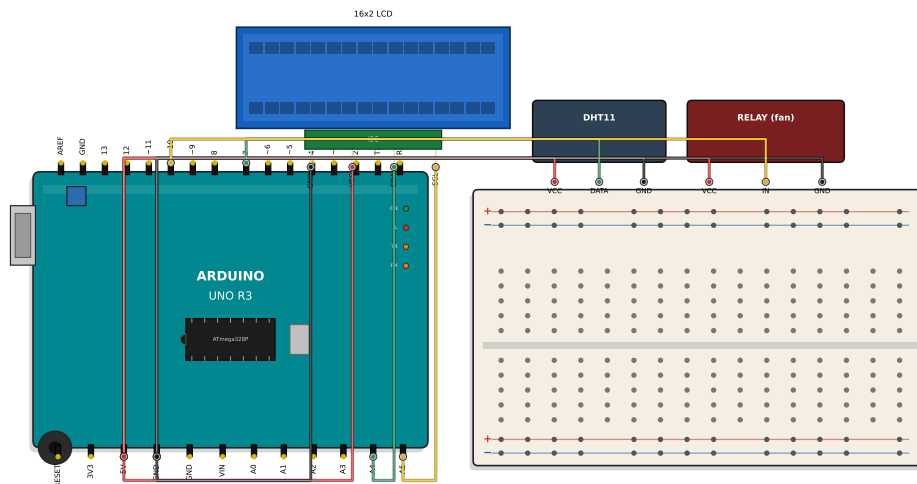


Figure 45.1: Experiment 45 circuit: DHT11 on D7, relay (fan/heater demonstration) on D10, LCD via I²C.

Connection Table

DHT11 as Experiment 23 (D7); relay VCC/GND/IN on 5V/GND/D10; LCD as Experiment 18.

Working Principle – Signal Flow

Temperature reading → passed through the (float-adapted) `hysteresis()` function together with the current fan state → drives the relay; LCD shows temperature and fan state.

Algorithm

1. Read temperature (respecting the DHT11's minimum read interval).
2. Call `hysteresis(t, fanOn, LOW_TEMP, HIGH_TEMP)`.
3. Drive the relay from the returned state; display temperature and fan state.

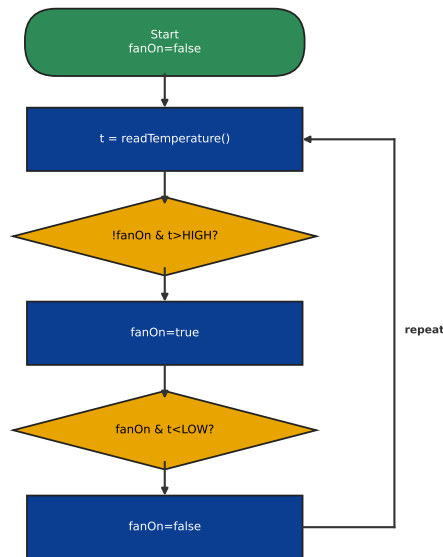


Figure 45.2: Flowchart: Experiment 45.

Arduino Program

```

1  #include <Wire.h>
2  #include <LiquidCrystal_I2C.h>
3  #include <DHT.h>
4  #define DHT_PIN 7
5  #define DHT_TYPE DHT11
6  LiquidCrystal_I2C lcd(0x27, 16, 2);
7  DHT dht(DHT_PIN, DHT_TYPE);
8  const int RELAY_PIN = 10;
9  const float LOW_TEMP = 28.0, HIGH_TEMP = 30.0;    // 2C hysteresis gap
10 const unsigned long READ_INTERVAL_MS = 2000;
11 unsigned long lastRead = 0;
12 bool fanOn = false;
13 float lastTemp = 0;
14
15 bool hysteresis(float value, bool state, float low, float high) {
16     if (!state && value > high) return true;    // turn fan ON once
17         clearly hot
18     if (state && value < low) return false;    // turn fan OFF once
19         clearly cool
20     return state;
21 }
22
23 void setup() {
24     pinMode(RELAY_PIN, OUTPUT);
25     lcd.init(); lcd.backlight();
26     dht.begin();
27 }
28
29 void loop() {
30     if (millis() - lastRead >= READ_INTERVAL_MS) {
31         lastRead = millis();
32         float t = dht.readTemperature();
33         if (!isnan(t)) lastTemp = t;
34         fanOn = hysteresis(lastTemp, fanOn, LOW_TEMP, HIGH_TEMP);
35     }
36 }

```

```

33     digitalWrite(RELAY_PIN, fanOn ? HIGH : LOW);
34
35     lcd.setCursor(0, 0);
36     lcd.print("T:"); lcd.print(lastTemp, 1);
37     lcd.print(fanOn ? " FAN ON " : " FAN OFF");
38 }
39 }

```

Code Explanation

Note the `hysteresis()` function body here is logically the *mirror image* of Experiment 44's (turning ON above a high threshold rather than below a low one, since heat rising is the "trigger" condition rather than darkness) — demonstrating that the same general pattern serves both "turn on when a value gets low" and "turn on when a value gets high" scenarios, just by swapping which comparison is used with which threshold.

Experimental Procedure

1. Wire the DHT11, relay, and LCD.
2. Upload; warm the DHT11 gently to cross 30°C and confirm the relay activates, then cool it below 28°C and confirm it deactivates.

Expected Output

LCD shows "T:NN.N FAN OFF" normally, switching to "T:NN.N FAN ON" once temperature exceeds 30°C, remaining on until it drops below 28°C.

Observation Table

Temperature (°C)	Fan/relay state	Matches design?

Calculations and Analysis

Compare this experiment's transition temperatures against Experiment 25's, noting that both use a 2°C gap but arrived at via a shared, reusable function here rather than duplicated inline logic.

Result

The temperature-controlled relay activated at _____°C and deactivated at _____°C, using the reused hysteresis function unmodified.

Troubleshooting

Problem	Likely cause	Fix
Relay never activates	LOW/HIGH temperature thresholds unreachable in your room	Warm the sensor directly to test
Fan state and LCD disagree	Update ordering bug	Re-check <code>fanOn</code> is set before the LCD print

Viva Questions and Extension Challenge

Conceptual: (1) What changed in `hysteresis()`'s body compared to Experiment 44, and why? (2) Why does reusing the same function (with a mirrored comparison) still count as "the same pattern"? (3) Why gate the fan decision behind the DHT11 read interval? (4) What would happen with `LOW_TEMP > HIGH_TEMP`? (5) Why use `float` parameters here versus `int` in Experiment 44? **Troubleshooting:** (1) The fan turns on but the LCD still shows "FAN OFF" momentarily — what statement ordering issue could cause this? (2) The relay clicks rapidly near 29°C — what gap width should you revisit? **Programming:** (1) Make the two thresholds configurable via Serial input. (2) Add a minimum ON-time for the relay (e.g. at least 30s) to protect a real fan motor's start/stop cycling (a common real HVAC practice), even though this kit demonstrates it with an LED/relay only. **Extension Challenge:** Combine Experiments 44 and 45 into one sketch controlling two independent relays (light and fan) from two independent hysteresis calls sharing one function.

Chapter 46

Experiment 46: IR Remote Smart Controller

Experiment Identification

Experiment Number:	46
Title:	IR Remote Smart Controller
Learning Layer:	Layer 3 — Advanced
Estimated Completion Time:	30 minutes
Difficulty:	Advanced
Prerequisite Experiments:	Experiment 34

Aim

To use decoded codes from a household IR remote to independently toggle an LED and a relay-driven demonstration load, turning Experiment 34's passive decoder into an interactive controller.

Learning Objectives

1. Map specific, previously-recorded IR codes to specific actions.
2. Implement independent toggle state per output, driven by distinct remote buttons.
3. Handle repeated/held-button IR codes without unwanted rapid re-toggling.

Required Components

Arduino UNO R3, breadboard, 16×2 I²C LCD, 1 IR receiver, 1 LED, 1×220 Ω resistor, 1 relay module, jumper wires, plus the same household remote used in Experiment 34.

Technical Theory

Building on Experiment 34, this experiment hard-codes the specific hexadecimal values recorded from two chosen remote buttons (recorded during that experiment's procedure) and maps each to a distinct action. Many remotes send a special "repeat" code (rather than resending the full code) while a button is held down; this experiment ignores repeat codes for toggle actions, since a toggle should fire once per discrete press, not repeatedly while held — an IR-domain analogue of Experiment 7's debounced edge detection.

Component Working Principle

See Experiment 34 for IR receiver operation.

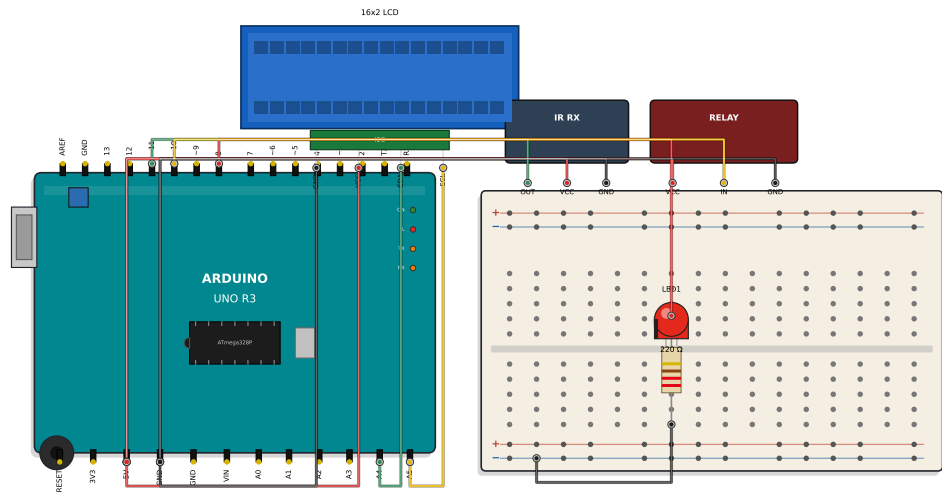


Figure 46.1: Experiment 46 circuit: IR receiver on D11, LED on D8, relay on D10, LCD via I²C.

Connection Table

IR receiver as Experiment 34 (D11); LED with 220 Ω on D8; relay VCC/GND/IN on 5V/GND/D10; LCD as Experiment 18.

Working Principle – Signal Flow

Remote button press → decoded hex code → matched against two known codes → toggles the LED or the relay accordingly; unrecognised or repeat codes are ignored.

Algorithm

1. Wait for a decoded IR signal.
2. If it matches the recorded "Button A" code, toggle the LED.
3. Else if it matches the recorded "Button B" code, toggle the relay.
4. Ignore repeat codes and any other button; resume receiving and repeat.

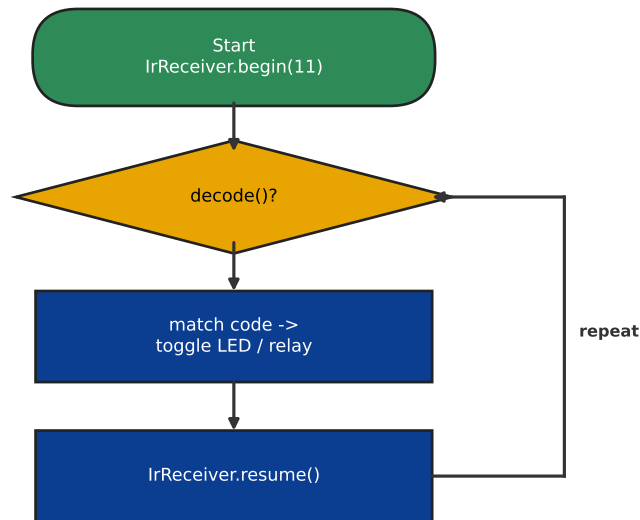


Figure 46.2: Flowchart: Experiment 46.

Arduino Program

```

1  #include <Wire.h>
2  #include <LiquidCrystal_I2C.h>
3  #include <IRremote.hpp>
4  #define IR_RECEIVE_PIN 11
5  LiquidCrystal_I2C lcd(0x27, 16, 2);
6  const int LED_PIN = 8, RELAY_PIN = 10;
7
8  // Replace these with codes YOU recorded in Experiment 34 for your own
   remote:
9  const unsigned long CODE_LED    = 0xFF6897UL;
10 const unsigned long CODE_RELAY = 0xFF9867UL;
11
12 bool ledOn = false, relayOn = false;
13
14 void setup() {
15   pinMode(LED_PIN, OUTPUT);
16   pinMode(RELAY_PIN, OUTPUT);
17   lcd.init(); lcd.backlight();
18   IrReceiver.begin(IR_RECEIVE_PIN);
19 }
20
21 void loop() {
22   if (IrReceiver.decode()) {
23     unsigned long code = IrReceiver.decodedIRData.decodedRawData;
24
25     if (code == CODE_LED) {
26       ledOn = !ledOn;
27       digitalWrite(LED_PIN, ledOn);
28     } else if (code == CODE_RELAY) {
29       relayOn = !relayOn;
30       digitalWrite(RELAY_PIN, relayOn);
31     }
32     // any other code (including repeat codes) is simply ignored
33   }

```

```

34     lcd.setCursor(0, 0);
35     lcd.print("LED:"); lcd.print(ledOn ? "ON " : "OFF");
36     lcd.print(" RLY:"); lcd.print(relayOn ? "ON " : "OFF");
37
38     IrReceiver.resume();
39 }
40 }

```

Code Explanation

The two `CODE_LED/CODE_RELAY` constants are explicitly commented as needing replacement with values the learner records themselves, since remote codes are remote-specific and cannot be predicted in advance (as discussed in Experiment 34). Because repeat codes from a held button do not equal either stored constant (they use a distinct, protocol-specific repeat marker), they naturally fall through to "ignored" without special-case code.

Experimental Procedure

1. Reuse Experiment 34's setup to record two button codes from your remote.
2. Update `CODE_LED/CODE_RELAY` in the sketch and wire the LED and relay.
3. Upload and test both buttons, confirming clean single toggles per press.

Expected Output

Pressing the chosen "LED" button toggles the LED on/off once per press; pressing the chosen "relay" button independently toggles the relay; the LCD reflects both states live.

Observation Table

Button pressed	Expected action	Observed action

Calculations and Analysis

Not applicable numerically; instead confirm each button reliably produces exactly one toggle per press across at least 5 repeated tests each.

Result

The IR remote smart controller correctly toggled the LED and relay independently, with _____ unwanted double-toggles observed across testing.

Troubleshooting

Problem	Likely cause	Fix
Nothing toggles	Stored codes do not match your actual remote	Re-record codes per Experiment 34
Holding a button causes rapid toggling	Repeat codes happen to match a stored constant	Verify your remote's repeat code differs (typical for most protocols)
Only one button works	Second constant not updated from the placeholder	Re-check both constants

Viva Questions and Extension Challenge

Conceptual: (1) Why are the two code constants placeholders that must be replaced? (2) Why do repeat codes not need special-case handling here? (3) How does this experiment relate to Experiment 7's debouncing conceptually? (4) Why use `unsigned long` for the codes? (5) What happens if two different remotes happen to share a code? **Troubleshooting:** (1) A third, unrelated household remote also happens to control this circuit — is this surprising given the protocol? (2) The relay toggles twice for one press occasionally — what quirk of some remotes/protocols could explain this? **Programming:** (1) Add a third mapped button that toggles both outputs together. (2) Print the raw code of any *unmapped* button press to the LCD, to help learners discover more codes. **Extension Challenge:** Build a tiny "channel" system: one button cycles through 3 modes (both off / LED only / both on) rather than two independent toggles.

Chapter 47

Experiment 47: Multi-Sensor Environmental Station

Experiment Identification

Experiment Number:	47
Title:	Multi-Sensor Environmental Station
Learning Layer:	Layer 3 — Advanced
Estimated Completion Time:	35 minutes
Difficulty:	Advanced
Prerequisite Experiments:	Experiments 36, 37, 20

Aim

To integrate DHT11, soil moisture, LDR, and HC-SR04 into one station using a formal non-blocking round-robin scheduler, generalising Experiment 37's two-sensor cycling to four independently-timed sensors on four LCD pages.

Learning Objectives

1. Design a round-robin scheduler that polls one sensor per scheduler "tick" rather than all sensors every pass.
2. Manage four independent cached readings and four display pages cleanly.
3. Recognise the scalability limit of hand-written per-sensor timers as sensor count grows.

Required Components

Arduino UNO R3, breadboard, 16×2 I²C LCD, 1 DHT11, 1 soil moisture sensor, 1 LDR, 1×10 kΩ resistor, 1 HC-SR04, jumper wires.

Technical Theory

System architecture: four independent sensor inputs feed one Arduino UNO acting as a scheduler, which drives one shared LCD output (block diagram, Figure 47). Rather than four separate hand-written `millis()` timers (which becomes unwieldy as sensor count grows — Experiment 26 already used two, Experiment 37 effectively managed a similar structure informally), this experiment introduces a single **round-robin scheduler**: one shared timer advances a `slot` index (0–3) every fixed interval, and on each tick, only the sensor at the current slot is read and its page redrawn — spreading the sensing workload evenly over time instead of

reading everything every pass.

Component Working Principle

See Experiments 23 (DHT11), 27 (soil), 10 (LDR), and 16 (HC-SR04) for individual sensor operation.

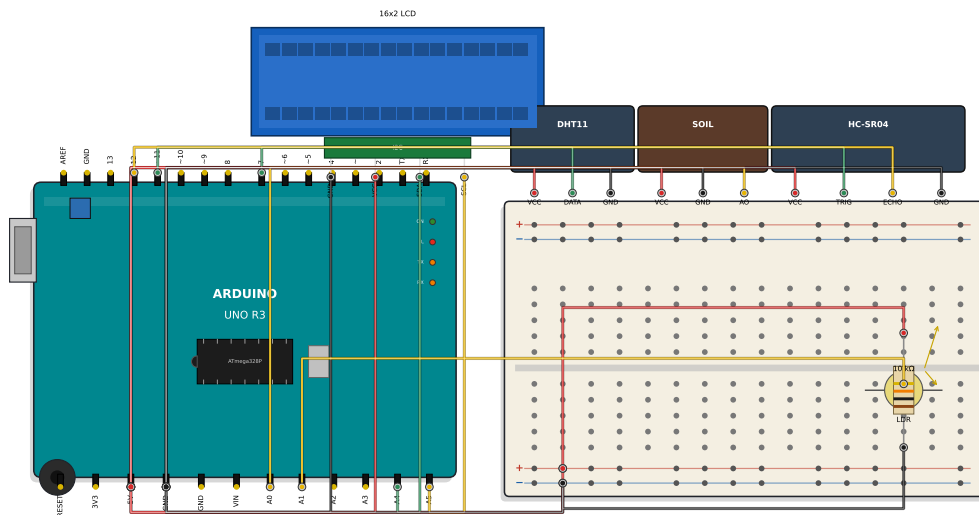


Figure 47.1: Experiment 47 circuit: DHT11, soil sensor, HC-SR04, and LDR combined with the LCD.

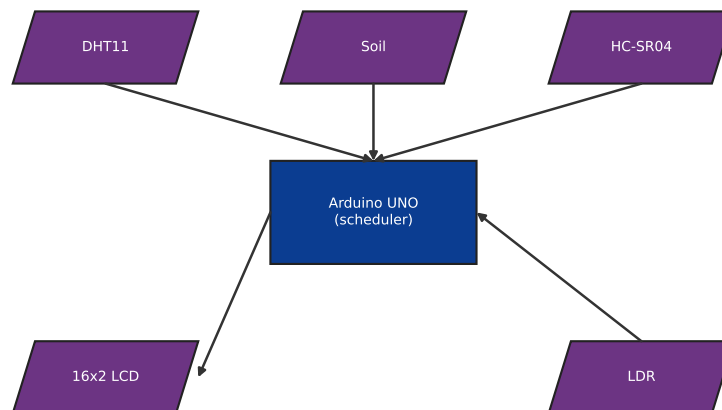


Figure 47.2: Experiment 47 system block diagram: four sensors feeding one scheduler, driving one LCD.

Connection Table

Component	Arduino pin
DHT11 DATA	D7
Soil sensor AO	A0
HC-SR04 TRIG/ECHO	D11/D12
LDR divider	A1
LCD	GND/5V/A4/A5

Working Principle – Signal Flow

A shared scheduler timer advances `slot` (0–3) every `SLOT_INTERVAL_MS`; each tick reads and caches only that slot's sensor and redraws the LCD from the corresponding cached value.

Algorithm

1. Every `SLOT_INTERVAL_MS`: read the sensor for the current `slot`, cache its value, redraw the LCD for that slot, then advance `slot = (slot+1) % 4`.
2. Repeat indefinitely.

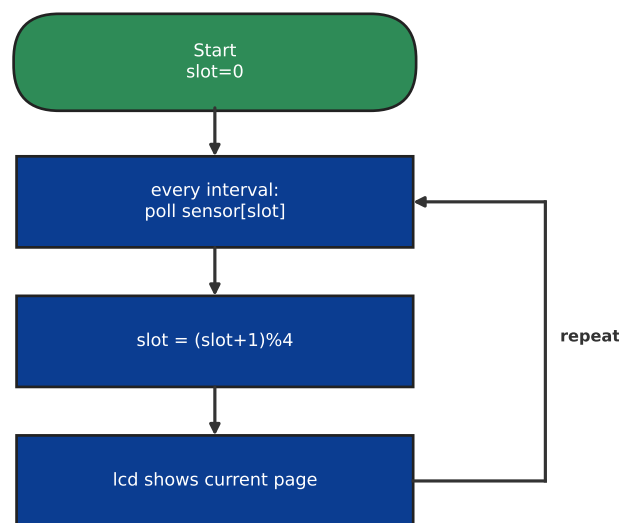


Figure 47.3: Flowchart: Experiment 47.

Arduino Program

```

1  #include <Wire.h>
2  #include <LiquidCrystal_I2C.h>
3  #include <DHT.h>
4  #define DHT_PIN 7
5  #define DHT_TYPE DHT11
6  LiquidCrystal_I2C lcd(0x27, 16, 2);
7  DHT dht(DHT_PIN, DHT_TYPE);
8  const int SOIL_PIN = A0, LDR_PIN = A1, TRIG_PIN = 11, ECHO_PIN = 12;
9  const unsigned long SLOT_INTERVAL_MS = 2500;
10 unsigned long lastSlot = 0;

```



```

11 int slot = 0;
12
13 float measureDistanceCm() {
14     digitalWrite(TRIG_PIN, LOW); delayMicroseconds(2);
15     digitalWrite(TRIG_PIN, HIGH); delayMicroseconds(10);
16     digitalWrite(TRIG_PIN, LOW);
17     unsigned long duration = pulseIn(ECHO_PIN, HIGH, 30000UL);
18     return (duration == 0) ? -1 : duration / 58.0;
19 }
20
21 void showSlot0() { float t = dht.readTemperature();
22     lcd.clear(); lcd.print("Temp:"); if (!isnan(t)) lcd.print(t, 1);
23     else lcd.print("err"); }
24 void showSlot1() { int raw = analogRead(SOIL_PIN);
25     int pct = constrain(map(raw, 800, 350, 0, 100), 0, 100);
26     lcd.clear(); lcd.print("Soil:"); lcd.print(pct); lcd.print("%"); }
27 void showSlot2() { int raw = analogRead(LDR_PIN);
28     lcd.clear(); lcd.print("Light:"); lcd.print(map(raw, 0, 1023, 0,
29     100)); lcd.print("%"); }
30 void showSlot3() { float d = measureDistanceCm();
31     lcd.clear(); lcd.print("Dist:"); if (d > 0) lcd.print(d, 0); else
32     lcd.print("--"); lcd.print("cm"); }
33
34 void (*pages[4])() = { showSlot0, showSlot1, showSlot2, showSlot3 };
35
36 void setup() {
37     pinMode(TRIG_PIN, OUTPUT); pinMode(ECHO_PIN, INPUT);
38     lcd.init(); lcd.backlight();
39     dht.begin();
40 }
41
42 void loop() {
43     if (millis() - lastSlot >= SLOT_INTERVAL_MS) {
44         lastSlot = millis();
45         pages[slot](); // call the function for the current
46             slot
47         slot = (slot + 1) % 4;
48     }
49 }

```

Code Explanation

`void (*pages[4])()` is an array of function pointers — exactly the generalisation previewed in Experiment 36's Extension Challenge. `pages[slot]()` calls whichever function corresponds to the current slot, so adding a fifth sensor later means adding one more function and one more array entry, with *no change* to the scheduling logic itself.

Experimental Procedure

1. Wire all four sensors and the LCD.
2. Upload and confirm the display cycles through all four readings in order, each staying visible for 2.5 s.

Expected Output

LCD cycles: "Temp:NN.N" → "Soil:NN%" → "Light:NN%" → "Dist:NNcm" → repeats, each for 2.5 s.

Observation Table

Slot / sensor	Value displayed	Plausible?

Calculations and Analysis

Confirm the full 4-slot cycle takes close to $4 \times 2500 = 10000$ ms, timed with a stopwatch.

Result

The multi-sensor station correctly cycled through all four sensors using the round-robin scheduler, with a full cycle time of _____ ms.

Troubleshooting

Problem	Likely cause	Fix
Only one page ever shows	<code>slot</code> not advancing	Re-check <code>slot = (slot+1)%4</code> runs every tick
Compile error on function pointer array	Syntax typo	Match the declaration exactly
One sensor's page shows implausible values	That sensor's own wiring/calibration	Re-check the corresponding earlier experiment

Viva Questions and Extension Challenge

Conceptual: (1) What problem does round-robin scheduling solve compared to reading all sensors every pass? (2) What is a function pointer, conceptually? (3) Why does adding a 5th sensor not require touching the scheduler logic? (4) Why is a single shared timer used instead of four independent ones? (5) What does the block diagram (Figure 47) communicate that the code alone does not? **Troubleshooting:** (1) The HC-SR04 page occasionally shows "-" — is this necessarily a fault? (2) Adding a 5th `showSlotX()` function but forgetting to extend the array size — what compiler or runtime symptom would you expect? **Programming:** (1) Make `SLOT_INTERVAL_MS` shorter for sensors that change quickly (e.g. distance) and longer for slow ones (temperature), using a per-slot interval array instead of one shared constant. (2) Add a 5th page showing all four cached values compactly. **Extension Challenge:** Add a button that jumps directly to a chosen slot on demand, in addition to the automatic cycle.

Chapter 48

Experiment 48: Smart Classroom Automation Prototype

Experiment Identification

Experiment Number:	48
Title:	Smart Classroom Automation Prototype
Learning Layer:	Layer 3 — Advanced
Estimated Completion Time:	35 minutes
Difficulty:	Advanced
Prerequisite Experiments:	Experiments 44, 47

Aim

To combine automatic light control, environmental monitoring, and an occupancy proxy into one classroom-automation prototype, demonstrating how independently-developed subsystems compose into a coherent application.

Learning Objectives

1. Use an IR obstacle sensor at a doorway as a simple occupancy proxy (not a true occupancy sensor).
2. Combine light automation (hysteresis), environment monitoring (DHT11), and an occupancy-gated comfort alert.
3. Practice integrating three previously separate subsystems into one coherent `loop()`.

Required Components

Arduino UNO R3, breadboard, 16×2 I²C LCD, 1 LDR, 1×10 kΩ resistor, 1 DHT11, 1 IR obstacle sensor, 1 relay module, 1 active buzzer, jumper wires.

Technical Theory

A doorway IR obstacle sensor cannot truly measure "is the room occupied" (a real classroom system would use a dedicated PIR motion sensor or people counter, not in this kit) — it can only detect "something crossed the doorway recently". This experiment uses that as an honest, explicitly-labelled **occupancy proxy**: a latch that is set on any doorway crossing and only cleared after a timeout with no further crossings, giving a reasonable (if imperfect) approximation of "someone is probably in the room". Automatic lighting uses Experiment 44's

hysteresis; the buzzer sounds a "reminder" only when the room is both occupied (by the proxy) *and* uncomfortably warm, avoiding nuisance alerts in an empty room.

Component Working Principle

See Experiments 10 (LDR), 23 (DHT11), 15 (IR obstacle), and 29 (relay).

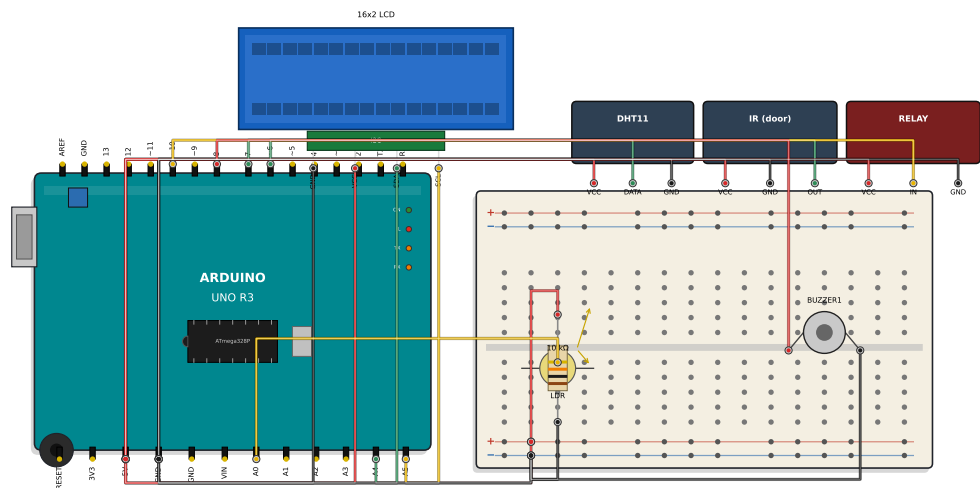


Figure 48.1: Experiment 48 circuit: LDR, DHT11, IR obstacle (doorway), relay, and buzzer combined with the LCD.

Connection Table

Component	Arduino pin
LDR divider	A0
DHT11 DATA	D7
IR obstacle (doorway) OUT	D6
Relay IN (light)	D10
Buzzer + (reminder)	D8
LCD	GND/5V/A4/A5

Working Principle – Signal Flow

Doorway crossing → occupancy proxy latched for a timeout window; light relay driven by LDR hysteresis regardless of occupancy; buzzer sounds only if occupied *and* temperature exceeds a comfort threshold.

Algorithm

1. On a doorway crossing, record the crossing time (occupied = true for the next OCCUPANCY_TIMEOUT_MS).
2. Drive the light relay from LDR hysteresis (Experiment 44 function), independent of occupancy.

3. If occupied and temperature > comfort threshold, sound the buzzer; else silence it.
4. Display light/occupancy/temperature status on the LCD.

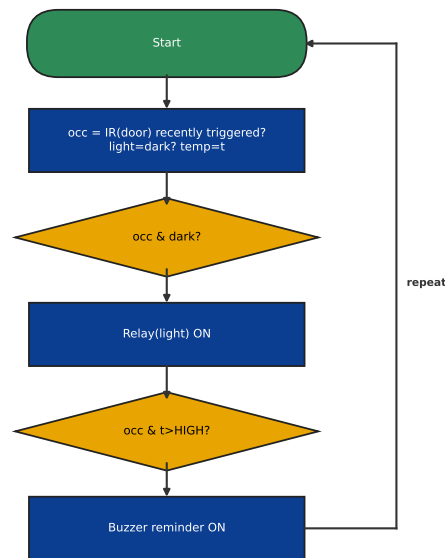


Figure 48.2: Flowchart: Experiment 48.

Arduino Program

```

1  #include <Wire.h>
2  #include <LiquidCrystal_I2C.h>
3  #include <DHT.h>
4  #define DHT_PIN 7
5  #define DHT_TYPE DHT11
6  LiquidCrystal_I2C lcd(0x27, 16, 2);
7  DHT dht(DHT_PIN, DHT_TYPE);
8  const int LDR_PIN = A0, IR_PIN = 6, RELAY_PIN = 10, BUZZER_PIN = 8;
9  const unsigned long OCCUPANCY_TIMEOUT_MS = 60000; // 1 minute
10 const float COMFORT_TEMP = 29.0;
11 unsigned long lastCrossing = 0;
12 bool lightOn = false;
13
14 bool hysteresis(int value, bool state, int low, int high) {
15     if (!state && value < low) return true;
16     if (state && value > high) return false;
17     return state;
18 }
19
20 void setup() {
21     pinMode(IR_PIN, INPUT);
22     pinMode(RELAY_PIN, OUTPUT);
23     pinMode(BUZZER_PIN, OUTPUT);
24     lcd.init(); lcd.backlight();
25     dht.begin();
26 }
27
28 void loop() {
29     if (digitalRead(IR_PIN) == LOW) lastCrossing = millis();

```

```

30  bool occupied = (millis() - lastCrossing) < OCCUPANCY_TIMEOUT_MS;
31
32  lightOn = hysteresis(analogRead(LDR_PIN), lightOn, 350, 450);
33  digitalWrite(RELAY_PIN, lightOn ? HIGH : LOW);
34
35  float t = dht.readTemperature();
36  bool remind = occupied && !isnan(t) && (t > COMFORT_TEMP);
37  digitalWrite(BUZZER_PIN, remind ? HIGH : LOW);
38
39  lcd.setCursor(0, 0);
40  lcd.print(occupied ? "Occ " : "Empt");
41  lcd.print(lightOn ? " Light" : " Dark ");
42  lcd.print(remind ? " Warm!" : " ");
43  delay(500);
44  }

```

Code Explanation

Occupancy is a simple *latched timeout*, not a continuous sensor reading: `lastCrossing` records the last doorway event, and `occupied` is derived freshly each pass from how long ago that was — no separate "occupied" variable needs manual clearing, since the timeout comparison naturally becomes false once enough time has passed with no new crossings.

Experimental Procedure

1. Wire the LDR, DHT11, IR obstacle sensor (doorway), relay, and buzzer.
2. Upload; simulate a doorway crossing, confirm "Occ" appears and persists for roughly 1 minute after the last crossing, then reverts to "Empt".
3. Independently confirm the light relay follows LDR hysteresis regardless of occupancy, and the buzzer only sounds when both occupied and warm.

Expected Output

LCD shows a combination of "Occ"/"Empt", " Light"/" Dark ", and " Warm!" as appropriate, with the buzzer sounding only in the occupied+warm combination.

Observation Table

Scenario	Occupied?	Light state	Buzzer state

Calculations and Analysis

Time how long "Occ" persists after the last doorway crossing and confirm it is close to the programmed 60000 ms.

Result

The classroom automation prototype correctly combined occupancy, lighting, and comfort-alert logic across _____ tested scenarios.

Troubleshooting

Problem	Likely cause	Fix
Occupancy never times out	<code>lastCrossing</code> being repeatedly reset by sensor noise	Check IR sensor for false triggers
Buzzer sounds in an empty room	<code>remind</code> condition missing the <code>occupied</code> term	Re-check the boolean expression
Light ignores occupancy entirely	Correct behaviour (by design)	Not a fault

Viva Questions and Extension Challenge

Conceptual: (1) Why is the IR sensor called a "proxy" rather than a true occupancy sensor? (2) Why does the light relay ignore occupancy by design? (3) How is "occupied" derived without an explicit clearing step? (4) Why require both conditions for the reminder buzzer? (5) What real sensor would improve occupancy detection, and why isn't it in this kit? **Troubleshooting:** (1) The room reads "Empt" immediately after someone remains inside without moving — what does this reveal about the proxy's limitation? (2) The buzzer chatters near the comfort threshold — what earlier experiment's fix applies? **Programming:** (1) Add hysteresis to the comfort-temperature check. (2) Log every doorway crossing with a timestamp for a simple usage report. **Extension Challenge:** Add a second doorway IR sensor and count net crossings (in minus out) for a slightly better occupancy estimate than a simple timeout, discussing its remaining limitations.

Chapter 49

Experiment 49: Security and Environment Monitoring Station

Experiment Identification

Experiment Number:	49
Title:	Security and Environment Monitoring Station
Learning Layer:	Layer 3 — Advanced
Estimated Completion Time:	35 minutes
Difficulty:	Advanced
Prerequisite Experiments:	Experiments 41, 47

Aim

To combine Experiment 40/41's security FSM with environmental sensing on one shared LCD, introducing explicit **alarm prioritisation**: security alerts must always pre-empt routine environmental display, never the reverse.

Learning Objectives

1. Combine a security FSM with routine sensor display sharing one output device.
2. Implement display priority: security state overrides normal page-cycling whenever active.
3. Justify a priority ordering from a systems-safety perspective.

Required Components

Arduino UNO R3, breadboard, 16×2 I²C LCD, 1 DHT11, 1 IR obstacle sensor (security zone), 1 active buzzer, 1 push button, jumper wires.

Technical Theory

When multiple subsystems share one limited output (a single 16×2 LCD), a design decision is required for what happens when they conflict — here, a security ALARM and a routine "show the temperature" page cannot both occupy the display simultaneously. This experiment establishes an explicit **priority rule**: the LCD *always* shows ALARM status when `state == ALARM`, regardless of what the environmental display would otherwise be showing, and only reverts to environmental display once the alarm is cleared. This is a simplified instance of a general embedded-systems principle: safety-relevant information must never be silently starved by routine information.

Component Working Principle

See Experiment 40 (security FSM) and Experiment 23 (DHT11).

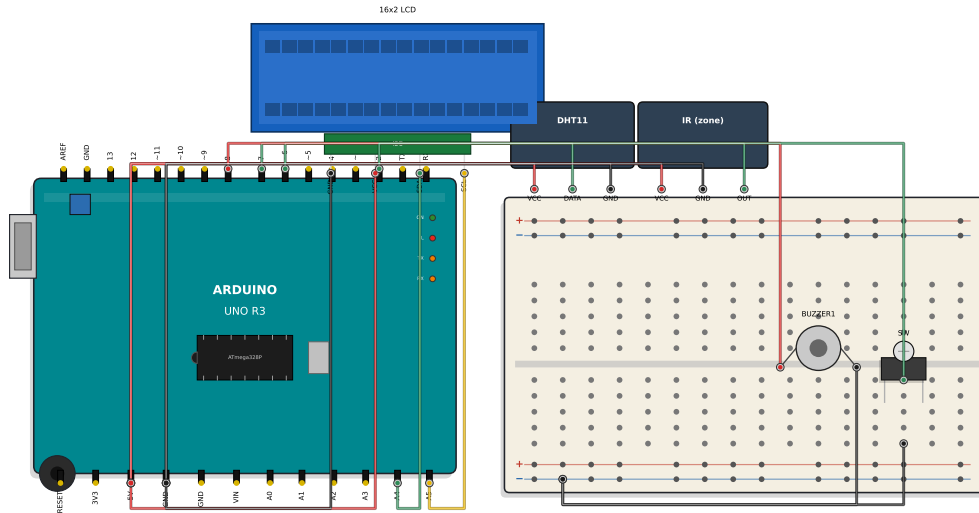


Figure 49.1: Experiment 49 circuit: DHT11, IR obstacle (security zone), buzzer, arm button, LCD.

Connection Table

Component	Arduino pin
DHT11 DATA	D7
IR obstacle (zone) OUT	D6
Buzzer +	D8
Arm button	D2
LCD	GND/5V/A4/A5

Working Principle – Signal Flow

Security FSM (as Experiment 40, using D6 for the sensor) runs independently of environmental sensing; the LCD-drawing code checks the security state *first* — if ALARM, it shows only alarm status; otherwise, it shows the environmental reading.

Algorithm

1. Update the security FSM (button arm/disarm, sensor-triggered alarm) exactly as Experiment 40.
2. Update the cached temperature on its own timer, regardless of security state.
3. If security state is ALARM, display alarm status (highest priority).
4. Else, display the cached temperature.

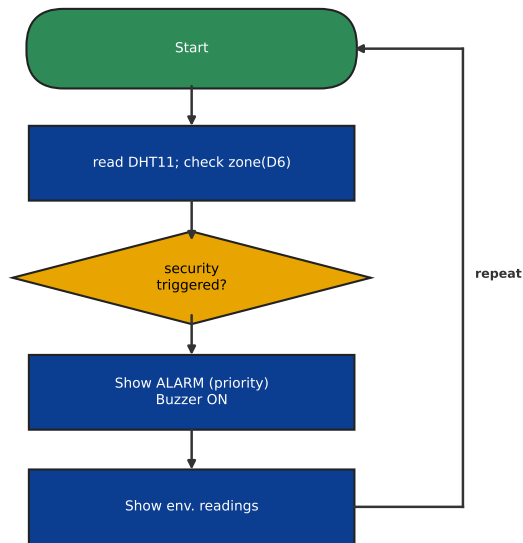


Figure 49.2: Flowchart: Experiment 49.

Arduino Program

```

1  #include <Wire.h>
2  #include <LiquidCrystal_I2C.h>
3  #include <DHT.h>
4  #define DHT_PIN 7
5  #define DHT_TYPE DHT11
6  LiquidCrystal_I2C lcd(0x27, 16, 2);
7  DHT dht(DHT_PIN, DHT_TYPE);
8  const int IR_PIN = 6, BUZZER_PIN = 8, BUTTON_PIN = 2;
9  enum State { DISARMED, ARMED, ALARM };
10 State state = DISARMED;
11
12 int lastReading = HIGH, stableState = HIGH;
13 unsigned long lastChangeTime = 0;
14 const unsigned long DEBOUNCE_MS = 30, READ_INTERVAL_MS = 2000;
15 unsigned long lastRead = 0;
16 float cachedTemp = 0;
17
18 bool buttonEdge() {
19     int reading = digitalRead(BUTTON_PIN);
20     if (reading != lastReading) lastChangeTime = millis();
21     bool edge = false;
22     if ((millis() - lastChangeTime) > DEBOUNCE_MS && reading !=
23         stableState) {
24         stableState = reading;
25         if (stableState == LOW) edge = true;
26     }
27     lastReading = reading;
28     return edge;
29 }
30
31 void setup() {
32     pinMode(IR_PIN, INPUT);
33     pinMode(BUZZER_PIN, OUTPUT);
34     pinMode(BUTTON_PIN, INPUT_PULLUP);

```

```

34  lcd.init(); lcd.backlight();
35  dht.begin();
36  }
37
38  void loop() {
39      if (buttonEdge()) {
40          state = (state == DISARMED) ? ARMED : DISARMED;
41      }
42      if (state == ARMED && digitalRead(IR_PIN) == LOW) state = ALARM;
43      digitalWrite(BUZZER_PIN, (state == ALARM) ? HIGH : LOW);
44
45      if (millis() - lastRead >= READ_INTERVAL_MS) {
46          lastRead = millis();
47          float t = dht.readTemperature();
48          if (!isnan(t)) cachedTemp = t;
49      }
50
51      lcd.setCursor(0, 0);
52      if (state == ALARM) {
53          lcd.print("** ALARM! **"); // security always wins the
                                   display
54      } else {
55          lcd.print("Temp: "); lcd.print(cachedTemp, 1); lcd.print(" C");
56      }
57  }

```

Code Explanation

The temperature is still read and cached on its own schedule *even while ALARM is showing*, so that the moment the alarm clears, an up-to-date reading is already available — the environmental subsystem is never truly paused, only its *display* is pre-empted, which is an important distinction for keeping both subsystems genuinely concurrent.

Experimental Procedure

1. Wire the DHT11, IR sensor, buzzer, button, and LCD.
2. Upload; confirm normal temperature display, then arm and trigger the sensor, confirming the LCD immediately switches to alarm status regardless of what page was showing.
3. Disarm and confirm the display correctly reverts to (still up-to-date) temperature.

Expected Output

"Temp: NN.N C" during normal operation; "** ALARM! **" immediately and exclusively once triggered while armed, reverting cleanly to temperature display once disarmed.

Observation Table

Scenario	Security state	LCD content	Priority correct?

Calculations and Analysis

Confirm the temperature reading resumes immediately (not after a fresh 2s delay) once the alarm clears, proving it was cached continuously throughout the alarm period.

Result

The security state correctly took display priority over environmental readings in _____ out of _____ tested scenarios.

Troubleshooting

Problem	Likely cause	Fix
Temperature briefly shows during an alarm	<code>if (state==ALARM)</code> check missing/misplaced	Re-check it is the first condition in the display block
Display never returns to temperature	Disarm transition not reachable	Re-check the button-edge branch

Viva Questions and Extension Challenge

Conceptual: (1) Why must security displays pre-empt routine information rather than share time with it? (2) Why does temperature continue being cached during an alarm? (3) What general embedded-safety principle does this illustrate? (4) Why is this "prioritisation" rather than just "two separate systems"? (5) What would go wrong if the priority check were reversed?

Troubleshooting: (1) The alarm text and temperature seem to alternate rapidly instead of the alarm winning cleanly — what condition ordering issue fits this? (2) The system seems to ignore the DHT11 entirely once armed — what should you check about the read-timer guard?

Programming: (1) Add a third priority tier: a low-battery-style "sensor fault" message that outranks temperature but not alarm. (2) Flash the alarm text (using `millis()`) instead of a static message. **Extension Challenge:** Extend to the full Experiment 41 multi-zone security logic combined with Experiment 47's four-sensor station, as direct preparation for the Experiment 50 capstone.

Chapter 50

Experiment 50: Mini Smart Home Automation System

Experiment Identification

Experiment Number:	50
Title:	Mini Smart Home Automation System
Learning Layer:	Layer 3 — Advanced (Capstone)
Estimated Completion Time:	60–90 minutes
Difficulty:	Advanced
Prerequisite Experiments:	Experiments 35–49

Aim

To integrate every sensor and actuator technique developed across this manual — security FSM, environmental monitoring, automatic lighting, distance sensing, and alarm prioritisation — into one coherent, non-blocking Mini Smart Home Automation System, built entirely from kit components.

Learning Objectives

1. Specify system requirements and a pin allocation plan before writing code.
2. Integrate 5 sensors and 2 actuators under one non-blocking scheduler with no polling conflicts.
3. Apply a finite-state machine, hysteresis, and alarm prioritisation together in one program.
4. Validate the complete system against a written checklist rather than ad hoc testing.
5. Critically state the system's real limitations and credible future enhancements.

Required Components

Arduino UNO R3, breadboard, 16×2 I²C LCD, 1 LDR + 1×10 kΩ resistor, 1 DHT11, 1 soil moisture sensor, 1 IR obstacle sensor (doorway/security), 1 HC-SR04, 1 relay module, 1 active buzzer, 1 push button, jumper wires (all components used are drawn from the single kit inventory in Chapter ; no component is used twice simultaneously beyond kit stock).

System Requirements

1. The system shall automatically turn a light (relay demonstration) ON in darkness and OFF in daylight, using hysteresis.
2. The system shall monitor temperature, humidity, and soil moisture, cycling their display when not showing a higher-priority page.
3. The system shall report distance to the nearest object (e.g. at an entrance) on request.
4. The system shall support arming a security zone (doorway IR sensor); a trigger while armed shall latch an audible+visual alarm until disarmed.
5. Security alarm status shall always take display priority over routine environmental pages.
6. No experiment-specific behaviour shall require blocking delays that stall any other subsystem.

Technical Theory

System architecture (Figure 50): five sensors (LDR, DHT11, soil moisture, IR obstacle, HC-SR04) feed one Arduino UNO running a combined FSM + round-robin scheduler (generalising Experiments 40 and 47), which drives three shared outputs (LCD, relay, buzzer). **Control logic** layers three previously independent patterns: (1) hysteresis-based automatic lighting (Experiment 44), (2) a security FSM with entry-delay and latching alarm (Experiment 40), and (3) round-robin environmental display (Experiment 47) — combined under the display-priority rule from Experiment 49 (security always wins the LCD).

Component Working Principle

Every component's working principle was covered in its introductory experiment (cross-referenced in the Technical Theory above); no new component behaviour is introduced in this capstone — only their integration is new.

Pin Allocation

Function	Arduino pin	Source experiment
Arm/disarm button	D2	40
Doorway/security IR obstacle	D6	15, 40
DHT11 DATA	D7	23
Buzzer (alarm)	D8	12
Relay (light demonstration)	D10	29
HC-SR04 TRIG	D11	16
HC-SR04 ECHO	D12	16
LDR divider	A0	10
Soil moisture AO	A1	27
LCD (SDA/SCL)	A4/A5	18

This uses 7 of 14 digital pins and 2 of 6 analog pins (plus the fixed I²C pair), comfortably within the Arduino UNO's I/O budget with no conflicts.

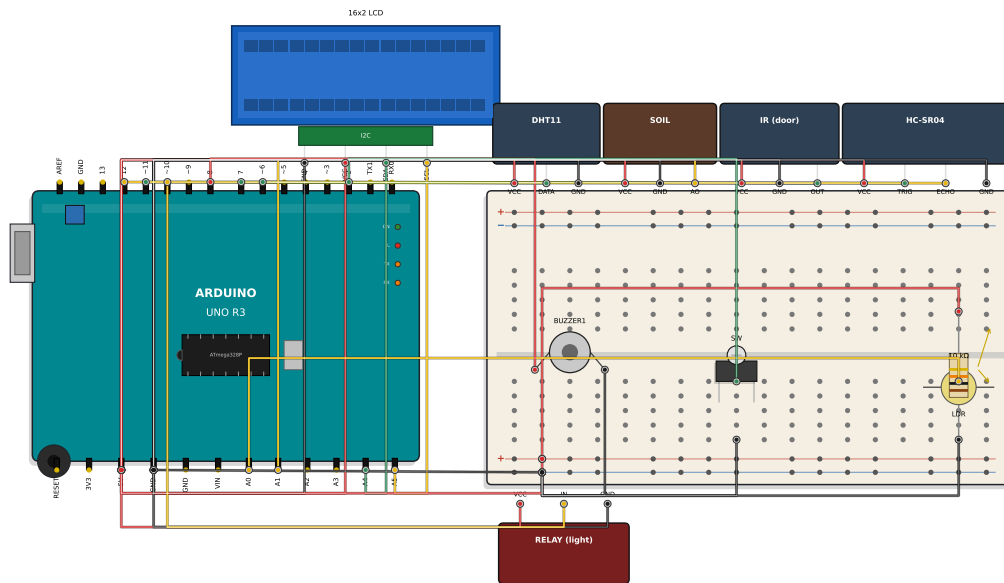


Figure 50.1: Experiment 50 capstone circuit: all five sensors and two actuators integrated on one breadboard, with the LCD via I²C.

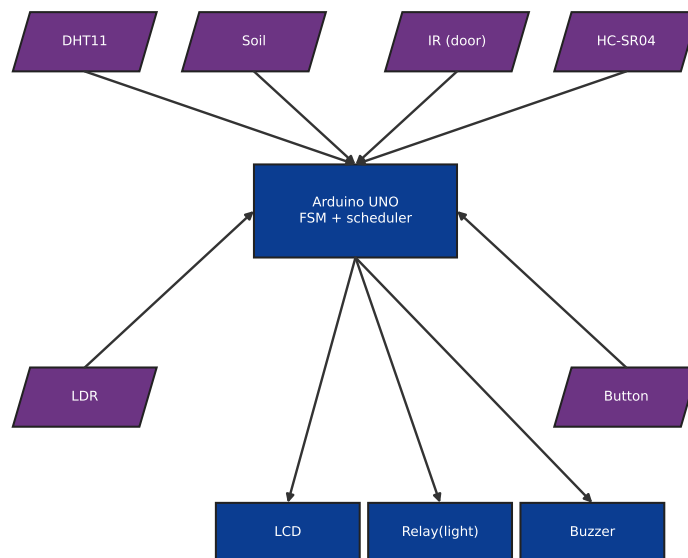


Figure 50.2: Experiment 50 system block diagram.

Connection Table

See the Pin Allocation table above; each connection reuses the exact wiring already validated in its source experiment (VCC to 5V, GND to GND, signal pin as listed).

Working Principle – Signal Flow

Security FSM and lighting hysteresis run every `loop()` pass (fast, safety/comfort-relevant); the round-robin environmental scheduler runs on a slower tick; the LCD is drawn last each pass, with security state checked first and given exclusive display priority whenever active.

Algorithm

1. Update the debounced arm/disarm button and the security FSM (DISARMED/ARMED/ALARM), including the entry delay.
2. Update the lighting relay from LDR hysteresis, unconditionally.
3. On the round-robin scheduler's tick, read and cache the next environmental sensor in sequence (temperature/humidity, soil, distance).
4. Drive the buzzer from the security state.
5. Redraw the LCD: if ALARM, show alarm status exclusively; else show the current round-robin page.

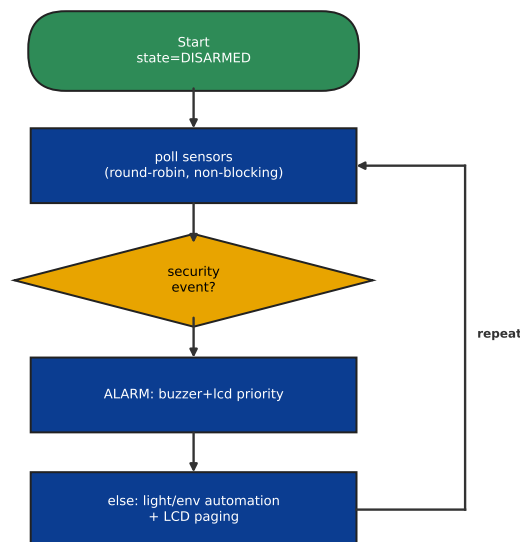


Figure 50.3: Flowchart: Experiment 50 (top-level).

Arduino Program

```

1  #include <Wire.h>
2  #include <LiquidCrystal_I2C.h>
3  #include <DHT.h>
4  #define DHT_PIN 7
5  #define DHT_TYPE DHT11
6  LiquidCrystal_I2C lcd(0x27, 16, 2);
7  DHT dht(DHT_PIN, DHT_TYPE);
8
9  const int BUTTON_PIN = 2, IR_PIN = 6, BUZZER_PIN = 8, RELAY_PIN = 10;
10 const int TRIG_PIN = 11, ECHO_PIN = 12, LDR_PIN = A0, SOIL_PIN = A1;
11 const unsigned long ENTRY_DELAY_MS = 5000, SLOT_INTERVAL_MS = 2500;
12
13 enum State { DISARMED, ARMED, ALARM };

```



```

14 State state = DISARMED;
15 unsigned long armedAt = 0;
16
17 int lastReading = HIGH, stableState = HIGH;
18 unsigned long lastChangeTime = 0;
19 const unsigned long DEBOUNCE_MS = 30;
20 bool lightOn = false;
21 unsigned long lastSlot = 0;
22 int slot = 0;
23 float cachedTemp = 0, cachedHum = 0;
24 int cachedSoil = 0;
25 float cachedDist = -1;
26
27 bool buttonEdge() {
28     int reading = digitalRead(BUTTON_PIN);
29     if (reading != lastReading) lastChangeTime = millis();
30     bool edge = false;
31     if ((millis() - lastChangeTime) > DEBOUNCE_MS && reading !=
        stableState) {
32         stableState = reading;
33         if (stableState == LOW) edge = true;
34     }
35     lastReading = reading;
36     return edge;
37 }
38
39 bool hysteresis(int value, bool st, int low, int high) {
40     if (!st && value < low) return true;
41     if (st && value > high) return false;
42     return st;
43 }
44
45 float measureDistanceCm() {
46     digitalWrite(TRIG_PIN, LOW); delayMicroseconds(2);
47     digitalWrite(TRIG_PIN, HIGH); delayMicroseconds(10);
48     digitalWrite(TRIG_PIN, LOW);
49     unsigned long d = pulseIn(ECHO_PIN, HIGH, 30000UL);
50     return (d == 0) ? -1 : d / 58.0;
51 }
52
53 void setup() {
54     pinMode(BUTTON_PIN, INPUT_PULLUP);
55     pinMode(IR_PIN, INPUT);
56     pinMode(BUZZER_PIN, OUTPUT);
57     pinMode(RELAY_PIN, OUTPUT);
58     pinMode(TRIG_PIN, OUTPUT); pinMode(ECHO_PIN, INPUT);
59     lcd.init(); lcd.backlight();
60     dht.begin();
61 }
62
63 void loop() {
64     // 1) Security FSM
65     if (buttonEdge()) {
66         if (state == DISARMED) { state = ARMED; armedAt = millis(); }
67         else state = DISARMED;
68     }
69     if (state == ARMED && (millis() - armedAt) > ENTRY_DELAY_MS
        && digitalRead(IR_PIN) == LOW) {
70

```

```

71     state = ALARM;
72 }
73 digitalWrite(BUZZER_PIN, (state == ALARM) ? HIGH : LOW);
74
75 // 2) Automatic lighting (always active)
76 lightOn = hysteresis(analogRead(LDR_PIN), lightOn, 350, 450);
77 digitalWrite(RELAY_PIN, lightOn ? HIGH : LOW);
78
79 // 3) Round-robin environmental scheduler
80 if (millis() - lastSlot >= SLOT_INTERVAL_MS) {
81     lastSlot = millis();
82     if (slot == 0) {
83         float t = dht.readTemperature(), h = dht.readHumidity();
84         if (!isnan(t)) cachedTemp = t;
85         if (!isnan(h)) cachedHum = h;
86     } else if (slot == 1) {
87         cachedSoil = constrain(map(analogRead(SOIL_PIN), 800, 350, 0,
88                                 100), 0, 100);
89     } else {
90         cachedDist = measureDistanceCm();
91     }
92     slot = (slot + 1) % 3;
93 }
94
95 // 4) Display (security has absolute priority)
96 lcd.setCursor(0, 0);
97 if (state == ALARM) {
98     lcd.print("** ALARM! **");
99 } else if (slot == 1) {
100     lcd.print("T:"); lcd.print(cachedTemp, 1); lcd.print(" H:"); lcd.
101         print(cachedHum, 0); lcd.print("% ");
102 } else if (slot == 2) {
103     lcd.print("Soil: "); lcd.print(cachedSoil); lcd.print("% ");
104 } else {
105     lcd.print("Dist: "); lcd.print(cachedDist, 0); lcd.print(" cm ");
106 }

```

Code Explanation

The four numbered comments in `loop()` mirror the Algorithm section's four steps exactly, so the code's structure is traceable back to the design without needing external documentation. Notice the lighting (step 2) and security (step 1) subsystems run *unconditionally* every pass (they are safety/comfort-critical and must never wait behind the slower environmental scheduler), while only the environmental readings (step 3) are gated behind the round-robin timer — a direct application of the "what needs to run every pass vs. what can be scheduled" reasoning built up since Experiment 26.

Experimental Procedure / Testing Procedure

1. Wire all components per the Pin Allocation table, double-checking shared GND and 5V rails.
2. Upload the sketch; confirm the LCD cycles through Temp/Humidity, Soil, and Distance pages every 2.5 s.
3. Test automatic lighting: cover/uncover the LDR and confirm the relay switches with hysteresis

(no chatter).

4. Arm the system (button press), wait past the 5 s entry delay, then trigger the doorway IR sensor; confirm the LCD immediately shows "*** ALARM! ***" regardless of which environmental page was showing, and the buzzer sounds.
5. Disarm (button press); confirm the LCD reverts to environmental cycling and the buzzer silences.
6. Repeat steps 3–5 in different combinations (e.g. trigger the alarm while the light is also on) to confirm subsystems do not interfere with each other.

Expected Output

Under normal conditions, the LCD cycles Temp/Humidity → Soil → Distance every 2.5 s, the relay silently tracks ambient light with hysteresis, and arming/triggering/disarming produces an immediate, exclusive "*** ALARM! ***" display and buzzer during the ALARM state only.

Observation Table

Test performed	Result (Pass/Fail)
Environmental page cycling	
Automatic lighting hysteresis	
Arm → entry delay → trigger → alarm	
Alarm display priority over environment page	
Disarm returns to normal display	

Calculations and Analysis

Confirm the $2.5\text{ s} \times 3 = 7.5\text{ s}$ full environmental cycle time, the $\approx 5\text{ s}$ entry delay, and the $\approx 350/450$ ADC hysteresis band all measure close to their programmed values.

Result

The Mini Smart Home Automation System successfully passed _____ out of 5 validation checklist items (below).

Validation Checklist

- ☐ All 5 sensors and 2 actuators wired per the pin allocation table with no pin conflicts.
- ☐ Automatic lighting shows no relay chatter across at least 10 light-level transitions.
- ☐ Security FSM correctly ignores triggers during the entry delay.
- ☐ Alarm state always pre-empts the LCD, in every tested combination.
- ☐ No subsystem uses a blocking `delay()` longer than a few milliseconds.

Troubleshooting

Problem	Likely cause	Fix
One subsystem seems to freeze others	A leftover blocking <code>delay()</code> somewhere	Search for and remove any <code>delay()</code> beyond microsecond-scale sensor timing
LCD flickers between alarm and environment text	Priority check not first in the display block	Re-check <code>if (state==ALARM)</code> is evaluated first
Relay or buzzer chatters	Hysteresis/entry-delay constants too tight	Revisit Experiments 44/40's tuning guidance
One sensor reads implausibly	Pin conflict or miswiring	Re-check the pin allocation table for that sensor

Limitations

- Occupancy/security detection uses a single IR obstacle sensor as a presence *proxy*, not a calibrated intrusion-detection sensor.
- Lighting, soil, and temperature thresholds are fixed constants tuned for one classroom; they are not automatically calibrated.
- All "actuation" (pump, light, barrier) is a safe low-voltage LED/relay demonstration, not a real-world load, per this manual's hardware constraints (Chapter).
- The system has no persistent memory (EEPROM) — all state resets on power loss.
- Only one security zone is included here; Experiment 41's multi-zone approach was not merged into this capstone for clarity, but could be.

Future Enhancements

- Integrate Experiment 41's second security zone (laser/LDR beam-break) for multi-zone coverage.
- Integrate Experiment 46's IR remote control as an additional manual override input.
- Add EEPROM-backed persistence for calibration constants and security arm state.
- Replace fixed thresholds with a guided on-device calibration routine (button-driven, storing results to EEPROM).
- Add a data-logging mode over Serial for long-term environmental trend analysis.

Viva Questions and Extension Challenge

Conceptual: (1) Why must security and lighting logic run unconditionally every pass while environmental sensing is scheduled? (2) What does the pin allocation table demonstrate about hardware feasibility on one Arduino UNO? (3) Why is occupancy called a "proxy" here? (4) What would happen to display behaviour if the priority check were removed? (5) Which three earlier design patterns (name the experiments) does this capstone integrate? **Troubleshooting:** (1) After merging code from several experiments, the sketch no longer compiles — what is the most likely class of error? (2) The system works on the bench but resets randomly — what electrical check from Chapter should you revisit first? **Programming:** (1) Refactor `loop()` into named helper functions (`updateSecurity()`, `updateLighting()`, `updateEnvironment()`,

`updateDisplay()` called in sequence, and discuss whether this improves readability without changing behaviour. (2) Add unit-style manual test routines (e.g. a compile-time flag that exercises each subsystem alone) to ease future debugging. **Advanced Extension Challenge:** Implement one item from the Future Enhancements list in full, and write a short design note (half a page) explaining the change, its testing, and any new limitations it introduces.

Part IV

BONUS EXPERIMENT

Chapter 51

Bonus Experiment: Servo-Controlled Automatic Barrier

Why this experiment is separate from the numbered 50

The JR Learners Arduino Starter Kit also includes an **SG90 micro servo motor** (Appendix B). None of the 50 numbered experiments in this manual require it, so that the core curriculum stays reproducible for any reader working from the component list in Chapter alone. This bonus experiment is provided so the servo is not left unused, and reuses Experiment 16's distance-measurement function directly.

Aim

To build an automatic barrier that opens (servo swings to 90°) when a vehicle is detected within range by the HC-SR04, and closes (servo returns to 0°) once the vehicle passes, extending Experiment 33/39's parking-sensor pattern with a genuine mechanical actuator instead of an LED/relay demonstration.

Learning Objectives

1. Install and use the built-in **Servo** library.
2. Command a servo to a specific angle with `servo.write()`.
3. Combine a previously verified sensor function with a new actuator type.

Required Components

Arduino UNO R3, breadboard, 1 HC-SR04, 1 SG90 micro servo motor, jumper wires.

Technical Theory

The SG90 is a small hobby servo motor: it accepts a 50 Hz PWM control signal on its SIG wire, where pulse width (typically 1–2 ms within each 20 ms period) commands a specific output shaft angle (0–180°), holding that angle against moderate load using an internal feedback potentiometer and control loop — unlike the DC-only PWM brightness control of Experiment 9, a servo interprets pulse *width*, not average voltage, which is why the standard **Servo** library (not `analogWrite()`) is used. The SG90 draws up to ≈650 mA under stall load, well beyond what the Arduino 5V pin should supply directly under load; for this brief demonstration (light, unloaded horn) powering it from the Arduino 5V pin is acceptable, but a real deployment should use a separate 5V supply for the servo, sharing only a common ground with the Arduino.

Component Working Principle

See Experiment 16 for the HC-SR04's time-of-flight operating principle. The servo's three wires are conventionally GND (brown/black), VCC (red), and SIG (orange/yellow).

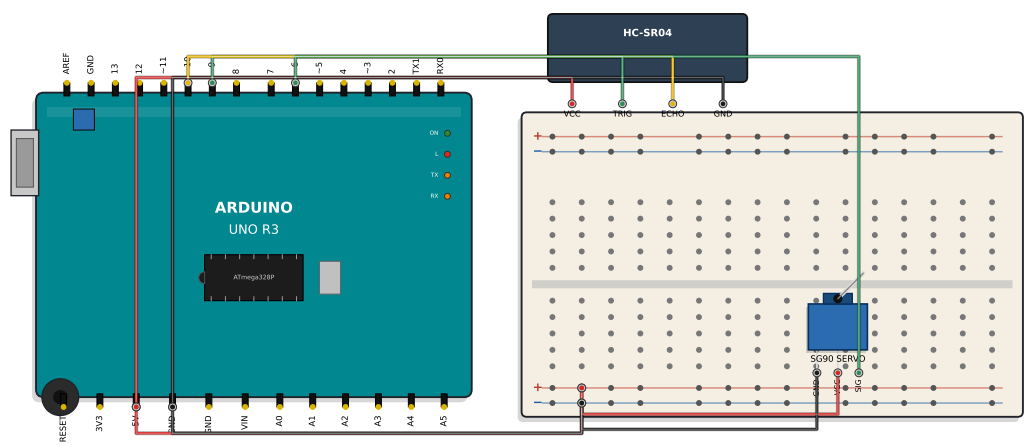


Figure 51.1: Bonus Experiment circuit: HC-SR04 (TRIG=D9, ECHO=D10), SG90 servo SIG on D6.

Connection Table

Component pin	Arduino pin	Notes
HC-SR04 VCC/TRIG/ECHO/GND	5V/D9/D10/GND	As Experiment 16
Servo GND (brown)	GND	Share ground with the Arduino
Servo VCC (red)	5V	Light/unloaded demonstration only
Servo SIG (orange)	D6	PWM control signal (via <code>Servo</code> library)

Working Principle – Signal Flow

Distance measured → compared against a 15 cm threshold → below threshold: servo commanded to 90°(barrier open); above threshold: servo commanded to 0°(barrier closed).

Algorithm

1. Attach the servo on D6.
2. Measure distance (Experiment 16 function).
3. If closer than 15 cm, command the servo to 90°; else command it to 0°.
4. Repeat.

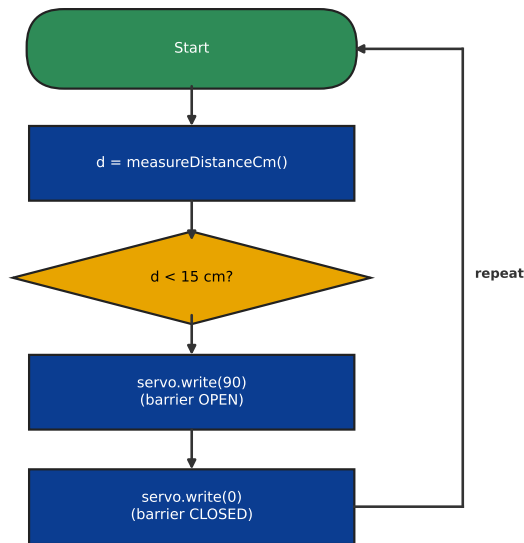


Figure 51.2: Bonus Experiment flowchart.

Arduino Program

```

1  #include <Servo.h>
2
3  const int TRIG_PIN = 9, ECHO_PIN = 10, SERVO_PIN = 6;
4  const unsigned long TIMEOUT_US = 30000UL;
5  Servo barrierServo;
6
7  float measureDistanceCm() {
8      digitalWrite(TRIG_PIN, LOW); delayMicroseconds(2);
9      digitalWrite(TRIG_PIN, HIGH); delayMicroseconds(10);
10     digitalWrite(TRIG_PIN, LOW);
11     unsigned long duration = pulseIn(ECHO_PIN, HIGH, TIMEOUT_US);
12     return (duration == 0) ? 999 : duration / 58.0;
13 }
14
15 void setup() {
16     pinMode(TRIG_PIN, OUTPUT);
17     pinMode(ECHO_PIN, INPUT);
18     barrierServo.attach(SERVO_PIN);
19     barrierServo.write(0);    // start closed
20 }
21
22 void loop() {
23     float d = measureDistanceCm();
24     if (d < 15.0) {
25         barrierServo.write(90);    // open
26     } else {
27         barrierServo.write(0);    // closed
28     }
29     delay(200);
30 }

```

Code Explanation

`barrierServo.attach(SERVO_PIN)` configures D6 to generate the 50 Hz servo control signal internally (handled by the `Servo` library's own timer, independent of `analogWrite()`'s PWM). `barrierServo.write(angle)` takes a plain angle in degrees, not a raw pulse width, making the code read directly in terms of the physical barrier position.

Experimental Procedure

1. Wire the HC-SR04 and servo per the connection table.
2. Upload and confirm the barrier is closed (0°) with nothing nearby.
3. Bring an object within 15 cm and confirm the servo swings to 90°, returning to 0° once removed.

Expected Output

Barrier servo rests at 0°; swings smoothly to 90° within $\approx 0.2\text{--}0.3\text{ s}$ of an object crossing the 15 cm threshold, and returns to 0° once it is clear.

Troubleshooting

Problem	Likely cause	Fix
Servo jitters continuously	Insufficient current from the shared 5V pin	Use a separate 5V supply for the servo, common ground only
Servo does not move at all	SIG wire on a non-existent/wrong pin, or library not attached	Re-check <code>attach()</code> and wiring
Barrier opens/closes at the wrong distance	Threshold constant, or HC-SR04 miswired	Re-check against Experiment 16

Viva Questions and Extension Challenge

Conceptual: (1) How does a servo's control signal differ from `analogWrite()` PWM? (2) Why is a separate power supply recommended for a loaded servo? (3) What do the servo's three wires carry? (4) Why is this kept as a bonus experiment rather than experiment 51? (5) What prior experiment's function is reused unchanged here? **Extension Challenge:** Combine this bonus experiment with Experiment 33's 3-LED parking-status bar for a complete visual+mechanical parking barrier system.

Appendix A

Arduino UNO R3 Technical Specifications

Parameter	Value
Microcontroller	ATmega328P (Microchip/Atmel), 8-bit AVR
Operating voltage	5 V
Input voltage (recommended, via barrel jack)	7–12 V
Clock speed	16 MHz
Flash memory	32 KB ($\approx$ 0.5 KB used by the bootloader)
SRAM	2 KB
EEPROM	1 KB
Digital I/O pins	14 (6 provide PWM output: D3, D5, D6, D9, D10, D11)
Analog input pins	6 (A0–A5), 10-bit ADC
DC current per I/O pin	20 mA recommended, 40 mA absolute maximum
DC current on 3.3V pin	50 mA
USB connector	Type-B
Communication	UART (D0/D1), I ² C (A4/A5), SPI (D10–D13)

These figures are standard published values for the Arduino UNO R3; always cross-check against the official Arduino documentation (Appendix O) for the most current specification.

Appendix B

Complete Kit Inventory

See Chapter for the full component inventory table with assumed variants and key specifications.

Appendix C

Arduino Pin Reference

See Chapter for the full pinout diagram and digital/PWM/analog pin summary table.

Appendix D

Resistor Colour-Code Reference

Four-band resistors (used throughout this kit) encode value as: [1st digit][2nd digit][multiplier][tolerance].

Colour	Digit value	Multiplier ($\times$)	Tolerance
Black	0	1	—
Brown	1	10	$\pm 1\%$
Red	2	100	$\pm 2\%$
Orange	3	1,000	—
Yellow	4	10,000	—
Green	5	100,000	$\pm 0.5\%$
Blue	6	1,000,000	$\pm 0.25\%$
Violet	7	—	$\pm 0.1\%$
Grey	8	—	—
White	9	—	—
Gold	—	0.1	$\pm 5\%$ (most common in this kit)
Silver	—	0.01	$\pm 10\%$

Worked example — **220 Ω :** red (2), red (2), brown ($\times 10$), gold ($\pm 5\%$) $\rightarrow 22 \times 10 = 220 \Omega$. **10 k Ω :** brown (1), black (0), orange ($\times 1000$), gold $\rightarrow 10 \times 1000 = 10,000 \Omega$.

Appendix E

Common Electronic Symbols

Symbol (as drawn in this manual)	Meaning
Zigzag / banded rectangle	Resistor
Triangle-and-bar	Diode (LED or PN-junction); bar = cathode
Two parallel lines (long/short)	Battery / DC supply
Circle with wiper arrow	Potentiometer
Ground symbol	Common 0 V reference (GND)
Two concentric circles	LED (dome view, this manual's breadboard style)
Labelled rectangle with pin stubs	Sensor/actuator module (this manual's convention for multi-pin breakout boards)

This manual deliberately favours realistic breadboard-style component drawings (Chapter 1 onward) over abstract schematic symbols, so that what a student sees on the page matches what they see on their physical breadboard; this table is provided so students can also recognise the standard schematic symbols used in datasheets and other references.

Appendix F

Common Arduino Programming Functions

Function	Purpose
<code>pinMode(pin, mode)</code>	Configure a pin as INPUT, OUTPUT, or INPUT_PULLUP
<code>digitalWrite(pin, val)</code>	Drive a digital pin HIGH or LOW
<code>digitalRead(pin)</code>	Read a digital pin's state
<code>analogRead(pin)</code>	Read a 0–1023 ADC value from an analog pin
<code>analogWrite(pin, val)</code>	Output a 0–255 PWM duty cycle on a ~ pin
<code>tone(pin, freq)</code> <code>noTone(pin)</code>	/ Generate / stop a square-wave tone on any digital pin
<code>delay(ms)</code>	Blocking pause
<code>millis()</code>	Milliseconds since power-up (non-blocking timing)
<code>map(x, inMin, inMax, outMin, outMax)</code>	Linearly rescale a value between ranges
<code>constrain(x, low, high)</code>	Clamp a value to a range
<code>pulseIn(pin, value, timeout)</code>	Measure a pulse's duration in microseconds
<code>Serial.begin(baud)</code>	Start the USB serial link
<code>Serial.print()</code> <code>println()</code>	/ Send text/values to the Serial Monitor
<code>isnan(x)</code>	Test whether a float is "Not a Number" (failed sensor read)
<code>Wire.begin()</code>	Initialise the I ² C bus

Appendix G

Arduino Library Installation Guide

See Chapter for the full list of libraries used in this manual and installation instructions via the Library Manager.

Appendix H

Sensor Calibration Procedures

- **LDR (Experiments 10, 20, 43):** record raw ADC values at two known conditions (e.g. bright room light, hand-covered) and use these as reference points; never assume a fixed threshold works across rooms or LDR units (Chapter on Technical Theory, Experiment 10).
- **Soil moisture (Experiments 27–29, 37–38):** record `DRY_RAW` with probes in open air and `WET_RAW` with probes in a cup of water; recalibrate periodically as probes corrode with use (Experiment 27).
- **HC-SR04 (Experiments 16, 21–22, 33, 39, 42, 47, 50):** no user calibration is possible or needed beyond confirming the assumed speed of sound (343 m/s); validate against a physical ruler at several distances (Experiment 16).
- **DHT11 (Experiments 23–26, 36–38, 45, 47–50):** no user calibration is possible; compare against a reference thermometer/hygrometer if precision beyond $\pm 2^\circ\text{C}/\pm 5\%\text{RH}$ is required, and consider a more precise sensor for that use case.
- **I²C LCD address (Experiment 18 onward):** run the I²C scanner (Chapter) once per physical LCD module and hard-code the found address.

Appendix I

Common Compiler Errors and Solutions

Error message (excerpt)	Likely cause and fix
'lcd' was not declared	Missing <code>#include <LiquidCrystal_I2C.h></code> or the object was not constructed before use
expected ';' before ...	Missing semicolon on the previous line
'DHT' does not name a type	DHT sensor library (and its Adafruit Unified Sensor dependency) not installed
a function-definition is not allowed here	Mismatched or missing closing brace <code>}</code> above
stray '\ ' in program	A curly/smart quote pasted instead of a straight quote (common when copying from word processors)
'IrReceiver' was not declared	Missing <code>#include <IRremote.hpp></code>
redefinition of ...	The same constant/variable declared twice, often after copy-pasting code blocks

Appendix J

Complete Program Index

Every experiment's complete Arduino program is listed in its own chapter under the "Arduino Program" heading. The table below cross-references each program to its primary pins for quick lookup.

#	Title	Key pins
1	LED ON/OFF Control	D8
2	LED Blinking	D8
3	Multiple LED Control	D8, D9, D10
4	LED Chaser	D8–D11
5	Traffic Light System	D8, D9, D10
6	Push Button LED Control	D2, D8
7	Push Button Toggle Switch	D2, D8
8	Five-Button LED Controller	D2–D6, D8–D12
9	Potentiometer LED Dimmer	A0, D9
10	LDR-Based Automatic Light	A0, D8
11	LDR Light-Level Indicator	A0, D8–D10
12	Buzzer ON/OFF Control	D8
13	Electronic Doorbell	D2, D8
14	Variable-Tone Buzzer	A0, D9
15	IR Obstacle Detector	D7, D8
16	Ultrasonic Distance Measurement	D9, D10
17	Ultrasonic Distance Alarm	D8–D10
18	LCD I ² C Hello World	A4, A5
19	Potentiometer Value Display	A0, A4, A5
20	LDR Light Meter	A0, A4, A5
21	Ultrasonic Distance Display	D9, D10, A4, A5
22	Digital Ruler	D9, D10, A4, A5
23	Temperature Display	D7, A4, A5
24	Humidity Display	D7, A4, A5
25	Temperature Alarm	D7, D8, A4, A5
26	Temperature and Humidity Monitor	D7, A4, A5
27	Soil Moisture Monitor	A0, A4, A5

#	Title	Key pins
28	Plant Moisture Alarm	A0, D8, A4, A5
29	Automatic Plant-Watering Controller Demonstration	A0, D10, A4, A5
30	Smart Night Lamp	A0, D9
31	IR Security Alarm	D2, D7, D8
32	Object Counter	D7, A4, A5
33	Smart Parking Indicator	D8–D13, A4, A5
34	IR Remote Controller Demonstration	D11, A4, A5
35	Smart Room Light Controller	A0, D2, D10
36	Room Environment Monitor	D7, A4, A5
37	Smart Plant Monitoring System	D7, A0, A4, A5
38	Automatic Plant Protection System	D7, A0, D8, D10, A4, A5
39	Smart Parking Assistant	D8, D11, D12, A4, A5
40	Electronic Security System	D2, D7, D8, D13
41	Multi-Zone Security Detector	D7, A1, D8, A4, A5
42	Digital Distance Warning System	D8–D13
43	Light Intensity Measurement System	A0, A1, A4, A5
44	Automatic Day/Night Controller	A0, D10
45	Temperature-Controlled System	D7, D10, A4, A5
46	IR Remote Smart Controller	D8, D10, D11, A4, A5
47	Multi-Sensor Environmental Station	D7, A0, A1, D11, D12, A4, A5
48	Smart Classroom Automation Prototype	A0, D7, D6, D10, D8, A4, A5
49	Security and Environment Monitoring Station	D7, D6, D8, D2, A4, A5
50	Mini Smart Home Automation System	D2, D6, D7, D8, D10, D11, D12, A0, A1, A4, A5

Appendix K

Viva Answer Key (Instructor Use)

Answers are intentionally concise model answers for quick instructor cross-checking, in the same order as each experiment's Conceptual (C), Troubleshooting (T), and Programming (P) questions. Extension Challenges are open-ended and not scored against a single answer.

Experiment 1 — LED ON/OFF Control

C1: A bare LED has almost no internal resistance, so it would draw destructive current at 5 V; a resistor alone doesn't emit light so it doesn't need current limited to protect itself the same way. C2: The pin stays a floating input; `digitalWrite` on an input pin toggles the internal pull-up instead of driving output. C3: The LED current changes only slightly (Ohm's law with a small R change), staying within safe range either way. C4: $\approx 2.9\text{--}3.2\text{ V}$ (wider semiconductor bandgap) vs. $\approx 2.0\text{ V}$ for red. C5: Reversing the LED reverse-biases the junction, blocking current entirely regardless of the resistor. T1: Missing/undersized resistor, or a partially failed LED. T2: The LED is likely open-circuit (burnt out) or wired to the wrong node. T3: A short circuit (miswired power/ground) causing a brownout reset. P1: Change HIGH to LOW in `setup()`. P2: Loses type checking / IDE autocomplete; harder to scope.

Experiment 2 — LED Blinking

C1: `loop()` itself is called repeatedly forever by the core. C2: $f = 1000 / (300 + 700) = 1\text{ Hz}$. C3: `delay()` blocks all code, including a button read. C4: No, brightness is unaffected by blink rate at a fixed duty cycle. C5: `millis()`. T1: Both delays doubled by mistake. T2: No, D13's onboard LED is independent hardware, expected. T3: Old sketch not re-uploaded / wrong port selected. P1: Set unequal delay values for ON/OFF. P2: Magic numbers replaced by clearer named constants (no functional change).

Experiment 3 — Multiple LED Control

C1: Each branch only shares the return path, not the signal. C2: Only that LED's brightness/behaviour would change. C3: Only reduces code size/duplication, not part count. C4: Index 2. C5: Add one more pin to the array and increment `NUM_LEDS`. T1: The third LED itself is likely faulty or miswired. T2: `ledPins[]` order only affects sequence order, not correctness. T3: A short between two branches sharing GND incorrectly. P1: Loop that sets all HIGH together instead of sequentially. P2: No other code changes needed besides the array and constant.

Experiment 4 — LED Chaser

C1: 1, confirming wrap-around. C2: Ternary avoids nested `if/else` for a single condition. C3: Higher bandgap semiconductor needs higher forward voltage. C4: Wrong LED would appear to light, or none if out of range. C5: Identical wiring, only control logic differs. T1: Duplicate value in `ledPins[]`. T2: `current` decrement direction was likely reversed. T3: Likely a hardware brightness difference (LED batch variation), not code. P1: Decrement `current` with modulo-safe wraparound. P2: Track direction and reverse at the array bounds.

Experiment 5 — Traffic Light System

C1: Yellow is a brief transition warning, not a "go" state. C2: Signals imminent change to GREEN, cueing drivers/pedestrians to prepare. C3: The last phase needs RED to remain lit while YELLOW is added. C4: 8 combinations possible; only 4 used. C5: 11 s. T1: Missing `allOff()` call before the yellow-only phase. T2: Stopwatch reaction-time error, not a bug. T3: Expected millisecond-scale switching transients, not a fault. P1: Add a 4th LED lit only during RED. P2: An array of `{mask,duration}` pairs iterated by one loop.

Experiment 6 — Push Button LED Control

C1: The internal resistor removes the need for an external one. C2: Internal pull-up sources current when nothing pulls the pin low. C3: One side's pre-joined pair bridges to the other side. C4: The pin would float and read randomly. C5: LOW represents "pressed" in this wiring. T1: Legs from the same internal pair used. T2: Check breadboard column-group placement. T3: Mechanical bounce, not a code fault at this stage. P1: Swap the `if/else` bodies. P2: Needs a remembered previous state (built in Experiment 7).

Experiment 7 — Push Button Toggle Switch

C1: Mechanical contact chatter for a few ms. C2: One tracks raw noise, one tracks the accepted/stable value. C3: An edge is the instant of change; a level is the ongoing state. C4: It would reset every loop and lose the toggle memory. C5: Toggling would fire many times per press (chatter). T1: Debounce interval likely too short for that switch. T2: Different mechanical bounce characteristics per unit. T3: Comparison direction (HIGH/LOW) reversed. P1: Swap LOW/HIGH in the edge check. P2: Refactor into a function parameterised by pin.

Experiment 8 — Five-Button LED Controller

C1: Keeps pairing implicit and code compact. C2: 5, exactly kit stock. C3: 10 pin numbers across 10 GPIOs. C4: Level control has no persistent state to corrupt from bounce. C5: Each channel would need its own debounce state array. T1: Mismatched index between the two arrays. T2: A shared GND rail wiring fault. T3: Cumulative GPIO current draw, unlikely but worth checking. P1: Minimal change: extend both arrays by one. P2: Build and print a 5-character status string.

Experiment 9 — Potentiometer LED Dimmer

C1: Otherwise the wiper floats or reads a fixed rail. C2: 1024 discrete codes (0–1023). C3: D8 has no PWM timer. C4: Only appears analog via fast switching (PWM). C5: ≈ 490 Hz on most pins. T1: One outer leg not connected (open divider). T2: ADC reference/wiring fault limiting range. T3: Normal PWM-induced audible coil/piezo effect in some LEDs' leads, not a fault. P1: Print both values via Serial. P2: Add a small dead-zone near 0.

Experiment 10 — LDR-Based Automatic Light

C1: LDR on top means darker (higher R_{LDR}) pulls the node voltage down. C2: The logic would invert. C3: LDR units and lighting vary too much for one fixed value. C4: A single noisy reading near the threshold can flicker output. C5: Response lags light changes by tens of ms. T1: LDR/resistor swapped. T2: Ambient light naturally changes through the day; not a fault. T3: Divider not reaching the analog pin. P1: Add a second, lower "very dark" tier. P2: Print "DARK"/"BRIGHT" text.

Experiment 11 — LDR Light-Level Indicator

C1: Otherwise multiple LEDs could light simultaneously. C2: The condition would need a consistent boundary rule (this manual: uses $<$ then $\geq$). C3: Add one more threshold and LED. C4: No other timing-critical task competes yet. C5: Uncalibrated LDR readings are relative, not absolute lux. T1: An unreachable middle bucket suggests overlapping/misordered conditions. T2: Thresholds poorly separated for the room's actual range. P1: Refactor into a threshold array and loop. P2: Add a 4th "very bright" bucket.

Experiment 12 — Buzzer ON/OFF Control

C1: An active buzzer has its own oscillator; a passive one needs a driven signal. C2: A buzzer's typical current draw is within safe GPIO limits (unlike an LED needing voltage-drop limiting). C3: 20 mA design limit. C4: Reversing may reduce volume or silence a polarised unit. C5: An internal oscillator circuit. T1: Suggests it is actually the passive buzzer. T2: Excess current draw browning out the board. P1: Add three short beeps then silence in `setup()`. P2: Reuse Experiment 6's button pattern.

Experiment 13 — Electronic Doorbell

C1: Experiments 6 and 12. C2: It tracks a live level, not a counted event. C3: Would need debounced edge detection instead. C4: Yes, GND can be shared safely. C5: Output directly mirrors input state. T1: Likely power-up transient, not a fault. T2: Check for a loose/intermittent button connection. P1: Fix a 500 ms sound duration per press using `millis()`. P2: Timestamp each press over Serial.

Experiment 14 — Variable-Tone Buzzer

C1: `tone()` uses a dedicated timer independent of PWM hardware. C2: The frequency of the driving square wave. C3: 20 Hz–20 kHz; narrowed to a musically useful sub-range. C4: A new `tone()` call simply updates the ongoing frequency. C5: Timer2 (implementation-dependent). T1: Expected transient, not a fault. T2: Check the potentiometer's wiring range. P1: Add a button-triggered `noTone()`. P2: Round the mapped frequency to preset note values.

Experiment 15 — IR Obstacle Detector

C1: An onboard comparator already makes the decision in hardware. C2: The comparator's sensitivity/reference threshold. C3: Reflectivity changes how much IR returns to the phototransistor. C4: It only reports presence/absence, not distance. C5: OUT goes LOW when an obstacle is detected. T1: Dark/absorptive surfaces reflect less IR. T2: Normal comparator threshold flicker at the detection boundary. T3: Strong ambient IR (sunlight) can desensitise the receiver. P1: Add a running detection counter. P2: Add debouncing to the digital reading.

Experiment 16 — Ultrasonic Distance Measurement

C1: The pulse travels to the object and back (double distance). C2: The program hangs waiting indefinitely for an echo. C3: Speed of sound varies with temperature/humidity. C4: Around 2 cm datasheet minimum. C5: TRIG initiates the pulse; ECHO reports the reflected timing. T1: Missing division by 2 (or by 58 vs. 29 confusion). T2: Soft fabric absorbs rather than reflects ultrasound. T3: Speed of sound rises slightly with temperature. P1: Average 5 consecutive readings. P2: Add a cm/inch toggle.

Experiment 17 — Ultrasonic Distance Alarm

C1: Prevents a timeout sentinel from being misread as "very close." C2: The alarm would falsely trigger continuously when out of range. C3: Both use continuous sensor reading against a threshold to drive an output. C4: Static output, debounced input, analog PWM, threshold decisions, audio output, two sensor modules. C5: It's illustrative, not a validated safety distance. T1: Sentinel value not excluded from the comparison. T2: Ambient temperature affecting the speed-of-sound assumption. P1: Vary beep rate with distance (developed in Experiment 39). P2: Add a second, closer "danger" LED tier.

Experiment 18 — LCD I²C Hello World

C1: One shared bus regardless of device count via addressing. C2: A4=SDA, A5=SCL. C3: Translates I²C commands to the HD44780 parallel interface. C4: Row/column indices start at 0 by convention. C5: 0x27 and 0x3F. T1: Update the sketch's address constant to match. T2: Missing `setCursor` call for the second row. T3: Loose VCC/GND wiring. P1: Use `scrollDisplayLeft()` in a timed loop. P2: Prevents stale characters from a previous longer string.

Experiment 19 — Potentiometer Value Display

C1: Avoids full-screen flicker on every update. C2: It never changes, so reprinting wastes time/causes flicker. C3: Stale trailing digits remain visible. C4: Column 5 is right after the 5-character label "Pot: ". C5: 0–1023. T1: Missing trailing spaces after a shorter number. T2: The column index used in `setCursor`. P1: Convert to a 0.00–5.00 V display. P2: Show both raw and voltage on two rows.

Experiment 20 — LDR Light Meter

C1: The value is relative/uncalibrated, not true illuminance. C2: A reference light meter and per-unit calibration curve. C3: Manufacturing variation between individual LDR units. C4: Which raw extreme maps to which output extreme. C5: Repeatable for one device, not precise in absolute terms. T1: Not necessarily a fault; individual LDR variation is expected. T2: Verify the divider wiring orientation. P1: Add a moving average. P2: Display raw and % together.

Experiment 21 — Ultrasonic Distance Display

C1: The function was already verified and is display-agnostic. C2: Number of decimal places to print. C3: Sensor timeout or physically out-of-range object. C4: LCD refresh does not need microsecond timing like the trigger pulse. C5: Separating measurement logic from its display/output. T1: Check the sensor wiring, not the LCD. T2: Missing trailing spaces. P1: Show both cm and inches. P2: Add an LCD block-character bar graph.

Experiment 22 — Digital Ruler

C1: Resolution is the smallest distinguishable change; accuracy is closeness to truth. C2: Beam spread/surface angle/temperature dominate over timing resolution. C3: The datasheet range catches implausible non-zero pulses the timeout alone would miss. C4: The transmitted pulse burst duration and near-field effects. C5: Ambient temperature (speed of sound). T1: MIN_CM set too high. T2: The valid-range check is missing or wrong. P1: Show the spread of the last 5 readings. P2: Reject large single-step outliers.

Experiment 23 — Temperature Display

C1: The 1-wire protocol needs settling time between transactions. C2: Whether the last read failed ("Not a Number"). C3: $\pm 2^\circ\text{C}$. C4: It uses a proprietary timed pulse-train protocol. C5: An internal thermistor element. T1: Occasional errors are normal for DHT11, not necessarily a fault. T2: A very high implausible reading usually means a failed/noisy transaction. T3: Check DHT11 wiring before the LCD. P1: Update only on a meaningful change. P2: Count failed reads since power-up.

Experiment 24 — Humidity Display

C1: A capacitive/resistive polymer element's electrical property. C2: $\pm 5\%\text{RH}$. C3: Breath adds local water vapour near the sensor. C4: Relative, referenced to the maximum the air could hold at that temperature. C5: Because both come from one physical read transaction. T1: A stuck-at-0 usually indicates a fault distinct from a NaN error. T2: Sensor response is naturally somewhat slow; not necessarily a fault. P1: Add a comfort classification. P2: Track min/max since power-up.

Experiment 25 — Temperature Alarm

C1: Reading noise near a single hard boundary. C2: Only clears once clearly below the lower bound. C3: Named constants make intent and reuse clearer. C4: The alarm would never turn off (or never turn on). C5: It is a simple two-constant margin, not a generalised function yet. T1: Buzzer line and LCD state update likely out of sync/order. T2: The turn-off condition is unreachable or inverted. P1: Test a 5°C margin. P2: Log every alarm transition with a timestamp.

Experiment 26 — Temperature and Humidity Monitor

C1: To let `loop()` stay responsive for future concurrent tasks. C2: Reads faster than the DHT11 datasheet allows, risking errors. C3: Correctness across the `millis()` overflow point. C4: Buttons, other sensors, additional logic. C5: Both come from one sensor transaction, best kept together. T1: Check whether the guard condition fires twice due to a race in the comparison. T2: Standard rollover-safe subtraction handles this automatically; no action needed. P1: Add an independently blinking heartbeat LED. P2: Drive the interval from the potentiometer.

Experiment 27 — Soil Moisture Monitor

C1: More ions/water lowers resistance between the probes. C2: Soil type/mineral content vary widely. C3: An out-of-calibration-range mapped percentage. C4: Corrosion changes probe behaviour over time. C5: The two reference raw readings used for scaling. T1: WET_RAW likely needs updating for your specific probe. T2: Sign/order of the two calibration constants swapped. P1: Print raw and % together during calibration. P2: Conceptually store constants in EEPROM.

Experiment 28 — Plant Moisture Alarm

C1: Threshold-triggered alarm from a calibrated sensor reading. C2: Reused unchanged across Experiments 17, 25, and here. C3: The plant would be watered later than intended. C4: A commonly cited general houseplant guideline, not universal. C5: Same structural pattern, different sensor/actuator pairing. T1: Check the update order between the percentage calc and the alarm check. T2: An open sensor circuit typically reads as an extreme value, not an explicit error. P1: Add hysteresis around 30%. P2: Add an auto-silence timeout.

Experiment 29 — Automatic Plant-Watering Controller Demonstration

C1: The coil/contact side is electrically separate from the logic side. C2: Mains switching is outside this kit's safety scope. C3: A real irrigation pump or valve. C4: The relay's armature physically moving. C5: No real pump is included in the kit; only the decision logic is complete. T1: Check VCC/GND/IN wiring on the relay module. T2: Check the COM/NO branch wiring to the LED. P1: Add a maximum relay-on duration. P2: Add hysteresis at the 30% boundary.

Experiment 30 — Smart Night Lamp

C1: Swapping the input range's order inverts the mapped output. C2: Smooth response feels more natural and less jarring. C3: Experiments 9 and 10. C4: PWM brightness control requires a timer-capable pin. C5: Prevents an out-of-range mapped duty value. T1: Check the `map()` argument order assumption. T2: The update rate (delay value) controls responsiveness. P1: Add a manual override button. P2: Apply a simple smoothing filter.

Experiment 31 — IR Security Alarm

C1: This one latches until manually cleared; Experiment 17 tracked a live condition. C2: Named states are more self-documenting than several booleans. C3: Only the button branch is allowed to clear it, by design. C4: The alarm persists after the triggering condition ends. C5: Three: DISARMED, ARMED, ALARM. T1: Check whether the sensor branch can also affect state. T2: Debounce/edge logic likely not matching Experiment 7 exactly. P1: Add a status LED reflecting each state. P2: Add a timed entry delay (formalised in Experiment 40).

Experiment 32 — Object Counter

C1: Comparator noise can bounce similarly to a mechanical switch. C2: Counting every LOW reading would over-count one pass. C3: `unsigned int`, since counts are never negative. C4: Avoids unnecessary LCD writes on idle passes. C5: Every reading would count as a new edge (severe over-count). T1: Debounce interval likely needs adjusting for that pass speed. T2: Likely a power interruption, not a code bug. P1: Add a debounced reset button. P2: Timestamp each count event.

Experiment 33 — Smart Parking Indicator

C1: Avoids collision with the LED pins used elsewhere in the same circuit. C2: A safe low-voltage barrier demonstration, not a real actuator. C3: All three LEDs are explicitly reset before the bucket decision. C4: Three: FREE, PARKING, FULL. C5: An out-of-range distance is treated as "space clearly free." T1: Check the three threshold conditions for overlap. T2: Check that the relay line sits only inside the FULL branch. P1: Add hysteresis at the FULL boundary. P2: Log FULL events with timestamps.

Experiment 34 — IR Remote Controller Demonstration

C1: No IR transmitter/remote is included in the kit by design. C2: Distinguishes remote signals from ambient IR/interference reliably. C3: Prepares the receiver for the next signal; omitting it stalls further decodes. C4: Some protocols include a toggle bit that flips each press. C5: VCC, GND, and OUT. T1: Yes, this is typical IR range/angle behaviour. T2: The remote itself may be faulty/mismatched, or in a different protocol. P1: Map exactly three codes to LED on/off/toggle. P2: Print the protocol name alongside the hex code.

Experiment 35 — Smart Room Light Controller

C1: A human should always be able to override automation. C2: Three, cycling AUTO → MANUAL-ON → MANUAL-OFF → AUTO. C3: The current control mode (AUTO or MANUAL). C4: It is only recalculated from the LDR while in AUTO mode. C5: Thermostats, smart-home apps, industrial control panels. T1: Add hysteresis exactly as in Experiment 25/44. T2: Check that the third-press branch correctly returns to AUTO. P1: Add a mode-indicator LED. P2: Add an automatic MANUAL-mode timeout.

Experiment 36 — Room Environment Monitor

C1: Sensing and page-flip timing are legitimately independent concerns. C2: A momentary failed read does not blank/corrupt the display. C3: One place is responsible for rendering, called from two contexts. C4: Pages would flip before a fresh sensor value is even available. C5: It is simply clearer for a small, fixed set of pages; either works. T1: Check the cache is not being reset on each flip. T2: Check the read-timer guard condition. P1: Add a compact third page with both values. P2: Add an immediate manual flip via a button.

Experiment 37 — Smart Plant Monitoring System

C1: `analogRead` has no minimum-interval requirement. C2: Soil readings would lag unnecessarily behind real changes. C3: A single function keeps rendering logic centralised regardless of page count. C4: Storing a value once measured for reuse across passes. C5: It reuses two independently developed sensor experiments rather than inventing new theory. T1: Check it isn't accidentally placed inside the DHT11 guard. T2: Expected momentarily with `lcd.clear()`; not a fault. P1: Add a combined third page. P2: Add a button-triggered immediate flip.

Experiment 38 — Automatic Plant Protection System

C1: Otherwise "critical" could occur without also being "dry." C2: It is the more severe, higher-priority condition. C3: Escalate automatically before requiring human intervention. C4: Each reflects a distinct real-world response with its own threshold. C5: Both would engage/disengage simultaneously, losing the tiered response. T1: Check the `if/else if` message-priority order. T2: Check the `alarmOn` condition's threshold value. P1: Add hysteresis to both thresholds. P2: Add a maximum relay-on duration.

Experiment 39 — Smart Parking Assistant

C1: It would stall distance measurement between beeps. C2: Keeps the mapped interval within sensible playable bounds. C3: A toggle produces an audible beep pattern rather than a steady tone. C4: The system reports "SAFE" and stops beeping. C5: A fixed set of discrete buckets becomes a continuous function of distance. T1: Check the `constrain()` bounds are not too narrow. T2:

Check the buzzer pin/wiring, since the LCD update is independent. P1: Add a continuous tone below a fixed danger distance. P2: Smooth the distance reading before mapping.

Experiment 40 — Electronic Security System

C1: States, an initial state, events, and a transition function. C2: Keeps state-change reasoning separate from output-driving reasoning. C3: Gives the user time to leave the protected area after arming. C4: A timestamp allows an elapsed-time comparison each pass. C5: Each diagram transition corresponds to one `switch` case/branch. T1: Check the blink-rate divisor in the expression. T2: Check that ALARM and DISARMED are handled as genuinely distinct cases. P1: Add a post-trigger exit delay before sounding. P2: Log every transition with a timestamp over Serial.

Experiment 41 — Multi-Zone Security Detector

C1: It senses a broken light beam rather than reflected IR proximity. C2: Simplicity; continuous power is adequate for this beam-break use. C3: Lets a user or log identify the actual source of a trigger. C4: Stray ambient light or slight beam misalignment. C5: Two independent, distinguishable trigger sources feed one alarm. T1: Likely ambient light or slight misalignment, not a wiring fault. T2: The system is designed to latch until explicitly disarmed. P1: Merge with Experiment 40's full FSM including entry delay. P2: Add per-zone event counters.

Experiment 42 — Digital Distance Warning System

C1: To ensure hysteresis is anchored to the tier the system is actually in. C2: Avoids alarm fatigue from a merely cautionary tier. C3: Rapid flicker/chatter at that single boundary. C4: The boolean expression's 0/1 value drives the pin directly. C5: Because the most severe condition alone determines the alarm. T1: Check the two margin values are wide enough and correctly ordered. T2: Check the buzzer condition is restricted to the CLOSE tier only. P1: Add a 4th "very close" tier with rapid beeping. P2: Log tier changes with timestamps.

Experiment 43 — Light Intensity Measurement System

C1: It compares two sensors against each other, not against an absolute standard. C2: Manufacturing tolerance between individual LDR units. C3: Small, insignificant differences from normal analog noise. C4: The kit provides exactly two LDRs. C5: Not directly on one Arduino UNO's 6 analog pins beyond a few channels, though more are technically possible. T1: Suggests an inherent difference between the two LDR units, not wiring. T2: The dead-band threshold should be increased. P1: Display the numeric difference too. P2: Add a simple bar-graph comparison.

Experiment 44 — Automatic Day/Night Controller

C1: Keeps the function pure/stateless and reusable by different callers. C2: Wide enough to exceed sensor noise, narrow enough to stay responsive. C3: Same behaviour, refactored into one reusable, testable function. C4: It compares against a rising condition unchanged in structure. C5: The ON and OFF conditions could both become unreachable or reversed. T1: Check the threshold constants for an unintended narrow/zero gap. T2: Check whether the constants suit your actual room's range. P1: Make the gap width potentiometer-adjustable. P2: Log every state change with its triggering value.

Experiment 45 — Temperature-Controlled System

C1: The comparison direction is mirrored (turn on above high, not below low). C2: The same general margin-based state-change pattern still applies. C3: Prevents reading faster than the DHT11 allows. C4: The ON/OFF conditions would become unreachable or reversed. C5: DHT11 readings are inherently floating-point. T1: Check that the LCD print occurs after the state update, not before. T2: Check the temperature gap width. P1: Accept thresholds via Serial input. P2: Add a minimum relay-on duration.

Experiment 46 — IR Remote Smart Controller

C1: Remote codes are specific to the individual remote used. C2: Repeat codes differ from the stored button constants and simply don't match. C3: Both require recognising a discrete event rather than a continuous level. C4: IR codes can exceed a 16-bit integer's range. C5: Both would respond to whichever remote sends that code. T1: Not surprising; many remotes/protocols reuse simple codes. T2: A protocol quirk in repeat-code timing/detection. P1: Add a third combined-toggle mapped button. P2: Print any unmapped code to help discover new ones.

Experiment 47 — Multi-Sensor Environmental Station

C1: Spreads sensing load over time instead of polling everything every pass. C2: A variable holding the address of a function, callable via that variable. C3: Only a new function and array entry are needed. C4: Keeps scheduling logic simple and centralised. C5: It visually communicates data flow and system structure at a glance. T1: Not necessarily; out-of-range readings are a normal HC-SR04 outcome. T2: A compiler warning or an out-of-bounds runtime access. P1: Use a per-slot interval array instead of one shared constant. P2: Add a combined summary page.

Experiment 48 — Smart Classroom Automation Prototype

C1: It only detects doorway crossings, not continuous presence. C2: Lighting should work correctly even in an empty room. C3: It is derived fresh each pass from a timeout comparison. C4: Avoids nuisance alerts in an unoccupied room. C5: A dedicated PIR motion sensor, not included in this kit. T1: A single doorway sensor cannot distinguish "still present" from "left." T2: Add hysteresis around the comfort threshold, as in Experiment 25. P1: Add hysteresis to the comfort check. P2: Log doorway crossings with timestamps.

Experiment 49 — Security and Environment Monitoring Station

C1: Safety-relevant information must never be silently displaced by routine data. C2: So an up-to-date reading is ready the instant the alarm clears. C3: Prioritised information display in a shared-resource system. C4: It combines two subsystems under an explicit display-priority rule. C5: Environmental data could interrupt/hide a safety-critical alarm. T1: Check that the ALARM check is the first condition evaluated. T2: Check the DHT11 read-timer guard is independent of security state. P1: Add a third, lower-priority "sensor fault" tier. P2: Flash the alarm text using `millis()`.

Experiment 50 — Mini Smart Home Automation System

C1: They are safety/comfort-critical and must not wait behind slower scheduling. C2: It shows the whole design fits comfortably within one Arduino UNO's I/O budget. C3: It only approximates presence via doorway crossings, not true occupancy. C4: Routine data could hide or interrupt a security alarm. C5: Experiments 40 (FSM), 44 (hysteresis), and 47/49 (scheduling and priority).

T1: A duplicate variable/function name from merging multiple sketches' code. T2: Revisit the electrical limits and common-ground checks in Chapter . P1: Discuss trade-offs; behaviour should be unchanged if refactored correctly. P2: A compile-time flag isolating one subsystem for focused testing.

Appendix L

Student Observation Sheets

Each experiment already provides a blank, ruled Observation Table in its own chapter, sized for that experiment's specific readings. For additional practice or repeat trials, photocopy the relevant experiment's Observation Table, or use the generic blank sheet below for any experiment:

Trial #	Reading 1	Reading 2	Notes
---------	-----------	-----------	-------

Appendix M

Experiment Assessment Rubric

Criterion (weight)	Descriptors
Circuit correctness (25%)	Wiring matches the connection table; correct polarity/pin usage; no shorts
Code correctness (25%)	Compiles without modification; produces the expected output; follows the given algorithm
Understanding (20%)	Can explain the working principle and answer viva questions correctly
Observation & analysis (15%)	Observation table completed with plausible values; calculations shown
Troubleshooting & modification (10%)	Can diagnose an introduced fault; completes at least one programming exercise
Lab discipline (5%)	Safety rules followed; workspace/components returned to the storage box

Suggested grade bands: 90–100% Excellent, 75–89% Good, 60–74% Satisfactory, below 60% Needs Repeat.

Appendix N

Final Project Evaluation Checklist

For Experiment 50 (Mini Smart Home Automation System capstone), evaluate against:

- ☐ Written system requirements provided before implementation.
- ☐ Pin allocation table matches the actual wiring, with no conflicts.
- ☐ All 5 validation checklist items (Experiment 50) pass.
- ☐ Student can explain, without notes, why security display pre-empts environmental display.
- ☐ Student can identify at least 2 genuine limitations of their own build.
- ☐ Student proposes at least 1 credible future enhancement beyond the manual's list.
- ☐ Code is organised into logical sections/functions, not one undifferentiated block.
- ☐ No blocking `delay()` longer than a few milliseconds appears anywhere in the final sketch.

Appendix O

References and Further Reading

- Official Arduino documentation and language reference — `docs.arduino.cc`.
- ATmega328P datasheet — Microchip Technology Inc.
- HC-SR04 ultrasonic sensor datasheet (consult your module's printed/vendor datasheet for exact timing figures).
- DHT11 datasheet (consult your module's printed/vendor datasheet for exact accuracy figures).
- `LiquidCrystal_I2C` library documentation (Frank de Brabander), via the Arduino Library Manager.
- `DHT sensor library` and `Adafruit Unified Sensor` documentation — Adafruit Industries.
- `IRremote` library documentation (Armin Joachimsmeier), via the Arduino Library Manager.
- PCF8574 I/O expander datasheet (the chip on the I²C LCD backpack) — Texas Instruments / NXP.

Always prefer the current official documentation and your specific module's datasheet over any numeric value quoted in this manual, since component variants and library versions change over time.